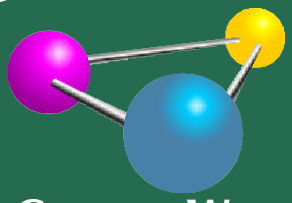
FINAL Version

Supports Through Android 9.0!

The Busy Coder's Guide to Android Development

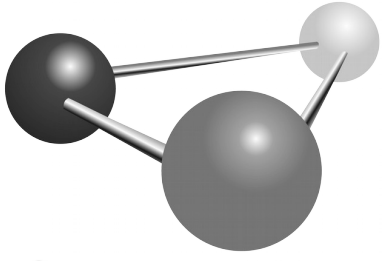
Mark L. Murphy



CommonsWare

Руководство по разработке на Андроид занятого программиста

by Mark L. Murphy



COMMONSWARE

Руководство по разработке на Андроид занятого программиста
by Mark L. Murphy

Copyright © 2008-2019 CommonsWare, LLC. Все права
зарегистрированы. Распечатано в США

История печати:
Февраль 2019 ФИНАЛЬНАЯ версия ISBN: 978-0-9816780-0-9

Название и логотип CommonsWare, “Busy Coder's Guide”, и относящиеся торговый вид - торговая марка CommonsWare, LLC.

Все другие торговые марки на которые ссылается данная книга - торговые марки соответствующих фирм.

Издатель и автор(ы) предполагают никакой ответственности за ошибки или упущения или за повреждения в результате использования информации содержащейся здесь.

Содержание

Заголовки отформатированы в ***bold-italic*** изменены по сравнению с прошлой версией.

- Предисловие
  - Добро пожаловать в Книгу!.....xliii
  - Книга первого поколения.....xliii
  - Структура книги.....xliv
  - Пути.....xlv
  - Об обновлениях.....l
  - О APK издании.....l
  - Исходный код и его лицензия.....li
  - Creative Commons и Four-to-Free (42F) гарантии.....li
  - Благодарности.....lii
- Ключевые концепты Андроид
  - Приложения Андроид.....1
  - Устройства Андроид.....6
  - Не пугайтесь.....10
- Выбор Вашего набора инструментов разработчика
  - Android Studio.....11
  - Eclipse.....11
  - IntelliJ IDEA.....12
  - Компиляция в командной строке через Gradle.....12
  - Ещё другие альтернативы.....12
  - IDE... и данная книга.....13
  - Что мы не охватываем.....13
- Руководство №1 - Установка инструментов
  - Но сперва некоторые заметки о эмуляторе Андроид.....15
  - Шаг №1: Проверьте Ваши аппаратные средства.....16
  - Шаг №2: Настройка Java и поддержки 32 битных Линукс.....17
  - Шаг №3: Установка Android Studio.....18
  - Шаг №4: Установка SDK и дополнений.....19
  - В нашем следующем эпизоде....27
- Андроид и проекты
  - Проекты и Android Studio.....29
- Руководство №2 - Создание проекта с заглушками
  - Про наш проект руководства.....37
  - Про остальное в руководстве.....38
  - О наших инструментах.....38

- Шаг №1: Импорт проекта 38
- Шаг №2: Подготовка к эмулятору x86 43
- Шаг №3: Настройка AVD 44
- Шаг №4: Настройка устройств 51
- Шаг № 5: Запуск проекта 56
- В нашем следующем эпизоде... 58

- Знакомство с Android Studio
  - Навигация в Project Explorer 59
  - Запуск проектов 62
  - Просмотр вывода 63
  - Доступ к инструментам Андроид 65
  - Android Studio и каналы релизов 66
  - Посетите пути! 68
- Содержание проектов Андроид
  - Что Вы получаете, в общем 69
  - Боле о структуре папок 72
  - Что вы получаете после этого 74
- Введение в Gradle и Manifest
  - Gradle: Главные вопросы 75
  - Получение Gradle 77
  - Версии Gradle и плагина Андроид Gradle 79
  - Переменные окружения Gradle 80
  - Изучение файлов Gradle 80
  - Введение в Manifest 83
  - Общие элементы между Manifest и Gradle 84
  - Другие примечательные элементы Gradle 86
  - Где GUI? 87
  - Остальное в Manifest 87
  - Изучаем о Gradle больше 89
  - Посетите пути! 90
- Руководство №3 - Изменения в Manifest
  - Некоторые заметки о относительных путях 91
  - Шаг №1: Поддерживаемые экраны 91
  - Шаг №2: Блокирование резервного копирования 92
  - Шаг №3: Игнорирование Lint 92
  - В нашем следующем эпизоде... 94
- Несколько слов о ресурсах
  - Теория строковых значений 95
  - Получение рисунка? 102
  - Размерности 105
  - Ресурсы, которые не должны именоваться... Пока 107

Значки приложение… И все остальное 109
- Создание зкачка приложения с применением Asset Studio 110
- Создание других значков с применением Asset Studio 116

- Значки
- Руководство №4 - Установка наших ресурсов
  - Шаг №1: Изменение имён 117
  - Шаг №2: Изменение значка.....118
  - Шаг №3: Запуск результатов.....123
  - В нашем следующем эпизоде… 123
- Теория виджетов
  - Что такое виджеты? 125
  - Size, Margins, and Padding (Размеры, поля и отступы) 127
  - Что такое контейнеры? 127
  - Антипаттерн абсолютного позиционирования.....128
  - Тема данного раздела: Темы 129
- Интерфейс пользователя Андроид
  - Активность (Activity) 135
  - Анализируем активность 136
  - Использования макетов (Layout) на основе XML 137
- Базовые виджеты
  - Общие концепции..... 141
  - Введение в графический редактор макетов 144
  - И сейчас, несколько заметок о простых проектах в книге 153
  - Присвоение надписей. 154
  - Командные кнопки 159
  - Быстрые рисунки 162
  - Поля зелёного Или других цветов 168
  - Больше общих концепций 171
  - Посетите пути!.....174
- Отладка Вашего приложения
  - Получение Stack Trace 176
  - Случаи ошибок вызова класса 177
  - Точки прерывания 178
- Классический контейнер классов
  - Введение в приложение шаблон 179
  - RTL и Ваши макеты..... 180
  - LinearLayout и модель Box 181
  - Все вещи относительны 207
  - Tabula Rasa 217
  - Эй, что о ConstraintLayout?..... 223
  - Возвращение к RTL 224

- Остальные стандартные виджеты и контейнеры
  - Просто Box to Check 227
  - Вам не нравятся Checkboxes? Как насчёт Toggles и Switches 231
  - Включение Radio 235
  - Scrollwork 238
  - Создаём прогресс с ProgressBars 242
  - Обрамление Scene 242
  - Посетите пути! 243
- Руководство №5 - Создание макета
  - Шаг №1: Создание нового ресурса макета 245
  - Шаг №2: Определение UI 246
  - В нашем следующем эпизоде… 250
- Построение GUI, продолжение
  - Создание собственных Selection 252
  - Включения включаются 252
  - Предварительный просмотр предстоящих Attractions 254
- AdapterViews и Adapters
  - Адаптация к условиям 256
  - Список Naughty и Nice 257
  - Clicks против Selections 260
  - Контроль вращения 263
  - Заприте в сетку своих львов (или что-то типа того..) 267
  - Поля: Теперь на 35% меньше печатания! 272
  - Изменяем Adapter 277
  - Посетите пути! 285
- Виджет WebView
  - Роль WebView 287
  - Папа, откуда появились WebViews? 288
  - Добавление виджетов 288
  - Загрузка контента через URL 289
  - Ссылки и перенаправления 291
  - Поддержка JavaScript 291
  - Альтернативы загрузки контента 292
  - Перехват событий 293
  - Поведение адресации ссылок/перенаправлений 296
  - Выходим из Google Monitoring 298
  - Посетите пути! 299
- Определение и использование стилей
  - Стили: DIY DRY 301
  - Элементы стилей 303
  - Темы: Будет ли стиль под любым другим именем … 306

- Что случится, если у Вас отсутствует темы 306
- Редактор тем Android Studio 307

- Зависимости
  - Что такое зависимость? 313
  - Применения зависимостей 314
  - Зависимости в локальном JAR 315
  - Что такое артефакт 316
  - Артефакты и репозитории 316
  - Главное семейство библиотек от Google 316
  - Запрос зависимостей 317
  - Библиотека поддержки Андроид 320
- Руководство №6 - Добавление библиотек
  - Шаг №1: Получение инструментов существующих остатков 325
  - Шаг №2: Запрос новых зависимостей 326
  - В нашем следующем эпизоде... 328
- Введение в ConstraintLayout
  - Почему другой контейнер? 329
  - Сравнение с классикой 331
  - Получение ConstraintLayout 331
  - Using Widgets and Containers from Libraries 332
  - Using a ConstraintLayout 332
  - Конвертирование существующих слоёв 346
  - Посетите пути! 348
- RecyclerView
  - AdapterView и его недовольство 349
  - Вход RecyclerView 350
  - Стандартный список 351
  - Настройки разделителя 359
  - Перехват событий нажатия 366
  - Посетите пути! 373
- Action Bar
  - Тур по барам 375
  - Ещё один урок истории 382
  - Настройки Вашего Action Bar 382
  - Установка цели 384
  - Определение ресурсов 386
  - Присвоение ресурсов 388
  - Отклик на события 388
  - Остальное в простой активности 389
  - Кнопка МЕНЮ, мы едва знали тебя 396
  - Action Bars, живи в живых цветах! 397

- Посетите пути!.....407
- Векторные отображения
  - Получение рисунка..... 409
  - VectorDrawableCompat.....414
- Руководство №7 - Настройка Action Bar
  - Шаг №1: Добавление некоторых значков.....421
  - Шаг №2: Определение некоторых настроек 423
  - Шаг 3: Загрузка и отклик на наши опции.....426
  - Шаг №4: Поддержка старых устройств 427
  - Шаг №5: Попробуйте это 429
  - В нашем следующем эпизоде... 430
- Модель процессов Андроид
  - Когда создаётся процесс 431
  - BACK, HOME, и Ваш процесс 432
  - Завершение..... 433
  - Передний план означает "Я люблю тебя" 434
  - Вы и Ваше множество 434
- Активные и их жизненные циклы
  - Создание Вашей второй (и третьей и ...) активности.....436
  - Внимание! Содержит явные вызовы 440
  - Использование неявных вызовов.....442
  - Дополнительно! Дополнительно! 448
  - Продумывание Parcelable.....450
  - Асинхронности и результаты 451
  - Активности Шредингера 451
  - Жизнь, смерть и Ваша активность 452
  - Когда активность умирает.....454
  - Проходим через жизненный цикл 455
  - Восстановление активности.....458
  - Приложение: Преодолевая активность.....459
  - Случай невидимой активности 460
- Руководство №8 - Настройка активности
  - Шаг №1: Создание пустого класса активности и записи в манифест463
  - Шаг №2: Запуск нашей активности.....465
  - В нашем следующем эпизоде... 466
- Тактика фрагментов
  - Шесть вопросов 467
  - Откуда Вы берете свои франменты 470
  - Ваш первый фрагмент 471
  - Методы жизненного цикла фрагментов 475
  - Ваш первый динамический фрагмент 477

- Фрагменты и Action Bar 480
- Руководство №9 - Запуск наших фрагментов
  - Шаг №1: Создание SimpleContentFragment 483
  - Шаг №2: Изучаем SimpleContentFragment 485
  - В нашем следующем эпизоде... 486
- Пролистывание с ViewPager
  - Части Pager 487
  - Располагаем на странице фрагменты 488
  - Располагаем другие вещи 493
  - Индикаторы 494
  - Пересмотр примера контейнеров 497
- Руководство №10 - Оснащаем ViewPager
  - Шаг №1: Добавляем ViewPager на Layout 505
  - Шаг №2: Создаём ContentsAdapter 506
  - Step #3: Setting Up the ViewPager 507
  - В нашем следующем эпизоде... 510
- Наборы ресурсов и конфигурации
  - Что такое конфигурации? И как они изменяются 511
  - Конфигурации и наборы ресурсов 512
  - Размер экрана и ориентация 513
  - Справляемся со сложностью 516
  - Выбор правильного ресурса 517
  - Ресурсы в зависимости от версии API 521
  - Поведение при изменениях по умолчанию 523
  - Сценарии сохранения состояний 525
  - Ваши возможности по изменениям конфигурации 526
  - Блокировка вращения 540
  - И сейчас мир от вида из проекта Андроид 541
  - Проблемы конфигураций 543
- Основы Material Design
  - Ваше приложение в техноцвете 549
- Договариваемся с потоками
  - Основной поток приложения 555
  - Выполнение в фоновом режиме 557
  - Ощущения асинхронности 557
  - Альтернативы к AsyncTask 567
  - И сейчас, Предостережения 569
  - Шины событий 569
  - Посетите пути! 578
- Запрос разрешений
  - Часто задаваемые вопросы о разрешениях 580

- Характеристики разрешения 585
- Новые разрешения в старых приложениях 588
- Система разрешений на лету в Android 6.0+ 589
- Простая абстракция разрешений на лету 603

- Наборы, файлы и парсинг данных
  - Упаковка файлов в вашем приложении 607
  - Файлы и андроид 609
  - Работы с внутренним хранилищем 610
  - Работы с внешним хранилищем 618
  - Извлекаемые хранилища 624
  - Множественные аккаунты пользователей 625
  - Файловые системы линукс: Вы синхронизируете, Вы выигрываете 626
  - StrictMode: Избегание мусорного кода 628
  - Файлы и Ваша машина разработки 630
  - Опции парсинга XML 637
  - Опции парсинга JSON 638
  - Использование файлов и Implicit Intents 638
  - Посетите пути! 639
- Руководство №11 - Добавление простого наполнения
  - Шаг №1: Добавление некоторого контента 641
  - Шаг №2: Использование SimpleContentFragment 642
  - Шаг №3: Запуск нашей активности, в действительности на этот раз 643
  - Шаг №4: Получение немного больше Material 644
  - Шаг №5: Просмотр результатов 646
  - В нашем следующем эпизоде... 648
- Руководство №12 - Отображение книги
  - Шаг №1: Добавление книги 649
  - Шаг №2: Создание ModelFragment 650
  - Шаг №3: Определение нашей модели 651
  - Шаг №4: Изучение нашей модели 653
  - Шаг №5: Определение наших событий 653
  - Шаг №6: Загрузка нашей модели 654
  - Шаг №7: Регистрация событий 657
  - Шаг №8: Адаптация содержимого 658
  - Шаг №9: Отображение содержимого по загрузке 659
  - Шаг №10: Прикрепление нашего ModelFragment 660
  - Шаг №11: отображение содержимого после изменения настроек 661
  - Шаг №12: Настройка StrictMode 662
  - В нашем следующем эпизоде... 663
- Использование предпочтений
  - Получение того, чего хотите 665

- Заявление Ваших предпочтений 666
- Сбор предпочтений с PreferenceFragment 666
- Редактор предпочтений Android Studio 675
- Типы предпочтений 675

- Руководство №13 - Использование некоторых предпочтений
  - Шаг №1: Определение файлов предпочтений XML 683
  - Шаг №2: Создание нашей активности предпочтений 687
  - Шаг №3: Добавление к нашему Action Bar 688
  - Шаг №4: Запуск активности предпочтений 688
  - Шаг №5: Загрузка предпочтений 691
  - Шаг №6: Сохранение положения последнего чтения 695
  - Шаг №7: Восстановление положения последнего чтения 696
  - Шаг №8: Оставляем экран включенным 697
  - В нашем следующем эпизоде... 699
- Базы данных SQLite
  - Введение в SQLite 701
  - Думаем о Schemas 702
  - Начинаем с Helper 702
  - Получение данных 707
  - Остальное в CRUD 714
  - Эй, как насчёт Hibernate? 720
  - Но, как насчёт Room? 720
  - Посетите пути! 720
- Руководство №14 - Сохранение заметок
  - Шаг №1: Добавление DatabaseHelper 723
  - Шаг №2: Изучение DatabaseHelper 724
  - Шаг №3: Создание NoteFragment 725
  - Шаг №4: Изучение NoteFragment 726
  - Шаг №5: Создание NoteActivity 726
  - Шаг №6: Изучение NoteActivity 727
  - Шаг №7: Добавление заметок в Action Bar 728
  - Шаг №8: Определение NoteLoadedEvent 730
  - Шаг №9: Загрузка заметок из базы данных 731
  - Шаг №10: Загрузка заметки в Fragment 732
  - Шаг №11: Обновление базы данных 733
  - Шаг №12: Сохранение заметки 734
  - Шаг №13: Добавление и удаление элемента Action Bar 736
  - Шаг №14: Закрытие NoteFragment при удалении 738
  - В нашем следующем эпизоде... 744
- Доступ к интернету
  - DIY HTTP 745

    - Что насчёт HttpClient? 755
    - HTTP через DownloadManager 756
    - Использование сторонних библиотек 756
    - SSL 757
    - Использование библиотек HTTP Client 757
    - Посетите пути! 778
- Intents, Intent Filters
    - Что такое Ваш Intent? 779
    - Заявляем Ваш Intent(ions) 781
    - Отклик на неявные Intents 781
    - Запрос неявных Intents 784
    - ShareActionProvider 789
    - Практикуем безопасные разрешения контента 792
- Broadcasts и Broadcast Receivers
    - Остановленное состояние 797
    - Примеры системных Broadcasts 798
    - Порядок вещей 807
    - Оставляйте их локальными 808
    - Посетите пути! 809
- Руководство №15 - Обмен Вашими заметками
    - Шаг №1: Добавление ShareActionProvider 811
    - Шаг №2: Поделиться заметкой 812
    - Шаг №3: Тестирование результатов 815
    - В нашем следующем эпизоде… 818
- Services и паттерны комманд
    - Почему Services 819
    - Настойка Service 820
    - Связь с Services 823
    - Сценарий: Музыкальный проигрыватель 825
    - Сила PendingIntent 828
    - Связь от Services 831
    - Сценарий: Загрузчик 833
    - JobIntentService 840
    - IntentService или JobIntentService? 844
    - Services и изменения конфигураций 844
    - Где заканчиваются Services? 845
    - Ограничение фонового режима Service 846
- Руководство №16 - Обновление книги
    - Шаг №1: Добавление пустой DownloadCheckService 851
    - Шаг №2: Привязка Service к Action Bar 852
    - Шаг №3: Определение наших событий 854

- Шаг №4: Определение нашего JSON 854
- Шаг №5: Определение нашего интерфейса Retrofit 855
- Шаг №6: Получение нашего JSON через Retrofit 856
- Шаг №7: Загрузка обновления 858
- Шаг №8: Распаковка обновления 859
- Шаг №9: Использование обновления 862
- В нашем следующем эпизоде... 867

- Руководство №17 - Поддержка больших экранов
  - Step #1: Creating Our Layouts 869
  - Шаг №2: Загрузка нашего Sidebar Widgets 874
  - Шаг №3: Открытие Sidebar 874
  - Шаг №4: Загрузка содержимого в Sidebar 875
  - Шаг №5: Удаление содержимого из Sidebar 878
- Обратная совместимость стратегии и тактики
  - Думай наперёд, не назад 885
  - Цель, куда Вы идёте 887
  - Окружение богатой цели 887
  - Lint: это не только для Belly Buttons 888
  - Небольшая помощь от Ваших друзей 889
  - Избегайте нового на старом 889
  - Тестирование 893
  - Продолжайте трассировать изменения 893
- Системные сервисы
  - Что такое System Service? 895
  - Какие System Services находятся там? 896
- Google Play Services
  - Что такое Google Play Services? 899
  - Что внутри Play Services SDK? 900
  - Добавление Play Services в Ваш проект 906
- Получение помощи
  - Вопросы. Иногда с ответами. 917
  - Направление к источнику 918
- Работа с модулями библиотек
  - Предпосылки 921
  - Создания модуля библиотеки 921
  - Использование модуля библиотеки 924
  - Модули библиотек и Manifest 924
  - Модули библиотек и переходящие зависимости 925
  - Ограничения модулей библиотек 926
- Gradle и задачи
  - Задачи относящиеся к Key Build 927

    - Результаты 930
- Варианты сборок Gradle
    - Предпосылки 931
    - Задачи структуры проекта 931
    - Терминология 932
    - Конфигурирование стоковых типов сборок 935
    - Добавление типов сборок 939
    - Добавление Product Flavors и получение сборок VariantsBuild 939
    - Выполнение разделений 941
    - Gradle и Android Studio 943
- Правила слияния Manifest
    - Предпосылки 947
    - Сценарии Manifest 947
    - Части генерации Manifest 949
    - Изучение результатов слияния 951
    - Просмотр составных Manifest в Android Studio 951
    - Слияние элементов и аттрибутов 952
    - Использование заполнителей ? 962
- Подпись вашего приложения
    - Предпосылки 967
    - Роль подписи кода 967
    - Что происходит в режиме отладки 968
    - Ключи подписи этапа производства 969
- Распространение
    - Предпосылки 977
    - Будь готов выйти на Market 977
- Написание дополнения Gradle
    - Предпосылки 983
    - Кастомизация Gradle Build 983
    - Некоторые примеры использования для собственных дополнений 984
    - Написание дополнения 985
    - Распространение плагина 992
    - Использование плагина 995
    - Создание настоящего плагина 998
- Генерация кода
    - Предпосылки 1003
    - Что движет пользовательским кодом? 1003
    - Java как поэзия 1005
    - Написание дополнения генерации кода 1006
    - Использование сгенерированного кода 1017
- Продвинутые советы Gradle для Android

    - Предпосылки 1019
    - Gradle, DRY..... 1019
    - Автоматизация информации версий APK 1022
    - Добавление в BuildConfig.....1024
- Тестирование с JUnit4
    - Предпосылки 1029
    - Тестирование инструментария и модульное тестирование 1029
    - Написание JUnit4 Test Cases 1031
    - Конфигурирование Gradle 1037
    - Запуск вашего инструментального тестирования 1038
    - Тестирование андроид библиотек проектов 1044
    - Тестовые разрешения и разрешения в процессе выполнения .. 1044
    - Оркестраторы андроид теста.....1045
- Тестирование с Espresso
    - Предпосылки 1049
    - Добавление Shot из Espresso 1049
    - Написание тестов в Espresso.....1052
    - Запись тестов Espresso 1058
    - Более крепкое Espresso..... 1062
    - Настройка опций Analytic 1072
    - Ожидание изменений в мире.....1072
- Тестирование с UI Automator
    - Предпосылки 1079
    - Что такое UI Automator? 1079
    - Кто выбирает UI Automator над альтернативами 1079
    - Настройки Gradle и Android Studio..... 1080
    - Создание Test Case 1081
    - Выполнение действий на уровне устройства..... 1081
    - Запуск Вашей активности 1082
    - Получение прокси для Widgets и Containers 1082
    - Взаимодействие с Widget 1085
    - Утверждение состояний.....1085
    - Запуск Ваших тестов..... 1086
    - Поиск Ваших Widget..... 1086
    - Применение просмоторщика UI Automator..... 1086
- Измерение покрытия тестами
    - Предпосылки 1089
    - Кто тестирует тестировщиков 1089
    - Некоторые типы покрытия тестами 1090
    - Покрытие и Ваши инструментальные тесты.....1092
- Модульное тестирование

    - Предпосылки 1097
    - Я думал мы уже выполняем модульное тестирование 1098
    - Сценарий: Чистая архитектура 1098
    - Настройка модульного тестирования 1099
    - Написание модульных тестов POJO 1101
    - Запуск модульных тестов 1105
    - Пародирование андроид 1109
- MonkeyRunner и Test Monkey
    - Предпосылки 1123
    - MonkeyRunner 1123
    - Случайное тестирование вокруг 1125
- Java 8 лямбда выражения
    - Предпосылки 1131
    - Базовая идея 1132
    - Использование лямбда выражений 1133
    - Альтернатива: метод отсылок 1137
- Rx основы
    - Предпосылки 1139
    - Жизнь - это всего лишь поток 1139
    - Действие и реакция 1140
    - Rx за то, что вас беспокоит 1142
    - Rx и лямбды 1143
    - Простой поток 1143
    - Будь своим собственным потоком 1145
    - Удаление AsyncTask 1147
    - Лямбды и жизненный цикл 1153
    - Поток из ресурсов 1154
    - Обработка ошибок 1155
    - Метаморфоза 1156
    - Библиотеки с включённым Rx 1157
    - Дальнейшее чтение 1159
    - Что насчёт LiveData? 1159
- Уведомления
    - Предпосылки 1163
    - Что такое уведомления 1163
    - Уведомления и каналы 1166
    - Отображение простого уведомления 1167
    - Сценарий Activity-Or-Notification 1172
    - Большие (и сложные) уведомления 1173
    - Фоновые сервисы 1180
    - Отключение уведомлений 1183

- Продвинутые уведомления
  - Предпосылки 1187
  - Быть хорошим гражданином 1187
  - Больше о каналах 1188
  - Где? Там! 1195
  - Стек уведомлений 1200
  - Избежание Wear 1206
  - Другие связанные с Wear параметры уведомление 1207
  - Удалённый ввод, на устройстве 1223
  - Группы уведомлений 1228
  - Уведомления экрана блокировки 1230
  - Приоритет, важность, и уведомления предупреждений 1240
  - Полноэкранные уведомления 1243
  - Уведомления прогресса 1245
  - Пользовательские View 1254
  - Жизнь после удаления 1257
  - Мистические случаи пропавших чисел 1258
  - Уведомления и MessagingStyle 1258
  - Изменение в API Level 23 1264
  - Звуки и Android 7.0 1264
  - Авто-Тайм-аут 1265
- Многооконная поддержка
  - Предпосылки 1269
  - История Windows 1269
  - Что видит пользователь 1270
  - Что видит Ваш код 1272
  - Opting Out 1274
  - Opting In 1274
  - Конфигурирование Layout 1275
  - Избежание прерывистости 1277
  - Управление фоном 1278
  - Как низко ты можешь пройти? 1278
  - Параллельная обработка 1279
  - Разделение экрана, HOME, и Ваша Activity 1281
  - Разделённые экраны и ориентация 1282
  - Принудительное Multi-Window/Multi-Instance в Вышем приложении 1282
  - Поддержка устаревшего проприетарного многооконного режима 1290
  - Свободные формы многооконного режима 1293
  - Picture-in-Picture 1300
  - Поддержка многооконности 1305
- Продвинутый ConstraintLayout

- Предпосылки 1315
- Барьеры для входа 1319
- Раскрытие вашей предвзятости 1324
- Центрирование себя 1329
- Сохранение вещей пропорциональными 1333
- Ограничивающие размеры 1334
- Цепи, без кнутов 1339
- Движение по кругу 1343
- Группы View... но не ViewGroups 1346

• GridLayout
- Предпосылки 1351
- Проблемы с классическими Container 1351
- Новый претендент: GridLayout 1353
- GridLayout и пакет Android Support 1353
- Наше тестовое приложение 1354
- Замена классики. 1357
- Неявные строки и столбцы 1363
- Интервалы строк и столбцов 1365

• Dialogs и DialogFragments
- Предпосылки 1371
- DatePickerDialog и TimePickerDialog 1371
- AlertDialog 1378
- DialogFragments 1378
- DialogFragment: Другие особенности 1383
- Dialogs: модальные, не блокирующие 1384

• Продвинутый ListViews
- Предпосылки 1385
- Типы множественных строк и саморазвёртование 1386
- Выбор режимов и активированный стиль 1392
- Пользовательское содержание изменяемых строк 1394
- От головы до ног 1399
- Вход RecyclerView 1402

• Режимы действий
- Предпосылки 1404
- Дела Context 1404
- Ручные режимы действий 1405
- Режимы действий модального множественного выбора 1410
- Длинный щелчок для инициации режима действий 1414

• Другие продвинутые техники Action Bar
- Предпосылки 1421
- Layout действия 1421

    - Action Views и Action Providers 1422
    - Поиск с SearchView 1423
    - Плавающие Action Bars 1429
- Панель инструментов
    - Предпосылки 1433
    - Базовые механики панели инструментов 1433
    - Примеры использования: Разделённый Action Bar 1434
    - Пример использования: замена Action Bar 1440
- AppCompat: Официальный обратно совместимый Action Bar
    - Предпосылки 1443
    - Хмммммм. Зачем? 1443
    - Основы использования AppCompat 1446
    - Другие эффекты AppCompat 1453
    - Панель инструментов и AppCompat 1460
    - Material, или не Material 1463
- The Android Design Support Library
    - Предпосылки 1467
    - GUI и другие Support Package 1468
    - Добавление библиотеки ... и что приходит с ней. 1468
    - Введение в CWAC-CrossPort 1469
    - Snackbars: Слаще чем Toasts 1470
    - Абсолютно сказочный (FAB) 1477
    - Вкладки Material с TabLayout 1485
    - Плавающие надписи 1492
- Продвинутый RecyclerView
    - Предпосылки 1505
    - Что насчёт Cursors? 1505
    - Grids 1512
    - Вариации элементов 1516
    - Изменяемое содержание строк 1529
    - Изменение содержания 1557
    - Порядок вещей 1563
    - Другой кусочек доброты 1571
    - Анимация разницы 1572
    - ListAdapter. Не, не тот ListAdapter 1580
    - Раскрывающиеся строки 1583
    - RecyclerView как Pager 1589
    - Декларирование LayoutManager в Layout 1603
    - Режим транскрипции 1603
- Продвинутое использование WebView
    - Предпосылки 1607

    - Друзья с преимуществами 1607
    - Навигация по водам 1622
    - Настройки, предпочтения , и параметры (Oh, My!) 1622
    - Безопасность и Ваш WebView..... 1623
    - Android 8.0 изменения в WebView 1626
    - Пользовательские закладки Chrome 1629
- Framework методов ввода
    - Предпосылки 1631
    - Клавиатуры, реальные и программные..... 1631
    - С учётом ваших потребностей 1632
    - Расскажите Android куда он может идти.....1637
    - Подгонка..... 1640
    - Джейн, остановить это сумасшедшие вещи! 1642
- Шрифты и текст
    - Предпосылки 1643
    - Люби того с кем ты. 1643
    - Ага, но действительно мы должны делать это в Java? 1647
    - Здесь Glyph там Glyph..... 1648
    - Авто размер TextView.....1649
    - Выровненный текст.....1654
- Rich Text
    - Предпосылки 1657
    - Концепция интервалов 1657
    - Загрузка Rich Text 1659
    - Редактирование Rich Text 1661
    - Сохранение Rich Text..... 1666
    - Манипуляции с Rich Text 1666
- Animators
    - Предпосылки 1669
    - ViewPropertyAnimator..... 1669
    - Основы: вес и аниматоры объектов.....1674
    - Анимация пользовательских типов 1677
    - Аппаратное ускорение 1678
    - Проблема трёх фрагментов 1679
- Устаревшие анимации
    - Предпосылки 1691
    - Это не только для мультфильмов теперь 1691
    - Причудливый перевод.....1692
    - Исчезать до чёрного. Или какого-то другого цвета..... 1696
    - Когда все сказано и выполнено 1698
    - Потеря заливки..... 1699

    - Стукнуть ускоритель 1699
    - Анимировать Задать Соответствовать 1700
    - Активные анимации 1701
- Пользовательские Drawables
    - Предпосылки 1703
    - Куда все это девается? 1704
    - ColorDrawable 1705
    - AnimationDrawable 1705
    - StateListDrawable 1709
    - ColorStateList 1710
    - LayerDrawable 1712
    - TransitionDrawable 1713
    - LevelListDrawable 1714
    - ScaleDrawable и ClipDrawable 1715
    - InsetDrawable 1725
    - ShapeDrawable 1726
    - BitmapDrawable 1737
    - Составные Drawables 1746
    - Стежок, сделанный вовремя, стоит девяти 1749
- Работа с картами Maps V2
    - Предпосылки 1761
    - Краткая история работы с картами на Андроид 1762
    - Где вы можете использовать Maps V2 1762
    - Условия лицензирования для Maps V2 1763
    - Что Вам необходимо, чтобы начать 1763
    - Примеры из книги... и Вы! 1766
    - Настройка базовой карты 1766
    - Поиграем с картами 1771
    - Плитки карты 1774
    - Расположение простых маркеров 1774
    - Просмотр всех маркеров 1778
    - Сплющивание и поворот маркеров 1781
    - Приводим в порядок Ваше “Info Windows” 1785
    - Рисунки и Ваше Info Window 1790
    - Настройка значка маркера 1796
    - Отклик на тапы 1798
    - Перетаскивание маркеров 1799
    - "Финальные" ограничения 1802
    - Немного больше о IPC 1805
    - Поиск пользователя 1805
    - Отрисовка линей и областей 1811

- Жесты и управление 1814
- Отслеживание изменений камеры 1815
- Карты в Fragment и Pager 1819
- Анимация движения маркеров 1824
- Карты разнообразия помещений 1831
- Получение снимка карты 1831
- SupportMapFragment против MapView 1832
- О том AbstractMapActivity классе 1833
- Helper Libraries для Maps V2 1837
- Проблемы с картами при выполнении 1841
- Проблемы с развёртыванием Maps V2 1841
- Что показывают неподдерживаемые устройства 1842
- Альтернативы работы с картами 1842

• Построение собственных View
- Предпосылки 1843
- Выбери свой яд 1843
- Цвета, смешивай как тебе нравится 1845
- ReverseChronometer: простой пользовательский класс 1855
- AspectLockedFrameLayout: пользовательский Container 1862
- Отзеркаливание и MirroringFrameLayout: Нарисуй его сам 1865

• Продвинутые предпочтения
- Предпосылки 1879
- Введение в PreferenceActivity 1879
- Intent для заголовков и предпочтений 1884
- Условные заголовки 1885
- Зависимые предпочтения 1890
- Вложенные экраны 1893
- Отслеживание изменения предпочтений 1896
- По умолчанию, и по умолчанию 1899
- Отслеживание изменений значений предпочтений 1900
- Динамическое содержание ListPreference 1900
- Поведение при внешних изменениях в предпочтениях 1905
- Предпочтения в приложении настроек устройства 1907

• Пользовательские Dialog и предпочтения
- Предпосылки 1915
- Ваш Dialog, покрытый шоколадом 1915
- Предпочтение Ваших собственных предпочтений, предпочтительно 1919

• Индикаторы прогресса
- Предпосылки 1927
- Progress Bars 1927
- ProgressBar и Threads 1932

    - Пошив Progress Bars 1935
    - Progress Dialogs 1945
    - Индикаторы Title Bar и Action Bar Progress 1947
    - Индикаторы Direct Progress 1949
- Больше веселья с Pager
    - Предпосылки 1951
    - Хостинг ViewPager в Fragment 1951
    - Pages и Action Bar 1953
    - ViewPagers и Scrollable Contents 1955
    - Отображение больше Page 1956
    - Columns для Large, Pages для Small 1961
- Управление фокусом и доступность
    - Предпосылки 1967
    - Подготовка для тестирования 1968
    - Контроль фокуса 1968
    - Доступность и фокус 1978
    - Доступность за пределами фокуса 1979
    - Доступность за пределами Impairment 1989
- Разнообразные хитрости UI
    - Предпосылки 1991
    - Полноэкранный и Lights-Out режим 1991
    - Предложение отложенного тайм-аута 2004
- Альтернативы Event Bus
    - Предпосылки 2009
    - Короткие заметки о приложениях примерах 2009
    - Стандартные Intents как Event Bus 2009
    - LocalBroadcastManager как Event Bus 2010
    - greenrobot's EventBus 3.x 2021
    - Эй, что об Otto? 2026
- Tasks
    - Предпосылки 2029
    - Сперва, немного терминологии 2029
    - А сейчас, немного о Task Killers 2035
    - Canary для Task's Coal Mine 2037
    - Опыт пользователя по умолчанию 2039
    - Обьяснение поведения по умолчанию 2043
    - Базовые сценарии для изменения поведения 2047
    - Решение непрерывных задач 2061
    - Документы как Task 2063
    - Другие относящиеся к Task свойства Activity 2066
    - Другие относящиеся к Task методы Activity 2075

- The Assist API (“Now On Tap”)
  - Предпосылки 2078
  - Какие данные раскрываются 2078
  - Добавление к данным 2080
  - Удаление из данных 2083
  - Блокировка Assist как пользователь 2086
  - Реализация своего собственного Assistant 2088
- The Autofill API
  - Предпосылки 2104
  - Части пазла 2104
  - Опыт пользователя 2105
  - Какие данные раскрываются 2107
  - Блокировка Autofill как пользователь 2121
  - Поддержка Autofill в стандартных Widget 2123
  - Поддержка Autofill с пользовательским Widget 2127
  - Работа с динамическими изменениями 2127
  - Требования безопасности с Autofill Services 2128
- The Data Binding Framework
  - Предпосылки 2135
  - Что, теперь? 2136
  - Базовые шаги 2137
  - Расширенные ресурсы Layout 2143
  - Привязка языка выражений 2147
  - Observables и обновление Binding 2150
  - Binding в две стороны 2163
  - Другие функции Note 2166
- Drag and Drop
  - Предпосылки 2193
  - Сфера действия Drag and Drop 2193
  - Части Drag-and-Drop 2195
  - Drag-and-Drop, внутри Activity 2199
  - Drag-and-Drop, между приложениями 2208
  - Обнаружение событий Drag между приложениями 2221
  - Междуоконное Drag-and-Drop внутри приложения 2221
  - Размышление о устаревшем Multi-Window 2224
  - Простые вещи Dragging and Dropping 2224
  - Многофункциональный Drag-and-Drop 2225
  - Вложенные цели Drop 2231
  - Обдумывание стандартов 2240
  - Обдумывание стабильности 2240
- Ввод из клавиатуры и мыши

    - Предпосылки 2243
    - Предлагаем горячие клавиши клавиатуры 2243
    - Пользовательские копирование и вставка 2253
    - Аппаратная клавиатура и фокус 2257
    - Предлагаем контекстное меню мыши 2261
    - Предлагаем Tooltips 2267
    - Захват указателя 2275
- Просмотр PDF
    - Предпосылки 2277
    - Критерии 2277
    - Классическое решение: ACTION_VIEW 2279
    - Действительно плохая идея: Google Docs 2280
    - Встроенные опции: PdfRenderer 2281
    - Выбор громовой ящерицы: PDF.js 2289
    - Нативные подходы: Pdfium 2294
    - Что выбрать? 2298
- Виджеты приложения на домашний экран
    - Предпосылки 2301
    - Виджеты приложения и безопасность 2301
    - Большой рисунок для маленького виджета приложения 2302
    - Создание виджетов приложения 2303
    - Ещё и ещё 2310
    - Виджеты приложения: их жизнь и время 2311
    - Управляй своей (виджеты приложения) судьбой 2311
    - Один размер может не подходить всем 2312
    - Виджеты экрана блокировки 2318
    - Предпросмотр изображений 2325
    - Быть хорошим хозяином 2326
- Основанные на Adapter виджеты приложения
    - Предпосылки 2329
    - AdapterView для виджетов приложения 2329
    - Создание основанных на Adapter виджетов приложения 2330
- Публикация Slices
    - Что такое Slice? 2345
    - Сказка о двух Slices 2346
    - Размер Slice 2346
    - Настройка Slice 2348
    - Привязка Slice 2352
    - Пробуем Slice 2355
    - Шаблон действий пользователя 2357
    - SliceAction 2358

    - Шаблоны элементов Slice 2359
    - Действия и размеры 2363
    - Асинхронные Slice..... 2364
    - Предупреждение: не выполнении допущений рендеринга..... 2371
    - Другие функции Slice Viewer..... 2373
- Хостинг Slice
    - Что Вам необходимо знать?.....2375
    - Зачем?.....2375
    - Как? 2376
    - The Catch: Обнаружение Slice 2383
    - Slice… из интернета? 2384
- Продвинутые разрешения
    - Предпосылки 2385
    - Обезопасьте себя..... 2385
    - Разрешения подписи 2388
    - Уязвимости пользовательских разрешений..... 2389
    - Опасные пользовательские разрешения, и Андроид 6.0..... 2397
    - Поиск доступных разрешений.....2401
- Ограниченные профили и UserManager
    - Предпосылки 2407
    - Планшеты Андроид и множественные аккаунты пользователей2407
    - Определение что пользователь может делать.....2413
    - Влияние на ограничения на уровне устройства 2416
    - Активация пользовательских ограничений 2416
    - Неявные Intent могут вызвать “Boom”.....2428
- Аутентификация устройства
    - Предпосылки 2431
    - Устройство защищено? 2431
    - Переподтверждение пользователя 2436
    - Отпечатки пальцев 2439
- Ключи и хранение ключей
    - Предпосылки 2455
    - Терминология..... 2456
    - Получение хранилища ключей..... 2457
    - Создание ключа..... 2457
    - Привязка ключа к аутентификации устройства 2458
    - Изучаем больше о Вашем ключе 2459
    - Зашифрованные данные 2460
    - Ограниченная по времени аутентификация устройства..... 2470
    - Зашифрованные кодовая фразы..... 2474
    - Ограничения (хранилища)ключей 2483

- Прочие техники безопасности
  - Предпосылки 2485
  - Валидация публичного ключа 2485
  - Выбор размера Вашего ключа подписи 2499
  - Избегайте случайных API 2500
  - Другие пути раскрытия данных 2505
  - Джек-атаки 2506
  - Использование FLAG_SECURE 2514
- Теория Content Provider
  - Предпосылки 2519
  - Использование Content Provider 2519
  - Создание Content Provider 2527
  - Проблемы с Content Provider 2535
- Паттерны реализации Content Provider
  - Предпосылки 2537
  - Однотабличный с базой данных в основе Content Provider 2537
  - Content Provider локального файлы 2545
  - Защищённый Provider 2554
  - Потоковый Provider 2556
  - FileProvider 2559
  - StreamProvider 2564
- Loader Framework
  - Предпосылки 2575
  - Введение в Loader Framework 2575
  - Использование CursorLoader 2577
  - Что другое потеряно? 2579
  - Что случится если...? 2579
  - Написание пользовательского Loader 2580
- ContactsContract и CallLog Providers
  - Предпосылки 2587
  - Представляем Вам Ваши контакты 2588
  - Выбор кучи выбранных людей 2589
  - Прокрутка через Ваши контакты 2591
  - Создание котнакта 2599
  - Просмотр CallLog 2603
- CalendarContract Provider
  - Предпосылки 2610
  - Вы не можете быть обманщиком 2610
  - Есть ли у Вас Room для Вашего Calendar? 2610
  - Разрешения Calendar 2611
  - Запрос событий 2611

    - Штриховка события 2617
- MediaStore Provider
    - Предпосылки 2619
    - Что такое MediaStore? 2620
    - MediaStore и “другие” внешние накопители 2621
    - Как моё содержимое индексируется? 2622
    - Как я получаю видео из MediaStore? 2622
- Пользовательские документы
    - Предпосылки 2629
    - Доступ к накопителю… Что? 2629
    - Участники Framework доступа к хранилищу 2631
    - Сбор урожая (фотографии маринованного перца) 2631
    - Открытие документа 2632
    - Зачем нам, чтобы вещи были Openable 2636
    - Остальное в CRUD 2637
    - DocumentFile Helper 2638
    - CWAC-Document и DocumentFileCompat 2638
    - Получение продолжительного доступа 2639
    - Другой продолжительный пример: Diceware 2648
    - Деревья документов 2662
    - Получение дерева: Примеры 2663
    - Рассмотрение ошибки доступа к директории 2680
    - Изменения Android 8.0 2681
- Предоставление документов
    - Предпосылки 2683
    - Получение Вашего содержимого и предоставление его 2683
    - Ключевые концепты Provider 2685
    - Части Provider 2686
    - Опциональные возможности Provider 2701
- Зашифрованные хранилища
    - Предпосылки 2708
    - Сценарии для шифрования 2708
    - Получение SQLCipher 2708
    - Использование SQLCipher 2709
    - Ограничения SQLCipher 2710
    - Пароли и сессии 2711
    - Про те кодовые фразы 2712
    - Зашифрованные предпочтения 2719
    - IOCipher 2721
- Упаковка и распространение данных
    - Предпосылки 2723

- Упаковка баз данных на ходу 2723
- Продвинутые техники баз данных
  - Предпосылки 2727
  - Полнотекстовая индексация 2727
- Резервное копирование данных
  - Предпосылки 2745
  - Сперва, немного терминологии 2746
  - Различное определение "резервного копироваиня" 2746
  - Внедрение резервного копирования в стиле ИТ 2748
  - Google Backup Bootstrap 2773
  - Повышение безопасности резервного копирования 2781
  - Альтернативные возможности 2783
- SSL Предпосылки 2789
  - Базовые операции SSL 2789
  - Проблемы в раю 2790
  - Введение в сетевые настройки безопасности 2793
  - Проблемы SSL и сетевые настройки безопасности 2796
  - Другие техники усиливающие SSL 2806
  - Продвинутое использование CWAC-NetSecurity 2809
  - NetCipher 2812
- NetCipher
  - Предпосылки 2813
  - Сетевая безопасность получает "луковые слои" 2813
  - NetCipher HTTP API интеграции 2815
  - Остальное в Builder API 2822
- Прочие сетевые темы
  - Предпосылки 2833
  - Загрузка файлов 2833
  - Сохранение данных 2847
- Воспроизведение аудио
  - Предпосылки 2851
  - Включить Ваши медиа 2851
  - MediaPlayer для аудио 2852
  - Другие пути создать шум 2857
- Запись аудио
  - Предпосылки 2861
  - Запись через Intent 2861
  - Запись в файлы 2864
  - Запись в потоки 2869
  - Необработанный ввод аудио 2872

    - Запрос микрофона 2872
- Воспроизведение видео
    - Предпосылки 2873
    - Двигающиеся картинки 2873
- Использование камеры через сторонние приложения
    - Предпосылки 2879
    - Конкретные особенности 2879
    - Все ещё фото: Позволим приложению для камеры делать это 2880
    - Разрешения и сторонние приложения для камеры 2887
    - Дела ориентации 2888
    - Сканирование с ZXing 2895
    - Видео: позволим нашему приложению для камеры сделать это 2896
    - Использование библиотеки камеры 2899
    - Непосредственная работа с камерой 2911
- Работаем напрямую с камерой
    - Предпосылки 2913
    - Заметки о примерах кода 2913
    - Сказка о двух API 2913
    - Выполнение базовых операций с камерой 2915
    - Настройка тихой камеры 2938
    - И сейчас, проблемы 2946
- Путы медиа
    - Предпосылки 2953
    - Терминология 2953
    - Сказка о двух MediaRouters 2954
    - Прикрепление к MediaRouter 2955
    - Использование Route Selection с MediaRouteActionProvider ... 2957
    - Использование путей Live Video 2972
    - Использование путей удалённого воспроизведения 2972
- Поддержка внешних мониторов
    - Предпосылки 2993
    - История внешних мониторов 2993
    - Что такое презентация? 2994
    - Воспроизведение с внешними мониторами 2995
    - Обнаружение мониторов 3001
    - Простая презентация 3002
    - Более простая презентация 3008
    - Презентации и изменение конфигурации 3014
    - Презентации как Fragment 3015
    - Другой простой проект: Слайды 3026
    - Поддержка устройств для презентации 3035

    - Презентации из Service 3035
    - Эй, что о Chromecast? 3038
- Google Cast и Chromecast
    - Предпосылки 3041
    - Здесь Cast, там Cast 3041
    - Общие заметки разработчика о Chromecast 3043
    - Выбор Вашего API 3043
    - Senders и Receivers 3044
    - Поддержка типов медиа 3045
    - Cast SDK Dependencies 3046
    - Разработка приложения Google Cast 3048
- "Десяти футовый UI"
    - Предпосылки 3049
    - Что такое "Десяти футовый UI" 3050
    - Overscan 3050
    - Навигация 3051
    - Рассмотрение стилистики 3052
    - Leanback UI 3054
    - Тестирование Ваших теорий 3071
- Держим все TV вместе: Decktastic
    - Предпосылки 3074
    - Введение в Decktastic 3074
    - Реализация Decktastic 3078
- Создание MediaRouteProvider
    - Предпосылки 3113
    - Терминология 3113
    - DIY Chromecast 3114
    - Создание MediaRouteProvider 3116
    - Потребление MediaRouteProvider 3126
    - Реализация этого “For Realz” 3131
- Media Projection API
    - Предпосылки 3133
    - Запрос скриншота 3133
    - Запись экрана 3144
- AlarmManager и паттерн запланированных Service
    - Предпосылки 3163
    - Сценарии 3163
    - Опции 3164
    - Простой пример 3166
    - Пять наборов... () разновидностей 3168
    - Четыре типа будильников 3170

- Когда планировать будильники 3170
- Archetype: Scheduled Service Polling 3172
- Предупреждение: не все устройства андроид играют хорошо ... 3178
- Отладка будильников 3178
- Андроид 6.0 и война фоновых процессов 3181
- Андроид 7.0 и OnAlarmListener 3181

• PowerManager и WakeLocks
- Предпосылки 3185
- Держим экран включённым, стили UI 3185
- Роль WakeLock 3186
- Что WakefulIntentService делает 3187

• JobScheduler
- Предпосылки 3189
- Ограничения AlarmManager 3189
- Вход в JobScheduler 3190
- Используем JobScheduler 3190
- Размышляем о критериях отсрочки 3205
- Другие функции JobScheduler 3206
- Ограничения периодов JobScheduler 3207
- GcmNetworkManager 3207
- Периодическая работа, через версии устройств 3207
- Андроид 6.0 и “война фоновых процессов” 3212
- Запланированных мониторинг содержания 3224

• Доступ к местоположению - Базовые Service
- Предпосылки 3239
- Provider местоположения: Они знают где ты прячешься 3240
- Поиск себя 3240
- В движении 3242
- Получение местоположения через PendingIntent 3250
- А мы ещё там? А мы ещё там? А мы ещё там? 3250
- Тестирование... тестирование 3251
- Альтернативные особенности обновления 3251
- Объединённые опции 3253
- Местоположение и функции 3253

• Provider объединённого местоположения
- Предпосылки 3255
- Зачем использовать Provider объединённого местоположения 3255
- Зачем не использовать Provider объединённого местоположения? 3256
- Поиск Вашего местоположения, единожды 3256
- Периодическое получение местоположения 3264

• Работа с буфером обмена

- Предпосылки 3269
- Работа с буфером обмена 3269
- Данные буфера обмена и Drag-and-Drop 3274
- Мониторинг буфера обмена 3275
- Ошибка буфера обмена Андроид 4.3 3276

• Телефония
- Предпосылки 3279
- Отчёт менеджеру 3280
- Вы делаете звонок 3280
- Нет, действительно, Вы делаете звонок! 3283

• Работа с SMS
- Предпосылки 3285
- Отправка SOS, Give или Take a Letter 3286
- Мониторинг и получение SMS 3293
- Входящие SMS 3300
- Запрос на изменения по умолчанию 3301
- SMS и Emulator 3302
- SMS Tokens 3302

• NFC Предпосылки 3309
- Что такое NFC? 3309
- NDEF, или не NDEF 3310
- NDEF Modalities 3311
- NDEF структуры и трансляция Андроид 3312
- Реальность NDEF 3313
- Источники тагов 3315
- Написание тага 3315
- Ответ на таг 3324
- Ожидаемый шаблон: Bootstrap 3325
- Мобильные устройства мобильные 3325
- Активация и деактивация 3326
- Android Beam 3326
- Beaming Files 3334
- Другой пример: SecretAgentMan 3335
- Дополнительные ресурсы 3345

• Администрирование устройств
- Предпосылки 3347
- Цели и объем 3347
- Определение и регистрирование компонента Admin 3348
- Переход в блокировку 3354
- Пароли и администрирование устройств 3361

    - Ладить с другими 3366
- Базовое использование датчиков
    - Предпосылки 3367
    - Абстрактная модель датчиков 3367
    - Учитывая ставки 3368
    - Чтение датчиков 3369
    - Пакетное чтение датчиков 3381
- Печать и генерация документа
    - Предпосылки 3383
    - Система печати андроид 3384
    - Об простом приложении 3384
    - Печать растрового изовражения 3385
    - Печать HTML Документа 3388
    - Печать PDF файла 3392
    - Печать с применением Canvas 3401
    - Печать заданий 3403
    - Печать, потоки и Services 3404
    - Печать до андроид 4.4 3407
    - Генерация HTML 3407
    - Опции генерации PDF 3411
- Базовый Bluetooth RFCOMM
    - Предпосылки 3413
    - Быстрая часть объема 3413
    - Об простом приложении 3414
    - Bluetooth и разрешения 3420
    - Rx для Вашего Bluetooth 3421
    - Я могу получить Bluetooth? 3422
    - Я чувствую связь между нами 3425
    - Путешествие открытий 3428
    - Обслуживание и Shouting 3431
    - Дотянутся и прикоснутся к кому-то 3436
    - Пинг и понг 3440
    - Разница с Андроид вещами 3444
- Работа с различным аппаратным обеспечением
    - Предпосылки 3447
    - Фильтрация устройств 3447
    - Определение совместимости на лету 3450
    - Работа с ошибками устройств 3451
- Написание и использование Parcelables
    - Предпосылки 3455
    - Роль Parcelable 3455

    - Написание Parcelable 3456
    - Ограничения Parcelable 3462
    - Будь осторожен PendingIntent 3465
- Ответ на URL
    - Предпосылки 3471
    - Модификации манифеста 3471
    - Создание пользовательского URL 3473
    - Реакция на подключение 3474
    - Подключения приложения 3476
- Ярлыки приложения
    - Предпосылки 3487
    - Активация глубокого погружения 3488
    - Ярлыки приложения, из пользовательского POV 3488
    - Предоставления ярлыков приложения манифеста 3491
    - Предоставление динамических ярлыков приложения 3497
    - Приватность, безопасность и ярлыки приложения 3509
- Трюки PackageManager
    - Предпосылки 3511
    - Поспрашивать вокруг 3511
    - Предпочтительные Activities 3515
    - Менеджмент по середине 3521
- Remote Services и Binding Pattern
    - Предпосылки 3525
    - Binding Pattern 3526
    - Когда атакует IPC! 3532
    - Service из Afar 3534
    - Ужесточение безопасности 3540
    - Servicing Service 3545
    - Размышления о безопасности 3549
    - Анти-паттерн “Everlasting Service” 3550
- Продвинутые советы манифеста
    - Предпосылки 3553
    - Просто ищем некоторое пространство для локтей 3553
    - Применение Alias 3563
    - Получение Meta (Data) 3564
- Другие советы интеграции
    - Предпосылки 3569
    - Поделиться непосредственно 3569
    - Получение ярлыка 3578
    - Самонаводящиеся маяки для Intent 3585
    - Интеграция с выбором текста 3585

    - Быстрые настройки и TileService 3596
    - Установка пакетов 3605
    - Удаление пакетов 3608
    - Определение изменений в пакете 3609
- Редакторы и диалоги Android Studio
    - Предпосылки 3611
    - Структура проекта 3611
    - Редактор переводов 3622
- Продвинутые возможности эмулятора
    - Предпосылки 3629
    - Другие заметные опции конфигурации 3629
    - Боковая панель эмулятора 3635
    - Операции с окном эмулятора 3645
    - Операции без заголовка 3646
- Lint и поддержка аннотаций
    - Предпосылки 3647
    - Что это такое 3648
    - Когда это запускается 3648
    - Что исправляет 3649
    - Что настраивает 3650
    - Аннотации поддержки 3654
- Инспектирование Layout
    - Запуск инспектора Layout 3667
    - Просмотр иерархии View 3668
    - Инспекции и захваты 3668
- Скриншоты и записи экрана
    - Предпосылки 3671
    - Сбор данных из Android Studio 3672
    - Запись экрана 3675
    - Сбор данных из командной строки 3675
    - Сбор данных из другого приложения 3677
    - Советы и трюки 3678
- Советы и трюки ADB
    - Предпосылки 3679
    - Это дроид, которого Вы искали 3679
    - Установка и удаление приложений 3680
    - Играем с разрешениями 3680
    - Запуск и остановка компонент 3681
    - Остановка процессов и очистка данных 3682
    - Изменение метрик отображения 3682
- Stetho

    - Подожди, что? Chrome? 3685
    - Базовая интеграция Stetho 3685
    - Подключение Chrome к Вашему приложению 3693
    - Что ты получаешь в инструментах разработчика Chrome 3693
    - Получение помощи с Stetho 3702
    - Эй, что о Sonar 3702
- Проблемы со скоростью
    - Предпосылки 3707
    - Приступая к делу 3707
    - Ваш UI кажется... "некачественным" 3708
    - Не достаточно в фоне 3708
    - Игра со скоростью 3709
- Поиск узких мест ЦПУ
    - Предпосылки 3711
    - Android Studio's Profiler 3712
    - Трассирование методов 3714
    - Другие общие техники измерения ЦПУ 3718
    - UI "Jank" Measurement 3720
- Фокусируемся: NDK
    - Предпосылки 3739
    - Роль NDK 3740
    - Введение в CWAC-AndDown 3743
    - Установка NDK 3743
    - Содержимое проекта NDK 3744
    - Создание своей библиотеки 3747
    - libhoudini и NDK 3753
- Повышение производительности CPU в Java
    - Предпосылки 3755
    - Уменьшение загрузки ЦПУ 3755
    - Уменьшение задержек в основном потоке приложения 3760
    - Улучшение пропускной способности и отзывчивости 3768
- Поиск и устранение "мусорных процессов"
    - Предпосылки 3771
    - Задача: ThreePaneDemoBC 3771
    - Мы "мусорные" 3772
    - Поиск источника "мусорного процесса" 3772
    - Когда дела идут неправильно 3784
    - Удаление "мусорного процесса" 3784
    - API измерения фреймов 3786
- Проблемы с пропускной способностью
    - Предпосылки 3793

    - Вы используете слишком много медленных вещей..... 3793
    - Вы используете слишком много тяжеловесных вещей..... 3794
    - Вы используете слишком много чьих-то вещей..... 3795
    - Вы используете слишком много... и ничего..... 3796
- Фокусируемся на: TrafficStats
    - Предпосылки 3797
    - Основы TrafficStats..... 3797
    - Пример: TrafficMonitor 3799
    - Другие способы применения TrafficStats.....3808
- Измерение потребления пропускной способности
    - Предпосылки 3811
    - Измерения на устройстве 3811
    - Измерение вне устройства 3815
    - Android Studio Profiler 3817
- Будь умнее о пропускной способности
    - Предпосылки 3819
    - Сохранения пропускной способности.....3819
    - Формирование пропускной способности 3823
    - Избегайте измерений(?) подключений..... 3827
    - Сохранение данных 3827
- Проблемы с хипом приложения
    - Предпосылки 3829
    - Вы в хипе проблем.....3830
    - Определение размера Вашего хип при выполнении программы3830
    - Фрагменты памяти 3831
    - Получение усечения хип 3832
    - Предупреждение: Содержит графические изображения..... 3833
    - Освобождение памяти SQLite.....3845
    - Обманки.....3845
    - Ограничение транзакций IPC в 1MB.....3846
- Поиск утечек памяти
    - Предпосылки 3849
    - Android Studio Profiler 3850
    - Получение дампов хипа 3853
    - Анализирование дампов хипа в Android Studio.....3854
    - Частые сценарии утечек 3861
    - Канарейка в угольной шахте с утечками 3868
- Проблемы с системной RAM
    - Предпосылки 3877
    - Разве мы не можем просто ладить? 3877
    - Вклад в потребление системной RAM.....3878

    - Измерение потребления системной памяти: Инструменты ...3879
    - Измерение потребления системной памяти: В процессе выполнения 3895
    - Научитесь отпускать (из своего хип).....3896
- Проблемы с временем жизни батареи
    - Предпосылки 3899
    - Вас обвиняют.....3900
    - Не все батареи созданы равными 3901
    - Растягивая последние мВтч 3901
- Опции измерения мощности
    - Предпосылки 3903
    - batterystats иBattery Historian.....3904
    - PowerTutor.....3916
    - Экран батареи в Приложении настройки 3921
    - Дамп BatteryInfo.....3922
- Источники утечки энергии
    - Предпосылки 3925
    - Экран 3926
    - Дисковые ввод/вывод.....3927
    - WiFi и мобильные данные.....3928
    - GPS 3931
    - Камеры 3932
    - Дополнительные источники.....3932
- Проблемы с адресацией размеров приложения
    - Предпосылки 3935
    - APK Analyzer.....3935
    - Код Java и ограничение в 64K методов.....3938
    - Нативный код.....3943
    - Изображения 3944
    - Файлы расширения APK.....3946
- Отчёт об ошибках с применением ACRA
    - Предпосылки 3947
    - Что происходит, когда вещи "Бабахают" 3947
    - Введение в ACRA 3948
    - Где ACRA создаёт отчёт о сбоях 3949
    - Основы интегрирования ACRA 3951
    - Что видит пользователь.....3957
    - Что видите Вы.....3963
    - ACRA и процессы 3969
- Диагностика внутри приложения
    - Предпосылки 3973
    - Activity диагностики.....3974

    - Наложение диагностики 3985
- Антипаттерны
    - Предпосылки 4001
    - Утечки в потоках... Или вещи прикреплённые к потокам 4001
    - Использование большого хип излишне 4003
    - Неиспользование кнопки меню 4005
    - Наложение с навигацией 4006
    - Применение android:sharedUserId 4008
    - Реализация кнопки "Выход" 4009
    - Завершение Ваших процессов 4011
    - Пытаемся спрятаться от пользователя 4012
    - Использование множественных процессов 4013
    - Захватчик системных ресурсов 4015
- Каталог виджетов: AdapterViewFlipper
    - Ключевые советы использования 4019
    - Простое применение 4020
    - Визуальная репрезентация 4020
- Каталог виджетов: CalendarView
    - Ключевые советы использования 4021
    - Простое применение 4022
    - Визуальная репрезентация 4024
- Каталог виджетов: DatePicker
    - Ключевые советы использования 4027
    - Простое применение 4028
    - Визуальная репрезентация 4031
- Каталог виджетов: ExpandableListView
    - Ключевые советы использования 4037
    - Простое применение 4038
    - Визуальная репрезентация 4045
- Каталог виджетов: SeekBar
    - Ключевые советы использования 4047
    - Простое применение 4047
    - Визуальная репрезентация 4050
- Каталог виджетов: SlidingPaneLayout
    - Декларирование SlidingPaneLayout 4053
    - Визуальная репрезентация 4055
    - Взаимодействие с SlidingPaneLayout 4057
- Каталог виджетов: StackView
    - Ключевые советы использования 4059
    - Простое применение 4060
    - Визуальная репрезентация 4062

- Каталог виджетов: TabHost и TabWidget
  - Deprecation Notes 4065
  - Ключевые советы использования 4065
  - Простое применение 4066
  - Визуальная репрезентация 4069
- Каталог виджетов: TimePicker
  - Ключевые советы использования 4071
  - Простое применение 4071
  - Визуальная репрезентация 4074
- Каталог виджетов: ViewFlipper
  - Ключевые советы использования 4077
  - Простое применение 4078
  - Визуальная репрезентация 4079
- Каталог устройств: Chrome и Chrome OS
  - Предпосылки 4083
  - Как это работает 4084
  - Форм факторы Chrome OS 4085
  - Тестирование Вашего приложения на Chrome OS 4086
  - Быть готовым быть стёртым 4097
  - Совместимости в Ваше приложение 4097
  - Ваше приложение на Chrome OS 4098
  - Опции распространения 4106
  - Получение помощи 4107
- Каталог устройств: BlackBerry
  - Я думал у BlackBerry своя ОС 4110
  - Что ещё отличается? 4110
  - Что мы создаем 4113
  - Создание вашего окружения разработки 4113
  - Как работает распространение? 4115
- Каталог устройств: Android TV
  - Предпосылки 4119
  - Эй, подожди минуту... Я думал название было “Google TV”? 4119
  - Некоторое аппаратное обеспечение Android TV 4120
  - Какие функции и конфигурации он использует? 4122
  - В чем действительно отличие? 4123
  - Создание вашего окружения разработки 4125
  - Как работает распространение? 4127
- Каталог устройств: Amazon Fire TV и Fire TV Stick
  - Предпосылки 4131
  - Введение в устройства Fire TV 4131
  - Какие функции и устройства они используют? 4138

- В чем действительно отличие? 4139
- Вывод и Fire TV 4140
- Создание вашего окружения разработки 4141
- Работа с ДУ и контроллером 4142
- Как работает распространение? 4144
- Получение помощи 4144

• Каталог устройств: Samsung DeX
- DeX Screen Modes 4150
- Влияние других приложений 4152
- Получения дополнительной информации 4157

• Приложение A: CWAC Libraries
- cwac-document 4161
- cwac-layouts 4161
- cwac-netsecurity 4162
- cwac-presentation 4162
- cwac-provider 4162
- cwac-saferoom 4162
- cwac-security 4162

• Приложение B: Андроид 8.0
- Война обработки в фоне, продолжение 4163
- Улучшения JobScheduler 4170
- Авто заполнение 4171
- Каналы уведомлений 4172
- Другие изменения и уведомления 4172
- Мульти-оконные изменения 4172
- Изменения WebView 4173
- Изменения ContentProvider 4173
- Изменения фреймворка доступа к хранилищу 4173
- Управление пакетами 4173
- Шрифты как ресурсы 4174
- Другие главные изменения в Андроид 8.0 4174
- Другие незначительные изменения в Андроид 8.0 4183

• Приложение C: Андроид 9.0
- Основные критические изменения 4189
- Основные функции 4196
- Другие примечательные изменения 4202
- Заметки о Support Library 4224
- Получение помощи с 9.0 4226

• Приложение D: Общественный театр и Appinars
- Просмотр Appinar Roster 4228
- Управление Appinars 4230

◦ Просмотр Appinar 4234

Предисловие

Добро пожаловать в Книгу!

Благодарности

Благодарим за Ваш интерес к разработке приложений для Андроид! Андроид выросла из ничего в аргументировано наиболее популярную ОС в мире за короткие два года. Когда Вы разрабатываете приложение для всех, для Вашего бизнеса и организации, или только экспериментируете самостоятельно, Я думаю Вы найдёте, что Андроид - увлекательная и сложная область для исследования.

И, кроме того, благодарим за Ваш интерес к данной книге! Я искренне надеюсь Вы найдёте её полезной и как минимум иногда занимательной.

Книга первого поколения

Разработка Андроид приложений может быть разделена на два поколения:

- Первое поколение разработки приложений использовало Java как язык программирования и усиливалось Android Support Library и `android.arch` версией архитектурных компонентов.
- Второе поколение разработки приложений более часто применяет Kotlin как язык программирования и усиливается AndroidX и остальной частью Jetpack (которое включает AndroidX версию архитектурных компонентов)

Это книга первого поколения. Она исследует `android.arch` версию архитектурных компонентов и она использует Java для большинства примеров.

Материалы второго поколения могут быть найдены в книжной серии “Elements”

CommonsWare Особо следует отметить, *Elements of Android Jetpack* - замена книги Введение в разработку андроид приложений от CommonsWare.

Структура книги

Как Вы могли отметить, это скорее большая книга.

Чтобы сделать данное огромное количество материалов структурированным, главы разделены на
ключевые главы и серии из *путей*.

Ключевые главы представляют много ключевых концептов, которые необходимы для понимания Андроид разработчикам, чтобы создать приложение. В случаи, если случайная тема "хорошо бы была" переходит в ключевую - чтобы помочь проиллюстрировать пункты, для примера - ключевые главы безусловно необходимы.

Ключевые главы выполнены для чтения последовательно и чередуют и стандартный технический текст и главы инструкции, чтобы предоставить Вам вспомогательный опыт с обсуждаемыми концептами. Большинство руководств может быть пропущено, хотя первые два - охватывающие настройку окружения SDK и создание проекта - каждый должен прочитать.

Набор глав разделен на пути, охватывающие определённые отдельные общие темы, от хранения данных к продвинутым эффектам UI и измерения производительности и улучшений. Каждый путь содержит несколько глав. Тем не менее, такие главы, и сами пути не обязательно написаны для чтения в каком-либо порядке. Каждая глава в пути указывает предварительные главы или концепты, которые вам необходимо предварительно изучить. Следовательно эти части являются в большей степени справочным материалом, когда Вы специально хотите изучить что-то об определённой теме.

Ключевые главы ссылаются на главы в пути, показывая где Вы можете найти материал относящийся к главе, которую Вы сейчас читаете. Так Вы можете использовать следующие способы поиска материалов для чтения - содержание книги, данное предисловие, инструмент поиска в программе чтения цифровой книги или пересекающие ссылки в главах.

На Ваше усмотрение вы можете читать книгу в обратном порядке, если пожелаете. Пути представлены после ключевых глав. Эти пути будут в

логически обоснованном порядке, потому Вы можете немного поперепрыгивать, чтобы охватить все предварительные главы.

Пути

Здесь представлен список всех путей и глав, которые принадлежат этим путям в порядке появления (за исключением тех, что появляются в списке несколько раз, где они пересекаются в основных категориях)

Организация кода и Gradle

- Работа с библиотеками проекта
- Gradle и задачи
- Варианты сборок Gradle
- Правила слияния Manifest
- Подпись вашего приложения
- Распространение
- Продвинутые советы Gradle для Android

Тестирование

- Тестирование с JUnit4
- Тестирование с Espresso
- Тестирование с UIAutomator
- Измерение покрытия тестами
- Модульное тестирование
- MonkeyRunner и Test Monkey

Rx

- Java 8 лямбда выражения
- Rx основы

Продвинутый UI

- Уведомления
- Продвинутые уведомления
- Многооконная поддержка
- Продвинутый ConstraintLayout
- GridLayout
- Dialogs и DialogFragments
- Продвинутый ListViews
- Режимы действий и контекстные меню

- Другие продвинутые техники Action Bar
- Панель инструментов
- AppCompat: Официальный обратно совместимый Action Bar
- Продвинутый RecyclerView
- The Android Design Support Library
- Продвинутое использование WebView
- Framework методов ввода
- Шрифты
- Rich Text
- Animators
- Устаревшие анимации
- Пользовательские Drawables
- Работа с картами Maps V2
- Построение собственных View
- Продвинутые предпочтения
- Пользовательские Dialog и предпочтения
- Индикаторы прогресса
- Больше веселья с Pager
- Управление фокусом и доступность
- Разнообразные хитрости UI
- Альтернативы Event Bus
- Tasks
- The Assist API (“Now On Tap”)
- API автозаполнения
- Привязка данных
- Drag-and-Drop
- Ввод из клавиатуры и мыши
- Просмотр PDF

Встроенные UI

- Виджеты приложения на домашний экран
- Основанные на Adapter виджеты приложения
- Публикация Slices

Безопасность

- SSL
- NetCipher
- Зашифрованные хранилища

- Продвинутые разрешения
- Ограниченные профили и UserManager
- Аутентификация устройства
- Ключи и хранение ключей
- Прочие техники безопасности

Хранилища данных и извлечение данных

- Теория Content Provider
- Паттерны реализации Content Provider
- Loader Framework
- ContactsContract Provider
- CalendarContract Provider
- MediaStore Provider
- Пользовательские документы
- Предоставление документов
- Зашифрованные хранилища
- Упаковка и распространение данных
- Продвинутые техники баз данных
- Резервное копирование данных

Продвинутые темы сети

- SSL
- NetCipher
- Дополнительные сетевые возможности

Медиа

- Воспроизведение аудио
- Запись аудио
- Воспроизведение видео
- Использование камеры через сторонние приложения
- Работаем напрямую с камерой
- MediaStore Provider
- Путы медиа
- Поддержка внешних мониторов
- Google Cast и Chromecast
- “10 футовый UI”
- Держим все TV вместе: Decktastic

- Создание MediaRouteProvider
- Media Projection API

Аппаратное обеспечение и системные сервисы

- AlarmManager и паттерн запланированных Service
- PowerManager и WakeLocks
- JobScheduler
- Доступ к местоположению - Базовые Service
- Provider объединённого местоположения
- Работа с буфером обмена
- Телефония
- Работа с SMS
- NFC
- Администрирование устройств
- Базовое использование датчиков
- Печать и генерация документа
- Базовый Bluetooth RFCOMM
- Работа с различным аппаратным обеспечением

Интеграция и интроспекция

- Написание и использование Parcelables
- Ответ на URL
- Ярлыки приложения
- Трюки PackageManager
- Remote Services и Binding Pattern
- Продвинутые советы манифеста
- Другие советы интеграции

Другие инструменты

- Диалоги и редакторы Android Studio
- Продвинутые возможности эмулятора
- Lint и поддержка аннотаций
- Инспектирование Layout
- Скриншоты и записи экрана
- Советы и трюки ADB
- Stetho
- Поиск узких мест ЦПУ

- Поиск утечек памяти

Тюнинг приложений Андроид

- Проблемы со скоростью
- Поиск узких мест ЦПУ
- NDK
- Повышение производительности CPU в Java
- Поиск и устранение "мусорных процессов"
- Проблемы с пропускной способностью
- Фокусируемся на: TrafficStats
- Измерение потребления пропускной способности
- Будь умнее о пропускной способности
- Проблемы с хипом приложения
- Поиск утечек памяти
- Проблемы с системной RAM
- Проблемы с временем жизни батареи
- Другие опции измерения потребления
- Источники утечки энергии
- Проблемы с адресацией размеров приложения

Дополнительные темы

- Crash Reporting with ACRA
- Диагностика внутри приложения
- Антипаттерны

Каталог виджетов

- AdapterViewFlipper
- CalendarView
- DatePicker
- ExpandableListView
- SeekBar
- SlidingPaneLayout
- StackView
- TabHost
- TimePicker
- ViewFlipper

Каталог устройств

- Chrome и Chrome OS
- BlackBerry
- Google TV
- Amazon Fire TV
- Samsung DeX

Приложения

- Приложение A: CWAC Libraries
- Приложение B: Андроид 8.x
- Приложение C: P Developer Preview
- Приложение D: Общественный театр и Appinars

Об обновлениях

Данная книга регулярно обновляется, обычно каждые 6-8 недель.

Каждый релиз содержит заметки показывающие, что нового или изменённого по сравнению с непосредственно предыдущим релизом:

- Содержание показывает части с изменениями шрифтом с полужирный курсивом
- Данные части содержат панели изменений справа для обозначения специальных параграфов, которые новые или изменённые.

О APK издании

В дополнение к классическим форматам электронной книги (PDF, EPUB, MOBI/Kindle), данная книга доступна как Андроид приложение, в формате APK файла. Данное приложение содержит встроенную программу чтения электронных книг, показывая Вам то же содержание, которое Вы можене найти в EPUB версии книги. Кроме того, присутствую некоторые функции, которые уникальны.

Первое, присутствует очень быстрый встроенный индекс полно текстового поиска. Вы можете быстро искать по ключевым словам, названиям класса, и тому подобное, с откликом в доли секунды на большинстве Андроид устройств. Также Вы можете использовать булевы условия при поиске (например поиск `encryption OR decryption`).

Второе, содержит Community Theater, где Вы можете посмотреть appinar и основанные приложении

тренировочные модули. Это презентации, содержащие слайды, видео, записи экрана, исходный код, и более. Через Community Theater, вы можете посмотреть appinar, загрузить те, что интересны, и просмотреть их когда Вам захочется.

APK версия книги работает на Андроид 4.0.3 и выше, часть с Community Theater работает только с Андроид 4.4 и выше.

Инструкция по установке APK версии может быть найдена на веб сайте CommonsWare. Детали о использовании Community Theater может быть найдена в дополнении данной книги.

Исходный код и его лицензия

Исходный код примеров, указанных в данной книги доступны для загрузки из GitHub репозитория книги. Все Андроид проекты лицензированы под лицензией Apache 2.0, в случаи, если Вы пожелаете использовать какой-либо из них.

Если Вы захотите использовать исходный код из репозитория GitHub, пожалуйста следуйте инструкции главной страницы репозитория для деталей о том, как использовать проекты в различных окружениях разработчика, особенно Android Studio.

Копирование исходного кода напрямую из книги в редакторах PDF, наилучше работает с Adobe Reader, хотя это может работать с другими просмотрщиками PDF. Некоторые просмотрщики PDF, по причинам остающимися непонятными, запутывают исходный код в буфере обмена когда он выбирается.

Creative Commons и Four-to-Free (42F) гарантии

Каждая редакции книги CommonsWare доступна для использования по лицензии Creative Commons Attribution-Noncommercial-ShareAlike 3.0 после четвёртой годовщине от даты публикации, или после продажи 4,000 копий издания будут проданы, что наступит первым. Что означает, когда пройдёт четыре года (возможно ранее), Вы сможете использовать данный текст в некоммерческих целях. Это наши гарантии Four-to- Free для наших читателей и более широкого сообщества. С целями данных гарантий, новое Warescriptions и обновления будут определяться исходя из количества продаж данного издания, начиная с времени публикации данного издания

Данная редакция данной книги будет доступна по вышеупомянутой Creative

Commons лицензии *1 Февраля 2023*. Конечно, просматривайте веб сайт CommonsWare, так как данное издание может быть перелицензированно быстрее, основываясь на продажах.

Для дополнительной информации о Creative Commons Attribution-Noncommercial-ShareAlik 3.0 лицензии, посетите Creative Commons веб сайт

Отметьте, что будущие издания данной книги, станут бесплатными позже, каждые четыре года после публикации данного издания или основываясь на продажах этого специального издания.
Издание одного издания под лицензией Creative Commons, не издаёт автоматически *все* издания под данной лицензией.

Благодарности

Я хотел бы поблагодарить команду Андроид не только за выпуск хорошего продукта, но и за бесценную помощь в группах Android Google и Stack Overflow.

Я также хотел бы поблагодарить тысячи читателей предыдущих изданий этой книги, за их обратную связь, отчёты об ошибках, и общую поддержку.

Конечно, благодарности также всей экосистеме Андроид, особенно тем разработчикам, которые привнесли свои навыки в общедоступные библиотеки, писали посты в блоги, поддерживали ответами на вопросы, или другим способом способствовали силе Андроид

Часть данной книги воспроизводит работу созданную и выложенную Android Open Source Project и использует в соответствии с терминами описанными в лицензии Creative Commons 2.5 Attribution.

Ключевые главы

Ключевые концепты Андроид

Без сомнения, Вы спешите начать разработку Андроид приложений. В конце концов, Вы читаете данную книгу, взятую на вооружение занятыми программистами.

Тем не менее, прежде чем мы углубимся в получение инструментов, настройке и непосредственно программированию, важно чтобы мы "остались на той же странице" с уважением к нескольким высоко уровневым концепциям Андроид. Это упростить будущую дискуссию позже в данной книге.

Приложения Андроид

Данная книга фокусируется на написании Андроид приложений. Приложение - это что-то, что пользователь может установить из Play Store или другим способом загрузить на их устройство. Такое приложение должно включать в себя некоторый интерфейс пользователя, и содержать остальной код разработанный для работе в фоне (мультизадачность).

Данная книга не сосредоточена на модификацию аппаратного программного обеспечения Андроид, такую как написание драйверов устройств. Для этого, Вам необходимо искать другие ресурсы.

Данная книга предполагает, что у Вас есть некоторый положительный опыт с устройствами Андроид, и кроме того Вы знакомы с кнопками HOME и BACK, встроенными настройками приложений, концептами главного экрана и загрузчика и т.д. Если Вы никогда не использовали Андроид устройства, строго рекомендуем Вам получить одно (новое или б/у) и провести некоторое время с ним, прежде чем ВЫ начнёте изучать разработку Андроид

приложений.

Языки программирования

Подавляющее большинство приложений Андроид написано эксклюзивно на Java. Следовательно, это то на что тратится большая часть данной книги и что демонстрируется с

практически бесконечным количеством

примеров. Кроме того, присутствуют

другие опции:

- Вы можете писать части приложения на C/C++, для повышения производительности, портирования существующего кода, и т.д.
- Вы можете написать всё приложение на C/C++, в основном для игр применяющих OpenGL для 3D анимации.
- Вы можете написать внутреннюю часть приложения на HTML, CSS, и JavaScript, используя инструмент для упаковки материалов в Андроид приложение, которое может быть распространено через Play Store или похожим способом.
- И так далее.

Что то из этого будет рассмотрено позже в данной книге, но подавляющее большинство данной книги фокусируется на расработке приложений основанных на Java.

Автор предполагает, что Вы знаете Java с этой точки зрения. Если нет, Вам необходимо изучить Java прежде чем продвигаться дальше. Вы не должны знать *всё* о Java, поскольку Java огромна Лучше, сфокусируйтесь на:

- Основы языка (управление потоком, и т.д.)
- Классы и объекты
- Методы и типы данных
- Public, private, и protected
- Static и область экземпляра
- Исключения
- Потоки
- Коллекции
- Дженерики
- Файловый ввод/вывод
- Рефлексия
- Интерфейсы

Ссылки ведут на материалы из Wikibooks этих тем, также есть множество других Java ресурсов по Вашему выбору.

Google также поддерживает Kotlin как язык программированиядля приложений Android. Kotlin может быть использован как замена Java. Хотя

Kotlin содержит набор преимуществ, он также относительно новый язык программирования. Данная книга сфокусирована более на Android SDK, чем Java (или Kotlin), и вне зависимости который из этих двух языков Вы используете, метериал данной книги будет актуальным. Примеры из книги в большинстве

на Java, по причине его преобладания среди языком программирования приложений Андрс
на данный момент.

Компоненты

Когда Вы сперва учили Java - когда бы это не было вчера или во времена кричащих динозавров на Земле - Вы вероятно начинали с чего-то похожего на это:

```
class SillyApp {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Другими словами, точка входа в Ваше приложение будет: `public static void` метод с названием `main()` который получает `String` массив аргуметров. С этого момента, Вы ответственны за выполнения того, что необходимо.

Тем не менее, существуют другие паттерны, применяемые где-то в Java. К примеру, Вы не всегда пишете метод `main()` когда пишете Java сервлет. Вместо этого, вы наследуете определённый класс поддерживаемый фреймворком (пр. `HttpServlet`) для создания компонента, после пишете некоторые метаданные, которые перечисляют Ваши компоненты и говорят фреймворку когда и как их использовать (пр, `WEB.XML`).

Приложения Андроид близки по духу к подходам сервлетов. Вы не пишете метод `public static void main()` Вместо этого, вы создаёте подкласс некоторого поддерживаемого Андроид базового класса, который описывает различные компоненты приложения. В дополнение, вы создаёте некоторые метаданные, которые говорят Андроид об этих подклассах.

Существует четыре типа компонентов, каждый из которых будет рассмотрен широко в данной книге:

Activities (Акитвности)

Строительный блок интерфейса пользователя - *activity* Вы можете думать об activity как о Андроид аналоге окна или диалога в приложения для ПК, или страницы классического веб приложения. Она представляет кусок Вашего интерфейса пользователя и в некоторых случаях, дискретную точку входа для Вашего приложения (т.е. путь для других приложений ссылаться на Ваше

приложение).

Обычно activity занимает основную часть экрана, оставляя промежуток для некоторых частей "chrome" таких как часы, индикатора уровеня сигнала, и так далее.

Однако, держите в памяти, что на некоторых устройствах пользователь способен работать с более чем одной activity одновременно, как режим разделения экрана на телефоне или планшете. Так, пока просто думайте о activity как эквиваленте экрана, просто запомните что это упрощение, и что реальность более сложна (как часто и бывает).

Services (Сервисы)

Activities коротко живущие и могут быть закрыты в любое время, когда пользователь нажимает кнопку BACK *Services*, с другой стороны разработаны, что продолжать работу, при необходимости, независимо от любой activity, средний период времени. Вы можете применять service для проверки обновлений RSS, или проигрывания фоновой музыки даже если управляющая activity больше не доступна. Вы также можете использовать service для запланированных задач (как Linux или macOS “cron jobs”) или для доступа к пользовательскому API из других приложений на устройстве, хотя последняя возможность является относительно продвинутой.

Content Providers (поставщики контента)

Content providers предоставляют на уровне абстракции любые данные, сохранённые на устройстве, которые доступны многим приложениям. Модель разработки на Андроид поощряет Вас создавать собственные данные доступными другим приложениям, также как и себе - создание content provider позволяет Вам это делать, при поддержке уровня контроля как Ваши данные будут доступны.

Так, к примеру, если у Вас есть PDF файл, который вы загрузили на стороне пользователя, и Вы хотите предоставить пользователю возможность просматривать этот PDF файл, Вы должны создать content provider, чтобы сделать этот PDF файл доступным другим приложениям. Вы можете потом запустить activity, которая будет в состоянии просмотреть этот PDF, тогда как пользователь Андроид будет определять какая activity для просмотра PDF будет перехватывать этот запрос.

Broadcast Receivers (приёмники вещания)

Система или приложение, посылает *broadcasts* время от времени, для всего от низкого уровня заряда батареи, до того когда экран выключается, или когда изменяется статус подключения с WiFi к мобильным данным. Broadcast receiver могут выполнять ожидание данной передачи и отреагировать

соответственно.

Виджеты, Контейнеры и ресурсы

Основной фокус в разработке Андроид приложений на уровне интерфейса пользователя или activity. Большинство activities Андроид применяется, что известно как “фреймворк виджетов” для

рендеринга их интерфейса пользователя, так Вы можете использовать 2D (`Canvas`) и 3D (OpenGL) API также как и более специальные GUI.

В терминах Андроид *widget* - "микро" единица интерфейса пользователя. Поля, кнопки, текстовые поля, списки и т.д. - всё виджеты. Следовательно Ваш интерфейс пользователя activity создан из одного или более виджетов. Это общая возможность в большинстве наборов инструментов UI, и более вероятно, Вы уже работали с кнопками, текстовыми полями, полями и похожими типами виджетов где-либо до этого.

Если у Вас более одного виджета - что обычно стандартно - Вам необходимо сказать Андроид как данные виджеты организованы на экране. Чтобы сделать это, вы можете использовать различные классы контейнеров называемые *layout managers*. (менеджеры слоёв). Что позволяет Вам располагать элементы в строки, столбцы, и более сложные порядки при необходимости.

Что описать как контейнеры и виджеты расположены, Вы должны обычно созданы *layout resource file*. (файл ресурсов слоёв) *Ресурсы* в Андроид ссылаются на элементы такие как рисунки, строки, и другие материалы, которые использует Ваше приложение но не является одним из видом исходного кода языка программирования. Слои интерфейса пользователя - другой тип ресурсов. Вы создаёте данные слои с использованием инструмента структурирования, такого как IDE drag-and-drop GUI builder, или вручную в формате XML.

Иногда, Ваш интерфейс пользователя работает на всех типах устройств: телефоны, планшеты, телевизоры, и т.д. Иногда, Ваш интерфейс пользователя нуждается в адаптации для различного окружения. У Вас есть возможность расположить ресурсы в *resource sets* (наборы ресурсов), которые указывают при каких условиях эти ресурсы могут быть использованы (пример использовать эти для нормальных размеров экрана, а эти - для больших экранов).

Мы рассмотрим все эти концепты, с большей детальностью, как будем погружаться дальше в книгу.

Приложения и пакеты

Имея набор исходного кода и набор ресурсов инструмент компилирования Андроид выдаст Вам приложение, как результат. Приложение предоставляется в виде *APK файла*. Это тот APK файл, который Вы загружаете в Play Store или распространяете другим способом.

Каждое Андроид приложение содержит название пакета, также ссылающееся на ID приложения. Название пакета должно удовлетворять трём требованиям:

1. Это должно быть валидное название пакета Java, так как некоторый исходный код в данном пакете будет сгенерирован инструментом компиляции.

2. На устройстве не может существовать два приложения одновременно с одинаковым ID приложения.
3. В Play Store не могут быть загружены два приложения с одинаковыми ID приложения.

Когда вы создаёте Ваш Андроид проект - репозиторий этого исходного кода и эти ресурсы - Вы объявляете какое название пакета будет использовано для Вашего приложения. Обычно, Вы выбираете название пакета исходя из названия пакета Java в конвенции "обратного доменного имени" (пример `com.commonsware.android.foo`). Таким способом, система доменных имён определяет, что Ваш префикс название пакета (`com.commonsware`) уникально, и уже на своей стороне Вы должны убедиться что названия пакетов отличается от одного Вашего приложения к другому.

Устройства Андроид

Сегодня используется более одного миллиарда Android-устройств, представляющие тысячи разных моделей от десятков различных производителей. Андроид самостоятельно развивается начиная с Андроид 1.0 в 2008. Между различными типами устройств и различными версиями Андроид, многие медиа эксперты лоббируют термин "fragmentation" (фрагментация) в Андроид, предлагая, что создание приложения, которое запускается на всех этих различных окружениях невозможно.

В действительности не всё так плохо. Некоторые приложение имеют существенные проблемы, но большинство приложений работают достаточно хорошо, если вы следуете руководству представленному в данной книге и в других ресурсах.

Типы

Андроид устройства существую во всех формах, размерах и цветах. Тем не менее, существует три преобладающих "форм фактора":

- телефон
- планшет
- ноутбук (в основном, они запускают Chrome OS, которая частично поддерживает Андроид приложения)

За пределами этого, существует несколько менее распространенных форм факторов.

- телевизор (TV)
- носимые устройства (умные часы и т.д.)

- настольные (устройства разработанные для подключения монитора, клавиатуры и мыши)

Вы часто слышали, что разработчики и эксперты ссылаются на данные форм факторы, и данная книга делает то же время от времени. Тем не менее, важно чтобы вы понимали, что Андроид не содержит собственного концепта устройств, которые "телефон", "планшет" или "TV". Скорее, Андроид распознает устройства основываясь на возможностях Так Вы не увидите метод `isPhone()` где-либо, так Вы можете спросить Андроид:

- какой размер экрана?
- содержит ли устройство совместимость с телефоном?
- и т.д.

Аналогично, если Вы создаёте Ваше приложение, скорее чем думать об этих форм факторах, фокусируйтесь на необходимых совместимостях и функциях. Это не только поможет понять как Андроид предполагает Вы создаёте Ваше приложение, но и сделает проще адаптировать к другим форм факторам, которые представлены:

- центрами развлечения в сидениях самолёта
- автомобильными устройствами навигации и развлечения
- и так далее.

Эмулятор

Так как ~ 2 миллиарда Андроид устройств, которые представлены ~ 10,000 моделями устройств, у Вас вероятно нет каждого из представленных моделей. У Вас может быть только одно устройство аппаратного обеспечения Андроид. И даже если и его нет, вы конечно хотите получить одно до публикации приложения Андроид.

Чтобы помочь заполнить промежуток между устройствами, которые у Вас есть и возможными устройствами, инструменты разработчика Андроид приводят к *эмулятору*. Эмулятор работает как часть аппаратного обеспечения Андроид, но это программа, которая запускается на вашей машине разработчика. Вы можете использовать данный эмулятор для эмулирования различных устройств, с различными размерами экрана и версиями Андроид, создавая одно или несколько виртуальных устройств Андроид или *AVDs*.

В следующей главе мы обсудим как устанавливать инструменты разработчика Андроид и мы сможем создавать эти AVDs и запускать эмулятор.

Версия ОС и уровни API

Андроид прошёл большой путь начиная с ранней беты конца 2007. В каждую новую версию добавлялись больше возможностей в платформу и больше элементов, которые разработчики могли выполнять используя эти возможности.

Кроме того, основная команда разработчиков Андроид очень сильно старалась обеспечить прямую и обратную совместимости. Приложение, которое Вы пишете сейчас должно работать без изменений в будущих версиях Андроид (прямая совместимость), хотя возможно пропадание некоторых функций или работе в некотором типе "режим совместимости". И это хорошо проторённая дорога как создавать приложения работающие и на последней версии и на предыдущих версиях (обратная совместимость).

Чтобы помочь нам следить за всеми различными версиями ОС, что интересует нас как разработчиков, Андроид содержит *API levels* (уровни API). Новый уровень API описывается, когда версия Андроид поставляется с изменениями затрагивающими разработчиков. Когда вы создаёте эмулятор AVD для тестирования Вашего приложения, Вы можете указать какой уровень API эмулятор должен эмулировать. Когда Вы распространяете Ваше приложение, Вы указываете самый старый уровень API поддерживаемый Вашим приложением, так приложение не будет устанавливаться на устройства, которые старее.

На время написание, уровни API, значимые для большинства разработчиков Андроид следующие:

- API Уровень 19 (Андроид 4.4)
- API Уровень 22 (Андроид 5.1)
- API Уровень 23 (Андроид 6.0)
- API Уровень 24 (Андроид 7.0)
- API Уровень 25 (Андроид 7.1)
- API Уровень 26 (Андроид 8.0)

Здесь, "значимые" относиться к уровням API с резонным количеством Андроид устройств - 5% и более, как указано в “Platform Versions” dashboard chart.

Последняя версия Андроид 9.0, API Уровень 28.

Отметьте, что API Уровня 20 используется для версии Андроид 4.4 работающей на первом поколении устройств “Android Wear” (сейчас называется “Wear OS”). Если Вы не разрабатываете приложение специально

для Wear, Вам не стоит беспокоится сильно о API Уровень 20.

Dalvik и ART

В терминах Андроид, Dalvik и ART являются виртуальными машинами (VM). Виртуальные машины используются многими языками программирования, такими как Java, Perl, и Smalltalk. Dalvik и ART разработаны для работы похоже на Java VM, но оптимизированы для встроенного окружения Linux.

Главное, разница между этими двумя в том, что ART используется на Андроид 5.0 и выше, тогда как Dalvik использовался на более старых устройствах. По правде говоря, история более сложная, но пока этого достаточно.

Так, что в действительности происходит когда кто-либо пишет Андроид приложение:

1. Разроботчик пишет исходный код в синтаксисе Java, использует библиотеки класса опубликованные проектом Андроид и третьими лицами.
2. Разработчики компилируют исходный код в байткод Java VM, используя `javac` компилятор который поставляется с Java SDK.
3. Разработчики транслируют байткод Java VM в байткод Dalvik VM, который упаковывается с другими файлами в ZIP архив с разрешением `apk` (файл APK)
4. Андроид устройство или эмулятор запускает файл APK, вызывая выполнение байткода в экземпляре Dalvik или ART VM.

С вашей точки зрения,, большинство из этого скрыто за инструментом разработки. Ваш чистый исходный код Java наверху, файл APK - снизу.

Тем не менее, могут быть места где различие между Dalvik VM и традиционной Java VM влияют на приложение разработчиков, и данная книга указывает на некоторые из них если необходимо.

Процессы и потоки

Когда Ваше приложение выполняется, оно выполняется в собственном процессе. Нет значительных отличий от других стандартных операционных систем. Часть магии Dalvik делает возможным выполнение многих процессов многих Андроид приложение одновременно без потребления нелепого количества RAM.

Андроид также настраивает набор потоков для выполнения Вашего приложения. Потоки, которые Вы программируете будут выполнятся в,

большую часть времени, как различно называют “main application thread” (главный поток приложения) и “UI thread” (поток интерфейса пользователя). Вы не должны их настраивать, и как мы увидим дальше в кинге, Вам необходимо обратить внимание, что вы делаете, а что не делаете в

этом потоке. Вы можете использовать ваши собственные потоки для работы, что происходить часто, потопу в некоторых местах Андроид управляет этим за кулисами.

Не пугайтесь

Да, данная глава бросила в Вас много терминов. Мы погрузимся в них более детально в данной книге. Кроме того, Андроид как jigsaw пазл с большим количеством взаимосвязанных частей. Чтобы иметь возможность описать один концепт детально, мы должны как минимум ссылаться на некоторые другие. Следовательно, данная глава выбросила на Вас термины в надежде, что они будут звучать смутно знакомыми как мы погрузимся в детали.

Выбор Вашего Набора инструментов Разработчика

Прежде чем Вы продвинетесь гораздо дальше в Вашем стремлении В Андроид (или возможно стремлений, в зависимости как Вы предпочитаете писать), Вам необходимо определиться с набором инструментов, который будете использовать для разработки Андроид приложений.

Android Studio

Текущая поддерживаемая Google Андроид IDE - Android Studio. Основанная на IntelliJ IDEA, Android Studio - это новая учреждение усилий Google по предоставлению разработчикам Android первоклассных инструментов разработки.

Следующая глава содержит часть с инструкцией по настройке Android Studio.

Отметьте также, что Android Studio требует достаточно мощной машины для хорошей работы при разработки: быстрый ЦПУ, много RAM (8ГБ и более), и SSD - всё строго рекомендуется.

Eclipse

Eclipse также популярная IDE, особенно для Java разработки. Eclipse была исходной IDE Google для Андроид разработки, с помощью надстройки Android Developer Tools (ADT), которая предоставляет основную информацию для Eclipse об Android. Дополнение ADT, необходимо, для выполнения стандартных операций в Eclipse и расширяет её для работы с Андроид проектами.

Отметьте, также, что Google прекратила поддержку ADT. Eclipse Foundation создаёт проект "Andmore" пытаясь продолжить работу на возможности создания Андроид приложений. Данная книга не охватывает проекты Andmore в данное время, и мы **настоятельно** рекомендуем разработчикам не использовать Eclipse с ADT от Google.

IntelliJ IDEA

Так как Android Studio основана на IntelliJ IDEA, Вы всё ещё можете использовать оригинальную IntelliJ IDEA для разработки приложений Андроид. Большой набор возможностей Android Studio доступных в дополнении Андроид для IDEA. Плюс коммерческая IDEA Ultimate Edition уходит вперёд Android Studio во многих областях за пределами Андроид разработки.

В частности, если вы ищете "единственную настоящую IDE", которую Вы можете использовать для Андроид и не Андроид проектов, Вы должны принять во внимание IntelliJ IDEA. Android Studio хороша, но в большинстве для Андроид проектов.

Компиляция в командной строке через Gradle

И, конечно, нет необходимости использовать IDE вовсе. Хотя это может звучать кощунственно для некоторых, IDE - не единственный путь создания приложений. Большинство того, что выполняется через IDE, может выполнятся через эквивалент командной строки, означая, что оболочка и редактор - это все, что Вам действительно необходимо.

Рекомендуемый способ создания Андроид приложений за пределами IDE - посредством Gradle. Google опубликовала дополнение Gradle, которое учит Gradle как компилировать Андроид приложение. Android Studio сама использует Gradle для компиляции, так единственная конфигурация компиляции (например `build.gradle` files) могут быть использованы как из IDE так и из инструмента автоматической компиляции, по типу непрерывного интеграционного сервера.

Следующая глава предоставляет больше о Gradle (и Android Gradle Plugin).

Ещё другие альтернативы

Другие IDE содержат свои эквиваленты ADT, хотя и с минимальной поддержкой от Google. К примеру, NetBeans содержит поддержку через дополнение NBAndroid, и по сообщениям значительно продвинулся за последние пару лет.

Вы также можете услышать отсылки к применению Apache Ant для выполнения компиляции Андроид приложений в командной строке. Что было вытеснено Android Gradle Plugin в данное время, и Apache Ant под слабой поддержкой. Новички в Андроид предпочитают не вкладывать время в новой работе в Apache Ant при разработке Андроид проектов.

IDE… и данная книга

Вам рекомендуется использовать Android Studio, при работе с данной книгой. Вы можете использовать другую IDE, если пожелаете. Вы можете даже пропустить IDE, и использовать только редактор.

Данная книга фокусируется в основном на демонстрации возможностей и API эксплуатирующие данные возможности. Следовательно, код в примерах будет работать с любым IDE. Однако, данная книга охватывает некоторые специальные инструкции, поскольку это преобладающая Андроид IDE для использования в данное время.

Что мы не охватываем

В начале (а именно 2007), мы были счастливы получить хотя какие средства для создания Андроид приложений.

В настоящее время, кажется нет завершения средств, которыми мы можем создавать Андроид приложения.

Несколько из этих "средств", всё-таки, специально за пределами данной книги.

App Inventor

Вы также возможно слышали об инструменте называемом App Inventor и интересно, как это соотносится со всем этим.

App Inventor был изначально создан образовательной группой Google, как средство обучения студентов, как думать о конструктах программирования (разветвления, циклы и т.д.) и получать интересный результат (Андроид приложение) без классического программирования на Java или другом основанном на синтаксисе языке. App Inventor - чисто drag- and-drop, как с виджетами так и с *логикой приложения*, последний - посредством «блоков», которые соединяются вместе, образуя логические цепочки.

App Inventor создавался на пожертвования от Google MIT, который недавно открыл его в публичный доступ.

Однако, App Inventor закрытая система - в настоящее он не генерирует как либо Java код, который Вы можете дальше использовать. Это ограничивает Вас, чтобы App Inventor изначально не делал, хотя и впечатляет сам по себе, предлагает небольшую часть всех возможностей Android SDK.

App Generators

Существует казалось бы неограниченное количество “app generators” (генераторов приложения) доступные как сервисы в сети. Они разработаны в основном для разработки приложений для определённых вертикальных рынков, таких как приложения для ресторанов или приложения для магазинов. Результирующие приложения в большей степени “brochure-ware” (программное обеспечение - брошюра), с несколькими возможностями за пределами мобильного веб сайта, но ещё требует от пользователя найти, загрузить и установить приложение. Некоторые из этих генераторов предоставляют исходный код сгенерированного приложения, что позволяет изменять приложение за пределами того, что сгенерировал генератор.

Руководство №1 - Установка инструментов

Сейчас, позвольте предоставить Вам настройку с деталями и частями необходимыми для создания Андроид приложения.

Заметка: Предоставленная здесь инструкция актуальна на время написания. Так как инструменты меняются быстро и так представленные инструкции могут быть устаревшими на время прочтения книги. Пожалуйста посетите, веб сайт разработчиков Андроид для текущей инструкции, используя инструкцию как руководство, чего ожидать.

Но сперва некоторые заметки о эмуляторе Андроид

Инструменты Андроид включают в себя эмулятор, часть программного обеспечения, которое претендует быть Андроид устройством. Это очень полезно для разработки - что не только означает, что вы можете начать разработку без устройства, но что эмулятор может помочь тестировать конфигурации устройств, которых у Вас нет.

Существует два типа эмуляторов: x86 и ARM. Это два основных типа процессоров используемых а Андроид устройствах. Вам *действительно* захочется применять эмулятор x86, так как эмулятор ARM чрезвычайно медленный.

Однако, чтобы использовать эмулятор x86, Ваша машина для разработки должна содержать корректно настроенные вещи. Пользователям Linux необходим KVM, Mac и Windows пользователям необходим “Intel Hardware Accelerated Execution Manager” (т.е. HAXM) (менеджер выполнения аппаратного ускорения от Intel).

Также, это работает только с определёнными архитектурами процессоров,

которые поддерживают виртуализацию на аппаратном уровне.

- Функциями Intel Virtualization Technology (VT, VT-x, vmx)
- AMD Virtualization (AMD-V, SVM) (Linux only)

Данные функции виртуализации также должны быть включены в BIOS вашего устройства и могут быть необходимы другие изменения связанные с ОС.

Также, как минимум для новый уровней API, Ваш ЦПУ, должен поддерживать SSSE3 инструкции, хотя детали требований не задокументированные по сотоянию на Октябрь 2017г.

Часть инсталляционного процесса Android Studio, попытается настроить Вам возможность использования эмулятора x86. Отмечайте все сообщения, которые Вы видите в мастере установки относящиеся к “HAXM” (или, если Вы запускаете Linux, KVM) так как они будут важны после.

Шаг №1: Проверьте Ваши аппаратные средства

Компиляция Андроид приложения, сама по себе, может быть аппаратно интенсивным процессом, особенно для больших проектов. Кроме того, Ваше IDE и эмулятор Андроид может увеличить нагрузку на Вашу машину для разработки. Из двух, эмулятор представляет большую проблему.

Чем больше RAM у Вас, тем лучше. 8Гб или больше - очень хорошая идея если Вы предполагаете использовать эмулятор и IDE совместно. Если Вы можете получить SSD для Вашего хранения данных, вместо стандартного жёсткого диска, это может значительно увеличить производительность IDE.

Быстрый ЦПУ также хорошая идея. Эмулятор Андроид SDK, с 2006, поддерживает ЦПУ с несколькими ядрами - до этого поддерживалось только одно ядро. Однако, другие процессы на вашей машине для разработки будут выполняться с эмуляторов с процессорным временем, и чем быстрее Ваш ЦПУ, тем лучше вам будет. Идеально, Ваш ЦПУ содержит 2-4 ядра, каждое по 2.5 Ггц или быстрее их базовой скорости.

Кроме того, для использования эмулятора x86 - эмулятора, который работает хорошо - Вам нужен ЦПУ с определёнными функциями:

ОС разработки	Требования ЦПУ
Windows	Intel ЦПУ с поддержкой VT-x, EM64T, и “Execute Disable” (XD)
Mac	любой
Linux	Intel ЦПУ с поддержкой VT-x, EM64T, and “Execute Disable” (XD),или AMD ЦПУ с поддержкой AMD-V

Если Ваш ЦПУ не поддерживает данные требования, Вы заходите получить 1+ устройство Андроид доступное для Вас, чтобы Вы могли производить тестирование на аппаратном уровне.

Также, если Вы запускаете Windows или Linux, Вам необходимо убедится, что Ваш BIOS компьютера настроен для поддержки функций виртуализации Intel. К сожалению, многие производителя ПК отключают её по умолчанию. Датели того как войди в настройки BIOS, могут отличаться от ПК к ПК, но обычно включает в себя перезагрузку Вашего компьютера и нажатие некоторых функциональных клавиш при начальном экране загрузки. В настойках BIOS, ищите ссылки на “virtualization” или “VT-x”. Активируйте их, если они ещё не активированы. Машины с macOS предоставляются с предварительно активированной виртуализацией.

Шаг №2: Настройка Java и поддержки 32 битных Линукс

Когда Вы пишете Андроид приложение, Вы обычно пишете его на Java. Исходный код Java потом преобразуется в что-то, что Адроид в действительности запускает (Dalvik байткод в файле APK).

Android Studio, начиная с версии 2.2, поставляется с собственной приватной копией OpenJDK 8, и использует её по умолчанию. Вы можете установить и использовать Вашу собственную JDK, по желанию, в идеале это Java 8.

Если ваша ОС разработки Linux, убедитесь, что вы запускать 32-битные бинарные файлы Linux. Это может быть как активировано так и нет на Вашем дистрибутиве Linux. К примеру на Ubuntu 17.10, Вам необходимо запустите следующее, чтобы установить поддержку 32-битных бинарных файлов, которые необходимы для инструментов компиляции Андроид.

```
sudo apt-get install libncurses5:i386 libstdc++6:i386 zlib1g:i386
```

Шаг №3: Установка Android Studio

Как указано в предыдущей главе, существует несколько инструментов разработчиков из которых Вы можете выбирать.

Руководства в книге направлены на Android Studio. Вы можете попробовать использовать Eclipse, другую IDE, или не использовать IDE вообще для компиляции Андроид приложений. Кроме того, вам нужно будет преобразовать некоторые из руководств относящихся к IDE как это будет необходимо для выбранного инструмента разработки.

На время написания книги, текущая производственная версия Android Studio 3.1, и данная книга охватывает эту версию. Если Вы читаете это в будущем, Вы возможно будете на новой версии Android Studio, и могут быть некоторые различия между тем, что у Вас и что представлено здесь.

У Вас есть две основные опции для загрузки. Вы можете получить последнюю предоставляемую версию Android Studio из страницы загрузки Android Studio.

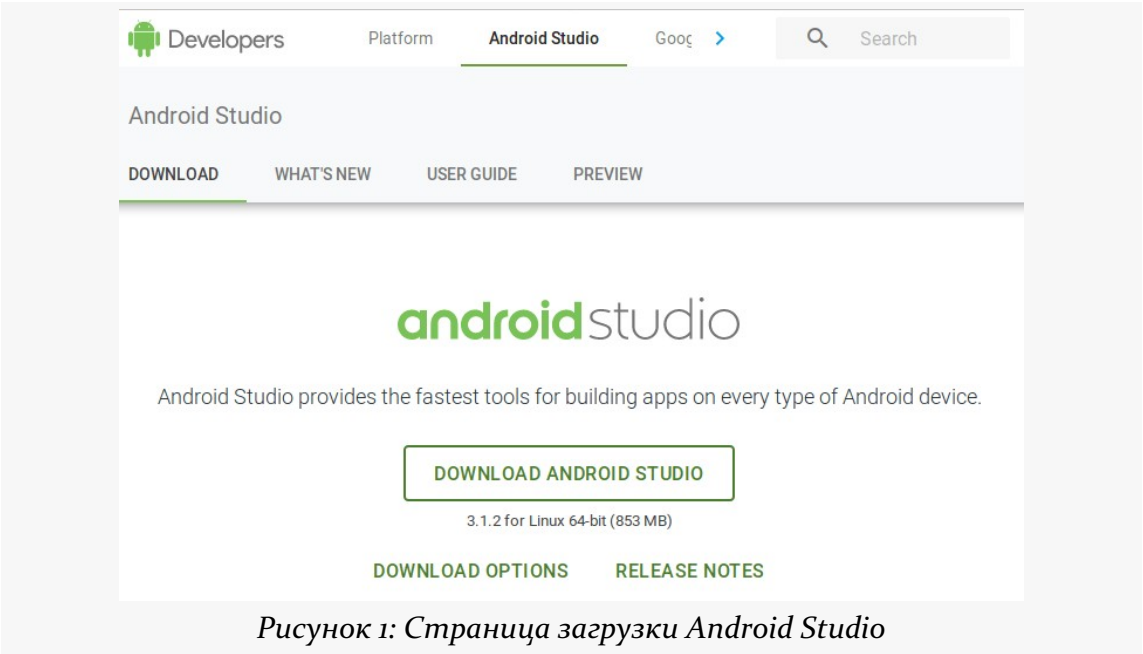


Рисунок 1: Страница загрузки Android Studio

Или Вы можете загрузить Android Studio 3.1 - версию применяемую в данном издании книги - напрямую для:

- Windows

- macOS
- Linux

Пользователи Windows могут загрузить самоустанавливающийся EXE, который добавит подходящие опции запуска для вас, что позволит запускать IDE.

Пользователи macOS могут загрузить образ диска DMG и установить его как и другие программы macOS, перемещая значок Android Studio в директорию Applications.

Пользователи Linux (и продвинутые пользователи Windows) могут загрузить файл ZIP, после разархивировать его в нужное место на диске. Android Studio может быть потом запущена из пакетного файла **`studio`** или скрипта оболочки из директории `bin/` . установки Вашей Android Studio.

Шаг №4: Установка SDK и дополнений

Дальше, нам необходимо просмотреть какие части из Android SDK мы уже установили и возможно установить некоторые новые элементы. Чтобы выполнить это, Вам необходим доступ к SDK Manager.

Когда Вы в первый раз запускаете Android Studio, у Вас спросят хитите ли Вы импортировать настройки из некоторой предыдущей установки Android Studio:

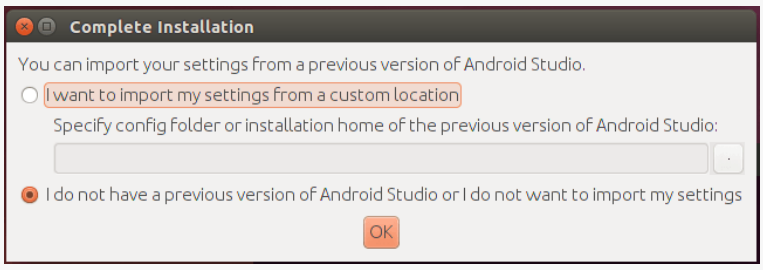


Рисунок 2: Диалог миграции при первом запуске Android Studio

Для большинства, особенно использующих Android Studio в первый раз, опция “I do not have...” - правильный выбор, чтобы сделать.

Потом, после короткого всплывающего экрана и потенциально долгого диалога прогресса “Finding Available SDK Components” (поиск доступных компонент SDK), Вы увидите Android Studio Setup Wizard (мастер настройки Android Studio):

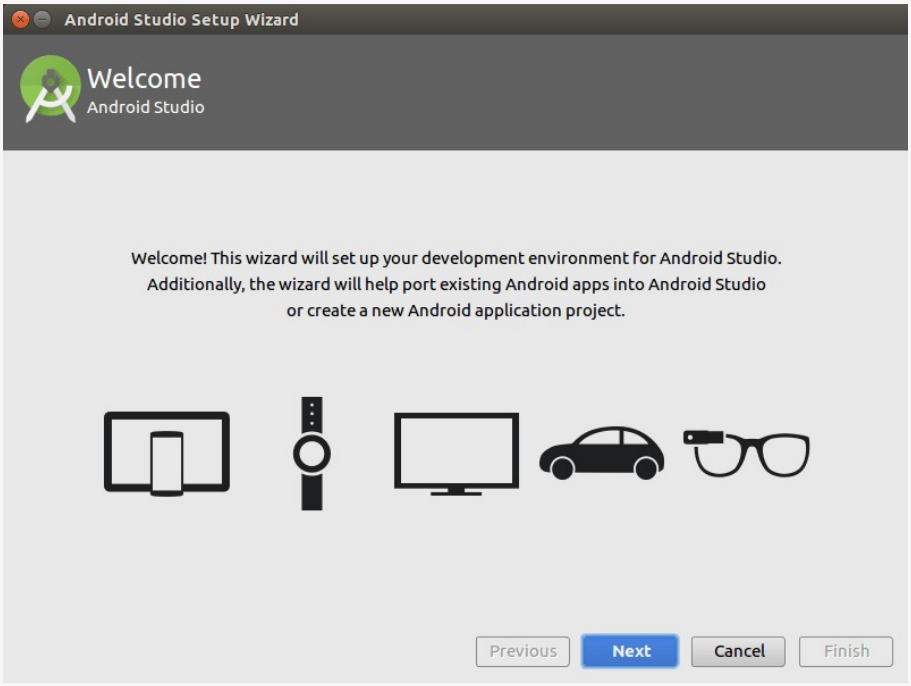


Рисунок 3: Первая страница мастера настройки Android Studio

Просто нажмите “Next” (далее) для перехода на вторую страницу мастера:

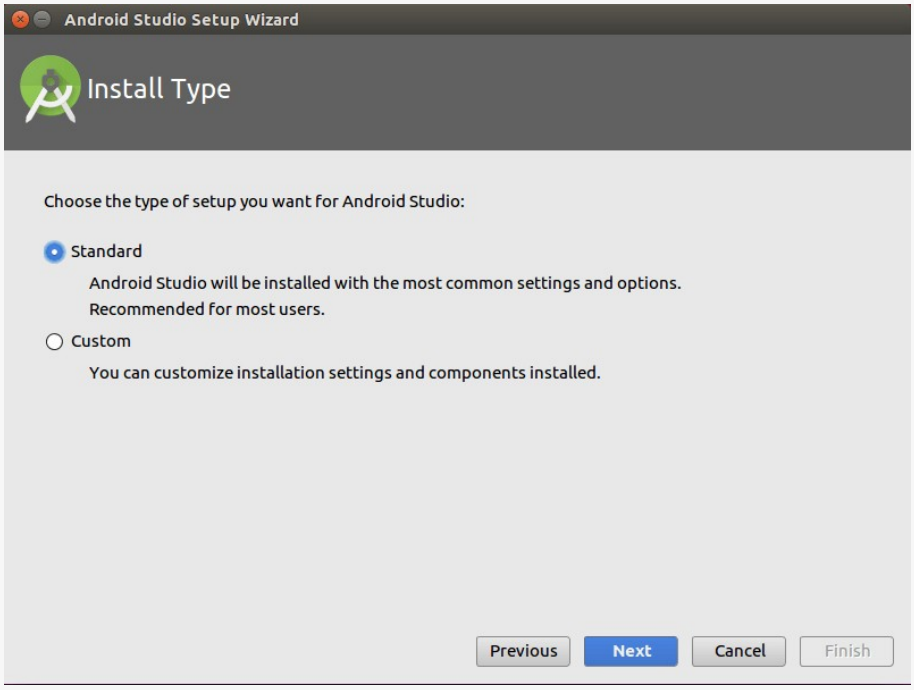


Рисунок 4: Вторая страница мастера настройки Android Studio

Здесь, у Вас есть выбор между “Standard” (стандартным) и “Custom” (пользовательским) режимами работы. Наиболее вероятно, сейчас, "Standard" путь будет лучше для вашего окружения.

Если Вы перешли на "Standard" путь и нажали “Next”, Вы перейдёте на страницу мастера, где Вы можете выбрать Вашу тему UI:

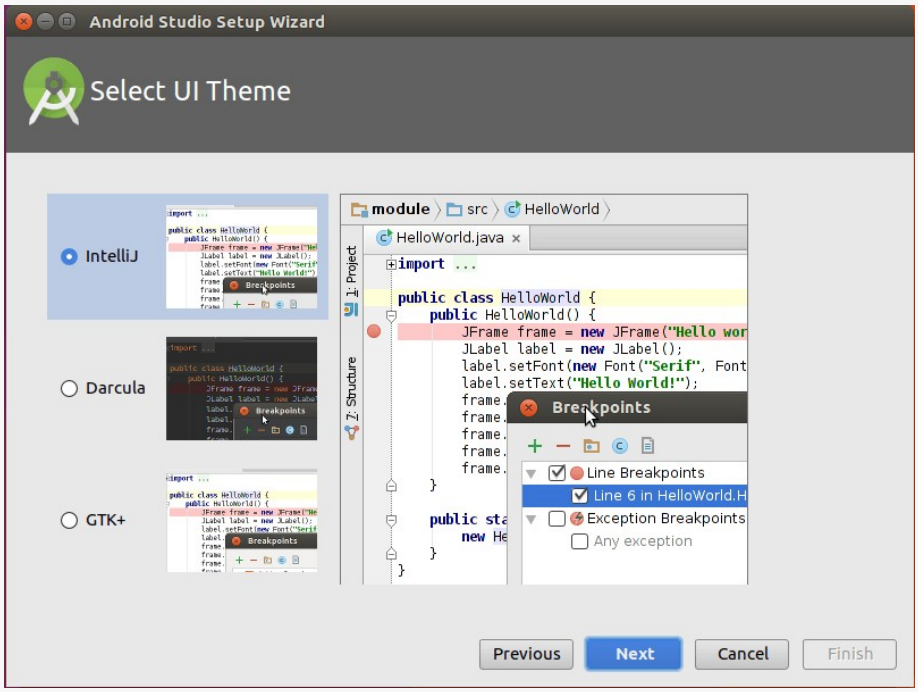


Рисунок 5: страница темы UI мастера настройки Android Studio

Выберите что Вам понравится и нажмите дальше, чтобы перейти на страницу мастера для верификации, что необходимо загрузить и установить:

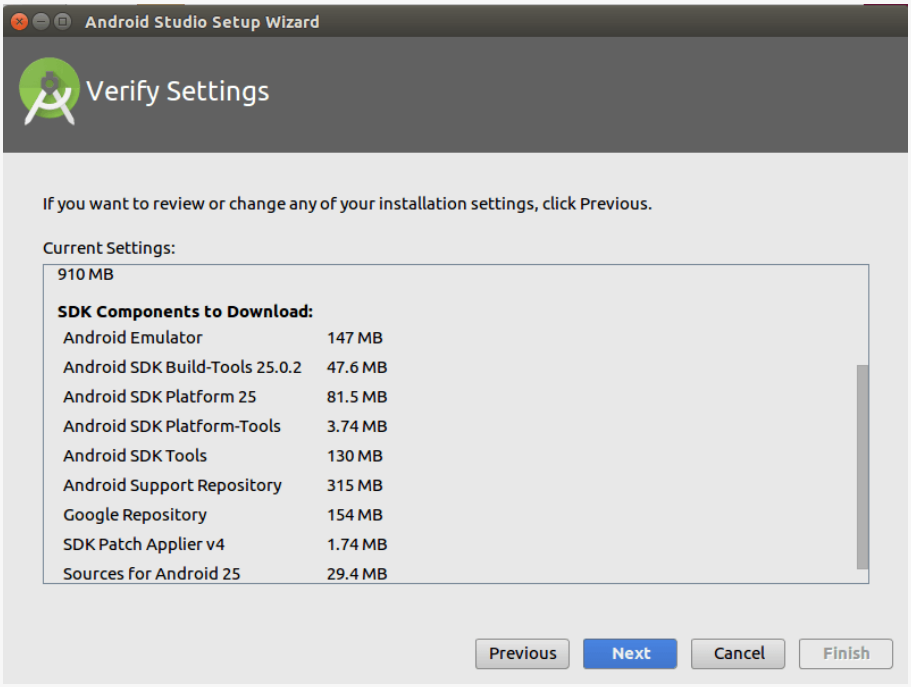


Рисунок 6: Страница верификации настроек мастера настройки Android

Нажатие Next направит Вас на страницу мастера, объясняющие некоторую информацию про Андроид эмулятор:

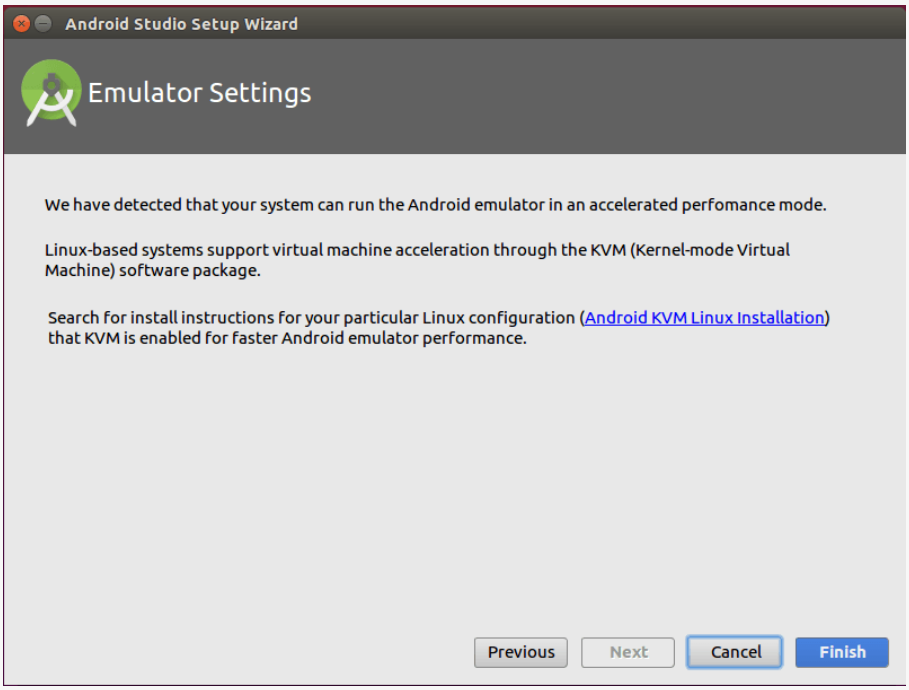


Рисунок 7: Страница информации эмулятора мастера настройки Android

То, что объясняется на данной странице может не иметь смысла для Вас. Это абсолютно нормально, и мы вернёмся к тому, что говорится на данной странице далее в книге. Просто нажмите "Finish" для начала процесса установки. Что включает загрузку копию Android SDK и установку её в директорию смежную, к той где установлена Android Studio.

После завершения и нажатия "Finish", Android Studio навязчиво запустит загрузку данных для вашей машины разработки. Если Вы запускаете Linux и Ваша установка прекращается с ошибкой "Unable to run mksdcard SDK tool", вернитесь к "Шагу №2" и настройте 32-битную поддержку в вашем окружению Linux.

Нажатие "Finish" приведёт Вас к приветственному диалогу Android Studio.

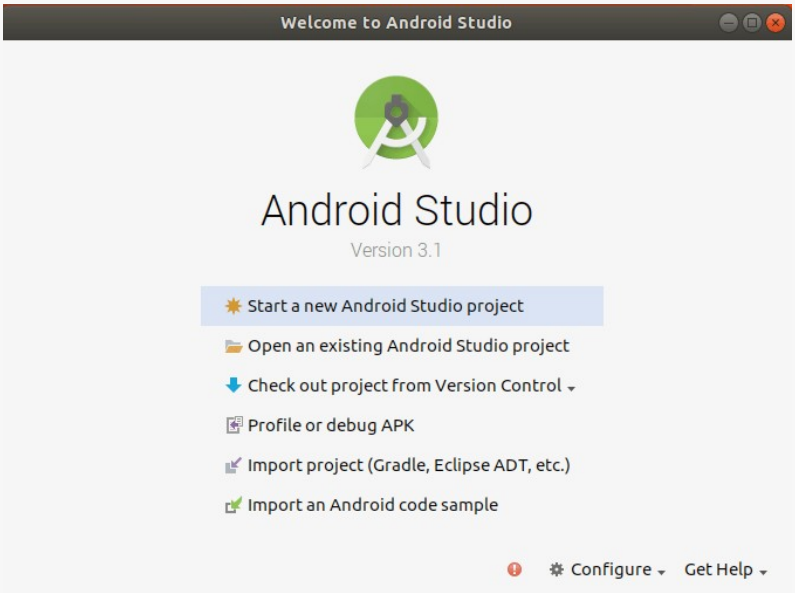


Рисунок 8: Диалог приветствия Android Studio

После в диалоге приветствия, нажмите Configure (настроить), для получения выпадающего списка настроек:

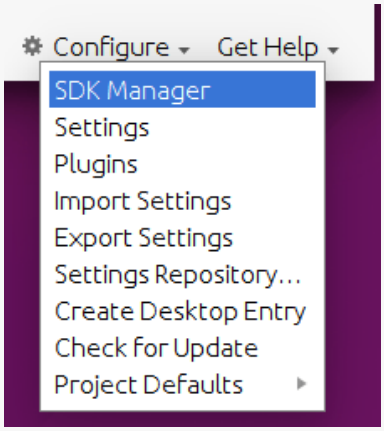


Figure 9: Диалог приветствие Android Studio выпадающий список

Здесь нажмите на SDK Manager для перехода к SDK Manager.

Использование менеджера SDK и обновление Вашего окружения

У Вас сейчас должен быть открыт SDK Manager как часть общих настроек по умолчанию Android Studio:

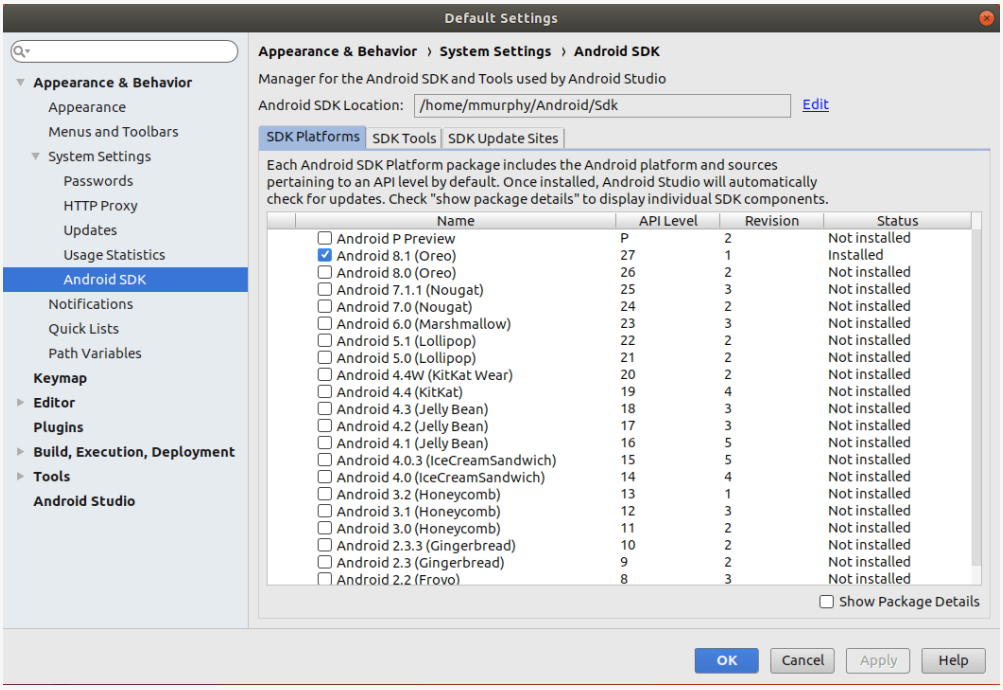


Рисунок 10: Менеджер Android SDK, вкладка “SDK Platforms”

Вкладка “SDK Platforms” предоставляет список версий Андроид на которые Вы можете перекомпилировать. Последняя версия Андроид (или возможно и предыдущая) обычно устанавливается, когда Вы изначально настраиваете Android Studio. Однако для данного руководства, пожалуйста также отметьте "Андроид 7.1 (Nougat)" в списке, если это уже не отмечено, и после нажмите кнопку "Apply" (Применить) для загрузки и установки данной версии. Вам необходимо принять информацию о лицензии в диалоге, как часть процесса - просмотреть лицензию, и если Вы принимаете её, нажмите "Next", чтобы начать установку.

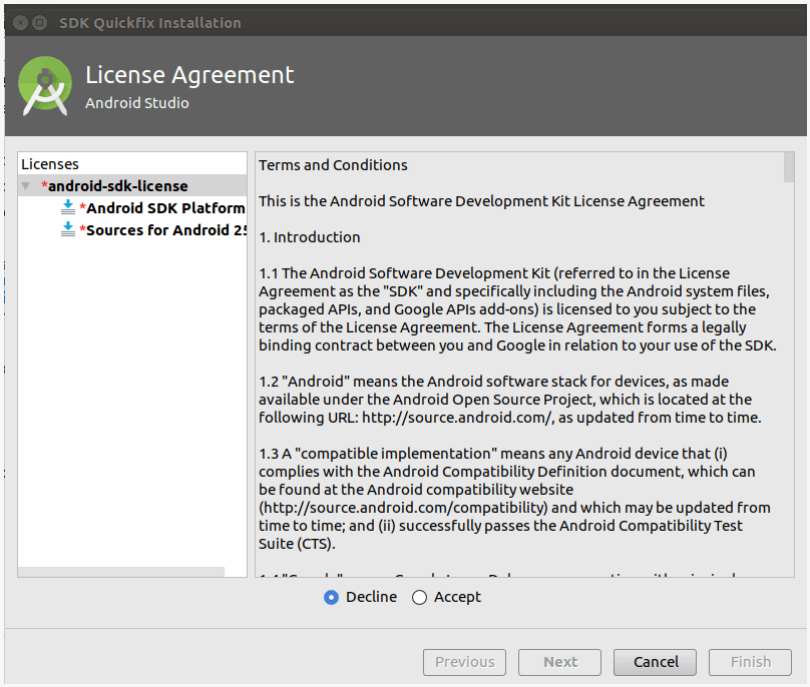


Рисунок 11: Менеджер Андроид SDK, Диалог принятия лицензии

Когда, это будет завершена, Вы можете нажать "Finish" (Завершить) для закрытия диалога, и после Вы вернётесь к менеджеру SDK. После, Вы можете закрыть менеджер SDK нажатием кнопки OK.

В нашем следующем эпизоде...

... мы создадим андроид проект который будет базовым для всех наших последующих руководств, плюс настроим эмулятор и устройства.

Андроид и проекты

Когда Вы работаете над созданием приложения для Андроид, Вы будете делать это работая в "проекте". Проект - это директория, содержащая Ваш исходный код и другие файлы, как картинки так и описания UI. Ваш IDE и другой инструмент компиляции возьмёт что есть из Вашего проекта и сгенерирует приложение Андроид (APK) на выходе.

Детали, как начать свой проект отличаются на основании того, какую IDE Вы используете, так данная глава рассматривает различные возможности.

Проекты и Android Studio

С Android Studio, для работы с проектом, Вы можете или создать новый проект с нуля, или скопировать существующий проект Android Studio в новый, или Вы можете импортировать существующий Андроид проект в Android Studio. Следующий раздел, рассматривает шаги необходимые для каждого из этого.

Создание New Project (нового проекта)

Вы можете создать проект из одного из двух мест:

- Если Вы на начальном диалоге, с которым Вы сталкиваетесь, когда открываете Android Studio, выберете пункт меню "Start a new Android Studio project...".
- Если Вы внутри Android Studio IDE Ю выберите File > New > New Project... из основного меню

Это покажет мастера нового проекта:

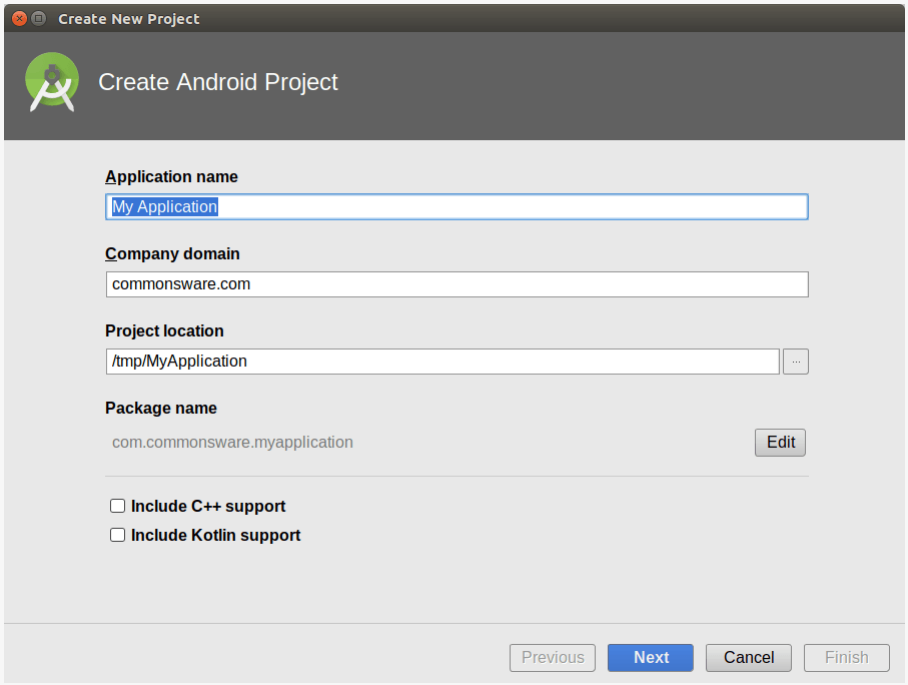


Рисунок 12: Мастер создангия проекта Android Studio, первая страница

Первая страница мастера это, где Вы можете задать:

- Название приложения, которое начальное название Вашего проекта, как виден пользователю, в местах как, значок запуска на домашнем экране и в списке установленных приложений.
- Название пакета, которое ссылается на название пакета Java (пример `com.commonsware.empublite`). Данное название пакета будет использоваться для генерирования некоторого исходного кода Java, и оно также используется как уникальный идентификатор данного приложения, как было указано ранее в данной книге.
- Директория, где размещаются файлы проекта.
- Знаете ли вы сейчас, будете ли вы использовать C ++ или нет, как часть Android Native Development Kit (NDK)
- Знаете ли Вы сейчас, будете Вы использовать Kotlin или нет

Ничто из того, что Вы выбираете здесь не постоянно, Вы можете пересмотреть всё позже по необходимости. Наиболее болезненно изменение названия пакета, так в идеале Вы выбираете хорошее значение наперёд.

По умолчанию, название пакета состоит из двух частей:

1. Доменного имени, которое Вы указываете в поле “Company Domain”
2. Название приложения, преобразованное в все строчные буквы без пробелов или другой пунктуации

Если это не то, что Вам необходимо, нажмите кнопку "Edit" в дальней правой части предлагаемого названия пакета, которое позволит Вам редактировать название пакета напрямую:

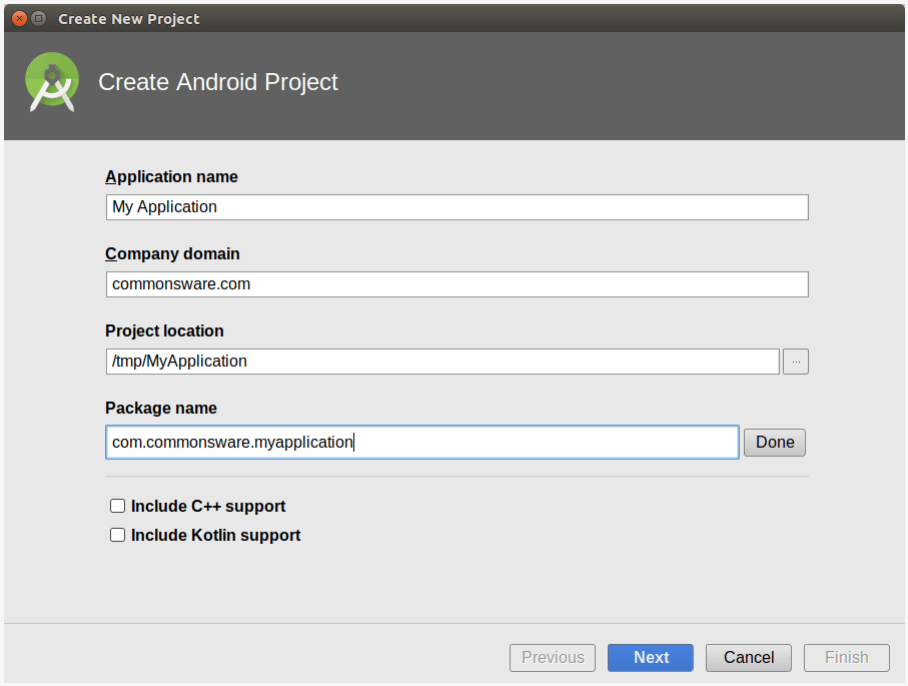


Рисунок 13: Мастер создания проекта Android Studio, первая страница, с редактируемым названием пакета

Нажатие "Next" продвинет Вас к странице мастера настройки, которое укажет какой тип проекта Вы создаёте, в терминах, на какое устройство направлено, (телефон/планшет, TB, и т.д.) и минимальный требуемый уровень SDK:

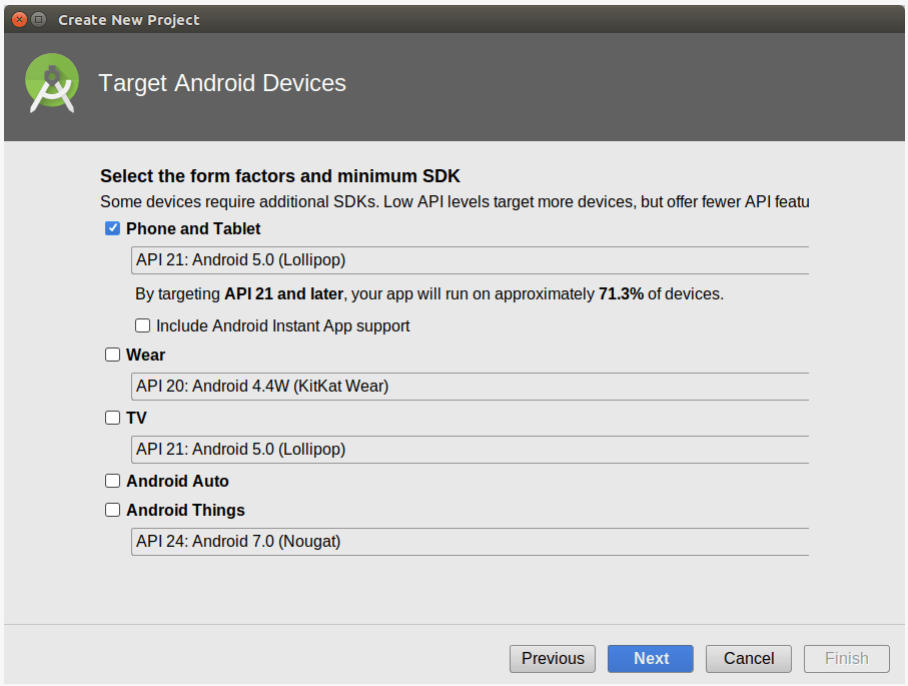


Рисунок 14: Вторая страница мастера создания проекта Android Studio

“Minimum SDK” относится к тому как далеко назад в историях Андроид версий Вы хотите сохранить поддержку. Чем меньше значение Вы укажите здесь, тем большее количество Андроид устройств смогут запускать Ваше приложение, но больше работы необходимо будет проделать для тестирования того, что ваше приложение поддерживает эти устройства.

Разработчики, только начинающие с Андроид должны только отметить "Phone and Tablet" (телефон и планшет) как тип устройства, оставляя остальные флажки невыбранными. По умолчанию значение “Minimum SDK” также обычно хороший выбор, и может быть изменён легко в Вашем проекте, как мы увидим позже в книге.

Нажатие "Next" продвигает Вас на третью страницу мастера, где Вы можете выбрать должна ли Android Studio создать начальную activity для Вас и если так, основываясь на каком шаблоне:

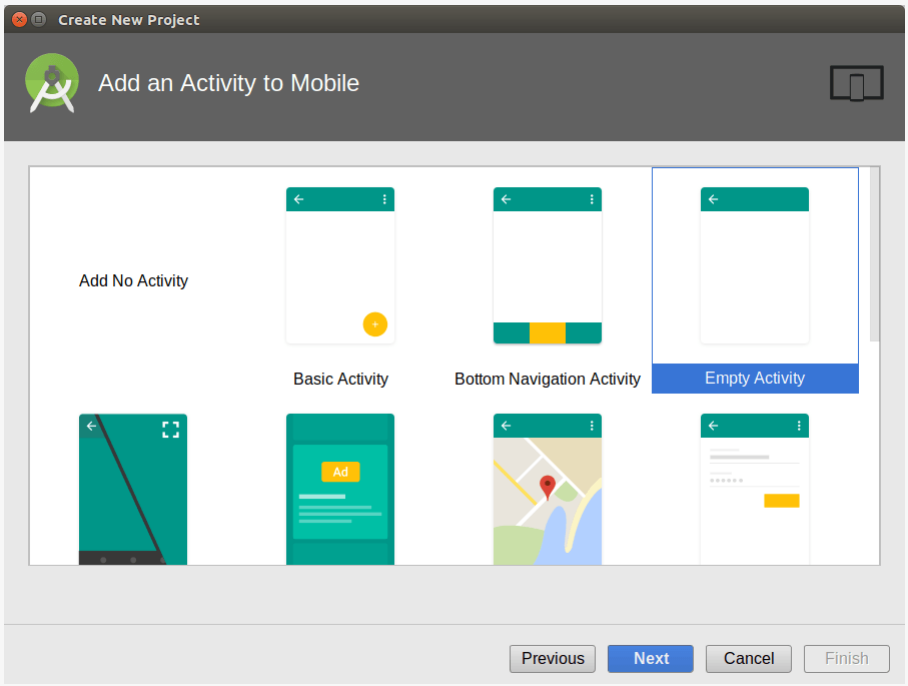


Рисунок 15: Третья страница мастера создания проекта Android Studio

Ни один из данных шаблонов особенно хорош, так как добавляют много материалов примера, которые вам очищать или заменять. “Empty Activity” (пустая активность) лучшая из представленных опций для разработчиков Андроид, выбирающих в первый раз, по простому потому, что добавляет минимальное количество "хлама".

Если Вы виберете любую опцию, кроме "Add No Activity", нажание "Next" продвинет Вас на страницу мастера, где Вы сможете предоставить дополнительные детали о создаваемой activity.

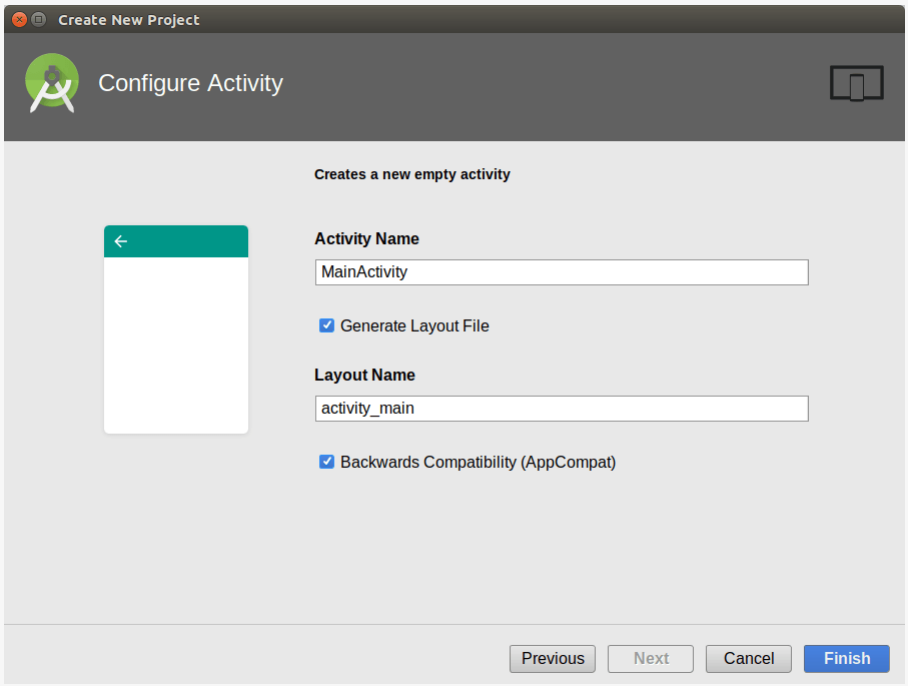


Рисунок 16: Мастер создания проекта Android Studio, четвёртая страница

Какие опции появляются здесь изменяются в зависимости от Вашего выбора на предыдущей странице. Общие опции включают “Activity Name” (название активности) (название Java класса для Вашей активности) и “Layout Name” (основное название файла XML, который будет содержать описание UI вышей активности).

"Обратная совместимость (AppCompat) " флажок указывает, что вы хотите использовать библиотеку известную как AppCompat. Мы обговариваем используемые библиотеки позже в книге также и что такое “AppCompat” Если вы не знаете наверняка, что хотите использовать AppCompat - и несколько примеров из данной книги - уберите данный флажок.

Нажатие “Finish” сгенерирует Ваши файлы проекта.

Копирование проекта.

Проекты Android Studio это простые директории с файлами, без специальных

метаданных где-либо. Следовательно, чтобы скопировать проект, просто скопируйте директорию.

Импортирование проекта

Вы можете импортировать проект из одного из двух мест:

- Если Вы на изначальном диалоге с которым Вы столкнулись при первом запуске Android Studio, выберете “элемент меню Import Project...”
- Если вы внутри Android Studio IDE, выберете File > New... > Import Project... из основного меню.

После, выберете директорию, содержащую импортируемый проект.

Что произойдёт сейчас зависит от природы Вашего проекта. Если проект уже был настроен для использования с Android Studio, или как минимум с Android Gradle Plugin, специальные файлы Android Studio будут созданы (или обновлены) в директории проекта.

Однако, если проект не настроен для Android Studio (или как минимум для Android Gradle Plugin), но содержит файлы проекта Eclipse (или как минимум файл `project.properties`), Вы будете направлены в мастер импорта Eclipse.Что будет непривычно в наши дни

Руководство №2 - Создание проекта с заглушками

Создание Андроид приложения сперва включает в себя Андроид "проект". Как и с многими другими окружениями разработчика, проект - это где Ваш исходный код и другие материалы (пример значки) находятся. И проект содержит инструкцию для Ваших инструментов для того как преобразовать исходный код и другие материалы в Андроид файл APK для применения с эмулятором, где APK - исполняемый формат файла.

Следовательно, в данном руководстве, мы отбрасываем разработку Андроид приложения из примера, чтобы дать Вам возможность заложить что-то из того, что вы изучаете практически.

Про наш проект руководства

Приложения, которые мы будем создавать в данном руководстве называется `EmPubLite`. `EmPubLite` будет электронной читалкой книг, позволяющей пользователям читать цифровые книги, как те, что Вы читаете сейчас.

`EmPubLite` будет практически внедрено в `EmPub` читалку, используемую в APK версии этой книги. `EmPub` сам по себе относительно обширное приложение, так `EmPubLite` будет содержать только часть этих функций.

“Em” от `EmPub` и `EmPubLite` остаётся “встроенным”. Данные читалки не спроектированные для чтения произвольных отформатированных EPUB или MOBI книг, которые Вы можете загрузить откуда либо. Скорее, содержимое книги (большой разархивированный файл EPUB) будет “запечён” в читалку APK сам по себе, так распространением APK, Вы распространяете книгу.

Про остальное в руководстве

Конечно, Вы можете проявлять небольшой интерес в написание приложения для чтения электронных книг.

Руководство представленное в данной книге безусловно опционально. Здесь нет ожидания, что Вы напишите любой код с целью получения значений из книги. Данное руководство здесь просто как способ помочь тем из Вас, кто "обучается через действие", давая возможность выполнять это.

Следовательно, много способов, которые Вы можете использовать в данном руководстве:

- Вы можете игнорировать его полностью. Это не лучший ответ, но Вы можете использовать его.
- Вы можете читать руководство, но в действительности не выполнять никакой работы. Это лучший не требующий усилий ответ, это как то, что вы изучаете темы из руководства, которые Вы могли пропустить простым чтением глав без руководств.
- Вы можете следовать вдоль шагов и в действительности создать приложение `EmPubLite`.
- Вы можете загрузить ответы из GitHub репозитория. Там, Вы найдёте одну директорию на руководство, отображающие результаты каждого из шага данного руководства. Для примера, вы найдёте директорию `T2-Project/` содержащую копию простого приложения `EmPubLite` после завершения шагов, указанных в данном руководстве. Вы можете импортировать данные проекты в Ваше IDE, проверить, что они содержат перекрёстные ссылки обратно в руководстве, и запустить их.

Любые из данных опций возможны - Вам только необходимо выбрать для Вас, что Вы хотите сделать.

О наших инструментах

Инструкция в оставшемся руководстве совпадает с Android Studio. 3.0.x. Руководства *должны* работать и для других версий IDE, но могут быть некоторые отличия.

Шаг №1: Импорт проекта

Мы должны создать проект Андроид для `EmPubLite`.

Обычно, Вы должны использовать мастер нового проекта для создания нового проекта. Однако, проблема с мастером нового проекта то, что Google продолжает изменять то, что генерирует мастер нового проекта. В большинстве ситуаций, это не большая проблема.
Однако, это становиться проблемой для руководства, такое как это, как если Google изменяет, что в новом проекте, инструкции руководства становятся устаревшими.

Так, вместо этого, мы импортируем существующей проект, так мы сможем начать с стабильной базы.

Посетите страницу релизов репозитория GitHub данной книги. После, пролистайте вниз до данной версии книги и загрузите `EmPubLite-Starter.zip` файл для этого. Разархивируйте данный проект в некоторое место на машине разработчика. Что разархивирует в директорию `EmPubLite/`

После импортируйте проект. Из начального диалога Android Studio, выбирая опцию “Import project (Eclipse ADT, Gradle, etc.)”. Из существующего окна Android Studio IDE, Вы можете использовать File > New > Import Project... из основного меню.

Импортирование проекта приведёт к стандартному диалогу выбора директории. Выберите `EmPubLite/` директорию и нажмите OK для начала процесса импорта. Это может занять длительное время, зависящее от скорости Вашей машины для разработки. Может появиться “Tip of the Day” диалог, который Вы можете отменить.

На данном этапе, у Вас есть пустое окно Android Studio IDE, возможно с сообщением об ошибке, такой как:

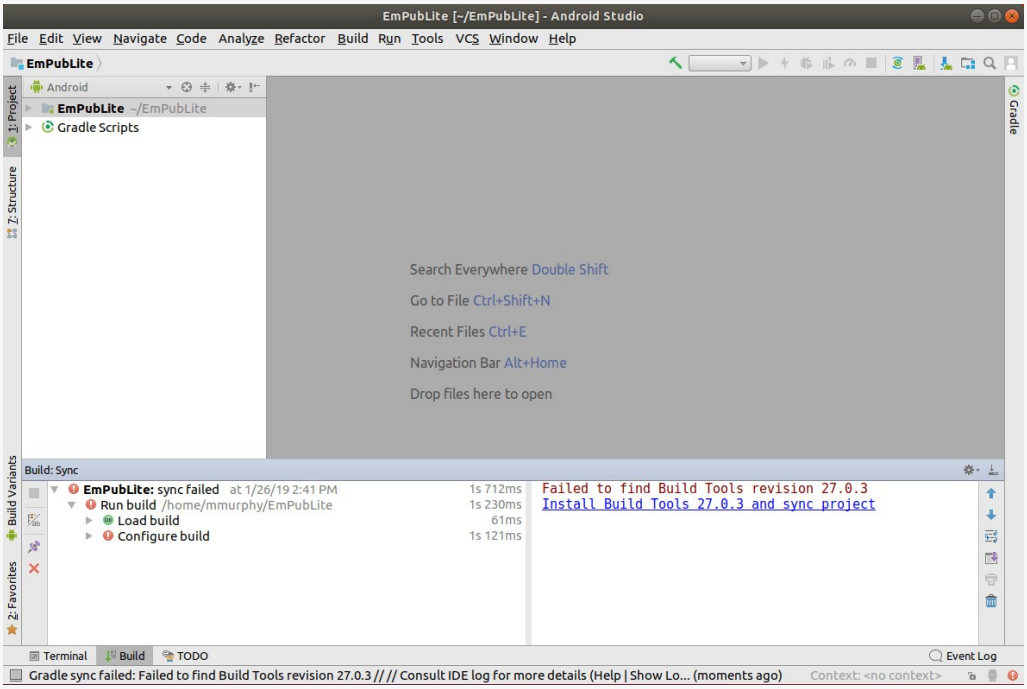


Рисунок 17: Android Studio, при начальном запуске, отображаемая ошибка

или:

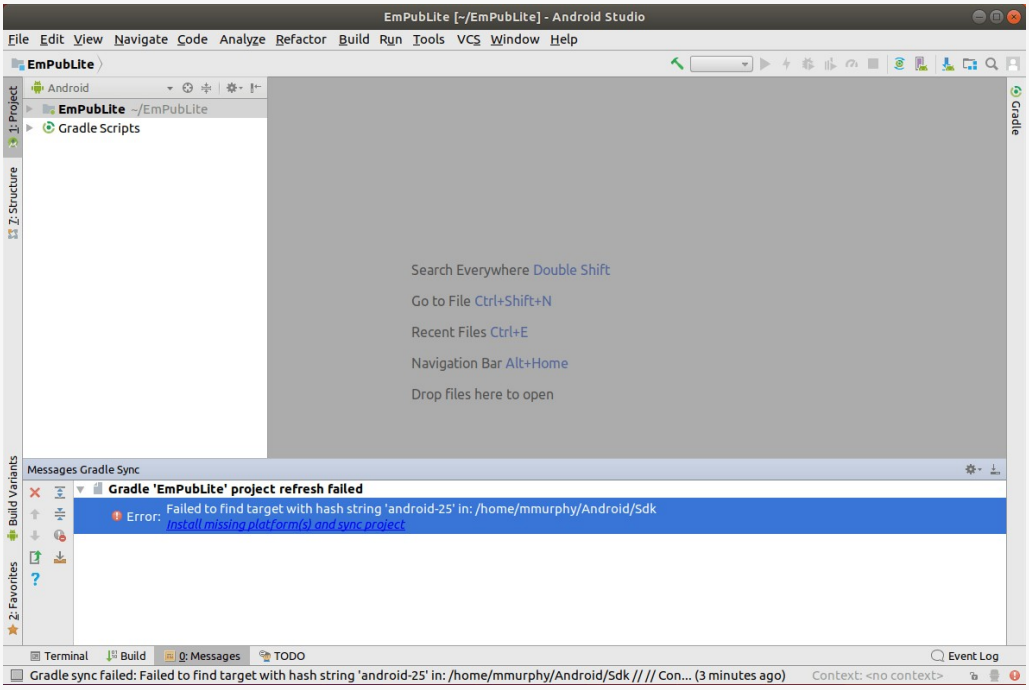


Рисунок 18: Android Studio, при начальном запуске, отображает другую

В панели инструментов “Messages Gradle Sync”, внизу экрана вы можете получить “Failed to find Build Tools revision 27.0.3”. Каждый Андроид проект, указывает, что используемая версия инструмента разработки, и данная немного старше, чем текущая. В том инструменте разработки, “Install Build Tools 27.0.3 and sync project” сообщение - гиперссылка - нажмите её, чтобы загрузить отсутствующие материалы. Это общий шаблон с данным сообщением об ошибке, где сообщение содержит ссылку для попытке исправить проблему.

Или, если вы “Failed to find target with hash string `android-25`...”, (ошибка поиска источника с хэш текстом Андроид 25) должна быть ссылка на проект “Install missing platform(s) and sync project” (Установка отсутствующих платформ и синхронизации проекта). Нажмите это, для попытки исправить проблему.

Далее, после нескольких мгновений, вы получите всплывающий диалог похожий на следующий:

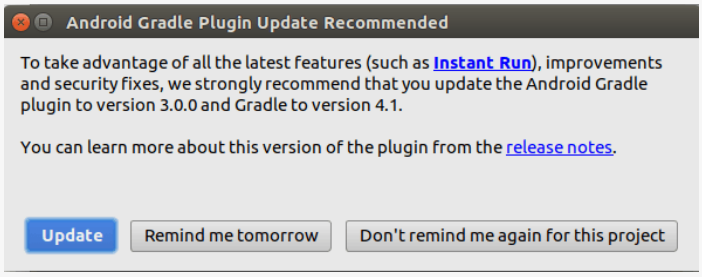


Рисунок 19: Android Studio, рекомендационный диалог

Триггер для данного диалога похож на триггер для предшествующие сообщению об ошибке: проект, который Вы импортируете использует некоторые старые инструменты, отличающиеся от последних. Для проекта, который Вы создаёте с нуля, Вы не получите это сообщение, так как мастер нового проекта всегда использует последние инструменты... даже если это не лучшая идея. Для проектов как этот, который Вы откуда-то импортируете, данный диалог обычное дело. Для целей данного руководства, нажмите кнопку "Don't remind me again for this project" (не напоминать мне снова для данного проекта). Для других проектов, которые Вы импортируете, Вы можете опционально нажать кнопку "Update" (обновить), что изменит проект на использование более новых инструментов, что может привести к проблемам с совместимостью, которые Вам необходимо будет отлаживать.

Инструмент "Project" - прикрепленный по умолчанию на левой стороне вверху - поможет Вам увидеть, что находиться в проекте. Android Studio содержит несколько способов просмотра содержимого Андроид проектов. По умолчанию, при создании или импортирования проекта Вам предлагается просмотр “Android view” (Андроид вид):

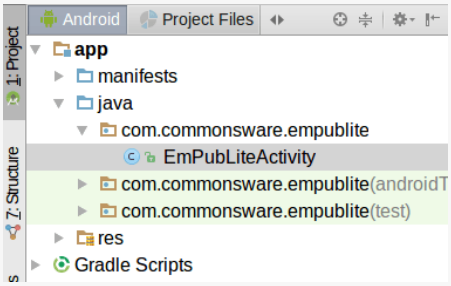


Рисунок 20: Android Studio “Android View”

Вам предлагается перемещаться по проекту с применением этого вида, руководства в главах данной книги, если они содержат скриншоты Android Studio, отображают вид проекта:

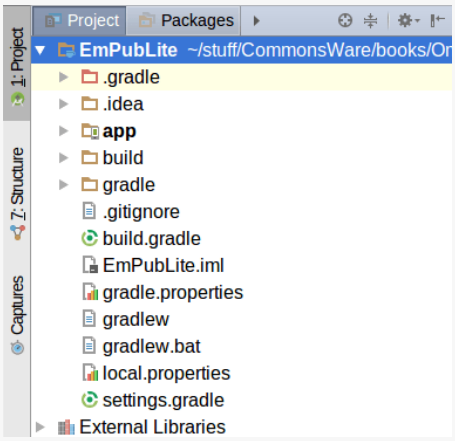


Рисунок 21: Android Studio "Project View"

Для переключения на этот вид - и следовательно совпадать с руководством показываемом Вам - нажмите элемент "Android" над деревом, для отображения выпадающего меню с различными видами. Выберите "Project", и переключит для Вас вид на вид проекта, применяемый в данном руководстве.

Шаг №2: Подготовка к эмулятору x86

Первое решение, которое Вам необходимо сделать - будете или нет настраивать образы эмулятора прямо сейчас. Если у Вас есть устройство Андроид, Вы можете предпочитать начать тестирование Вашего приложение на нём, и вернутся к настройке эмулятора позже. В данном случаи, перейдите к Шагу №4.

В другом случаи, здесь указано, что Вам необходимо, в зависимости от операционной системы на Вашей машине разработки.

Windows

Если Ваш ЦПУ соответствует требованиям, и Вы успешно активировали правильные настройки вашем системном BIOS, установка the Android Studio должна установить HAXM, и Вы будете готовы к работе.

Если, с другой стороны, Вы получаете некоторые сообщения об ошибке в

мастере установки, относящиеся к HAXM, Вам необходимо сперва обратится к ним.

Mac

Волшебники из Купертино, настроили их аппаратное обеспечение Мак способными запускать эмулятор Андроид x86, что ужасно мило с их стороны, понимая, что Андроид конкурирует с iOS. Мастер установки Android Studio должен успешно установить HAXM, и Вы будете способны продолжить со следующего шага в руководстве.

Linux

Эмулятор Андроид x86 на Linux не использует HAXM. Вместо этого, используется KVM, распространённое средство виртуализации.

Если в процессе установки Android Studio, мастер отображает Вам страницу, в которой говориться, что Вам необходимо настроить KVM, Вам необходимо это сделать до начала настройки и использования эмулятора x86. Детальная информация, как настроить KVM отличается в различных дистрибутивах Linux (пример Ubuntu).

Шаг №3: Настройка AVD

Андроид эмулятор, может эмулировать одно или несколько устройств Андроид. Каждая необходимая конфигурация сохраняется в “Android virtual device” (виртуальные настройки Андроид), или AVD. Менеджер AVD - это где Вы создаёте эти AVD.

Отметьте, что Android Studio сейчас содержит свою собственную реализацию Менеджера AVD отличную от той, что традиционно использовались разработчиками. Вы можете увидеть скриншоты старой версии Менеджера AVD в сообщениях блогов, ответах Stack Overflow, и других. Менеджер AVD, всё ещё выполняет ту же функцию, но содержит отличающийся интерфейс.

Чтобы открыть Менеджер AVD в Android Studio выберите, Tools > AVD Manager из основного меню.

Вы будете направлены на приветствующий экран:

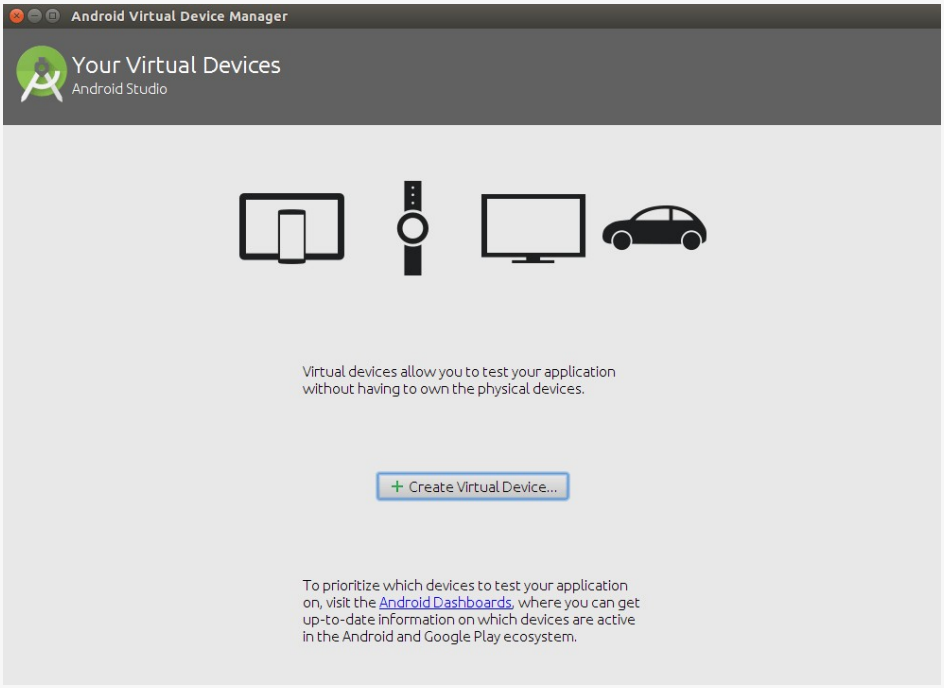


Рисунок 22: Менеджер AVD, Android Studio, Приветствующий экран

Нажмите кнопку "Create Virtual Device" (создать виртуальное устройство), которое направит Вас к мастеру “Virtual Device Configuration” (конфигурации виртуальных устройств):

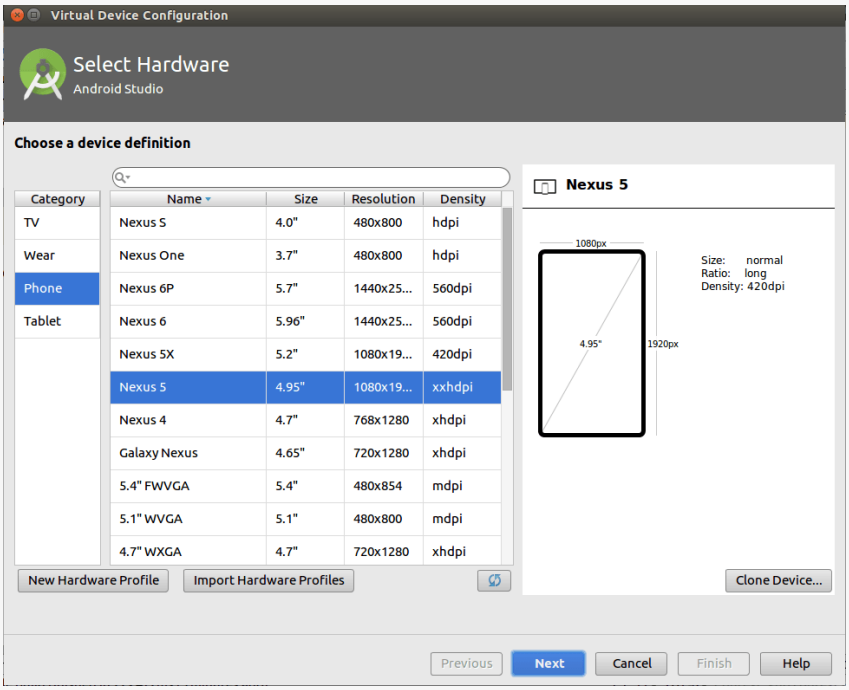


Рисунок 23: Мастер настроек виртуальных устройств Android Studio,

Первая страница мастера позволяет Вам выбрать профиль устройства, применяемое на начальном этапе вашего AVD. Кнопка “New Hardware Profile” (новый аппаратный профиль) позволяет Вам описать новый профиль, если не среди представленных нет соответствующего Вашим потребностям.

Так так, скорость эмулятора к какой-то степени привязана к разрешению их (виртуальных) экранов, вы в общем нацелены на профиль устройства нижнего сектора, но которое не полностью нелепо. Для примера, телефон 800x480 или 1280x768 рассматривается многими как относительно низкого разрешения. Тем не менее, много устройств выпускаются с таким разрешением (или ниже), что делает его обоснованным для запуска эмулятора.

Если Вы хотите создать профиль нового устройства основанное на существующем - изменить несколько параметров, но остальные использовать как в оригинальном профиле - нажмите кнопку “Clone Device” (клонировать устройство), после того как выбрали Ваш начальный профиль.

Однако, в общем, в самом начале, использование существующего профиля - прекрасный выбор. Образы Galaxy Nexus или Nexus 4 хороший выпор, чтобы начать.

Нажатие “Next” (далее) позволит выбрать образ эмулятора для использования:

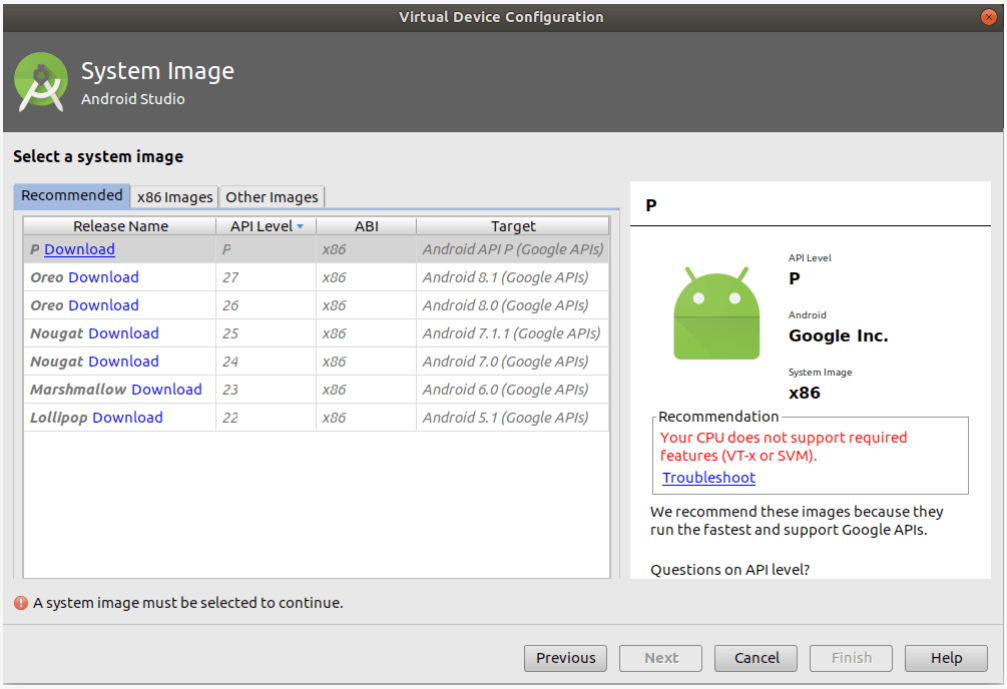


Рисунок 24: Мастер конфигураций виртуальных устройств Android Studio,

Образы эмулятора распределены по трём вкладкам:

- “Recommended” (рекомендуемые)
- “x86 Images” (образы x86)
- “Other Images” (другие образы)

Для целей данного руководства, Вам не нужны образы эмулятора с “Google APIs” - они для эмуляторов, которые содержат Google Play Services и относящиеся к ним приложения, например Google Maps. Тем не менее, в терминах уровня API, Вы можете выбрать всё что угодно начиная с 15 уровня API (Андроид 4.0.3) и выше. У Вас должно быть одно или более подходящих образов, уже настроенных для Вас, любезно установленных Android Studio.

Если Вы нажмёте на вкладку x86 Images, Вы сможете увидеть образы со ссылками на скачивание (“Download”), у другие без них.

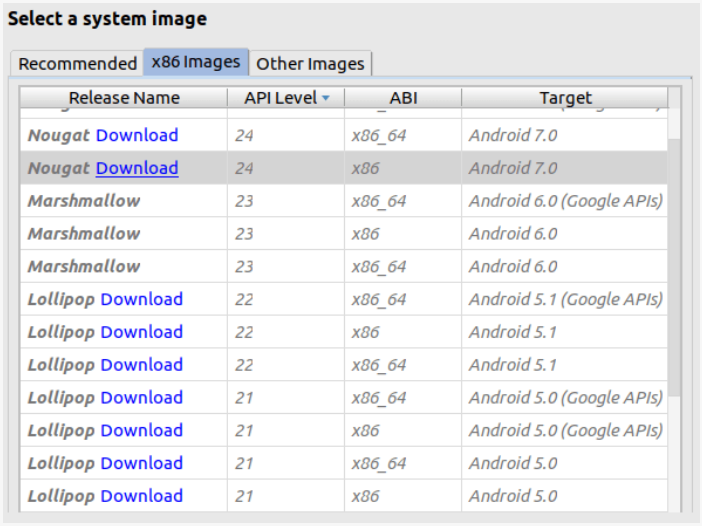


Рисунок 25: Мастер настройки конфигураций виртуальных устройтсв

Образы эмулятора с "Download" возле них, запускают загрузку в один клик необходимых для создания AVD файлов, для отдельной комбинации архитектур уровня API и ЦПУ, после дополнительных диалогов лицензии и прогресса.

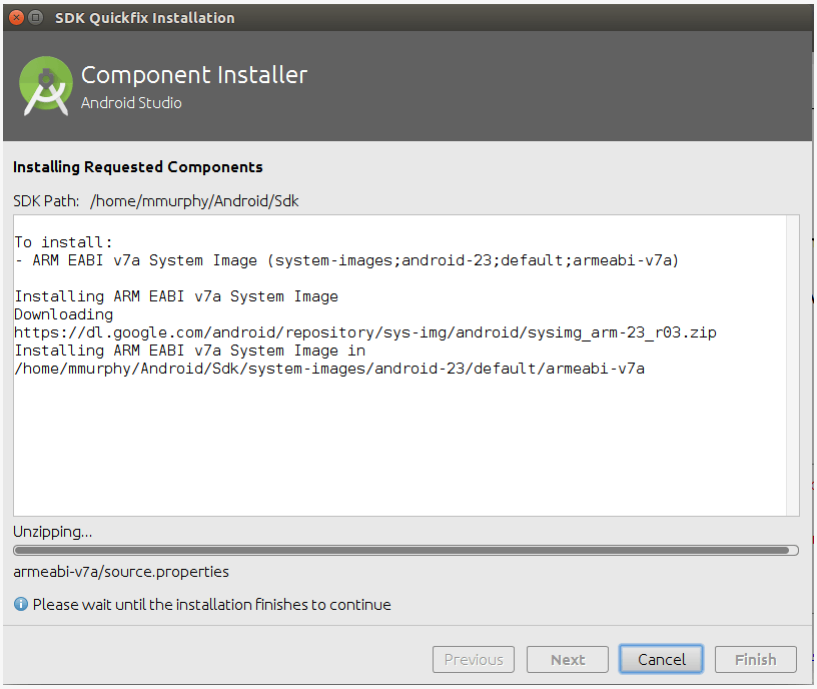


Рисунок 26: Диалог установки компонентов Android Studio, Загрузка API 23 ARM образа.

Если Вы определили образ(ы), которые вам необходимы - и загрузили те, которых у Вас ещё нет - нажмите на один из них в мастере:

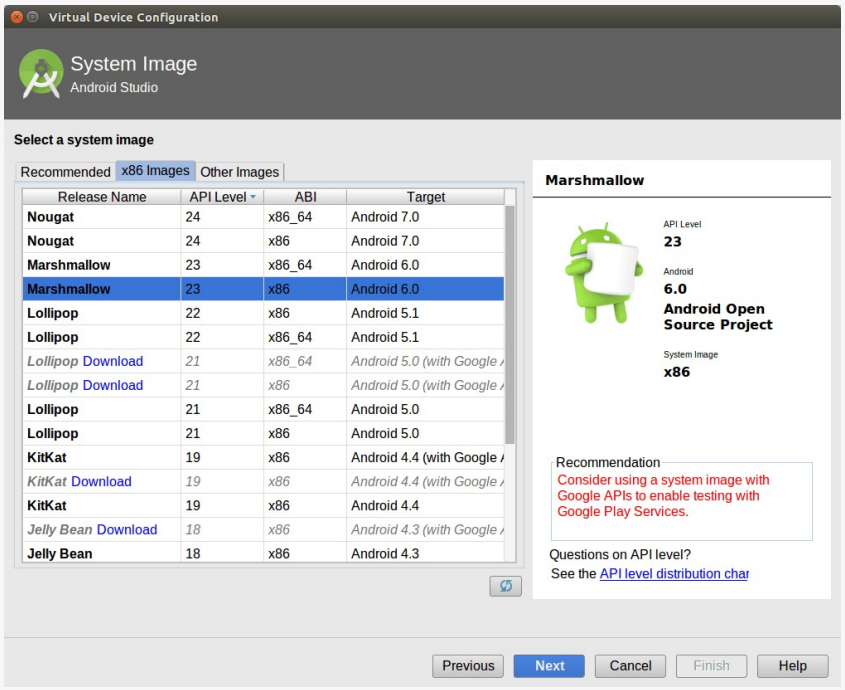


Рисунок 27: Мастер конфигураций виртуальных устройств Android Studio, после выбора образа.

Нажатие "Next" (далее) позволяет Вам завершить настройку Вашего AVD:

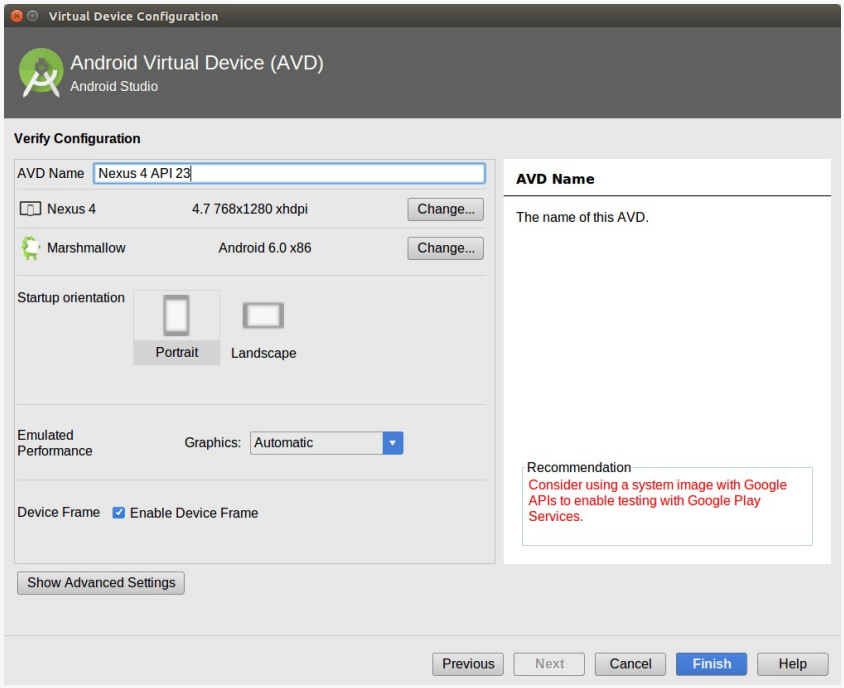


Рисунок 28: Мастер настройки виртуальных устройств Android Studio,

Предлагается название по умолчанию для AVD, так Вы можете изменить его своим собственным значением.

Измените название AVD, если необходимо, на что-то валидное: поддерживается только буквы, цифры, пробелы, и отмеченная пунктуация (., _, -, (,)).

Остальные значения по умолчанию пока подойдут.

Нажатие "Finish" возвратит Вас к основному окну AVD Менеджера, отображающего Ваше новое AVD. После Вы можете закрыть окно Менеджера AVD.

Шаг №4: Настройка устройств

Вам нет необходимости в Андроид устройстве для начала разработки Андроид приложений. Наличие одного - хорошая идея - прежде чем вы начнёте распространять приложение (пример, загружать в Play Store). И возможно, у Вас уже есть устройство - возможно это то, что подстёгивает Ваш

интерес к разработке под андроид.

Если у Вас нет Андроид устройства, которое Вы хотите настроить для разработки, пропустите этот шаг.

Первый шаг - сделать Ваше устройство готовым для использования при разработке - перейти в приложение настройки на вашем устройстве. Что будет происходить немного зависит от версии Андроид:

- На Андроид 1.x/2.x, перейдите в Приложения (Applications), дальше в Разработку(Development)
- На Андроид от 3.0 до 4.1, перейдите в “Developer options” (опции разработчика) из основного экрана Настроек.
- На Андроид 4.2 и выше, перейдите в About (Об устройстве), нажмите на номер сборки семь раз, и нажмите BACK (назад), и перейдите в “Developer options” (опции разработчика) (которые обычно скрыты).

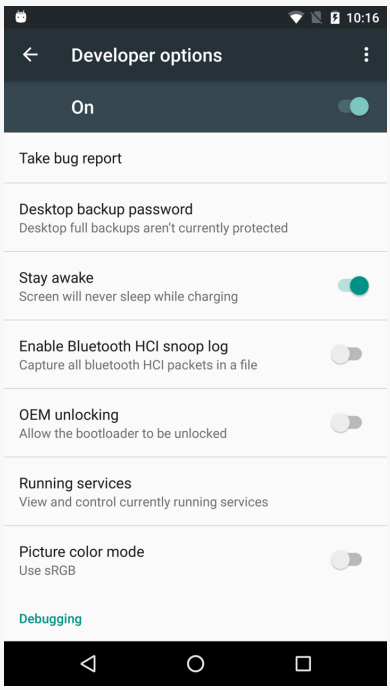


Рисунок 29: Опции разработчика, в приложении Настройки

Вам нужно переместить переключатель в верхнем правом углу экрана в положение "ON" (включено) для изменения значений на этом экране.

В общем, Вам необходимо пролистать вниз и включить отладку по USB, так Вы сможете использовать Ваше устройство с инструментами сборки Андроид:

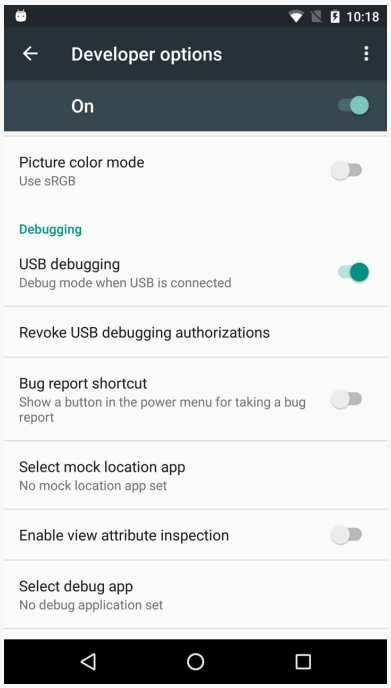


Рисунок 30: Опции отладки, в приложении настройки

Вы можете оставить другие настройки без изменений сейчас по желанию, хотя Вы можете полезной опцию “Stay awake” (оставаться включённым), которая сохранит Вас от необходимости разблокировать телефон каждый раз, когда Вы подключаете USB.

Отметьте, что в устройствах Андроид 4.2.2 и выше, прежде чем Вы сможете в действительности использовать настройки, которые вы только что включили, Вам предложат разрешить отладку по USB с вашей *определённой* машиной для разработки через диалог:

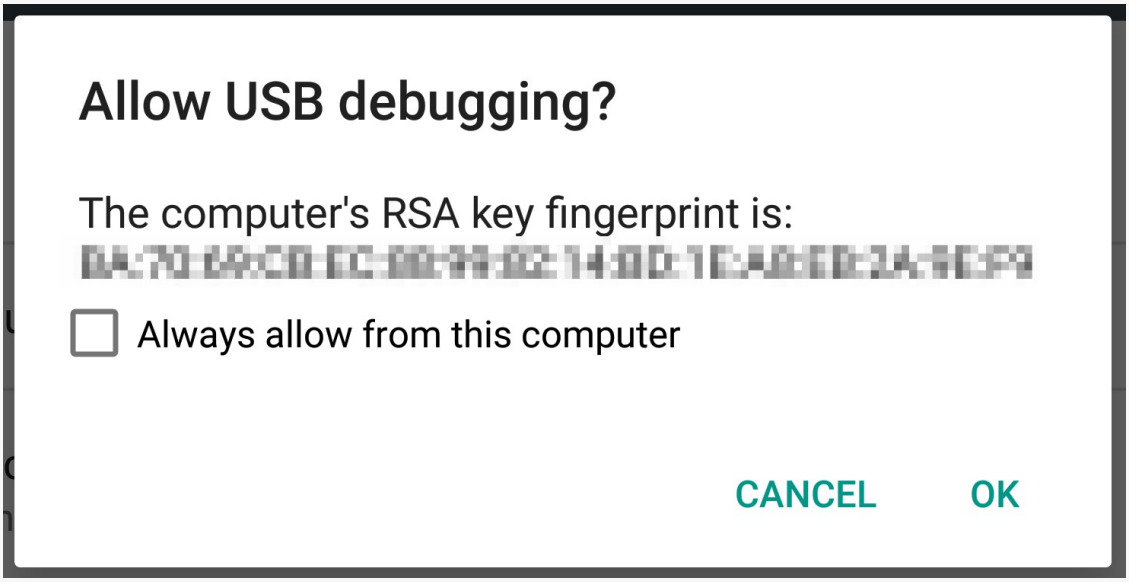


Рисунок 31: Диалог разрешения отладки по USB

Данное происходит, когда Вы подключаете устройство через USB кабель и драйвер настроен соответствующе. Данный процесс варьируется от операционной системы Вашей машины разработки, как показано в следующем разделе.

Windows

Когда Вы впервые подключаете Ваше устройство Андроид, Windows выполнит попытку найти драйвер для него. Возможно, что благодаря другому установленному вами программному обеспечению драйвер готов к использованию. Если драйвер найден, Вы, вероятно, готовы к работе.

Если драйвер не найден, здесь представлены несколько опций его получений.

Windows Update

Некоторые версии Windows (например Vista) предложи Вам поиск драйвера в Windows Update. Это определённо стоит попробовать, хотя не каждое устройство поддерживает этот драйвер Microsoft.

Стандартный драйвер Андроид

В Вашей установке Андроид SDK, если Вы выбрали установку пакета “Google USB Driver” (драйвер USB от Google) из Менеджера SDK Вы найдете директорию `extras/google/usb_driver/` , содержащую универсальный драйвер Windows для Андроид устройств. Вы можете указать мастеру драйверов на эту директорию,что проверить подходящий ли это драйвер для Вашего устройтсва. Это часто работает для устройств Nexus.

Драйвер поддерживаемый производителем.

Если у Вас до сих пор не установлен драйвер, OEM USB Drivers в документации может помочь Вам найти подходящий для загрузки от производителя Вашего устройства. Отметьте, что Вам нужна номер модели Вашего устройства, взамен от названия модели используемой в маркетинге (например GT-P3113 взамен “Samsung Galaxy Tab 2 7.0”).

macOS и Linux

Высоки шансы, что простое подключение Вашего устройства будет работать. Вы можете увидеть, что Андроид устройство распознаётся через запуск **`adb devices`** в командной строке (например терминал macOS), `adb` - в директории `platform-tools/` Вашего SDK. Если Вы получите вывод похожий на следующий, инструмент сборки определил Ваше устройство:

```
List of devices attached
HT9CPP809576  device
```

Если Вы работаете в Ubuntu (или возможно другие варианты Linux), и данная команда не работает, Вам необходимо добавить некоторые правила `udev`. Для примера здесь представлен
5- файл `android.rules` который будет работать с устройствами нескольких производителе

```
SUBSYSTEM=="usb", SYSFS{idVendor}=="0bb4", MODE="0666"
SUBSYSTEM=="usb", SYSFS{idVendor}=="22b8", MODE="0666"
SUBSYSTEM=="usb", SYSFS{idVendor}=="18d1", MODE="0666"
SUBSYSTEMS=="usb", ATTRS{idVendor}=="18d1", ATTRS{idProduct}=="0c01", MODE="0666",
OWNER="<me>"
SUBSYSTEM=="usb", SYSFS{idVendor}=="19d2", SYSFS{idProduct}=="1354", MODE="0666"
SUBSYSTEM=="usb", SYSFS{idVendor}=="04e8", SYSFS{idProduct}=="681c", MODE="0666"
```

Поместите их в директорию `/etc/udev/rules.d` на Ubuntu, после или перегрузите компьютер или перезагрузите правила `udev` (например., **`sudo`**

`service udev reload`).
После отключите устройство и переподключите устройство, чтобы посмотреть будет ли он
определено.

Шаг № 5: Запуск проекта

Теперь, мы можем подтвердить, что наш проект настроен должным образом запустив его на устройстве или эмуляторе.

Чтобы выполнить это в Android Studio, просто нажмите кнопку в панели инструментов Run (запустить) (обычно изображающуюся зеленым направленным вправо треугольником).

Вам после представится диалог указывающий где Вы хотите запустить приложение: на некотором существующем устройстве или эмуляторе, или некотором недавно запущенном эмуляторе:

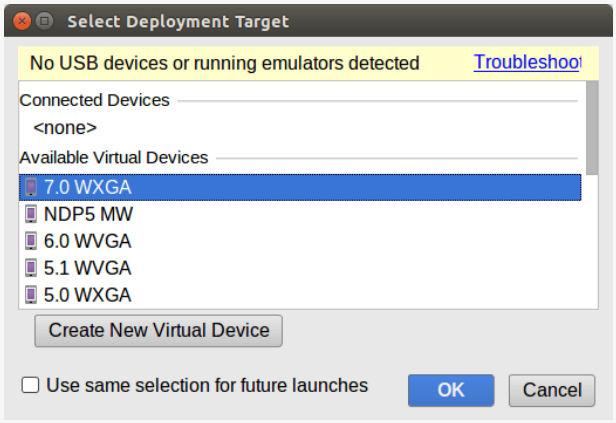


Рисунок 32:Диалог выбора устройства Android Studio

Если у Вас нет запущенного эмулятора выберите один из списка, далее нажмите OK. Android Studio запустит эмулятор для Вас.

И, когда Вы запускаете новый экземпляр эмулятора или используете уже существующий, Ваше приложение должно появится на нём:

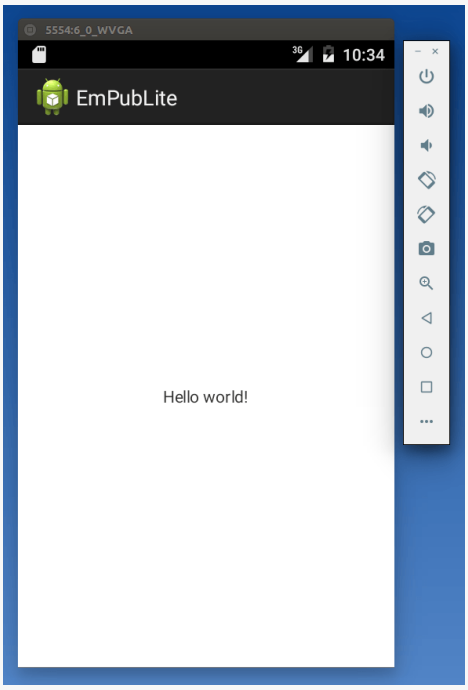


рисунок 33: Эмулятор Android 7.0 с EmPubLite

Заметьте, что Вы должны разблокировать Ваше устройство или эмулятор, чтобы действительно увидеть работающее приложение.

Когда Вы в первый раз запускаете эмулятор для отдельного AVD, Вы увидите следующее сообщение:

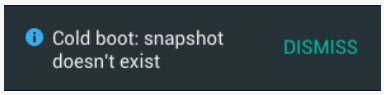


Рисунок 34: предупреждение холодной загрузки эмулятора Андроид

Эмулятор сейчас работает более похоже на устройство Андроид. Закрытие окна эмулятора раньше был похож на полное отключение телефона, но сейчас больше похоже на нажатие кнопки POWER для отключения экрана. В следующий раз, когда Вы запускаете определённое AVD, оно пробудиться в то состояние, в котором Вы его оставили, а не загрузиться с нуля ("холодная загрузка"). Это ускоряет запуск эмулятора. Время от времени, всё таки, у Вас будет необходимость запускать эмулятор как при включении питания

устройства.

on. Чтобы сделать это в выпадающем меню колонки Actions Менеджера AVD, выберите “Cold Boot Now”.

В нашем следующем эпизоде...

... мы модифицируем `AndroidManifest.xml` файл нашего проекта.

Знакомство с Android Studio

Данная глава служит быстрым туром по Android Studio IDE, чтобы помочь Вам начать. Другие специфичные для Андроид функции Android Studio будут рассмотрены в следующей главе.

Навигация в Project Explorer

Погле главной области редактирования - где Вы изменяете Ваш исходный код Java, Ваши ресурсы, и т.д. - часть Android Studio в которой Вы проведёте большую часть времени - просмотрщик проекта, обычно доступный в левой части окна IDE:

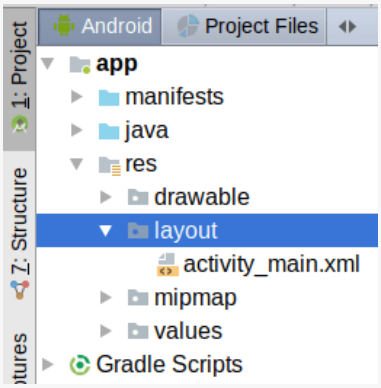


Рисунок 35: Просмотрщик проекта Android Studio, Показывающий в

Данная панель просмотрщика содержит две основных "вида проекта", которые Андроид разработчик использует: отображение Андроид проекта и классическое отображение проекта.

Отображение "Андроид проект"

По умолчанию, когда Вы создаёте проект или импортируете проект, Вы переходите в просмотр "Андроид проект".

В теории, отображение Андроид проекта разработано для упрощения работы с файлами Андроид проекта. На практике, это может быть так, но только для некоторых продвинутых разработчиков. В общем, это делает IDE значительно более сложной для новичков в Андроид, так как это значительно сложнее в том, где находятся данные и что относиться с чем.

Мы вернёмся к виду "Андроид проект" немного позже в книге и объясним его преимущества в отношении ресурсов и набора ресурсов Gradle.

Однако, для большей части книги - более важно, для руководства - мы будим использовать стандартный вид проекта.

Отображение "Классический проект"

Для переключения к классическому виду проекта, на пару наконечников стрелок справа от вкладки "Project Files" (Файлы проекта) над деревом просмотрщика,и выберите Project (Проект):

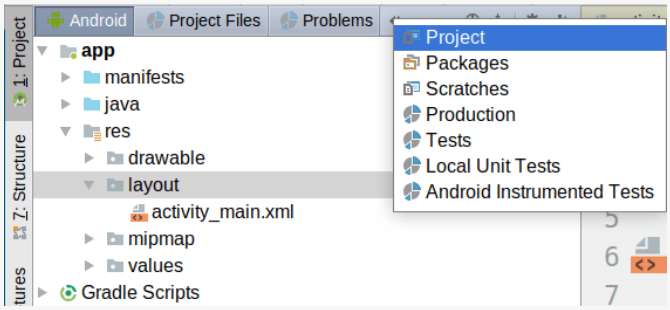


Рисунок 36: Просмотрщик проекта Android Studio, Отображающий

На некоторых машинах, в некоторых случаях, вместо пара стрелок отображается единичная раскрывающая стрелка далее от текущей выбранного вида (например Андроид).

Переключение на вид проект изменит содержание дерева для отображения всех файлов, в их соответствующих директориях:

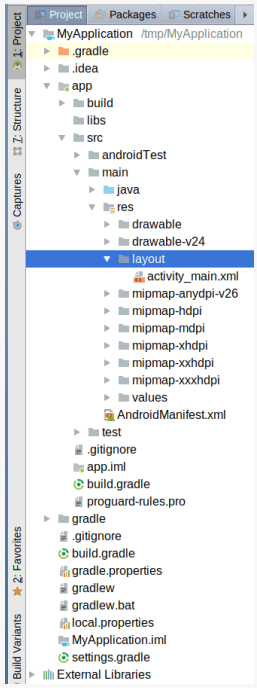


Рисунок 37: "Проводник" проекта Android Studio, отображенгие

Данное отображение проекта более похоже на эквивалентный в других IDE, позволяя Вам найти все части вашего Андроид проекта. Мы рассмотрим, что это за части, и как их файлы организованы в нашем проекте в следующей глава.

Контекстное меню в "проводнике"

Нажатие правой кнопки мыши на директории или файле предоставит Вам контекстное меню с набором опций. Некоторые их них типичны для любого рода файловых менеджеров, ткие как опции “вырезать”, “скопировать”, “вставить”. Некоторые из них организованы в соответствии с тем, как Android Studio управляет разработкой приложений, такие как подменю “Refactor” (рефактор), где вы можете переименовывать или перемещать файлы. Ещё другие будут специфичны для Android Studio, такие как возможность вызывать мастера для создания определённых типов компонент Андроид или других классов Java.

Открытие файлов из "проводника"

Двойное нажатие на файл обычно открывает данный файл во вкладке, которая позволяет В
редактировать этот

файл, применяя что-то подобное редактору.

Некоторые типы файлов, такие как изображения, могут быть открыты но не отредактированы, так как в Android Studio не содержит редакторов для всех типов файлов.

Запуск проектов

Конечно, если Вы изменили ваше приложение, Вы захотите испробовать его и увидеть, что оно работает, будь то устройство или эмулятор.

Основы

Как указано в Руководстве №2 для запуска Вашего проекта, просто нажмите кнопку Run (запустить) на панели инструментов:



Рисунок 38: управление запуском Android Studio, Отображение зелёной

Далее Вам представится диалог указывающий где Вы хотите, чтобы запустилось Ваше приложение.

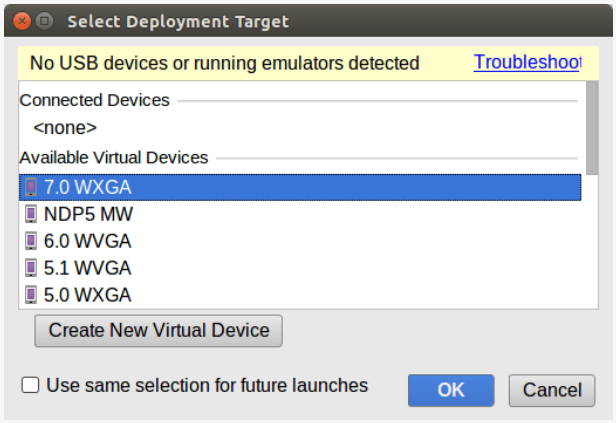


Рисунок 39:Диалог выбора устройства Android Studio

Категория “Connected Devices” (подключённые устройства) отображает список всех устройств или запущенных эмуляторов, которые инструмент сборки смог найти. Некоторые могут быть отключены по причинам совместимости, таких как эмулятор старой версии Андроид, тогда как Ваше

приложение требует новую версию Андроид.

Категория “Available Emulators” (доступные эмуляторы) содержит список всех AVD, которы
вы установили в AVD

менеджере, но ещё не запущены. Опять, Вы можете найти, что некоторые отключены по причинам совместимости.

Кнопка “Create New Emulator” (создать новый эмулятор) запустит мастер для создания нового AVD, как тот, что Вы можете запустить из менеджера AVD.

“Instant Run” (мгновенный запуск)

Следующая после кнопки стрелки “run” на панели инструментов находиться кнопка с молнией. Иногда она может быть серой и неактивной. В остальное время она показывается жёлтым:



Рисунок 40: Управление запуском Android Studio, С активированным

Данная карта представляет, то, что называется "Instant Run" (мгновенный запуск) Вместо создания вашего приложения и отправкой приложения на устройство или эмулятор, Instant Run совершает попытку пропатчить Ваше существующее приложение, основанное на том, какие изменения Вы сделали в проекте с последнего запуска.

С другой стороны, Instant Run очень быстрый. Кроме того, пропатченное приложение не тоже самое, как то что собранное с нуля. Не стесняйтесь использовать его для небольших изменений по умолчанию.

Просмотр вывода

Кроме непосредственно Вашего приложения, Android Studio генерирует другие типы диагностического вывода, в форме консольного вывода расшифровок вещей, которые происходят. Две из форм, которые, возможно значат наиболее для Вас это Build view (просмотр сборки) и Logcat (логгирование).

Build View

По умолчанию, прикреплённая в нижней границе Вашего окна Android Studio - элемент “Build” (сборки). Нажатие на неё откроет панель, отображающую вывод попытки сборки Вашего приложения.

По умолчанию вид - это дерево шагов, которые выполняют Android Studio и Gradle для сборки Вашего приложения:

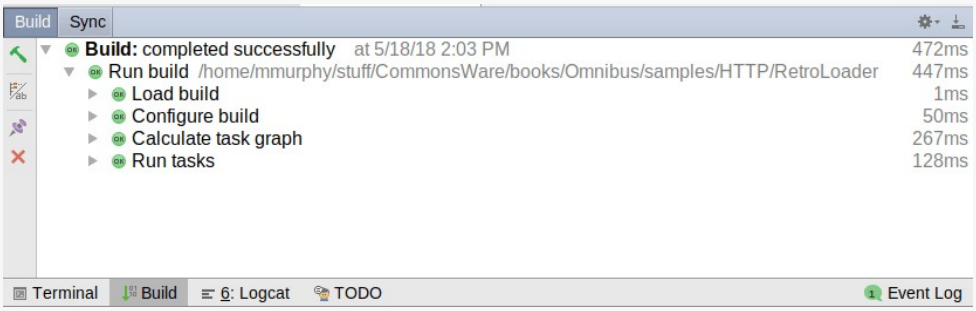


Рисунок 41: Вид сборки Android Studio, формат дерева

Второй элемент выпадающего меню, в строке инструмента переключиь Вас из вида дерева в "сырой" текстовый вывод из сборки Gradle:

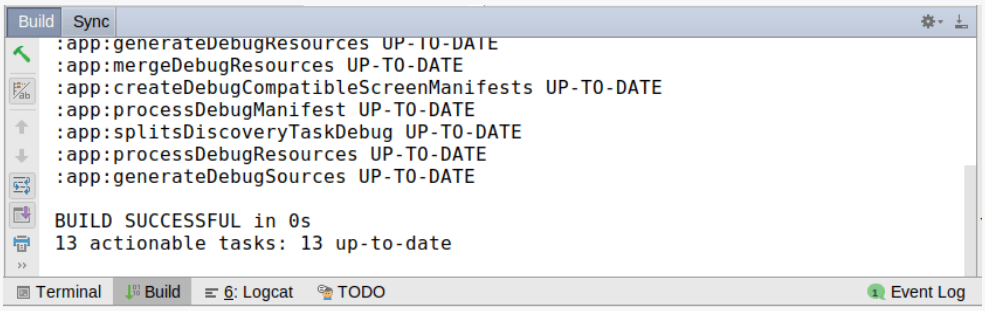


Рисунок 42: Вид сборки Android Studio, консольный формат

Если при запуске возникли проблемы, формат консольного вывода обычно более информативен.

Logcat

Сообщения, которые появляются при запуске - включают все возможные трассирования стека Java вызванные ошибкой в вашем коде - видны в Logcat. Инструмент Logcat прикреплён в нижнем левом углу Вашего окна Android Studio отображающего Logcat при нажатии:

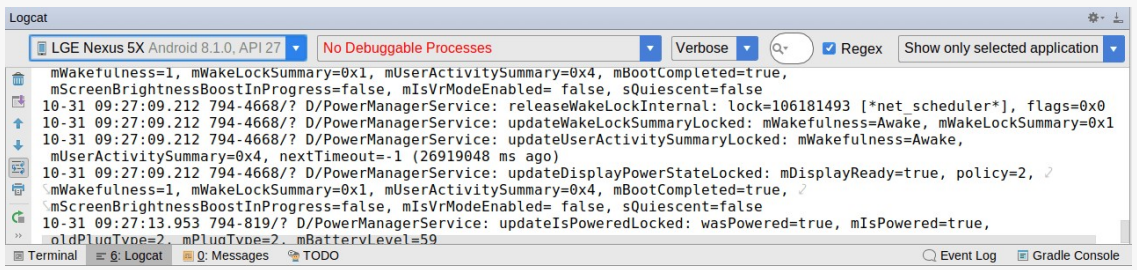


Рисунок 43: Инструмент Logcat Android Studio

Logcat объясняется более детально немного далее в данной книге.

Доступ к инструментам Андроид

Не все относящееся к Андроид - напрямую часть Android Studio. В некоторых случаях, инструмент которым необходимо поделится между пользователями Android Studio и пользователя Eclipse, и пользователями "не включённое в вышеперечисленное". В некоторых случаях, когда в перспективах добавление какого-либо инструмента функциональности напрямую в Android Studio, которое не работает просто не завершается к дате.

Здесь несколько заслуживающих внимание инструментов относящихся к Андроид, через которые Вы можете получить доступ через Tools (инструменты) опций главного меню.

Менеджеры SDK и AVD

Как мы видели в Руководстве №1 менеджер SDK - это инструмент Андроид для загрузки частей Андроид SDK, включающие в себя:

- редакции “SDK Platform”, позволяющие нам компилировать для определённого уров API.
- ARM и (иногда) x86 образы эмулятора
- Документацию

- Обновление основных инструментов сборки

- и т.д.

Вы можете запустить Менеджер SDK через Tools > SDK Manager (Инструменты > Менеджер SDK) из основного меню Android Studio, или по нажатию кнопки "дроид в коробке" на панели инструментов:



Рисунок 44: Значок Менеджера SDK панели инструментов Android Studio

Менеджер AVD инструмент для создания эмуляторов, который эмулирует определённое окружение Андроид, основанное на уровне API, размерах экрана, и других характеристиках.

Вы можете запустить Менеджер AVD через Tools > AVD Manager (Инструменты > Менеджер AVD) из основного меню Android Studio, или по нажатию кнопки "дроид и экран" на панели инструментов:



Рисунок 45: Менеджер AVD Android Studio, значок панели инструментов

Android Studio и каналы релизов

Когда Вы устанавливаете Android Studio в первый раз, Ваша установка будет

настроена на получение обновлений на канале релизов "stable". Здесь “release channel” (канал релизов) специальный набор возможных обновлений. Канал релизов "stable" означает, что Вы получаете полные

готовые к производству обновления. Android Studio проверяет обновления при запуске, и Вы можете вручную проверить обновления через главное меню (пример Help > Check for Update...(Помощь > Проверить обновления на Windows или Linux)).

Если обновления доступны, Вам представится диалоговое окно, отображающее Вам детали обновления:

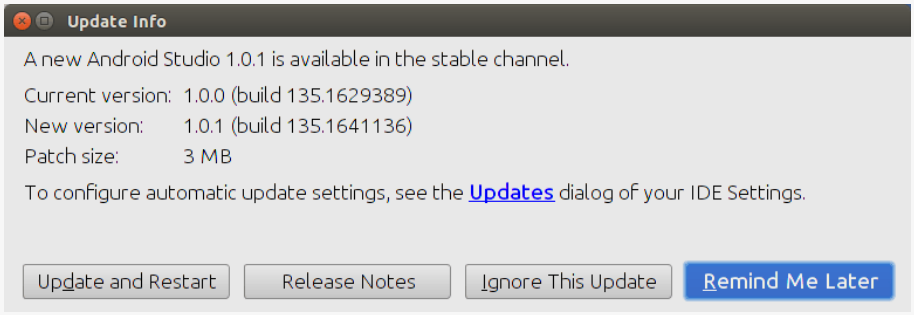


Рисунок 46: Диалог обновления Android Studio

Выбор “Release Notes” (Заметки релиза) переправит на Веб страницу с заметками релиза для нового релиза. Нажатие на “Update and Restart” (обновить и перезапустить) делает то, что название кнопки предлагает: загружает обновление и перезапускает IDE, применяя обновление в процессе.

Нажатие на гиперссылку “Updates” (обновления) в диалоге перенаправляет на еще один диалог, позволяющий выбрать, на какоё канал релиза Вы хотите подписаться:

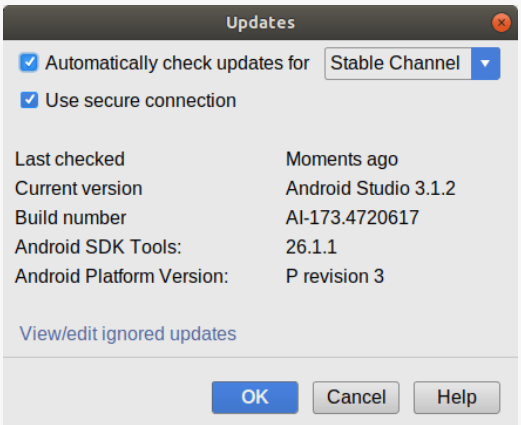


Рисунок 47: Диалог обновления канала релизов Android Studio

Вы можете выбрать из четырёх каналов:

- Stable (стабильный), который предпочтительный для большинства разработчиков.
- Beta (бета), который предоставляет обновления, которое сразу впереди за стабильными.
- Dev (разработка), которое далее впереди от канала бета.
- Canary (канарейка), который содержит *очень* ранние обновления (и название, предполагается от “canary in a coal mine” (канарейки в угольной шахте), который указывает, что Вы здесь, чтобы помочь в отладке IDE)

Посетите пути!

Структура диалога проекта и Translations Editor (редактор переводов) Android Studio рассмотрены далее в данной книге.

Содержание проектов Андроид

Система сборки организована вокруг определённой структуры дерева директорий Вашего проекта Андроид, более похоже на любой другой проект Java. Особенности, тем не менее, достаточно уникальны для Андроид - инструмент сборки Андроид выполняет несколько дополнительных действий для подготовки текущего приложения для запуска на устройстве или эмуляторе.

Здесь представлен быстрый пример структуры проекта, чтобы помочь Вам получить представление о нем в общем, особенно о примерах кода на которые ссылается данная книга.

Что Вы получаете, в общем

Детальная информация какие *именно* файлы в Вашем проекте зависят от множества вещей:

- Какую IDE и версию использовал проект при создании проекта
- Какой шаблон проекта был использован для создания проекта
- Какие другие изменения были сделаны в проекте после его

создания. Обычно это много элементов в общем.

Модули

Проект Android Studio по умолчанию содержит директорию `app/` в корневой директории проекта (директория которую Вы указали Android Studio для создания проекта). Данная директория `app/` представляет то, на что Android Studio указывает как на модуль. Большинство проектов содержат только модуль `app/` , хотя у Вас может быть больше чем один модуль при необходимости.

The Source Sets (наборы источников)

Внутри `app/` Вы можете найти директорию `src/main/` . Что представляет собой `main` (основной) набор исходников, содержит все “исходники” которые Ваш проект предоставляет для сборки приложения. Здесь, “source” означает больше чем просто исходный код языка программирования (пример Java). Он также содержит другие типы файлов, такие как ресурсы и manifest, которые предоставляются в Ваше приложение.

Manifest

Внутри `app/src/main/`, Вы найдёте файл называемый `AndroidManifest.xml` `AndroidManifest.xml` это XML файл описывающий приложение, которое будет собрано и из каких компонентов - activities (активности), services (сервисы), и т.д. - будут поддерживаться этим приложением. Вы можете думать об этом как о "таблице содержания" о чем Ваше приложение, похоже на то как книга содержит список “содержание” разных частей, глав, и дополнений, которые появляются в книге.

Мы рассмотрим manifest более близко начиная со следующей главы.

Java

Когда вы создаёте проект, если вы выбрали, чтобы Android Studio создала активность для Вас, Вам предоставляется название “основной” активности для приложения (например `MainActivity`). Что будет скомбинировано с названием пакета, который предоставляется Вам в проекте для определение полностью определённого название класса для данной активности (пример `com.commonsware.android.MainActivity`).

Вы обнаружите, что исходное дерево Вашего проекта также содержит расположение дерева директорий проекта, плюс класс представляющий запрошенную активность (пример `com/commonsware/android/MainActivity.java`). Файл будет внутри `основного` набора исходников, так полный путь от корневой директории проекта будет что-то похожее на `app/src/main/java/ com/commonsware/android/MainActivity.java`.

Вы можете модифицировать данный файл и добавлять Java классы при необходимости имплементировать Ваше приложение, и мы продемонстрируем это множество раз при прогрессе через данную книгу.

В обратном случаи - в директориях в которых Вы обычно не работаете -

инструмент сборки Андроид также сгенерирует некоторый исходный код для Вас каждый раз когда Вы собираете ваше приложение. Один из сгенерированных Java классов (`R.java`) будет важным для

контроля нашего интерфейса пользователя из нашего собственного Java кода, и мы увидим много отсылок к данному `R` классу когда мы начнём создание приложения всерьёз.

Resources (Ресурсы)

Вы также можете найти, что Ваш проект содержит директорию `res/` внутри `основного` (main) набора исходников Что содержит "ресурсы (resources)" - статические файлы, которые упаковываются в Ваше приложение, или в исходном виде или, иногда, в предобработанном виде. Некоторые их поддиректорий, которые вы можете найти или создать в `res/` включают:

1. `res/drawable/` и `res/mipmap-*/` (для нескольких значенийц `*`) для изображений (PNG, JPEG, и т.д.)
2. `res/layout/` для основаннх на XML спецификаций разметов интерфейса пользовате
3. `res/menu/` для основанных на XML спецификаций меню
4. `res/raw/` для файлов общего назначения (а именно аудио клипы, файлы CSV информации об аккаунте)
5. `res/values/` для строк, размерностей, и как
6. `res/xml/` для других файлов XML общего назначения которые вы ходите предостави

Некоторые из названий директорий могут содержать суффиксы, похожие на `res/mipmap-hdpi/`. Данное указывает, что директории ресурсов должны использоваться только в определённых обстоятельствах - в данном случаи, mipmap ресурсы разработаны для устройств в экранами с высокой плотностью пикселей.

Мы рассмотрим всё это, и больше, позже в данной книге.

Инструкции сборки

IDE необходимо знать как собрать все эти вещи и подойти к Андроид файлу APK. Что-то из этого уже "известно" IDE основываясь на том, как написана IDE. Но некоторые детали необходимо конфигурировать время от времени, и потому данные детали сохраняются в файлах, которые вы можете редактировать, в каком-то смысле, из Вашей IDE.

В Android Studio, большинство указанных данных содержаться в файле с названием `build.gradle`. Данные для средства сборки, известного как Gradle которое использует Android Studio для сборки APK и других выводов Android.

Боле о структуре папок

Все из указанных элементов сохраняются в определённой структуре директорий в проекте Android Studio ... как минимум по умолчанию. Android Studio и Gradle мощный инструмент и может быть сконфигурирован для другой структуры, хотя большинство проектов завязаны на стандартные установки.

Корневая директория

В корневой директории Вашего проекта, наиболее важные элемент это директория `app/` представляющая модули приложения. Мы рассмотрим это в следующей главе.

Кроме директории `app/` следующие стоящие упоминания файлы файлы в корневой директории Вашего проекта включают:

- `build.gradle`, который часть инструкций сборки для Вашего проекта, как описано Выше
- Различные относящиеся к Gradle файлы (`settings.gradle`, `gradle.properties`, `gradlew`, `gradlew.bat`, и т.д.)
- `local.properties`, который указывает на расположение инструментов Android SDK
- Файл `.iml` , где Android Studio содержит некоторые дополнительные метаданные об Вашем проекте
- Файл `.gitignore` , представляющие стандартные правила, что должно входить и что должно не входить в контроль версий, особенно в отношении к системе контроля версий Git.

В итоге, у Вас будет:

- Директория `build/` содержит скомпилированный выходные файлы Вашего приложения, плюс различные отчёты и другие файлы относящиеся к процессу сборки и тестирования приложения
- Директория `.gradle/` содержит исполняемый код Gradle
- Директория `.idea/` — вместе с файлом `.iml` , представляет данные represents необходимые IntelliJ IDEA, на которой основана Android Studio

Директория App (приложение)

Директория `app/` и её содержимое, где Вы будете проводить большую часть времени, как разработчик. Довольно нечасто возникает необходимость изменять

файлы в корневой директории.

Наиболее важные элементы в директории `app/` это директории `src/`, которая является

корневой набора ресурсов вашего проекта. Мы рассмотрим это детальнее в следующем разделе. За пределами директории `src/` , содержатся несколько других элементов, которые стоит отметить в `app/`:

- Директория `build/` , которая содержит выходные данные сборки вашего приложения, включая файл APK.
- Файл `build.gradle` где сохраняются большинство конфигураций специфичных для Gradle, для обучения Android Studio как собирать Ваше приложение.
- Файл `app.iml` содержащий большинство метаданных Android Studio
- Другой файл `.gitignore` перезаписывает правила управления версиями Git из файла `.gitignore` в корневой проекта.
- `proguard-rules.pro`, где правила для инструмента называемого ProGuard, который помогает уменьшить размер вашего приложения.

The Source Sets (наборы источников)

Набор источников - это где организованы "источники" Вашего проекта. Как было отмечено ранее, к "источникам" относятся не только исходный код языка программирования (пример Java), но и другие типы входных данных для сборки, такие как ресурсы.

Набор источников в котором Вы проведёте больше больше всего времени - `main/`. У Вас также есть стабильный набор источников называемый `androidTest` и `test`, для применения при тестирования Вашего приложения, и который будет рассмотрен далее в книге.

Внутри набора источников, включены общие элементы:

- код Java, в директории `java/`
- Ресурсы, в директории `res/`
- Данные в директории `assets/` представляют другие статические файлы, которые Вы хотите упаковать в приложение для развёртывания на устройстве.
- Ваш файл `AndroidManifest.xml`

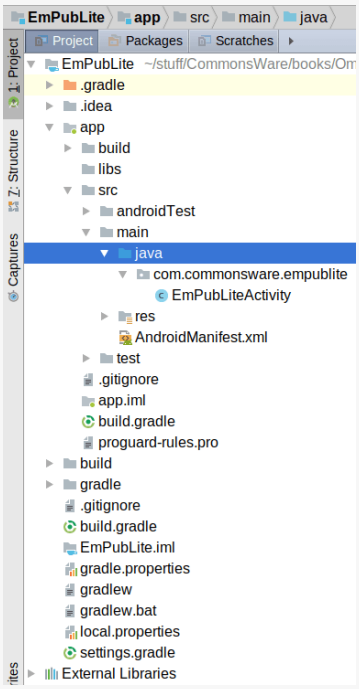


Рисунок 48: Android Studio просмотр проекта, Отображение EmPubLite

Некоторые проекты содержат дополнительные элементы, обычно привязанные к определённому дополнительному инструменту, который применяется при развёртывании проекта.

Что вы получаете после этого

Как часть запуска Вашего приложения на устройстве или эмуляторе, IDE сгенерирует файл APK. Вы найдёте его в директории `build/outputs/apk` Вашей директории модуля, (пример `app/build/outputs/apk` для стандартного проекта Android Studio).

APK - это ZIP архив, содержащий Ваши скомпилированные Java классы с упакованными версиями остальных Ваших источников (например ресурсы,

manifest). APK - это то, что распространяется Вашим пользователям, через сервис распространения приложений Google's Play Store или другим способом.

Введение в Gradle и Manifest

В обсуждении Android Studio, данная книга предлагает что-то называемое “Gradle”, без большого количества обсуждений.

В данной главе, мистерия Gradle будет рассмотрена для

Вас. (ну хорошо, *некоторая часть* мистерии...)

Мы также обратимся к предыдущей главе за концептом "manifest".
— `AndroidManifest.xml` — специальный файл в Вашем проекте Android.

С одной стороны, Gradle и манифест строго соотносятся. С другой, некоторые (но далеко не все) элементы, которые мы можем указать в manifest могут быть перезаписаны в Gradle.

Так, в данной главе, мы рассмотрим оба и что Gradle есть, и что manifest есть, какой роль каждого из них, и базовые концепты как они связаны.

Gradle: Главные вопросы

Сперва, позвольте нам "установить планку" рассматривая о чем это все, через серию часто спрашиваемых вопросов ответов (FAQs (ЧАВО))

Что такое Gradle?

Gradle - это программное обеспечение для сборки программного обеспечения, известное иначе как "программное обеспечение автоматической сборки" или "система сборки". Вы могли использовать другие системы сборки ранее в окружении, такие как **`make`** (C/C++), **`rake`** (Ruby), Ant (Java), Maven (Java), и т.д.

Данные инструменты знают - через внутренние возможности и правила, которыми вы обучаете их - как определить что необходимо создать (основываясь на изменениях файла) и как создать это. Система сборки не компилирует, не создаёт ссылки, не упаковывает и т.д. приложение напрямую, но вместо этого непосредственные отдельные компиляторы, линкеры, и упаковщики делают эту работу.

Gradle использует domain-specific language (DSL) (специальный для направления язык) построенные на основе Groovy для выполнения данных задач.

Что такое Groovy?

Существует много языков программирования, которые спроектированные для запуска поверх виртуальной машины Java (Java VM) Некоторые из них, такие как JRuby и Jython, являются имплиментацией других общих языков программирования (Ruby и Python, соответственно). Другие языки уникальны и Groovy один из них.

Скрипты Groovy выглядят слегка как смесь Java и Ruby. Как и Java, Groovy поддерживает:

- Определение классов с ключевым словом `class`
- Создание подклассов применяя `extends`
- Импортирование классов из внешних JAR применяя `import`
- Определение тела метода применяя скобки (`{` and `}`)
- Объекты создаются через оператор

`new`. Как в Ruby, также:

- Утверждения могут быть частью класса, или просто написаны в императивном стиле, как в скриптовом языке
- Параметры и локальные переменные безтиповые
- Значения могут быть автоматически пропатчены в строки, хотя и применяя слегка другой синтаксис (`"Hello, $name"` для Groovy вместо `"Hello, #{name}"` для Ruby)

Groovy - интерпретируемый язык, как Ruby, а не как Java. Скрипты Groovy запускаются и исполняются командой **`groovy`**, передавая ему скрипт для запуска. Выполнение Groovy, также как и Java JAR, требует JVM для работы.

Одна из сильных сторон Groovy - создание domain-specific language (or DSL) (языка специфического для области). Gradle, для примера, это Groovy DSL для

выполнения сборки программного обеспечения. Возможности, специфичные для Gradle, кажутся первоклассными языковыми конструкциями, обычно неразличимыми.

от внутренних возможностей Groovy. Да, Groovy DSL в основном декларативный, как файл XML.

В некоторой степени мы получаем лучшее из обоих миров: определения в стиле XML (обычно с меньшим количеством знаков препинания), но с возможностью «проникать в Groovy» и при необходимости создавать собственные сценарии.

Что Андроид должен делать с Gradle?

Google опубликовала Android Gradle Plugin, который предоставляет Gradle возможность сборки Андроид проектов. Google также использует Gradle и Android Gradle Plugin как систему сборки в основе Android Studio.

Получение Gradle

Как и в любой системе сборки, для её применения, Вам необходимо собрать сам системн движок.

Если Вы будете использовать Gradle в контексте с Android Studio, IDE позаботится о получение Gradle за Вас. Если, кроме того, Вы планируете использовать Gradle за пределами Android Studio (пример сборка в командной строке), Вы захотите определить откуда Gradle появляется. Это особенно важно для ситуаций когда вы хотите собрать приложение без применения IDE, как при применении сервера непрерывной интеграции (CI), как Jenkins.

Также путь, которым Android Studio работает с Gradle - называется Gradle Wrapper (обёртка Gradle) - создает проблемы безопасности для вашей машины разработки, если вы из тех, кто загружает проекты с открытым исходным кодом из таких мест, как GitHub, и пробует их использовать.

Прямая установка

Большинство разработчиков, желающих использовать Gradle вне Android Studio, в конечном итоге сделают установку Gradle напрямую.

Страница загрузки Gradle содержит ссылки на ZIP архивы непосредственно на бинарники Gradle, исходный код или оба.

Вы можете разархивировать архив в желаемое расположение на Вашей машине разработки.

Пакеты Linux

Вы можете получить пакеты Gradle через менеджер пакетов в окружении Linux. Для примера здесь, Ubuntu PPA для Gradle.

Обёртка gradlew (gradlew Wrapper)

Если Вы начали от проекта который опубликовал кто-либо ранее, Вы можете найти файлы `gradlew` и `gradlew.bat` в корневой проекта, вместе с директорией `gradle/` Они представляют "Gradle Wrapper".

Gradle Wrapper содержат три части:

- командный файл (`gradlew.bat`) или скрипт оболочки (`gradlew`)
- файл JAR используемый командным файлов или скриптом оболочки (в `gradle/wrapper/`)
  директории)
- файл `gradle-wrapper.properties` (также в директории `gradle/wrapper/`)

Android Studio использует файл `gradle-wrapper.properties` для определения откуда загружать Gradle, для применения в Вашем проекте, из свойства `distributionUrl` в данном файле:

```
#Thu May 10 08:57:09 EDT 2018
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-4.4-all.zip
```

(из Basic/Button/gradle/wrapper/gradle-wrapper.properties)

Когда Вы создаёте или импортируете проект, или если Вы изменяете версию Gradle на которую ведёт ссылка в файле предпочтений, Android Studio загрузит Gradle из свойства `distributionUrl` и установит его в директорию `.gradle/` (обратите внимание на ведущую) в вашем проекте. Это будет версия Gradle которую будет использовать Android Studio.

Правило №1 Используйте только `distributionUrl` (ссылку распространения) которой доверяете.

Если Вы импортируете проект Андроид извне - как примеры из данной книги - и они содержат файл `gradle/wrapper/gradle-wrapper.properties`, проверти

его и посмотрите куда указывает `distributionUrl`. Если загрузка из `сервисов gradle.org`, или из внутреннего корпоративного сервера, то это вероятно

достоверно. Если URL указывает куда-то в другое место, определите действительно Вы хотите использовать эту версию Gradle, понимая что она может быть поддельна.

Командный файл, консольный скрипт, и JAR файл здесь для поддержки сборки через командную строку. Если Вы запустите команду **`gradlew`**, она будет использовать локальную копию Gradle установленную в `.gradle/` внутри проекта. Если такой копии нет **`gradlew`** загрузит Gradle из `distributionUrl` как это делает Android Studio. Отметьте, что Android Studio не использует **`gradlew`** с этой целью - данная логика встроена непосредственно в Android Studio.

Правило №2: Используйте только `gradlew` которому вы действительно доверяете.

Это относительно легко проверить файл `.properties` для определения URL чтобы увидеть что он кажется валидным. Имеет смысл что командный файл и консольный скрипт могут быть громоздкими. Декомпиляция файла JAR и приведение его в смысловой может быть более сложно. Еще, если Вы применяете **`gradlew`** который ВЫ получаете от кого-либо, тот скрипт и JAR запускается на Вашей машине разработки, как и копия Gradle которую они устанавливают. Если данный код был поддельным, вредоносное ПО получит полный доступ к Вашей машине разработчика и всему к чему сможет добраться, как сервера в вашей организации.

Отметьте, что Вам нет необходимости применять Gradle Wrapper вовсе. Если Вы хотите не переживать об этом, установите версию Gradle на Вашу машину разработки самостоятельно и удалите файлы Gradle Wrapper. Вы можете использовать команду **`gradle`** для сборки Вашего приложения (если Ваша директория `bin/` в Вашем `PATH`), и Android Studio будет использовать Вашу установку Gradle (если Вы научите её где найти её, как например через переменную окружения `GRADLE_HOME`)

Версии Gradle и плагина Андроид Gradle

Андроид дополнение Gradle, который мы используем, что дать Gradle "супер Андроид силы" периодически обновляется. Каждое обновление содержит соответствующие требования к версии Gradle. Google поддерживает страницу список версий Gradle поддерживающей каждой версией Дополнения Андроид Gradle.

Если Вы используете обёртку Gradle, Вы используете установленный Gradle,

который локальный в проекте. Как долго версия Gradle в проекте совпадает с версией Дополнения Андроид Gradle требуемого в проектном файле `build.gradle` - как рассмотрим далее в данной главе - вы будете в хорошей позиции.

Если вы не используете Обёртку Gradle, Вам необходимо решить, когда применять новый выпуск Дополнения Андроид Gradle и планировать обновление Вашей установленного Gradle и файлов `build.gradle` в тандеме.

Переменные окружения Gradle

Если Вы устанавливали Gradle самостоятельно, Вам захочется описать переменные окружения `GRADLE_HOME`, указывающие где Вы установили Gradle, и добавить Вашу директорию `bin/` внутри Gradle к вашей переменной окружения `PATH`.

Вы можете также установить настройки переменной окружения `GRADLE_USER_HOME`, указывающую на директорию в которой Gradle может создать поддиректорию `gradle/`, для пользовательского кэша и соотносящихся материалов. По умолчанию, Gradle использует стандартную домашнюю директорию.

Изучение файлов Gradle

Проект Android Studio обычно содержит два файла `build.gradle`, один на уровне проекта, и один на уровне модуля (например в директории `app/`).

Файл уровня проекта

Файл `build.gradle` в директории проекта управляет конфигурацией Gradle для всех модулей в Вашем проекте. Сейчас наиболее вероятно Вы имеете один модуль, и многие приложения используют только один модуль. Тем не менее возможно для Вас добавить другие модули в данный проект, и мы рассмотрим причины делать это дальше в книге.

Типичный файл `build.gradle` верхнего уровня:

```
// Top-level build file where you can add configuration options common to all sub-
projects/modules.

buildscript {

    repositories {
        google()
        jcenter()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:3.0.0'
```

```
        // NOTE: Do not place your application dependencies here; they belong
        // in the individual module build.gradle files
    }
}

allprojects {
    repositories {
        google()
        jcenter()
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}
```

buildscript

В терминах Groovy "closure" (замыкание) это часть кода обёрнутая в скобки (`{ }`).

Замыкание `buildscript` в Gradle - это где Вы настраиваете JAR и что Gradle самостоятельно использует для интерпретации остатка файла. Следовательно, здесь Вы не конфигурируете Ваш *project* так же как Вы конфигурируете *непосредственно сборку*.

Замыкание репозиториев внутри `buildscript` указывает, откуда могут поступать плагины. Здесь `jcenter()` встроенный метод, который обучает Gradle из JCenter популярное местоположение для получения свободных библиотек. Похоже, `google()` встроенный метод который обучает Gradle о сайте где он может загрузить плагины от Google.

Замыкание зависимостей указывает что необходимо для возможности запуска остальной части скрипта сборки. `classpath 'com.android.tools.build:gradle:3.0.0'` не особенно хорошо задокументированно командой Gradle. Тем не менее `'com.android.tools.build:gradle:3.0.0'` чвчть означает:

- Найти группу плагинов `com.android.tools.build`
- Найти `gradle` артефакт внутри группы
- Убедится что у нас `3.0.0` версия плагина

При первом запуске Вашего скрипта сборки, с замыканием `buildscript` как показано выше, Gradle укажет что у Вас нет данной зависимости. It will then

download that artifact from Google, as Google serves up its plugin from its own site nowadays.

Иногда последний сегмент версии заменяется знаком `+` (например `3.0.+`) Что указывает Gradle загрузить последнюю версию, тем самым автоматически обновлять Вас на последнюю версию патча (например `3.0.1` в каком-то моменте)

allprojects

Замыкание `allprojects` указывает "применять данные настройки ко всем модулям данного проекта". Здесь мы настраиваем `jcenter()` и `google()` как места поиска библиотек применяемых в каждом модуле нашего проекта. Мы будем использовать много библиотек в нашем проекте - имея данные "репозитории" указанные в `allprojects` делает более простым для нас запрашивать их.

Файл Gradle уровня модуля

В Вашем модуле `app/` вы также можете найти файл `build.gradle`. Он содержит настройки уникальные для данного модуля, независимо от каких-либо других модулей, которые Ваш проект может содержать в будущем.

Здесь типичный файл уровня модуля `build.gradle`:

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 26
    defaultConfig {
        applicationId "com.commonsware.myapplication"
        minSdkVersion 21
        targetSdkVersion 26
        versionCode 1
        versionName "1.0"
        testInstrumentationRunner "android.support.test.runner.AndroidJUnitRunner"
    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-
rules.pro'
        }
    }
}

dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation 'com.android.support.constraint:constraint-layout:1.0.2'
```

```
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'com.android.support.test:runner:1.0.1'
    androidTestImplementation 'com.android.support.test.espresso:espresso-core:3.0.1'
}
```

Сейчас давайте сфокусируемся на некоторых ключевых элементах данного файла.

dependencies

Данный файл `build.gradle` также содержит замыкание `dependencies`. Тогда как замыкание `dependencies` в замыкании `buildscript` вершнего уровня файла `build.gradle` относится к библиотекам в процессе сборки, замыкание `dependencies` в файле `build.gradle` модуля относиться к библиотекам которые используются в Вашем коде в данном модуле. Мы рассмотрим концепты данных библиотек далее в книге.

android

Замыкание `android` содержит всю специфическую для Андроид информацию по конфигурированию. Данное замыкание это то, что андроид плагин активирует, тогда как сам плагин поступает из строки `apply plugin: 'com.android.application'` наверху, соединённый со строкой `classpath` из файла `build.gradle` уровня проекта.

Но прежде чем мы погрузимся в данное замыкание мы должны "переключить шестерёнки" и обговорить файл manifest, как что поступает в замыкание `android` связанное с тем что поступает в файл manifest.

Введение в Manifest

Основа для какого-либо Андроид приложения манифест файл: `AndroidManifest.xml`. Находиться в директории модуля `app src/main/` (`main` (основной) набор ресурсов) для типичных проектов Android Studio.

Здесь Вы указываете что внутри Вашего приложения - activities (активности), services (сервисы), и т. д.. Вы также указываете как данные части прикрепляются к общей Андроид системе, к примеру, вы указываете какая активность (или активности) должны появляться в главном меню устройства (также launcher (загрузчик)).

Когда Вы создаёте Ваше приложение, Вы получаете начальный манифест сгенерированный для Вас Для простых приложений, предоставляющих одну

активность и ничего более, автоматически сгенерированный манифест вероятно будет работать хорошо, или возможно требовать нескольких незначительных

модификаций. На другом конце спектра, файл манифеста для демонстрационного набора Android API более 1000 строк длиной. Ваше продуктное Андроид приложение будет вероятно лежать где-то по середине.

Как указано ранее, некоторые элементы могут быть определены и в файле манифест и в файле `build.gradle`. Возможность указывания данных элементов в манифест файле ещё работает. Для пользователей Android Studio вы вероятно применяете файл Gradle и не используете определение обычных элементов в манифест файле.

Общие элементы между Manifest и Gradle

Есть несколько ключевых элементов, которые могут быть определены в манифест файле и могут быть перезаписаны в утверждениях `build.gradle` . Данные элементы достаточно важны в разработке и выполнении наших Андроид приложений.

Название пакета и идентификатор приложения

Корневой элемент всех файлов манифест, не удивительно, элемент манифеста:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.commonsware.empublite">
```

Отметьте декларирование пространства имён Андроид. Вы можете только использовать пространство имён множества атрибутов, не элементов (пример, `<manifest>`, не `<android:manifest]`)

Большая часть информации, которую Вам необходимо поддерживать в элементе `<manifest>` это аттрибуты пакета.

Атрибуты пакета всегда должны быть в файле манифест, в том числе и для проектов Android Studio. Атрибуты пакета управляют где некоторый исходный код генерируется для нас, особенно некоторые классы `R` и `BuildConfig` , с которыми столкнётесь позже в книге.

Так как значение пакета применяется в генерировании Java кода, оно должен быть валидных наименованием пакета Java Конвенции Java утверждают, что наименование пакета основано на обратном наименовании доменов (пример `com.commonsware.myapplication`), где вы являетесь владельцем рассматриваемого домена. Таким образом, маловероятно, что кто-либо другой

случайно столкнётся с таким же именем.

`Пакет` также служит как “application ID” (идентификатор приложения по умолчанию) по умолчанию. Это должен быть уникальный идентификатор, так что:

- два приложения с одинаковым идентификатором приложения не могут быть установлены в одно время на одном устройстве
- два приложения с одинаковым идентификатором приложения не могут быть загружены в Play Store (и другие каналы распространения могут иметь те же ограничения)

По умолчанию идентификатор приложения, это значение пакета, но пользователи Android Studio могут перезаписать его в их файлах сборки Gradle. Особенно внутри замыкания `android` может быть замыкание `defaultConfig`, и внутри него может быть утверждение `applicationId`:

```
android {
    // other stuff

    defaultConfig {
        applicationId "com.commonsware.myapplication"
        // more other stuff
    }
}
```

Пользователи Android Studio могут не только переписать идентификатор приложение в замыкании `defaultConfig`, но существуют способы иметь различные идентификаторы приложения для различных сценариев, таких как сборка для отладки и сборка для релиза. Мы рассмотрим это детальнее далее в книге.

minSdkVersion и targetSdkVersion

Ваше замыкание `defaultConfig` внутри замыкания `android` в Вашем файле `build.gradle` модуля содержит пару свойств с названием `minSdkVersion` и `targetSdkVersion`. Технически, они перезаписывают значения, которые могут быть определены через элемент `<uses-sdk>` в манифест файле хотя в настоящее время мало проектов будет иметь такой элемент.

Из двух более важный `minSdkVersion`. Который указывает какая самая старая версия Андроид тестируется с Вашим приложением. Значение атрибута целочисленное представление уровня Android API Так, если Вы тестируете Ваше приложение только на Андроид 4.1 и новых версиях, Вы устанавливаете

`minSdkVersion` на `16`. Изначальное значение это которые Вы запрашиваете когда Android Studio создаёт проект.

Вы можете также установить `targetSdkVersion`. Что указывает о какой версии Андроид

вы думаете кгда пишете Ваш код. Если Ваше приложение запускается на более новых версиях Андроид, Андроид может выполнить некоторые действия в попытке улучшить совместимость Вашего кода в соответствии с изменениями сделанными в более новой версии. В настоящее время, большинство разработчиков Андроид должны указывать целевую версию SDK на `15` или выше. Мы начнём рассматривать более об `targetSdkVersion` когда погрузимся глубже в книгу, все равно Ваше IDE предоставляет Вам значение по умолчанию, которое вероятно хорошо как стартовое значение.

Версия кода и название версии

Аналогично замыкание `defaultConfig` (настойка по умочанию) имеют свойства `versionCode` (версия кода) и `versionName` (название версии) В принципе, они перезаписывают атрибуты `android:versionName` и `android:versionCode` в корне элемента `<manifest>` в манифест файле, потому Вы не найдете много проектов использующих эти XML атрибуты.

Данные два значения представляют версии Вашего приложения. Значение `versionName` это то, что видит пользователь в указании версии на экране деталей о Приложении в их приложении Настройки. Также, название версии используется в описании Play Store, если Вы распространяете Ваше приложении данным способом. Название версии может быть любым строковым значением.

Версия кода с другой стороны, должна быть целочисленной, и более новые версии должны содержать большее значение версии кода чем старые версии. Андроид и Play Store сопоставляют версию кода нового APK с версией кода установленного приложения для определения, что новое приложение действительно является обновлением. Типичный подход начинать с версии кода `1` и увеличивать его каждый продуктный релиз Вашего приложения, хотя Вы можете выбрать другие конвенции по желанию. В процессе разработки, Вы можете оставить их в покое, но когда Вы перейдёте к распространению эти атрибуты будут иметь большое значение.

Другие примечательные элементы Gradle

Замыкание `android` содержит свойство `compileSdkVersion` (версия Sdk компилятора) `compileSdkVersion` указывает уровень API компиляции, обычно как простое целочисленный уровень API (пример 19). Данное указывает какие Java классы, методы и т.д. доступны Вам при написании Вашего приложения. Обычно, Вы устанавливаете значение в последний продуктный релиз Андроид,

используя значения как указано ранее в книге.

Замыкание `android` *должно* содержать свойство `buildToolsVersion` (версия инструмента сборки). `buildToolsVersion`
указывает версию инструментов сборки Андроид SDK которые Вы желоете использовать в данном

проекте. Android Gradle Plugin действительно тонкая оболочка вокруг набора “инструментов сборки” которая обеспечивает большую часть работы по созданию APK для Вашего проекта. Если Ваше замыкание `android` не содержит `buildToolsVersion`, Android Gradle Plugin использует свою собственную версию по умолчанию инструментов сборки, и для большинства проектов этого будет достаточно. В данной книге, большинство проектов устанавливает их `buildToolsVersion`, хотя некоторые пропускают данную строку и применяют значение по умолчанию предоставляемую плагином.

Где GUI?

Вы можете задаваться вопросом почему мы продираемся через весь этот Groovy код и спрашивать существует ли некоторый GUI в применении к настройкам Gradle.

Ответ да ... и нет.

Здесь диалог структуры проекта, который позволяет Вам поддерживать некоторые из указанных действий. И Вы можете попробывать его. Тем не менее, чем более сложным становиться ваша сборка, тем более вероятно что GUI будет недостаточно, и Вам будет необходимо работать с файлами сборки Gradle напрямую. Следовательно, данная книга сфокусирована на указанные файлы сборки.

Остальное в Manifest

Не всё в манифест файле может быть перезаписано в файлах сборки Gradle. Здесь несколько ключевых элементов, которые всегда определяются в манифест файле, а не в фале `build.gradle`.

Приложение для Вашего приложения

В Вашем начальном проектном манифест файле главный потомок элемента `<manifest>` элемент `<application>` (приложение)

По умолчанию, когда вы создаёте новый проект Андроид, у Вас единственный элемент <activity>
внутри элемента `<application>`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.commonsware.myapplication"
  xmlns:android="http://schemas.android.com/apk/res/android">

  <application
```

```
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/AppTheme">
    <activity android:name="MainActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

Элемент `<activity>` поддерживает `android:name` для реализации класса активности, `android:label` для отображаемого названия активности и (иногда)
`<intent-filter>` элемент потомок описывающий при каких условиях данная активность будет отображена. Стоковый элемент `<activity>` настраивает появление Вашей активности в загрузчике, так пользователи смогут выбрать её для запуска. Как мы увидим далее в данной книге, у Вас может быть несколько активностей в одном проекте, по Вашему выбору.

Атрибут `android:name` в данном случаи содержит простое название Java класса (`MainActivity`). Иногда, Вы можете увидеть `android:name` с полным названием класса (пример `com.commonsware.myapplication.MainActivity`). Иногда, Вы можете увидеть название Java класса с единичной точкой как префикс (пример `.MainActivity`). Оба `MainActivity` и `.MainActivity` ссылаются на Java класс, который будет в пакета Вашего проекта - тот, который декларировался в аттрибуте `package` элемента `<manifest>` .

Поддержка множества экранов

Устройства Андроид существуют с широким диапазоном размеров экрана, от 2.8” маленьких смартфонов до 46” телевизоров. Андроиз разделяет их на четыре группы, основываясь на физическом размере и расстоянии с которого на них обычно смотрят:

1. Small (маленькие) (меньше 3”)
2. Normal (обычные) (3” до около 4.5”)
3. Large (большие) (4.5” до около 10”)
4. Extra-large (экстра большие) (более 10”)

По умолчанию, Ваше приложение поддерживает маленькие и обычные

экраны. Оно также может поддерживать большие и экстра большие экраны через некоторое автоматическое преобразование кода встроенное в Андроид.

Для желаемой действительной поддержки всех размеров экрана, Вы должны обоснованно добавить
элемент `<supports-screens>` в Ваш манифест. Данное перечисляет размеры экрана явно поддерживаемые Вами. К примеру, если Вы предоставляете пользовательскую поддержку для больших и экстра больших экранов, Вы захотите указать элемент `<supports-screens>` Так, хотя начальный манифест файл работает, управление множественными размерами экранов это то, что о чем Вы захотите подумать.

Вы получаете элемент, похожий на:

```
<supports-screens
  android:largeScreens="true"
  android:normalScreens="true"
  android:smallScreens="false"
  android:xlargeScreens="true" />
```

Гораздо больше информации о предоставлении целостной поддержки для всех размеров экранов включая примеры элемента `<supports-screens>` находятся далее в книге, где мы рассматриваем стратегии для больших экранов.

Другие вещи

Как мы будем продвигаться далее по книге, Вы найдете другие элементы, добавляемые в манифест файл, такие как:

- `<uses-permission>` (используемые разрешения), чтобы сказать пользователю, что Вам необходимы разрешения по использованию определённых возможностей устройства, таких как доступ к Интернету
- `<uses-feature>` (используемые функции), чтобы сказать Андроид, что Вам необходимо получить определённые функции устройства (пример камера), и потому Ваше приложение не может быть установлено на устройство с отсутствием указанных функций.
- `<meta-data>` (мета данные), для частей информации необходимых для специфических расширений Андроид, как для `FileProvider`.

Данные и другие элементы будут представлены в других частях книги.

Изучаем о Gradle больше

Эта книга более подробно расскажет о Gradle как в основных главах, так и в

путях. Но основное внимание будет уделено подключаемому модулю Android Gradle, а сам Gradle предлагает гораздо больше чем это. Веб сайт Gradle содержит ссылки документации, на относящиеся к Gradle книги, и ссылки на другие образовательные ресурсы Gradle.

Посетите пути!

В данной книге есть несколько глав уделяющих более детальное внимание использованию Gradle и Android Gradle Plugin.

- Gradle and Tasks (Gradle и задачи) объясняет как мы запрашиваем Gradle выполнение действий на нашей стороне (“задачи”), такие как компиляция нашего APK для нас.
- Gradle and Build Variants (Gradle и варианты сборки) описывает какие возможности получаем от структуры проекта Gradle включая возможности настраивать “build types” (типы сборки) и “product flavors” (версии продукта)

Также есть глава “Продвинутые секреты Gradle для Андроид” для других тем Gradle и глава о слиянии манифест файлов в Gradle

Руководство №3 - Изменения в Manifest

Как мы создаём `EmPubLite`, нам необходимо выполнить некоторое количество изменений в наш манифест проекта. В данном руководстве, мы позаботимся о паре указанных изменений, чтобы показать Вам как манипулировать файлом `AndroidManifest.xml` . Будущие руководства будут делать ещё больше изменений.

Пользователи Android Studio также получат их первый шанс поработать с build.gradle файлом.

Это продолжение работы, выполненной в предыдущем руководстве.

Вы можете найти результаты предыдущего руководства и результаты данного руководства в GitHub репозитории книги.

Некоторые заметки о относительных путях

В данном руководстве, Вы увидите ссылки на относительные пути, такие как `AndroidManifest.xml`, `res/layout/`, и так далее.

Вы должны интерпретировать данные пути как относительные к директории `app/src/main/` внутри проекта, если не указано обратное. Так, к примеру, Шаг №1 ниже просит Вас открыть `AndroidManifest.xml` — данный файл может быть найден в `app/src/main/ AndroidManifest.xml` из корня проекта.

Шаг №1: Поддерживаемые экраны

Наше приложение ограничивает поддержку размеров экрана. Планшеты

идеально подходят для чтения электронных книг. Телефоны также возможно использовать, но чем меньше телефон, тем сложнее будет

справиться с UI, который позволяет пользователю делать то, что уму нужно, ещё и иметь достаточно пространства для более чем одного или двух предложений книги за раз. Так, давайте добавим `элемент <supports-screens>` чтобы держать подальше наше приложение от устройств с "маленькими" экранами (с размером диагонали менее 3")

Пользователи Android Studio могут дважды щёлкнуть на `AndroidManifest.xml` в просмотрщике проекта.

Как потомка корневого элемента `<manifest>` добавьте элемент `<supports-screens]` как показано:

```
<supports-screens
    android:largeScreens="true"
    android:normalScreens="true"
    android:smallScreens="false"
    android:xlargeScreens="true"/>
```

Шаг №2: Блокирование резервного копирования

Если Вы посмотрите на элемент `<application>` , Вы увидите, что у него несколько атрибутов, включая `android:allowBackup="true"`. Данный аттрибут указывает, что EmPubLite должен участвовать в системе автоматического резервного копирования Андроид.

Это плохая идея, до тех пор пока Вы не поймёте технические и юридические последствия данного выбора, который мы рассмотрим гораздо далее в этой книге.

В краткосрочной перспективе, измените `android:allowBackup` на `false`.

Шаг №3: Игнорирование Lint

Даже после этих изменений название элемент `application` может иметь бежевый фон. Если Вы наведёте мышку на него и посмотрите на объясняющую подсказку, Вы увидите, что оно жалуется, что приложение не индексируемое, и потому Вы должны добавить `ACTION_VIEW` активность в приложение.

Это смешно. Данное приложение (надеемся) никогда не взлетит в Play Store, и потому возможность Google "app indexing" (индексирование приложения) никогда не будет уместной.

Наведите Ваш текстовый курсор где-то внутри названия элемента `application` и нажмите Alt-Enter (или Option-Return на macOS). Это отобразит всплывающее

окно, показывающее “quick fixes” (быстрые исправления) для проблемы:

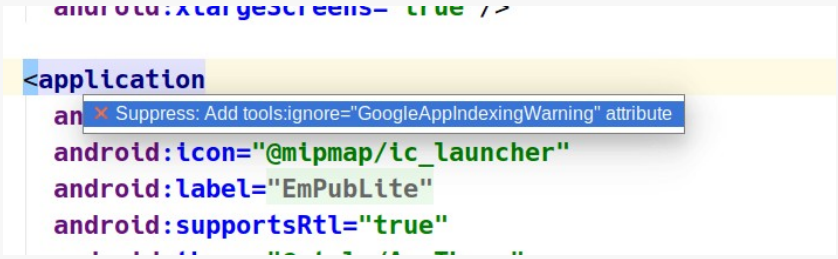


Рисунок 49: Быстрые исправления

Выберите опцию “suppress” (скрыть). После, нажмите Ctrl-Alt-L (Command-Option-L на macOS) чтобы реформатировать файл. Вы завершите с чем-то похожим:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.commonsware.empublite"
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

  <supports-screens
    android:largeScreens="true"
    android:normalScreens="true"
    android:smallScreens="false"
    android:xlargeScreens="true" />

  <application
    android:allowBackup="false"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:supportsRtl="true"
    android:theme="@style/AppTheme"
    tools:ignore="GoogleAppIndexingWarning">
    <activity android:name=".EmPubLiteActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

(из EmPubLite-AndroidStudio/T3-Manifest/EmPubLite/app/src/main/AndroidManifest.xml)

Элемент `<application>` сейчас содержит атрибут `tools:ignore="GoogleAppIndexingWarning"`, и корневой элемент `<manifest>` определяет пространство имён XML `tools` . В итоге мы говорим инструментам сборки, особенно утилите Lint, что они должны игнорировать эту проблему.

В нашем следующем эпизоде...

... мы произведём некоторые изменения в ресурсах нашего учебного проекта.

Несколько слов о ресурсах

Очень похоже, что в настоящее время вы "причмокиваете", чтобы перейти непосредственно к написанию некоторого кода. Это понятно. Это было сказано, прежде чем погрузится в исходный код нашего проекта, нам действительно необходимо коротко поговорить о ресурсах.

Ресурсы - это статические части информации, лежащие за пределами исходного кода Java. Как мы обсуждали ранее, ресурсы сохранены как фалы в директории `res/` вашего набора источников (пример `app/src/main/res/`) Здесь Вы найдёте все Ваши значки и другие ресурсы, Ваши внешние строковые значения для интернационализации, и больше.

Они отделены от исходного Java кода не только потому, что в другом формате. Они отделены потому, что у Вас может быть *множественные* определения ресурсов для различных состояний. Например, с интернационализацией, у Вас могут быть строковые значения для различных языков. Ваш Java код способен оставаться в большей степени не затрагивающим это, когда Андроид выбирает правильные ресурсы для использования, из Всех кандидатов в данных обстоятельствах (пример выбор Испанского строкового значения, если локализация устройства установлена на Испанский).

Вы рассмотрим все детали данного набора ресурсов позже в книге. Прямо сейчас, нам необходимо обсудить ресурсы применимо к нашему проекту, плюс ещё кое-что.

Данная глава ссылается на директорию `res/` как укорочено для директории `app/src/main/ res/` проекта.

Теория строковых значений

Держа Ваши текстовые метки и другие части текста за пределами основного исходного кода Вашего приложения в общем рассматривается как очень хорошая идея. Особенно это помогает с

интернационализацией (I18N) и локализацией (L10N). Даже если Вы не планируете переводить Ваши строковые значения на другой язык, легче производить корректировку, если все строки в одном месте, а не разбросаны по всему Вашему исходному коду.

Простые строки.

Говоря в общем, все что Вам необходимо это файл XML в директории `res/values` (обычно именуемый `res/values/strings.xml`) с корневым элементом `resources` и одним элементом потомком `string` (строка) для каждой строки, которую Вы хотите закодировать как ресурс. Строковый элемент содержит элемент `name` (название), которое является уникальным для данной строки, и единичный текстовый элемент содержащий текст строки.

```
<resources>
  <string name="quick">The quick brown fox...</string>
  <string name="laughs">He who laughs last...</string>
</resources>
```

Одна хитрый момент, если строковое значение содержит скобки или апостроф. В таких случаях, вы выделяете такие значения, ставя перед ними обратный слэш (пример `These are the times that try men\'s souls`) Или, если в тексте столько апостроф, Вы можете заключить значения в скобки (пример `"These are the times that try men's souls."`).

Для примера, проектный файл `strings.xml` может выглядеть так:

```
<resources>
  <string name="app_name">EmPubLite</string>
</resources>
```

(из EmPubLite-AndroidStudio/T3-Manifest/EmPubLite/app/src/main/res/values/strings.xml)

Вы можете ссылаться на данный строковый ресурс из различных локаций в Нашем исходном Java коде или где-либо ещё. Для примера, строковый ресурс `app_name` (название приложения) часто используется в файле `AndroidManifest.xml` :

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.commonsware.empublite"
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

  <supports-screens
    android:largeScreens="true"
    android:normalScreens="true"
    android:smallScreens="false"
```

```
    android:xlargeScreens="true" />

  <application
    android:allowBackup="false"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:supportsRtl="true"
    android:theme="@style/AppTheme"
    tools:ignore="GoogleAppIndexingWarning">
    <activity android:name=".EmPubLiteActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

(из EmPubLite-AndroidStudio/T3-Manifest/EmPubLite/app/src/main/AndroidManifest.xml)

Здесь атрибут `android:label` элемента `<application>` ссылается на строковый ресурс `app_name` Что появляется в нескольких местах приложения, в особенности в списку установленных приложений в Настойках. Так, если Вы хотите изменить, как название Вашего приложения появляется в данных местах, просто установите строковый ресурс `app_name` соответственно.

Синтаксис `@string/app_name` говорит Андроид "найди строковый ресурс с названием `app_name`". Это вызывает сканирование Андроидом соответствующий файл `strings.xml` (или какие-либо другие файлы в директории `res/values/` содержащие строковые ресурсы) в попытке найти `app_name`.

Хм, подождите, мой манифест не выглядит так.

Когда Вы просматриваете манифест в чем-то похожем на Android Studio, он отображается так, как будто Вы не используете ресурсы, Вы можете не увидеть `@string/...` ссылки:

```
<application
  android:allowBackup="false"
  android:icon="@mipmap/ic_launcher"
  android:label="EmPubLite"
  android:supportsRtl="true"
  android:theme="@style/AppTheme"
  tools:ignore="GoogleAppIndexingWarning">
  <activity android:name=".EmPubLiteActivity">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />

      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
</application>
```

Рисунок 50: AndroidManifest.xml, Начальное отображение в Android

Здесь `android:label` выглядит как жёстко запрограммированное значение “EmPubLite”.

Тем не менее, отметьте, что значение атрибута отформатировано иначе чем остальные. Остальные - зелёный текст на белом фоне, когда этот серый текст на затенённом фоне.

Это потому, что Android Studio обманывает Вас.

Если Вы поставите вашу мышку над значением, вы увидите *настоящий* атрибут появившийся как раз под ней:

```
<application
  android:allowBackup="false"
  android:icon="@mipmap/ic_launcher"
  android:label="EmPubLite"
  android:label="@string/app_name"
  android:theme="@style/AppTheme"
  tools:ignore="GoogleAppIndexingWarning">
  <activity android:name=".EmPubLiteActivity">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />

      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
</application>
```

Рисунок 51: AndroidManifest.xml, С мышкой находящейся над "EmPubLite"

И если Вы щёлкните на не настоящее значение, вы увидите настоящий XML, с настоящим значением строкового ресурса.

Случилось то, что Android Studio по умолчанию заменяет значение ресурса при его отображении в манифесте, других ресурсов, которые ссылаются на ресурсы и даже Java коде. Каждый раз, когда Вы видите это серое-на-светло-синем форматирование, помните, что это не настоящее значение, и что Вам необходимо раскрыть настоящее значение через наведение мышки или щелчком на него.

Стилизированный текст

Многие вещи в Андроид могут отображать форматированный текст, где текст может быть отформатирован с применением легковесных разметок HTML: `<b>`, `<i>`, и `<u>`. Ваши строковые ресурсы поддерживают это, просто применяя теги HTML, как Вы делаете на Веб странице.

```
<resources>
  <string name="b">This has bold in it.</string>
  <string name="i">Whereas this has italics!</string>
</resources>
```

CDATA. CDATA беги. Беги, DATA, беги.

Так как файл строковых ресурсов XML это XML файл, если ваше сообщение содержит символы `[`, `]`, или `&` (другие чем теги форматирования указанные выше), Вам необходимо использовать раздел `CDATA` .

```
<string name="report_body">
<![CDATA[
<html>
<body>
<h1>TPS Report for: {{reportDate}}</h1>
<p>Here are the contents of the TPS report:</p>
<p>{{message}}</p>
<p>If you have any questions regarding this report, please
do not ask Mark Murphy.</p>
</body>
</html>
]]>
  </string>
```

Наименование директорий

Наш строковый ресурс в нашем проекте в файле `res/values/strings.xml` Так как название директории (`values`) не содержит суффиксов, строковые ресурсы в данной директории будут валидны для всех типов ситуаций, включая любую локализацию устройства. Нам необходимы дополнительные директории, с различными файлами `strings.xml` для поддержки других языков. Мы рассмотрим как это делать далее в книге.

Редактирование строковых ресурсов

Если Вы дважды щёлкните на файл строкового ресурса, пример `res/values/strings.xml`, в Android Studio, Вам представится XML и его редактирование. Существует опция входа в просмотр предназначенный для перевода строк, рассматриваемый позже в данной книге.

Поддержка мульти-локализаций

Пользователи Android 7.0+ могут указывать, что они поддерживают более чем один язык.

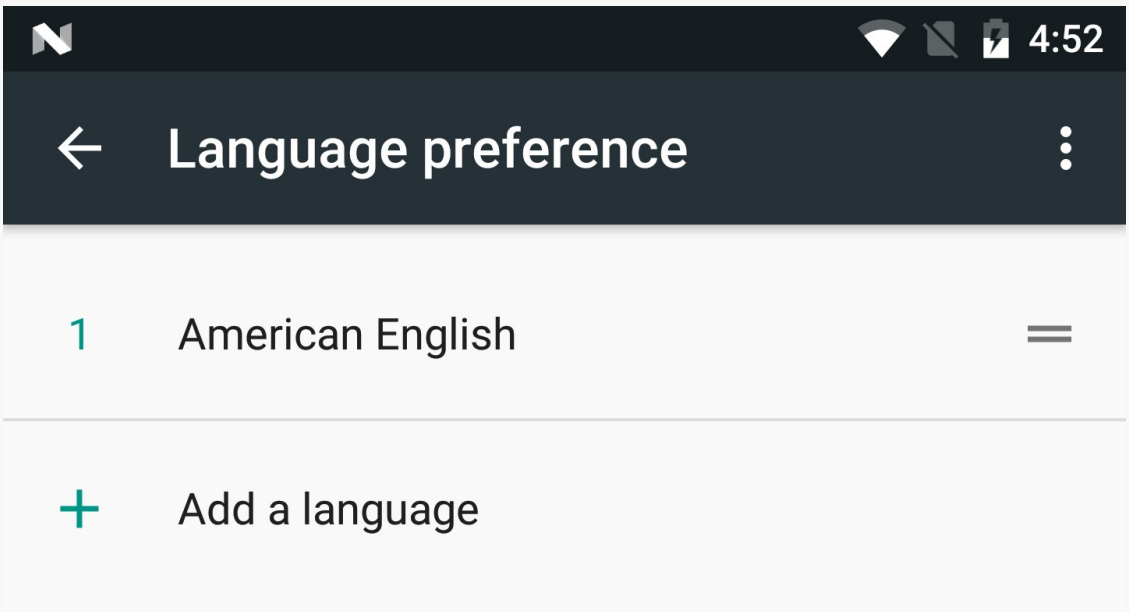


Рисунок 52: Android 7.0 Настройки языка

Пользователь может выбрать относительных приоритет этих языков, перемещая держатель в правой части строки и перетаскивая язык выше или ниже в списке.

Что влияет на разрешение ресурсов для всех зависящих от локализации ресурсов, таких как строки. Сейчас Андроид проверяет множество языков на совпадение ресурсов, прежде чем перейти на язык по умолчанию (например, что бы ни было в `res/values/ strings.xml`). Следовательно, это важно убедиться, что у Вас есть завершенный набор строк для каждого языка, который Вы поддерживаете, чтобы пользователи не столкнулись с смешанным набором языков в UI.

Вы можете убедиться какой язык пользователь запрашивает через класс `LocaleList` и его статический метод `getDefault()`. Данное, как предполагает название содержит список объектов `Локализация` представляющих предпочтительные языки пользователя. Если вы ранее использовали для этого только `Локализация` (например, для специализированной языковой поддержки в приложении помимо ресурсов), вам нужно будет переключиться на `LocaleList` для Android 7.0. и выше.

Получение рисунка?

Все версии Андроид поддерживают рисунки в форматах PNG, JPEG, и GIF. GIF официально не рекомендуется, тем не менее; PNG в общем предпочитаемый формат. Андроид также поддерживает некоторые собственные основанные на XML форматы рисунков, хотя мы не будем обсуждать их сейчас, а рассмотрим позже в книге. Много новых версий Андроид также поддерживает формат рисунка Google WebP, хотя он не особо популярный.

Присутствуют два типа ресурсов, которые используют так рисунки: drawables и mipmaps. По правде они почти идентичные. Mipmaps применяются больше для "значков загрузчика" - значки видимые на домашнем экране загрузчика, которые идентифицируют активность , которую может запустить пользователь. Drawables покрывают всё остальное.

(если вы опытный разработчик Android и читаете этот раздел: хотя ресурсы drawable могут быть удалены при упаковке APK, например, для разделённой системы Android Gradle Plugin для создания выпусков приложения с учётом плотности, ресурсы mipmap остаются неизменными, по-видимому)

В модуле Андроид возможно иметь директории `res/drawable/` и `res/mipmap/` Тем не менее вы обычно не найдёте изображения там. Вместо этого, они расположены в директориях `res/drawable-mdpi/` и `res/drawable-hdpi/`.

Что ссылается на различные наборы рисунков. Суффиксы (пример `-mdpi`, `-hdpi`) - фильтры, указывающие в каких условиях изображения хранящиеся в этих директориях будут использоваться. Конкретно, `-ldpi` указывает изображения, которые применяються на устройстве с экранами с низкой плотностью пикселей (около 120 dots-per-inch (точек на дюйм), или “dpi”)) Суффикс `-mdpi` указывает ресурсы для экранов со средней плотностью (около 160dpi), `-hdpi` указывает ресурсы для высокой плотности (около 240dpi). `-xhdpi` указывает ресурсы для экранов с экстра высокой плотностью (около 320dpi), `-xxhdpi` указывает ресурсы для экранов с экстра экстра высокой плотностью (около 480dpi), `-xxxhdpi` указывает ресурсы для экранов с экстра экстра экстра высокой плотностью (около 640dpi), и т.д.

В проекте `EmPubLite` руководства, Вы найдёте набор директорий `mipmap` с тем же набором суффиксов (пример `res/mipmap-hdpi`) В каждой из директории Вы увидите файл `ic_launcher.png` . Это стоковый значок , который будет использоваться для Вашего приложения в загрузчике домашнего экрана.

Каждое из изображение тот же значок, но с большей плотностью и содержит больше пикселей. Задача для изображения - иметь приблизительно одинаковый физический размер на каждом устройстве, применяя большую плотность для более детальных изображений.

Наш файл `AndroidManifest.xml` который ссылается на значок `ic_launcher` в элементе <application>

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.commonsware.empublite"
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

  <supports-screens
    android:largeScreens="true"
    android:normalScreens="true"
    android:smallScreens="false"
    android:xlargeScreens="true" />

  <application
    android:allowBackup="false"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:supportsRtl="true"
    android:theme="@style/AppTheme"
    tools:ignore="GoogleAppIndexingWarning">
    <activity android:name=".EmPubLiteActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

(из EmPubLite-AndroidStudio/T3-Manifest/EmPubLite/app/src/main/AndroidManifest.xml)

Обратите внимание, что манифест просто ссылается на `@mipmap/ic_launcher`, говоря Андроид найти mipmap ресурс с названием `ic_launcher`. Ссылка на ресурс не указывает тип файла ресурса - в идентификаторе ресурса нет `.png` . Это означает Вы не можете иметь `ic_launcher.png` и `ic_launcher.jpg` в одном и том же проекте, так они оба будут определятся по одинаковому идентификатору. Вам необходимо держать "базовое название" (имя файла без разрешения) различный для всех Ваших рисунков.

Также, ссылка `@mipmap/ic_launcher` не указывает какую плотность экрана использовать. Это потому, что *Андроид* выберет правильную плотность экоана для использования, основываясь на устройстве на котором запускается Ваше приложение. Вас не стоит беспокоится об этом явно, помимо наличия нескольких копий вашего значка. Если андроид определить, что плотность

экрана устройства

та, для которой нет значка, Андроид возьмёт следующий ближайший и масштабирует его.

Некоторое заметки о WebP

Андроид 4.0 добавляет частичную поддержку формата рисунка Google WebP и устройства Андроид 4.3 поддерживают ранее отсутствующую функцию (компрессию без потерь и прозрачность). WebP подаётся как замена и PNG и JPEG и при некоторых условиях может приводить в меньшему-на-диске размеру с почти-идентичному качеству изображения.

Android Studio начиная с версии 2.3, содержит специальную поддержку для помощи Вам в конвертировании ресурсов drawable и других изображений из JPEG и PNG в WebP. Просто щёлкните правой кнопкой над изображением в дереве проекта и выберите из контекстного меню "Convert to WebP" (конвертировать в WebP).

Вначале, Вам представится окно для контроля качества и вывода:

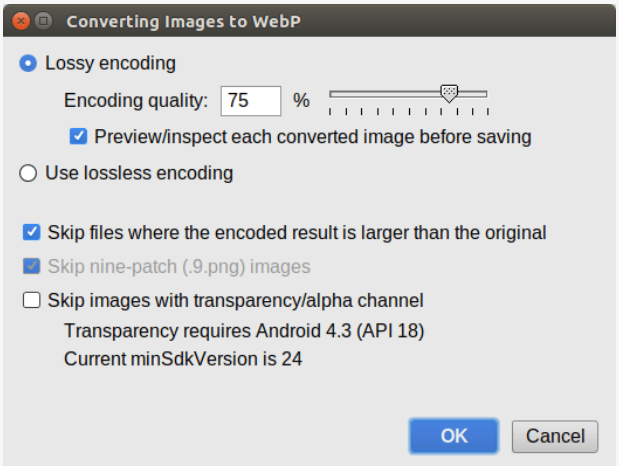


Рисунок 53: Конвертер WebP в Android Studio

"Кодировка с потерей" ссылается к типу, который представляет JPEG, с учётом того, что люди ограничены в возможностях распознавать похожие цвета для получения большего сжатия. "Кодировка без потерь" ссылается к типу, который представляет PNG, где сжатое изображение идентичное оригиналу, также как содержимое файла ZIP идентично файлам, перед тем как они были упакованы. Для кодировкой с потерей, Вы можете выбрать процент качества, когда изображения более высокого качества не сжаты так хорошо.

Вы можете также:

- Пропустить всё, что приводит к *большим* изображениям (так как иногда WebP будет больше, чем эквивалентные JPEG или PNG)
- Пропустить тип PNG, называемый nine-patch PNG (девяти патчевый), используемый для фонов виджета
- Пропустить изображения использующие прозрачность, в случаи, если Ваше `minSdkVersion` не поддерживает такие изображения

Если Вы выберите компрессию с потерей и оставите флажок “preview” (предпросмотр) нажатым, Вам представиться окно, отображающее результаты преобразования на Вашем запрашиваемом уровне качества:

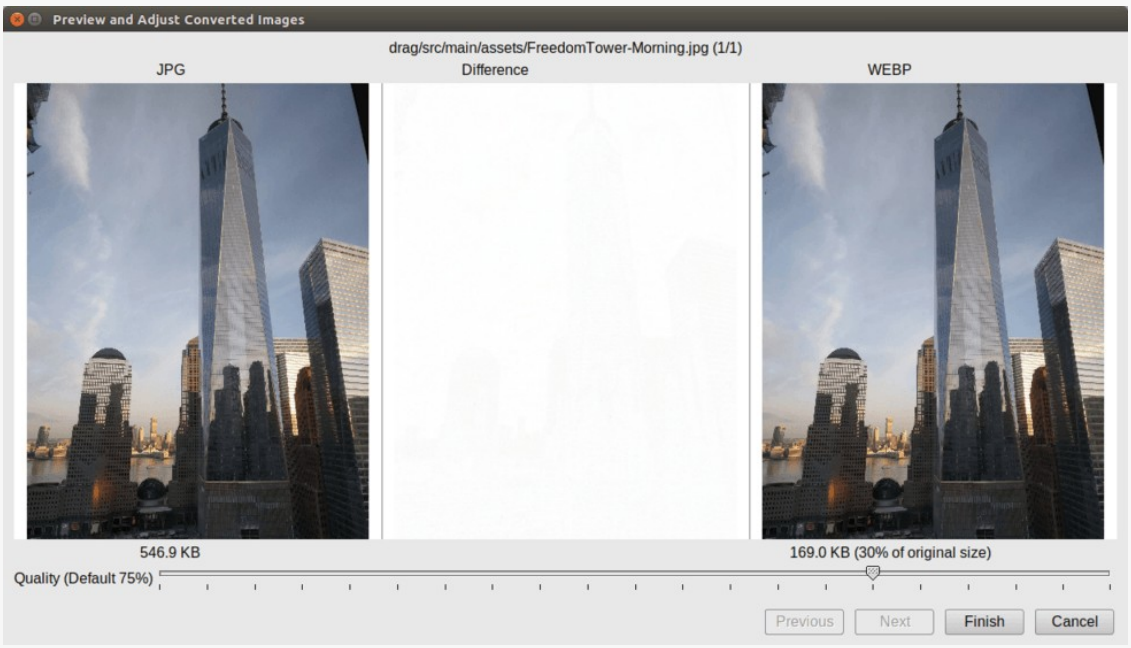


Рисунок 54: Предпросмотр преобразования WebP

Вы можете применить ползунок качества внизу изображения, чтобы увидеть изменения изображения с различными уровнями качества и какое количество дискового пространства сохраните при преобразовании в WebP.

Когда Вы закончите, преобразование WebP заменит Ваши старые фаайлы PNG или JPEG преобразованными изображениями WebP.

Размерности

Размерности применяются в нескольких местах в Андроид для описания расстояний, таких как

размеры виджетов. Несколько различных единиц измерения доступны для Вас:

1. `px` (pixels (пиксели)) означают аппаратные пиксели, размер которых изменяется устройством, так как не все устройства с одинаковой плотностью пикселей экрана
2. `in` и `mm` (inches и millimeters (дюймы и миллиметры)), соответственно основано на текущем размере экрана.
3. `pt` (points (точки)), термин типографии 1/72 дюйма (снова, основывается на текущем физическом размере экрана)
4. `dip` (or `dp`) (density-independent pixels (пиксели не зависящие от плотности)) - один `dip` равен одному аппаратному пикселю для разрешения экрана ~160dpi, но один `dip` равен двум аппаратным пикселям на экране ~320dpi
5. `sp` (scaled pixels (масштабируемые пиксели)), где один `sp` равен одному `dip` для нормального уровня масштабирования шрифта, увеличиваясь и уменьшаясь по необходимости основываясь на выборе пользователя уровня масштаба в Настройках

Ресурсы размерностей, по умолчанию, содержатся в файле `dimens.xml` в директории `res/values/`
которая также содержит Ваши строки.

Для кодировки размерности как ресурса, добавьте элемент `dimen` в `dimens.xml`, с атрибутом `name` (название) для Вашего уникального названия для данного ресурса и единичный текстовый элемент потомок представляющий значение:

```
<resources>
  <dimen name="thin">10dip</dimen>
  <dimen name="fat">1in</dimen>
</resources>
```

В layout (разметке) Вы можете ссылаться нп размерности как `@dimen/...`, где троеточие местоположение для Вашего уникального названия ресурса (пример `thin` и `fat` из примера Выше). В Java, Вы ссылаетесь на ресурсы размерности по уникальному названию с префиксом `R.dimen.` (пример, `Resources.getDimension(R.dimen.thin)`).

Хотя наш проект не использует ресурсы размерностей, мы скоро с ними столкнемся.

Редактирование ресурсов размерностей

Как и с большинством типов ресурсов XML, Android Studio позволяет

редактировать XML напрямую, при двойном щелчке на ресурсе в просмотрщике проекта.

Ресурсы, которые не должны именоваться... Пока

Ваш проект также содержит директорию `res/layout/` в дополнение тем, что описаны выше. Это для разметки UI, описывающих как должен выглядеть Ваш интерфейс пользователя.
Мы рассмотрим детально данный тип ресурсов когда начнём рассматривать наш интерфейс пользователя в следующей главе.

Значки

Значки это ресурсы `drawable` . За исключением если они не ресурсы `mipmap` . И за исключением если они не множественные ресурсы, комбинируемые вместе при выполнении для создания изображения с "квадрокруглым" фоном. И...

Вы ещё не запутались?

В теории, настройка значков для Вашего приложения будет лёгкой, и годами так и было. В настоящее время, настройка значков приложения излишне сложно, так что как минимум Android Studio содержит инструмент Asset Studio в попытке сделать это немного легче.

В данной главе, мы рассмотрим что необходимо для настройки одной из этих значков приложения.

Значки приложение... И все остальное

Каждое приложение имеет значок. Который используется в местах как список установленных приложений в Настройках. Традиционно - и чтобы уменьшить путаницу для пользователя - этот значок используется для загрузчика на домашнем экране.

Гугл сделал создание данного значка усложненным в последние несколько лет.

- Значок приложения ресурс `mipmap` , тогда как всё остальное ресурс `drawable`
- Андроид 7.1 внедряет концепт разделённых "круглых значков", которые может содержать приложение, которые будут использоваться в местах обычных значков приложения на конкретных устройствах Андроид 7.1... после убирает эту функцию с

Андроид 8.0

- Андроид 8.0 представляет концепт "адаптивных значков", где Вы должны предоставить отдельные изображения "переднего плана" и "фона", в большинстве чтобы загрузчик домашнего экрана мог сформировать фоновое изображение как хочет (квадрат, круг, "квадрокруг", и т.д.)

Большинство других значков - другие от значков уведомлений - сравнительно простые.

Создание закачка приложения с применением Asset Stud

Если Вы щёлкните правой кнопкой мышки на почти любой директории в дереве Android Studio появится контекстное меню с опцией “Image Asset” Это также доступно из File > New > Image Asset. Это запускает Asset Studio, чтобы помочь Вам собрать значки.

По умолчанию, Asset Studio содержит выпадающий набор “Icon Type” (тип значка) (Адаптивный или изначальный), таким образом Вы настраиваете значёк приложения в настоящее время:

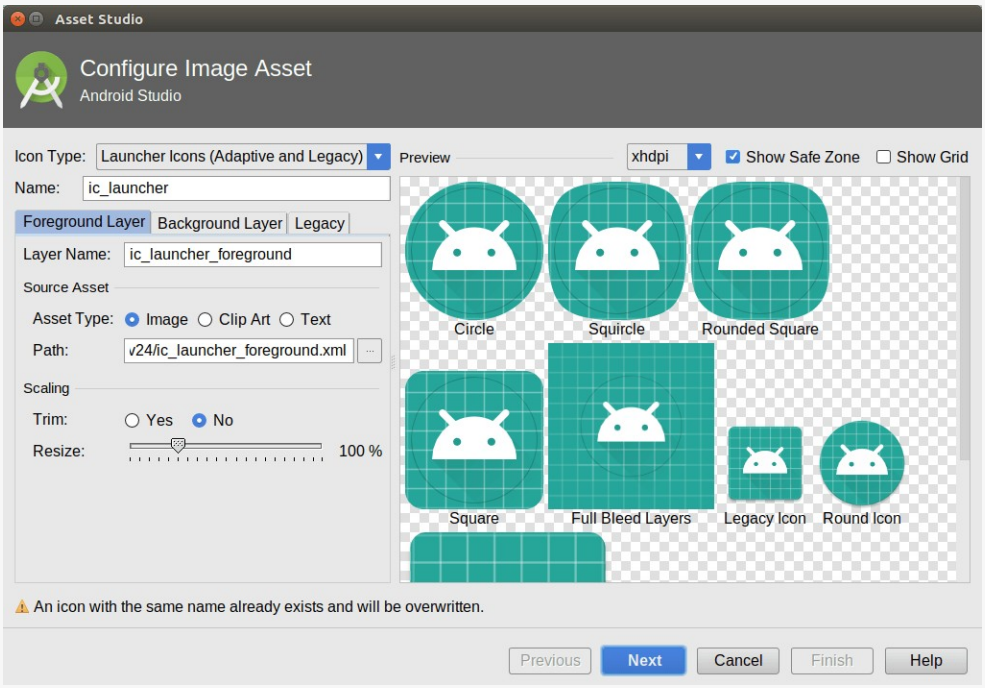


Рисунок 55: Asset Studio, начальный запуск

Общее название для значка загрузчика находиться в поле название над вкладками. По умолчанию `ic_launcher`, и до тех пор пока у Вас не появиться хорошая причина изменить его, лучше для Вас оставить его в покое.

Слой переднего плана

То про что Вы обычно думаете как значке - это то, на что Android Studio и

Андроид 8.0+ ссылается как “Foreground Layer” (слой переднего плана). Asset Studio начинает с вкладки Foreground Layer для настойки данного слоя.

Ваш значок может создаваться из трёх основных источников:

- Некоторое изображение созданное Вами, которое Вы загружаете во вкладку Foreground Layer, выбирая переключатель Image, после нажимая кнопку “...” возле поля Path, чтобы отобразить Вашу машину для разработки и найти это изображение.
- Часть сохранённого клип арта, который Вы можете выбрать нажимая на переключатель “Clip Art”, нажатие на кнопку “Clip Art” выбираете изображение, и нажатием на кнопку Color выбираете цвет применяемый к изображению.

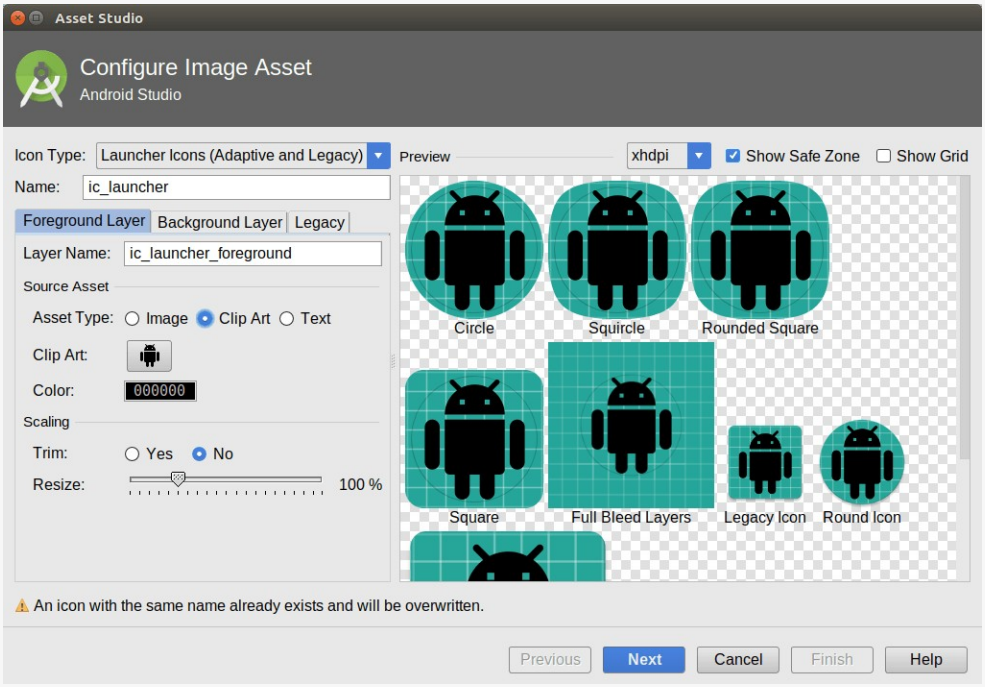


Рисунок 56: Asset Studio, Отображение опций Clip Art

- Буква или две, что Вы выбираете нажимая на переключатель Text, заполняя в 1-2 буквы в Текстовом поле, выбирая шрифт на Вашей машине разработки в соседнев выпадающем меню и на нажимая на кнопку Color для выбора цвета применяемого к шрифту:

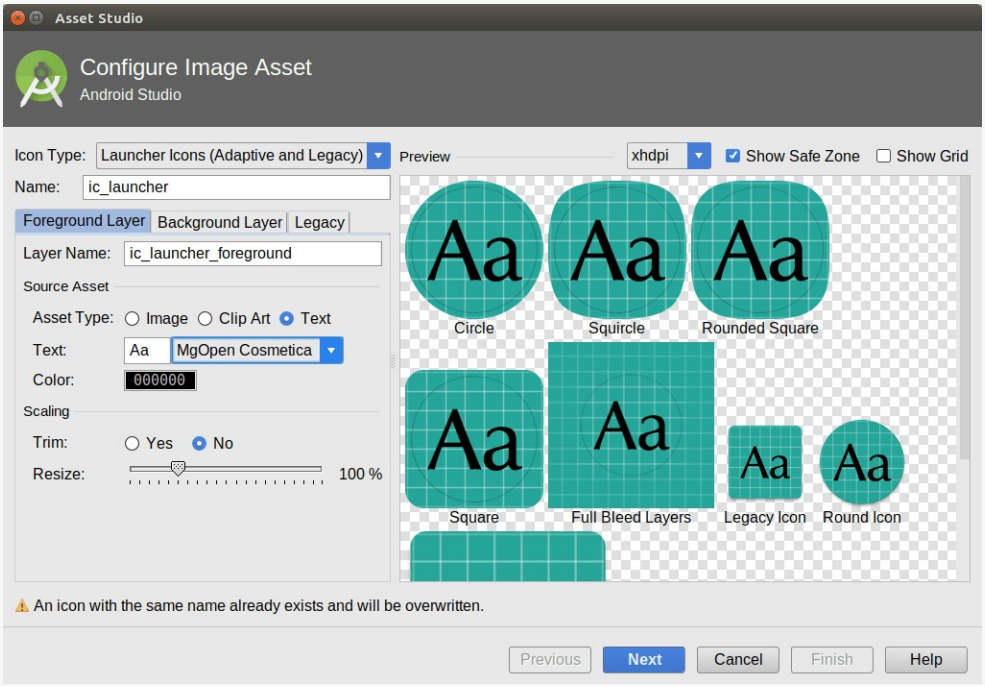


Рисунок 57: Asset Studio, Отображение текстовых опций

Для каждого из трёх Вы можете:

- Выберите название для использования для данного изображения в поле “Layer Name
- Выберите через переключатель Trim, убрать ли всё прозрачное пространство из границ изображения (“Yes”) или (“No”)
- Измените размер изображение из его размера по умолчанию

Это последний выбор особенно важный, если Вам необходимо держать содержимое слоя переднего плана внутри “safe zone” (безопасной зоны). Это отобразиться в превью как темно серый круг. До тех пор пока слой переднего плана в безопасной зоне, Вам не стоит переживать что загрузчик случайно обрежет часть слоя при рендеринге Вашего значка приложения.

Слой фона

В области предпросмотра, по умолчанию, Вы увидите сине-зелёную сетку позади слоя переднего плана. Это по умолчанию фоновый слой. Если Вы хотите использовать что-то другое, нажмите на вкладку “Background Layer”:

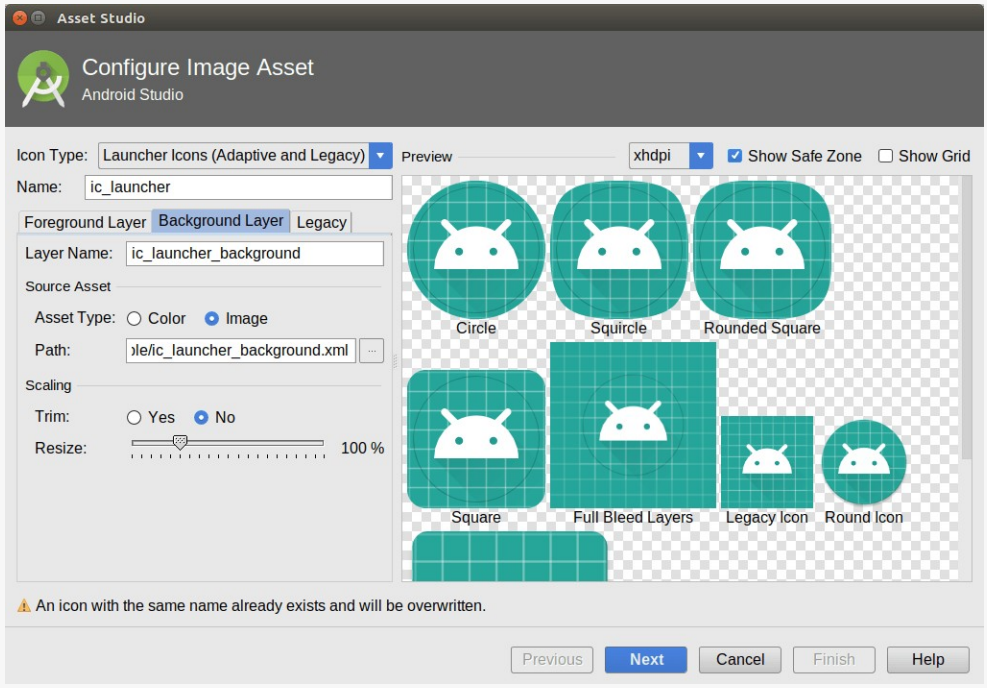


РИсунок 58: Asset Studio, отображение фонового слоя

Ваши две основные опции:

- простой цвет
- изображение по выбору (также как для переднего плана, когда Вы нажимаете кнопку “...” рядом с полем Path, чтобы выбрать фоновое изображение)

Фон должен создаваться с учётом обрезки в различные формы, которые отображены в предпросмотре (круг, различные скруглённые формы квадрата, и т.д.). Следовательно, кроме простого цвета, обычные фоны будут градиентами или простыми паттернами, как на сетке по умолчанию.

Изначальный

На устройствах Андроид 8.0+ слой переднего плана, фоновый слой и форма выбираемая

загрузчиком

(пример квадратокруг) комбинируются для создания значка приложения.

На более старых устройствах, значок приложения это что Вы выберите, и вкладка Legacy поможет Вам выбрать что это:

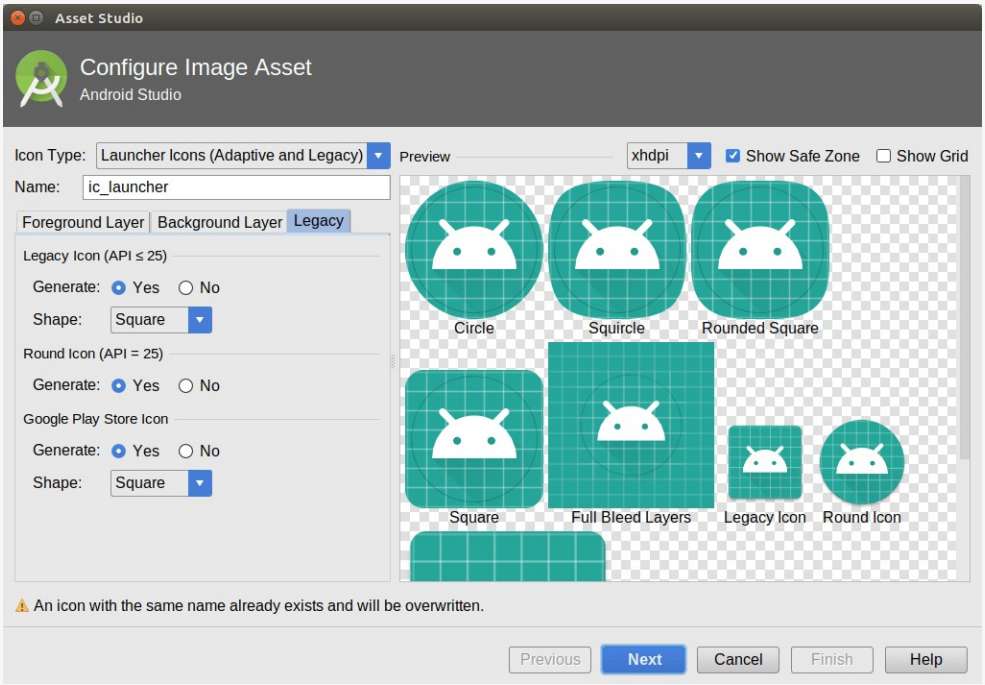


Рисунок 59: Asset Studio, отображение опций Legacy

Наиболее важная опция legacy - раздел “Legacy Icon (API <= 25)” Если Ваш `minSdkVersion` ниже 26 это будет изображение значка, которое будет использоваться как значок приложения по умолчанию. Asset Studio соединяет слои переднего плана, фона после применяет выбранную форму из выпадающего меню (пример квадрат).

Для устройств Андроид 7.1 также есть опция Asset Studio создать отдельный круглый значок, который Вы можете указать в манифест файле, как указано далее в главе.

Если Вы собираетесь распространять Ваше приложение через Play Store, Вы также можете сгенерировать изображение Play Store Вашего значка. Это напоминает изначальный значок , но в более высоком разрешении.

Генерация значков

После того как Вы установите Ваш значок приложения через три вкладки, нажмите на кнопку Next (далее) в мастере Asset Studio. Что перенесёт Вас на финальную страницу мастера, отображающую Вам что будет сгенерировано Вам мастером.

Дерево Output Directories (выходные директории) отобразит Вам каждый файл, который будет создан или заменён. Те что отображены красным будут заменены, те что отображены чёрным - новые файлы. Обычно, до тех пор пока Вы не измените названия слоя или значка, большинство файлов будет заменой файлов, которые уже существуют.

Некоторые из этих файлов будут типичными ресурсами bitmap стиля. Некоторые vector drawables (векторными изображениями)

Использование в Вашем манифесте.

После, в Вашем манифест, у Вас могут быть атрибуты `android:icon` и возможно `android:roundIcon` в элементе `<application>` чтобы связать Ваши значки с Вашим приложением. Если Вы не меняли названия значков, тогда Ваш манифест уже должен содержать подходящие значения атрибутов.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest package="com.commonsware.myapplication"
  xmlns:android="http://schemas.android.com/apk/res/android">

  <application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/AppTheme">
    <activity android:name=".MainActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

Создание других значков с применением Asset Studio

В принципе Вы можете использовать Asset Studio для создания других типов значков, выбирая другой тип значков из выпадающего меню, таких как “Action Bar and Tab Icons”.

На практике, Asset Studio не делает много для Вас здесь, кроме как создаёт множественные версии Вашего значка для различных плотностей экрана.

Для большинства разработчиков приложений в Андроид, имеется две опции:

1. Получить значки в правильном разрешении от Вашего графического дизайнера, возможно экспортированные из Adobe Photoshop
2. Создать векторные значки, как мы рассмотрим в дальнейшей главе.

Руководство №4 - Установка наших ресурсов

Наш проект `EmPubLite` содержит некоторые изначальные ресурсы. Тем не менее, по умолчанию это не то, что нам хочется в долгосрочном периоде. Так в дополнение для добавления новых ресурсов в будущих руководствах, мы исправим те, что уже имеем в данном руководстве.

Это продолжение работы, выполненной в предыдущем руководстве.

Вы можете найти результаты предыдущего руководства и результаты данного руководства в GitHub репозитории книги.

Шаг №1: Изменение имён

Наше приложение отображается везде как “EmPubLite”:

- В стоке заголовка нашей активности
- Как подпись под нашим значком на домашнем экране загрузчика
- В списке приложений а приложении Настройки
- И так далее.

Мы изменим это на “EmPub Lite”, добавив пробел для более лёгкого чтения, и чтобы проиллюстрировать, что это “lite” (облегчённая) версия полного приложения `EmPub`

Дважды щёлкните на файл `res/values/strings.xml` в нашем просмотрщике проекта.

В редакторе XML для строковых ресурсов, Вы найдёте элемент, который выглядит как:

```
<string name="app_name">EmPubLite</string>
```

Измените текстовый узел данного элемента на `EmPub Lite`. Далее сохраните Ваши изменен
получая

что-то похожее:

```
<resources>
  <string name="app_name">EmPub Lite</string>
</resources>
```

Шаг №2: Изменение значка

Инструмент сборки предоставляет нам стоковый значок для использования в загрузчике - актуальное изображение варьируется от релиза инструментов Андроид. Тем не менее мы можем изменить его на что-то другое. К примеру мы можем использовать часть значка от логотипа CommonsWare:

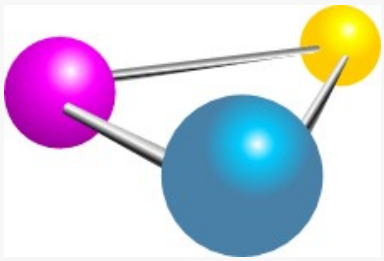


Рисунок 60: CommonsWare

Загрузите PNG файл молекулы из веб сайта CommonsWare и сохраните его где-то на машине разработчика.

После правый щелчок на директории `res/` в вашем `main` наборе ресурсов в просмотрщике проекта, и выберите New > Image Asset из контекстного меню. Это запустит мастер Asset Studio:

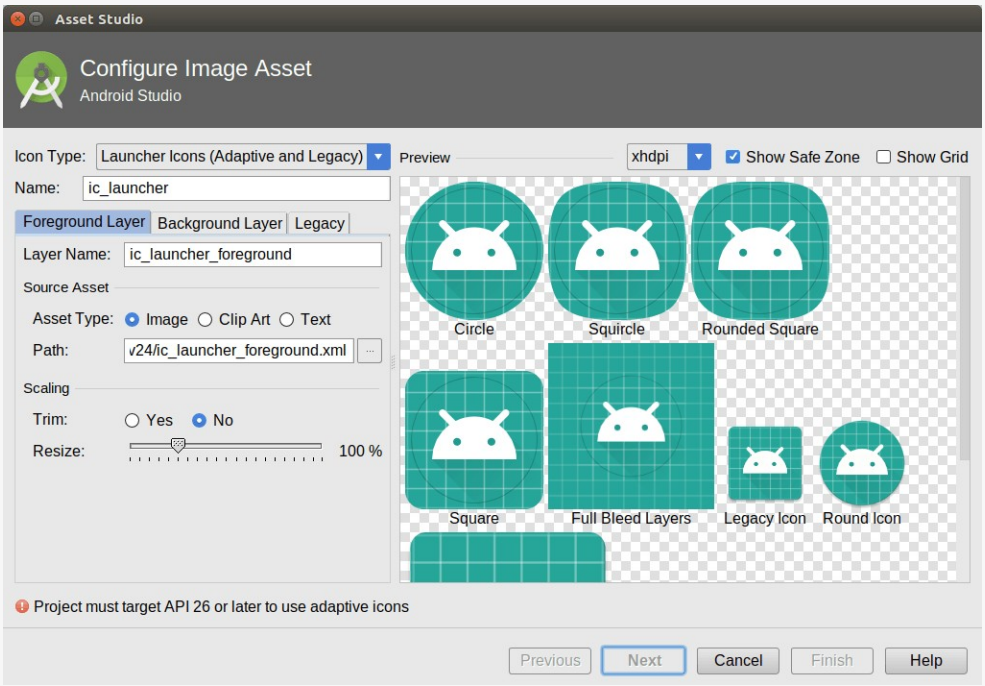


Рисунок 61: Мастер Asset Studio, Первая страница

В выпадающем меню Icon Type (тип значка) выберите “Launcher Icons (Legacy only)” (значки загрузчика). Это изменит мастер к следующему виду:

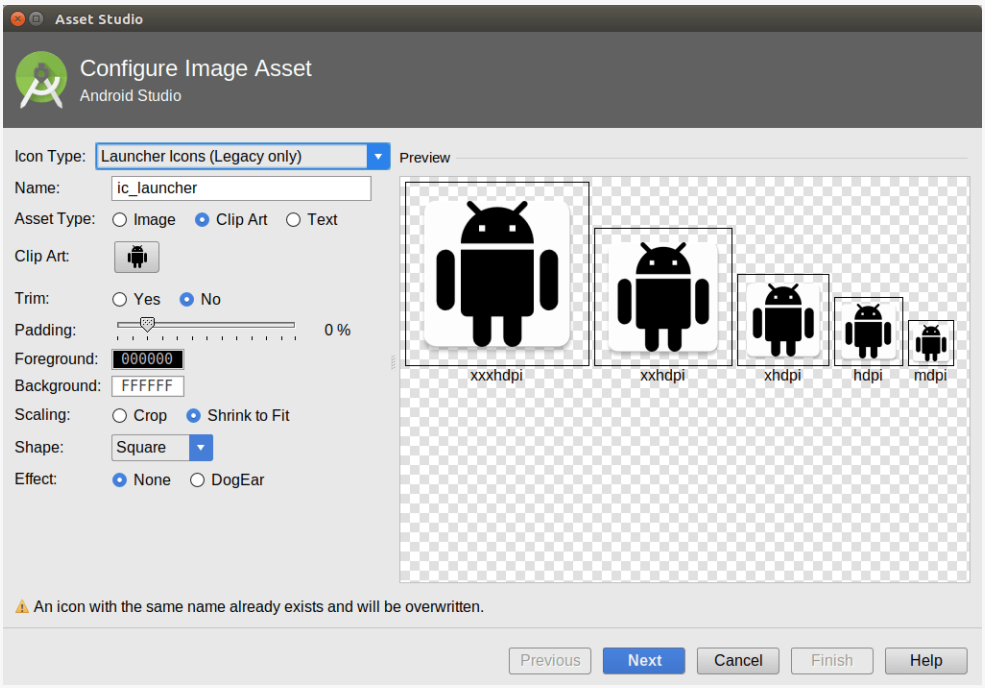


Рисунок 62: Мастер Asset Studio, Первая страница, режим Legacy

Нажмите на переключатель “Image” в стоке “Asset Type” После, нажмите на “...” справа от поля “Path” и выберите файл `molecule.png` который Вы загрузили. Также убедитесь, что “Scaling” установлено на “Shrink to Fit”, и выберите “None” из выпадающего меню “Shape”. Что должно привести Вас к предпросмотру как будет выглядеть значок:

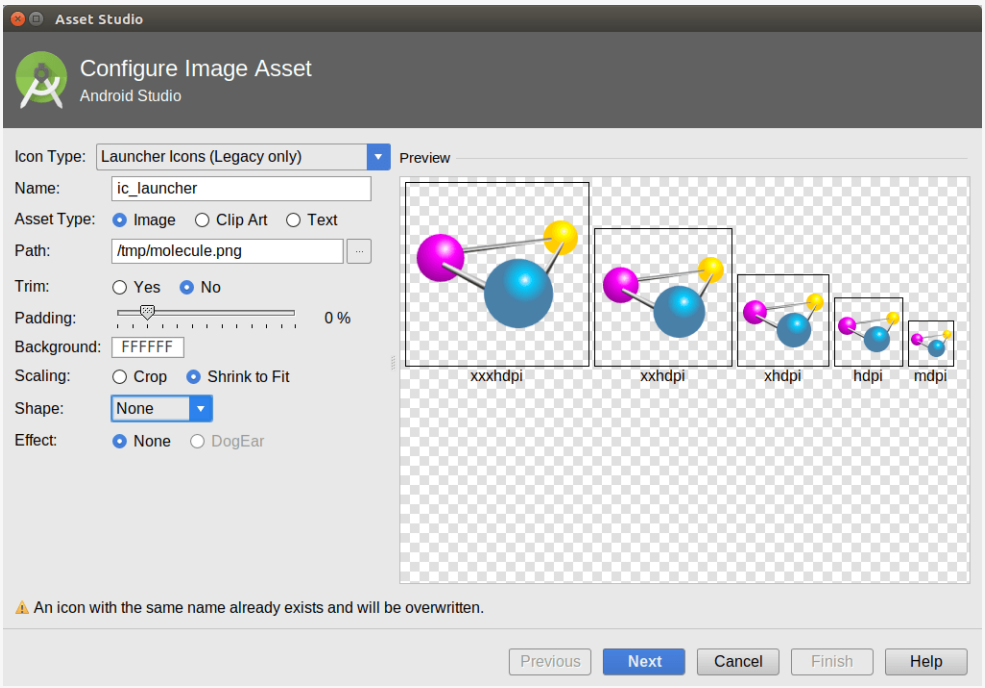


Рисунок 63: Мастер Asset Studio, Первая страница, после загрузки

Оставьте все остальное в мастере как есть и нажмите Next, что пройти к следующей страни

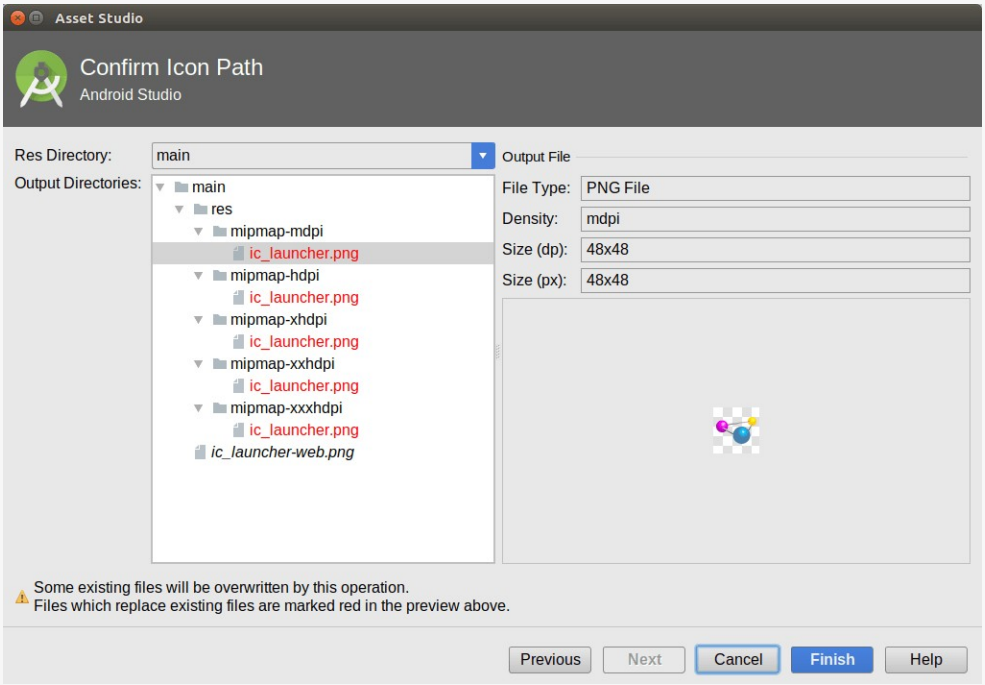


Рисунок 64: Мастер Asset Studio, Вторая страница

Вы должны получить предупреждение вверху, указывающее, что если Вы завершите мастер, то перепишете существующие файлы. Это ожидаемо, так как мы пытаемся заменить старые файлы `ic_launcher.png` новыми. Так пройдите далее и нажмите Finish.

Шаг №3: Запуск результатов

Если Вы запустите результирующее приложение, Вы увидите, что оно отображается с новым названием и значком, как в загрузчике:

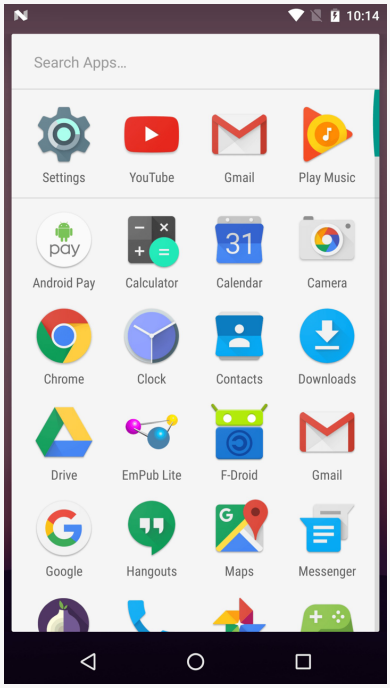


Рисунок 65: EmPubLite с новыми значками

В нашем следующем эпизоде...

... мы начнём изменять основной интерфейс пользователя нашего учебного проекта.

Теория виджетов

Есть не малый шанс, что Вы уже выполняли работы с основанными на виджетах фреймворках пользовательского интерфейса. В таком случаи, большая часть данной главы будет повторением, несмотря на это рассмотрите раздел анти паттерн абсолютного позиционирования это должно стоит того.

Есть шанс, также, что Ваш опыт пользовательского интерфейса, происходит из того, что Вы не использовали стандартный фреймворк виджетов, где Вы выполняли все отрисовки самостоятельно (пример игровой фреймворк) или где пользовательский интерфейс описывался больше в форме документа (пример классическая веб разработка). Эта глава направлена на вас, чтобы дать вам некоторое представление о том, о чем мы говорим, при обсуждении понятия виджетов и контейнеров.

Что такое виджеты?

Википедия содержит хорошее описание виджета:

> В компьютерном программировании, виджет (или контрол) элемент графического интерфейса пользователя (ГИП), который отображает информационное представление изменяемое пользователем, такие как окно или текстовое поле. Описательная характеристика виджета это предоставлять единичную точку взаимодействия для прямого воздействия на данный тип данных. Другими словами, виджеты - это базовые визуальные строительные блоки, которые комбинируя в приложении, содержат все обработку данных приложением и доступными взаимодействиями с этими данными.

(цитата из версии страницы от 7 Марта 2014)

Возьмите для примера, данный экран Андроид:

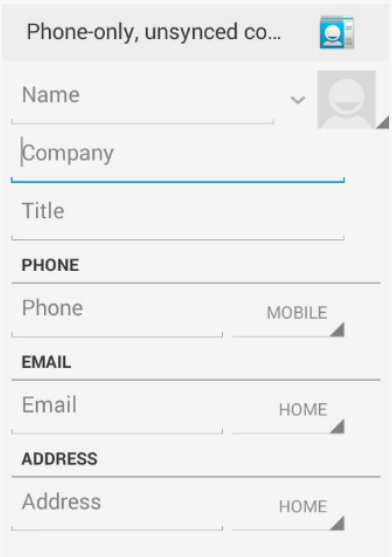


Рисунок 66: Простой экран Андроид

Здесь, мы видим:

- некоторый текст "Только телефон, несинхронизированное со..." и "ТЕЛЕФОН"
- и значок карты контакта "Rolodex"
- некоторые поля входных данных такие как "Имя" и "Компания"
- некоторый выпадающий список "spinner" (элементы со стрелкой указывающей на юго-восток)
- некоторые серые линии разделения

Всё указанное выше это виджеты. Пользовательский интерфейс для большинства экранов Андроид ("активности") сделаны из одного или более виджетов.

Это не означает, что Вы не можете использовать Ваши собственные отрисовки. Фактически, все существующие виджеты имплементируются через низко уровневую отрисовку шаблона, который Вы можете использовать везде от своих собственных пользовательских виджетов до игр.

Это также не означает, что вы не можете использовать веб технологии. Фактически, мы увидим это дальше в книге, виджет разработан позволяющим Вам встраивать веб контент в активность Андроид.

Тем не менее, для большинства не игровых приложений, Ваш пользовательский интерфейс Андроид будет создан

из нескольких виджетов.

Size, Margins, and Padding (Размеры, поля и отступы)

Виджеты имеют какой-то размер, поскольку высота от нулевого пикселя, и ширина от нулевого пикселя не особенно удобна. Иногда, данный размер будет продиктован тем что находится внутри виджета, как подпись (`TextView`) имеет размер продиктованный текстом подписи. Иногда, этот размер продиктован размером того что содержит виджет (контейнер, описанный в следующей части), где виджет стремится занят оставшуюся ширину или высоту. Иногда, размеры будут специфическим набором размерностей.

Виджет может содержать поля. Как и с CSS, поля представляют собой разделение между виджетами и чем-либо прилегающим к нему (пример другие виджеты, границы экрана). Поля в действительности спроектированы, чтобы помочь предотвратить запуск виджетов сразу после предыдущего, так они визуально различимы. Некоторые разработчики, тем не менее, пробуют применять поля как способ хакнуть "абсолютное позиционирование", что является анти-паттерном которые будет рассмотрен позже в главе.

Виджет может иметь отступы. Как и с CSS, отступы представляют собой разделение между содержимым виджета и границами виджета. Это в основном используется с виджетами, которые имеют некоторый тип фона, как кнопка, что содержимое виджета (надпись кнопки) не находится на границах кнопки, снова для визуального различия.

Что такое контейнеры?

Контейнеры - это способ организации многих виджетов в какой-то тип структуры. Виджеты не располагаются самостоятельно в какой-либо определённый паттерн - мы должны определить паттерн самостоятельно.

В большинстве наборов инструментов ГИП, контейнер предполагает наличие набора потомков. Эти потомки - виджеты или иногда другие контейнеры. Каждый контейнер имеет собственное правило, как размещать своих потомков на экране, возможно с настройками по запросу от потомков.

Распространённые паттерны контейнеров включают:

- расположить всех потомков в строку, один после второго
- расположить всех потомков в столбец, один под вторым

- расположить потомки в таблицу или сетку с некоторым количеством строк и столбцов.
- прикрепить потомков к сторонам контейнера, исходя из запроса сделанного этими потомками.
- прикрепить потомка к другому потомку в контейнере, исходя из запроса сделанного этим потомком
- сложить всех потомков один над другим
- и так далее.

В простой активности выше доминирующий паттерн это столбец, с элементами расположенными сверху вниз. Некоторые из этих элементов строки с содержимым расположенными слева направо. Тем не менее, когда они заканчиваются, пространство с большинством этих виджетов может прокручивается.

Андроид поддерживает набор контейнеров, разработанных обрабатывать большинство распространённых сценариев, включая всё в списке выше. Вы также можете создавать Ваши собственные пользовательские контейнеры, для реализации бизнес правил, которые напрямую не поддерживаются существующими контейнерами.

Отметьте, что контейнеры также имеют размер, поля и отступы как и виджеты.

Антипаттерн абсолютного позиционирования

Вы можете задаваться вопросом почему все эти контейнеры и такие необходимые. В конце концов, не можете ли Вы просто сказать такой-то виджет в таких пиксельных координатах, и этот другой виджет в таких пиксельных координатах и т.д.

Многие разработчики выбирают эту возможность - известную как абсолютное позиционирование - годами, к их возможным сожалению.

К примеру, многие из Вас использовали Виндовс приложения, возвращаясь в 1990-е когда если вы изменяли размер окна приложения, приложение в действительности не реагировало на это хорошо. Вы могли расширить окно, и пользовательский интерфейс не менялся, за исключением больших пустых пространств справа и снизу окна. Это потому, что разработчики просто сказали, что такие-то виджеты идут в таких пиксельных координатах, и этот другой виджет идёт в таких пиксельных координатах, **независимо от актуального размера окна**.

В современной веб разработке, вы видите это в дебатах о фиксированном против адаптивного веб дизайна. Консенсус похоже в том, что адаптивный дизайн лучше, хотя зачастую он более сложный в разработке. Адаптивный веб дизайн может лучше реагировать на различные

размеры окна браузера, когда эти окна изменяют размер, потому что пользователь изменил размер окна браузера вручную, или потому что размер окна диктуется разрешением экрана устройства на котором просматривается веб страница. Фиксированный веб дизайн = эффективно говорящий что такой-то элемент на таких-то пиксельных координатах и т.д. - имеет тенденцию быть проще в разработке но более слабо адаптируется к различным размерам окна.

В мобильной разработке, особенно с Андроид, мы имеем широкий диапазон возможных разрешений экрана, от QVGA (320x240) до сверх 1080p (1920x1080), и много значений между. Более того, любой производитель устройств может создавать устройства с любым разрешением, с которым решат - не существует правил ограничивающих производителей к конкретным разрешениям. Соответственно, как разработчикам, иметь Андроид эквивалент адаптивного веб дизайна критично, и способ, которым вы можете выполнить это разумное применение контейнеров, избегающее абсолютное позиционирование. Контейнеры (и в меньшей мере виджеты) определяют как дополнительное пространство заполняется, если экраны становиться больше и больше.

Тема данного раздела: Темы

В веб разработке у нас есть таблица стилей для практически всего. Через такие файлы каскадных таблиц стилей (CSS) мы можем предусмотреть различные правила о том как наша вею страница должна выглядеть. Что включает:

- Установку отображения по умолчанию для определённых HTML тегов по имени тега (пример установка шрифта и размера для всех элементов `[h1>` и `<h2>`)
- Установка отображения для отдельных HTML элементов по классу или ID (пример установка ширины для отдельного `<div>` в определённое количество пикселей CSS)

В Андроид, эквивалентная концепция может быть найдена в стилях и темах. Стили - это набор значений для свойств (пример иметь цвет переднего плана красный). Что может быть применено к отдельному виджету (пример эта надпись должна перенимать этот стиль), или они могут быть настроены "темами", которые предоставляют отображение по умолчанию для всех видов виджетов и других элементов нашего пользовательского интерфейса.

Конечно, Вы не обязаны описывать какую-либо тему для Вашего приложения. Андроид предоставит Вам отображение по умолчанию без какой-либо отдельной темы. Это отображение варьируется через года, влияя на

визуальные основы различных Андроид виджетов. Данные темы именуются по тому как мы ссылаемся к ним: `Theme`, `Theme.Holo`, и `Theme.Material`.

В начале была "Тема", и это было нейтрально

Возвращаясь к Андроид 1.0, тема по умолчанию была известна как `Theme` Технически, все темы наследуются из `Theme`, также как поздние таблицы стилей CSS эффективно "наследуют" настройки установленные предыдущей таблицей стилей.

Пользовательский интерфейс `Theme` в частности выглядит так:

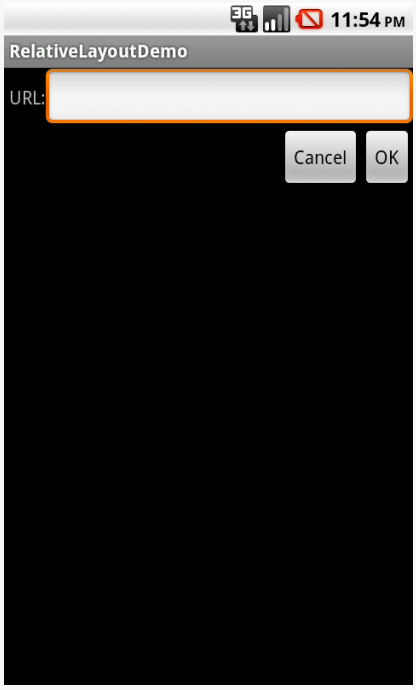


Рисунок 67: Надписи, поля, и кнопки Theme

Например:

- Вверху экрана тонкая серая “title bar” (полоска заголовка) с еазванием нашего приложения
- Поле на котором фокус (виджет `EditText`) имеет оранжевую рамку, когда обычно у него простой белый прямоугольник.
- Кнопки (“OK” и “Cancel”) ... просто... кнопки.

Holo, Там!

Андроид 3.0 (Уровень API 11) внедряет новую тему по умолчанию `Theme.Holo`, с вторым названием "голографическая тема виджетов". Это изменило отображение нашего интерфейса на такой:

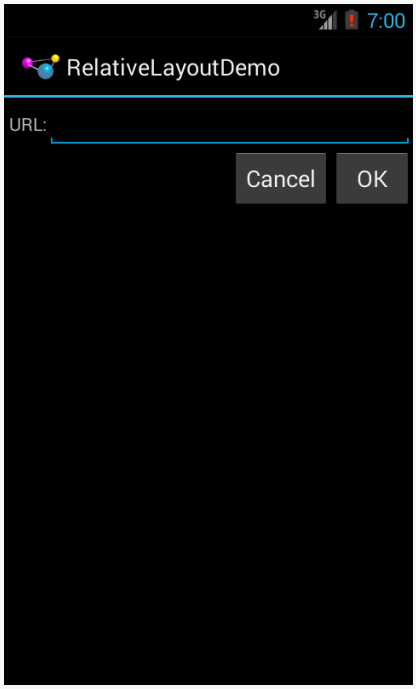


Рисунок 68: Надписи, поля, и кнопки Theme.Holo

Сейчас:

- Вверху экрана имеем “action bar” (активная полоска), содержащая лого приложения и название
- Поле на котором фокус имеет голубую "нижнюю скобку", которая обычно серая
- Стиль кнопок немного другой, с большим шрифтом, другим фоном, и т.д.

Учитывая Material

Андроид 5.0 изменил тему по умолчанию ещё раз к `Theme.Material`:

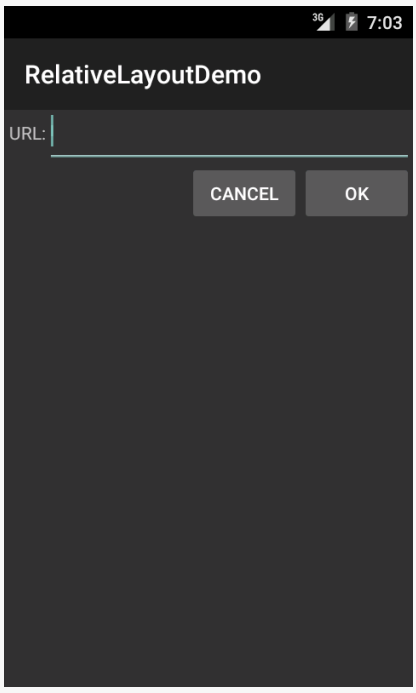


Рисунок 69: Надписи, поля, и кнопки Theme.Material

Сейчас:

- Активная полоска вверху экрана больше не показывает значок приложения
- Наши поля указываются подчёркиванием, который цвета чирок или серый когда нет фокуса.
- Кнопки сейчас принудительно с шрифтом со всеми заглавными, с слегка меньшим размером шрифта и тонко отличающимся фоном, чем был с `Theme.Holo`.

Делаем больше с Themes

Конечно, мы можем делать гораздо больше, чем просто использовать это. Существуют другие стоковые темы с другими характеристиками. Далее больше, мы можем кастомизировать темы, описывая наши собственные (наследуя от стоковой темы) и изменять некоторые свойства (пример заменить цвет чирок на какой-то другой).

Мы рассмотрим гораздо больше об создании кастомных стилей и тем позже в книге.

Тем не менее, мы увидим эффект `Theme`, `Theme.Holo`, и `Theme.Material` на стоковых виджетах в последующей главе.

Интерфейс пользователя Андроид

Проект, созданный в предыдущем руководстве были просто файлами по умолчанию сгенерированными инструментом сборки Андроид - Вы не писали какой-либо Java код самостоятельно. В данной главе мы рассмотрим базовый Java код и ресурсы, которые создают активность Андроид.

Активность (Activity)

Исходный Java код, который Вы поддерживаете будет в стандартном дереве директорий стиля Java основываясь на пакете Java, который Вы выбираете, когда создаёте проект (пример, `com.commonsware.android` в результате `com/commonsware/android/`). Android Studio содержит эти исходники по умолчанию в `app/src/main/java/` верхнего уровня дерева проекта.

Если в мастере нового проекта Вы выбрали создать активность, Вы получите во внутренней директории, файл исходного кода Java, представляющий класс активности.

Очень простая активность выглядит так:

```
package com.commonsware.empublite;

import android.app.Activity;
import android.os.Bundle;

public class EmPubLiteActivity extends Activity {

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
  }
}
```

(из EmPubLite-AndroidStudio/T2-Project/EmPubLite/app/src/main/java/com/commonsware/empublite/EmPubLiteActivity.java)

Анализируем активность

Давайте рассмотрим данный Java код кусок за куском:

```
package com.commonsware.empublite;

import android.app.Activity;
import android.os.Bundle;
```

(из EmPubLite-AndroidStudio/T2-Project/EmPubLite/app/src/main/java/com/commonsware/empublite/EmPubLiteActivity.java)

По умолчанию, описание проекта такое же как и то, что использовалось при создании проекта. И, как и в других проектах Java, Вам необходимо импортировать любые классы на которые ссылаетесь. Большинство классов специфичных для Андроид в пакете `android`.

Помните, что каждый класс Java SE доступны для Андроид программ. Посетите Android class reference(справочник Андроид класса) чтобы увидить, что доступно, а что нет.

```
public class EmPubLiteActivity extends Activity {
```

(из EmPubLite-AndroidStudio/T2-Project/EmPubLite/app/src/main/java/com/commonsware/empublite/EmPubLiteActivity.java)

Активности это публичные классы, наследованные от базового класса `android.app.Activity` (или, возможно от некоторого другого класса, который сам наследуется от `Activity`) Вы можете иметь какие-либо данных, которые решите, что вам нужно, хотя первоначальный код не имеет ничего.

```
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
```

(из EmPubLite-AndroidStudio/T2-Project/EmPubLite/app/src/main/java/com/commonsware/empublite/EmPubLiteActivity.java)

Метод `onCreate()` вызывается когда активность запускается. Мы обсудим параметр `Bundle` к `onCreate()` в поздних главах. В данный момент, рассматриваете его как непрозрачный обработчик, которые все активности получают при создании.

Первое что обычно Вы должны сделать в `onCreate()` это привязаться к суперклассу, так инициализация стоковой активности может быть завершена.

Единственное другое выражение в `onCreate()` нашего проекта вызов `setContentView()`. Это где мы говорим Андроид какой пользовательский интерфейс должен быть для нашей активности.

Это вызывает вопрос: Что значит `R.layout.main` ? Откуда взялось это `R` ?

Чтобы объяснить это, мы должны начать думать о ресурсах макетах и как на ресурсы ссылаются из Java кода.

Использования макетов (Layout) на основе XML

Как замечено в предыдущей главе, Андроид использует наборы виджетов и контейнеров для описания типичного пользовательского интерфейса. Это всё наследуется от базового класса `android.view.View` , для элементов, которые могут быть отображены в стандартной основанной на виджетах активности.

Так как технически возможно создать и прикрепить виджеты и контейнеры к нашей активности просто через Java код, более распространённый подход применять основанные на XML файлы макетов Динамическое создание экземпляров виджетов зарезервировано для более сложных сценариев, когда виджеты не известны во время компиляции (например заполнение столбца кнопками переключателями основываясь на данных полученных из интернета)

Держа это в памяти, время остановиться на XML и изучить как развернуть содержимое активности таким способом.

Сто такое макет основанный на XML?

Как предполагает название название основанный на XML макет - это спецификация соотношения виджетов друг к другу - и к контейнерам - закодированные в XML формат. Специфично, Андроид рассматривает основанные на XML макеты как ресурсы, и эти файлы макетов сохраняются в директории `res/layout/` внутри Вашего андроид проекта (или как мы увидим позже, другого набора ресурсов как `res/layout-land/` для макетов применяемых когда устройство в горизонтальном состоянии). Как было отмечено в другом месте книги, изначальное расположение `res/` в `app/src/main/` для Android Studio.

Каждый файл XML содержит дерево элементов определяющее макет виджетов и контейнеров, которые составляют одно `View`. Атрибуты XML элементов это свойства, описывающие как виджет должен выглядеть или как должен работать контейнер. К примеру, если элемент `Button` содержит значение атрибута `android:textStyle = "bold"`, это значит что текст, появляющийся на поверхности кнопки должен отображаться в стиле шрифта - жирный.

К примеру, здесь файл `res/layout/main.xml` который может быть использован в вышеуказанной активности:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout android:id="@+id/main"
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:paddingBottom="@dimen/activity_vertical_margin"
  android:paddingLeft="@dimen/activity_horizontal_margin"
  android:paddingRight="@dimen/activity_horizontal_margin"
  android:paddingTop="@dimen/activity_vertical_margin"
  tools:context="com.commonsware.empublite.EmPubLiteActivity">

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
</RelativeLayout>
```

(из EmPubLite-AndroidStudio/T2-Project/EmPubLite/app/src/main/res/layout/main.xml)

Название класса виджета или контейнера - такое как `RelativeLayout` или `TextView` – формирует название элемента XML. Так как `TextView` виджет поддерживаемый Андроид, мы можем использовать просто название класса. Если Вы создаёте сой собственный виджет как подкласс от `android.view.View`, Вам необходимо будет предоставить полное описание пакета (пример `com.commonsware.android.MyWidget`)

Корневой элемент должен объявлять XML пространство имён Андроид (`xmlns:android="http://schemas.android.com/apk/res/android"`). Все другие элементы будут потомками корня и будут наследовать это объявление пространства имён.

Атрибуты - это свойства виджета или контейнера, описывающие как он должен выглядеть или работать. К примеру, атрибуты `android:layout_width` и `android:layout_height` элемента `TextView` делают запрос от потомка (`TextView`) к родителю (`RelativeLayout`) кокой размер потомка должен быть.

Вы получите детали об этих атрибутах, их возможных значениях, и их использовании, в последующих главах. Отметьте, что те атрибуты которые в пространстве имён `tools` (пример `tools:context`) исключительно для поддержки инструмента сборки Андроид и не влияют на процесс выполнения Вашего проекта.

Андроид SDK поставляется с инструментом (**`aapt`**), который использует макеты. Этот инструмент будет автоматически вызван вашей цепочкой инструментов

Андроид (пример Android Studio). Определённая важность для Вас как разработчика, что **aapt** генерирует файл источников `R.java` .

позволяющий Вам получать доступ к макетам и виджетам в этих макетах напрямую из Вашего Java кода. Другими словами, это откуда магия значения `R` использующаяся в `setContentView()` приходит. Мы рассмотрим это немного более позже в данной главе.

Макеты XML и Ваше IDE

Если вы используете Android Studio, и дважды щёлкните на файле `res/layout/main.xml` в Вашем проекте, Вы изначально не увидите XML. Вместо этого Вы будете направлены в графических редактор макетов:

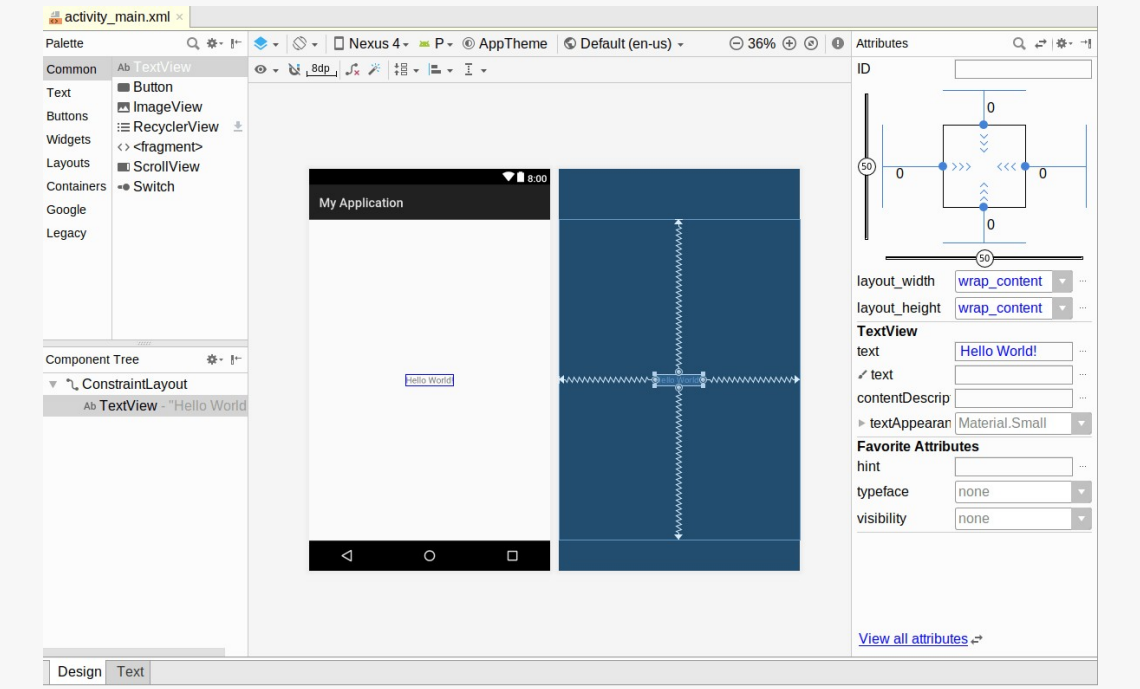


Рисунок 70: Android Studio Графический редактор макетов

Мы рассмотрим более детально о графическом редакторе макетов в последующей главе, когда мы начнём работать с определёнными виджетами и контейнерами.

Зачем использовать макеты основанные на XML?

Почти всё что делается с использованием макета XML можно достичь через Java код. Например, Вы можете использовать `setText()` чтобы кнопка отображала определённую надпись, вместо использования свойства в XML макете. Так как, XML макеты ещё один файл который вам отслеживать, нам

необходимы веские причины использовать такие файлы.

Возможно наибольшая причина это помощь в создании инструментов для описания отображения, как вышеупомянутый графический редактор макетов в Android Studio. Такие создатели ГИП могут, в принципе, генерировать Java код вместо XML. Проблема в повторном чтении описания поддерживаемых редактирований - это гораздо легче в структурированном формате как XML чем в языке программирования. Более того, хранение сгенерированных частей отдельно от написанного вручную кода создаёт меньшую вероятность, что чей-то созданный пользователем исходник будет случайно поломан при повторной генерации сгенерированных частей. XML формы это хорошая середина между тем что легко для создателей инструментов и легко для программистов для работы вручную по необходимости.

Также, XML как формат описания ГИП становиться более распространённым. Microsoft's XAML, Adobe's Flex, Google's GWT, и Mozilla's XUL все : берут аналогичный подход как и Андроид: располагать детали макета в файле XML и программные решения в файлах исходников (пример., JavaScript для XUL). Много менее известных фреймворков ГИП, такие как ZK, также используют XML для описание отображения. Хотя «Следуя за стадом» не обязательно является наилучшей политикой, у него есть преимущество в том, чтобы помочь облегчить переход на Андроид от любых других XML-центрированных языков описания отображения.

Использование макетов из Java

Имеем, что у Вас есть кропотливо настроенные виджеты и контейнеры для Вашего отображения в файле макета XML с названием `main.xml` сохранённом в `res/layout/`, всё что Вам необходимо одно утверждение в Вашем вызове `onCreate()` активности для использования данного макета, как мы видим в активности нашего проекта:

```
setContentView(R.layout.main);
```

Здесь, `R.layout.main` говорит Андроид загрузить макет (`layout`) ресурса (`R`) с названием main.xml (`main`).

Базовые виджеты

Каждый набор инструментов ГИП содержит некоторые базовые виджеты поля, надписи, кнопки и т.д. Набор инструментов Андроид не отличается в этом, и базовые виджеты предоставляют хорошее введение в то, как работают виджеты в активностях Андроид. Мы рассмотрим ряд из них в данной главе.

Общие концепции

Есть несколько ключевых функций виджетов, которые нам надо обсудить сначала, прежде чем мы погрузимся в детали отдельных типов виджетов.

Виджеты и атрибуты

Как замечено в предыдущей главе виджеты содержат атрибуты, которые описывают их поведение. В файле макета XML это буквально XML атрибуты в элементе виджета в файле. Обычно, они соответствуют методам get и set для манипулирования данными атрибутами при выполнении из Вашего кода Java.

Если вы посетите JavaDocs для виджета, как JavaDocs для `TextView`, Вы увидите почти наверху таблицу “XML Attributes”. Это список всех атрибутов определённых уникально в данном классе, и следующая таблица “Inherited XML Attributes” отображает список тех, которые виджет наследует от суперкласса, такого как `View`. Конечно, JavaDocs также предоставляет список полей, констант, конструкторов и public/protected методов, которые Вы можете использовать с виджетом.

Данная книга не пытается объяснить каждый атрибут каждого виджета. Мы, тем не менее, покрываем наиболее популярные виджеты и наиболее использующиеся атрибуты этих виджетов.

Ссылаемся на виджеты по ID

Многим виджетам и контейнерам необходимо появляться только в файле макета XML и нет необходимости ссылаться в Вашем коде Java. К примеру, статичная надпись (`TextView`) часто нужно быть только в файле макета для определения где ей появиться.

Всё, что Вы *хотите* использовать в Вашем исходном Java, тем не менее, нуждается в `android:id`.

Конвенции предполагают использование `@+id/...` как значение идентификатора (где представляется локально уникальное имя виджета) для первого случая нахождения данного значения `id` в Вашем файле макета. Второе и последующие случаи в том же файле макета должны пропускать знак `+` .

Андроид предоставляет несколько специальных значений `android:id` в форме `@android:id/...` - мы увидим некоторые из них в различных главах данной книги.

Для доступа к нашим идентифицированным виджетам, используйте `findViewById()`, передавая ему числовой идентификатор виджета в запросе. Этот числовой идентификатор будет сгенерирован Андроид в классе `R` как `R.id.что-то` (где что-то определённый виджет, который Вы ищите).

Данный концепт становиться важным, когда мы питаемся прикрепить слушатель к нашим виджетам (пример определение когда флажок нажат) или когда мы пытаемся ссылаться на виджеты из других виджетов в нашем файле макета XML (пример с `RelativeLayout`) Всё это будет рассмотрено позже в данной главе.

Любопытный случай

Большинство примеров кода, который вы увидите для Андроид будут отображать результат вызова
`findViewById()` приведённый к некоторому другому классу:

```
TextView tv=(TextView)findViewById(R.id.name);
```

Это потому, что для большинства сущностей в Андроид `findViewById()` возвращает `View`, так мы должны привести его к более подходящему классу.

Тем не менее, если Вы используете Android Studio 3.0+, с приложение с

`compileSdkVersion` 26 или выше, Вы будете уведомлены, что больше нет необходимости в таком приведении. Все существующие будут отображаться серым, со всплывающей подсказкой указывающей, что приведение ненужно.

Случилось то, что Андроид 8.0 изменил `findViewById()` для возврата `T`, используя Java дженерики для автоматического приведения его к типу данных, который Вы запрашиваете а утверждении.
Приведение это действия на этапе компиляции Java - они не появляются в скомпилированном коде и не влияют на выполнение. Как результат, код, который пропускает приведение работает идеально хорошо на более старых устройствах.

Размер

В большинстве случаев нам необходимо указать Андроид какими большими мы хотим чтобы наши виджеты были. Иногда, это управляется за нас - мы увидим примеры этого с `TableLayout` в последующих главах. Но в общем мы должны предоставить данную информацию сами.

Чтобы сделать это, Вам необходимо указать атрибуты `android:layout_width` и `android:layout_height` наших виджетов в файле макета XML. Данные значения атрибутов содержат три варианта:

1. Вы можете предоставить определённые размерности, как `125dip` чтобы указать, что виджет должен занимать точно определённый размер (здесь 125 независимых от плотности пикселей)
2. Вы можете предоставить `wrap_content`, что означает, что должен занять столько пространства, сколько требует его содержимое (пример содержимое надписи виджета `TextView` - текст который отображается)
3. Вы можете предоставить `match_parent`, что означает виджет должен заполнить все оставшиеся доступное пространство в заключаемом его контейнере.

Последние две возможности более частые, так как не зависят от размера экрана, позволяя Андроид устанавливать Ваше отображение по размерам доступного пространства.

Отметьте, что Вы также видите `fill_parent`. Это старый синоним для `match_parent`. `match_parent` - рекомендуемое значение в будущем, но `fill_parent` тоже работает.

Данная глава сфокусирована на отдельных виджетах. Размер становиться гораздо более важным, когда мы начинаем комбинировать множество виджетов на одном экране, и так мы проведём больше времени на сценариях установки размеров в последующих главах.

Префикс `layout_` на данных атрибутах означает, что данные атрибуты представляют запрос виджета к содержащему его контейнеру. Так как данные запросы зависят от того, какие остальные виджеты в контейнере и какие *их* запросы.

Введение в графический редактор макетов

Если Вы откроете ресурс макета в Android Studio, по умолчанию Вы увидите графический редактор макетов:

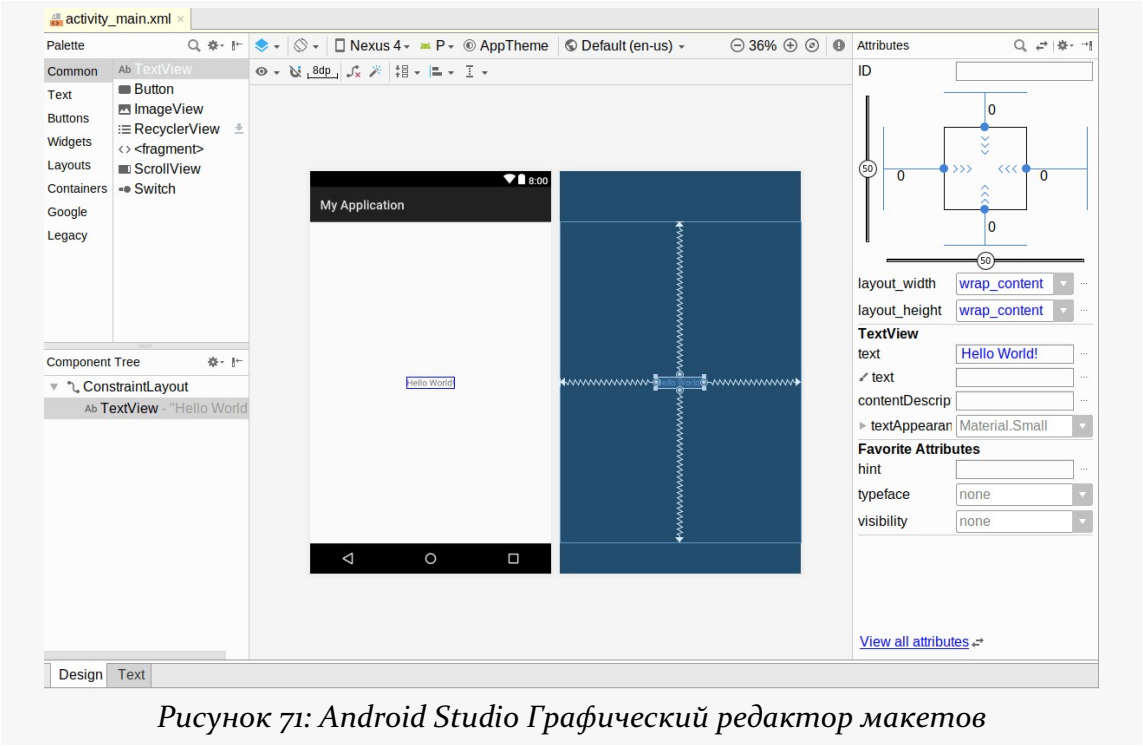


Рисунок 71: Android Studio Графический редактор макетов

Это позволяет перетаскивание для описания содержимого ресурса макета.

Android IDE содержит возможности построения ГИП перетаскиванием несколько лет, с времени когда Eclipse был официальным IDE. Тем не менее Android Studio 2.2 сделала некоторые существенные изменения в способе работы и внешнего вида построителя ГИП перетаскиванием, и последующие обновления изменили это дальше. Данная книга рассматривает текущий внешний вид, но более старые посты в блогах, ответы в Stack Overflow, и похожие ресурсы могут ссылаться на аспекты предыдущего построителя ГИП.

Учитывая это всё, давайте посмотрим на различные части графического редактора макетов

Палитра

Левая верхняя часть графического редактора макетов это инструмент Palette (Палитра):

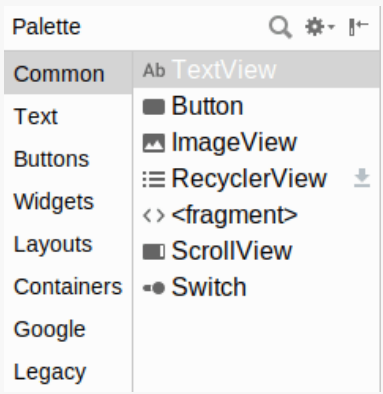


Рисунок 72: Инструмент Palette

Она содержит список всех виджетов и контейнеров, которые Вы можете перетаскивать. Они разделены в категории (“Widgets”, “Text”, “Layouts”, и т.д.) с многими опциями в каждой. Некоторые не строго виджеты или контейнеры, но скорее некоторый тип элементов XML, которые могут быть в ресурсе макета (пример `<fragment>`, `<requestFocus>`) Некоторые - такие как `RecyclerView` показанный в скриншоте выше - из библиотек и содержат значок "download" (загрузить) прикрепленный к ним чтобы проиллюстрировать это.

Когда мы рассмотрим как использовать графический редактор макетов, мы увидим как создавать и настраивать несколько из этих виджетов, контейнеров и других элементов.

Предпросмотр

Основное центральное пространство графического редактора макетов состоит из двух видов содержимого ресурса макета. Тот, что слева, предпросмотр того на что будет похож ваш интерфейс пользователя, если данный макет будет использован как интерфейс активности.

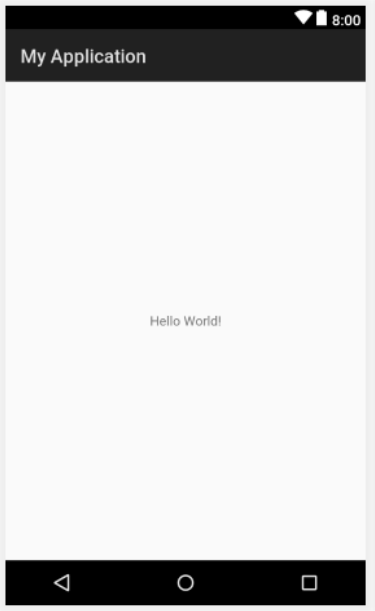


Рисунок 73: Предпросмотр интерфейса пользователя

Предпросмотр размещает содержимое ресурса макета в рамку предпросмотра, которая имеет те же аспекты, что и стандартное андроид устройство, какие как строку навигации внизу и строку состояния сверху.

Если Вы перенесёте элементы из палитры на предпросмотр, он будет добавлен к Вашему ресурсу макета.

Чертёж

Справа на области предпросмотра отображение чертежа. Это также визуальное изображение ресурса макета. Тем не менее, чем отображать Вам предпросмотр того как будет выглядеть Ваш интерфейс, он визуально отображает какие классы виджетов и контейнеров Вы используете. И для некоторых типов контейнеров, он отображает некоторые правила позиционирования и установки размеров, которые применяются для потомком данного контейнера.

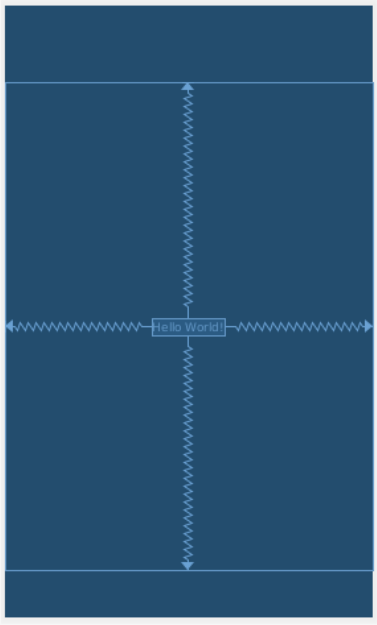


Рисунок 74: Чертёж

Для тривиального ресурса макета, отображение чертёжа не покажит Вам много. Он становиться более полезным с более сложными ресурсами макета. В частности, это очень полезно, когда Вы проектируете некоторые виджеты или контейнеры невидимыми, так они отображаются на чертеже и не отображаются на предпросмотре:

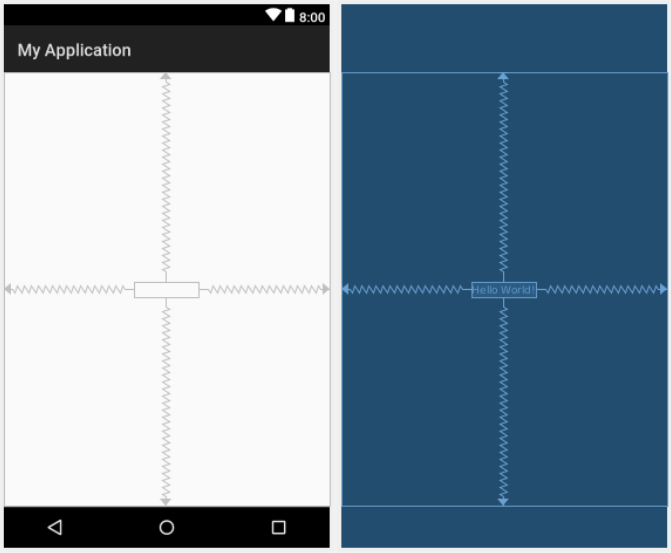


Рисунок 75: Ресурс макета с невидимым TextView

Панель инструментов предпросмотра

Над предпросмотром и чертежом двухуровневая панель инструментов, которая позволяет Вам настраивать различные аспекты отображение и поведения предпросмотра и чертежа.

Верхняя левая панель инструментов

Слева направо верхняя левая часть панели инструментов содержит:

- Выпадающее меню для переключения что вы видите предпросмотр, чертёж или оба
- Переключатель для управления видите ли вы макет применимо к портретной или альбомной перспективе
- Выпадающее меню для выбора кокой размер и разрешение устройства должно быть использовано для предпросмотра, отобранные из образов эмулятора и доступных описаний устройства
- Выпадающее меню для выбора какой уровень API должен использоваться для моделирования интерфейса в предпросмотре

- Кнопка для выбора какая тема используется для презентации интерфейса предпросмотра

- Кнопка для выбора какой язык использовать для определения какой строковый ресурс использовать в предпросмотре



Рисунок 76: Полоса инструментов предпросмотра, Верхний уровень

Некоторые из них - особенно выбор темы - относятся к темам, которые мы рассмотрим позже в книге.

Верхняя правая панель инструментов

В верхней правой части панели инструментов:

- Управление увеличением
- Кнопка для сброса увеличения к заполнению области доступной для предпросмотра и/или чертежа
- Значок "info" (информация) отображающий если ли какие-то предупреждения или ошибки связанные с Вашим макетом ресурсов.



Рисунок 77: Предпросмотр панели инструментов, Нижний уровень, правая

Левая сторона нижнего уровня панели инструментов предпросмотра изменяется, в зависимости от выбранного виджета или контейнера, предоставляя Вам возможность сделать простые изменения выбранного. Мы увидим примеры этого в следующих нескольких главах

Дерево компонентов

В направлении верхнего левого угла дерево компонентов:

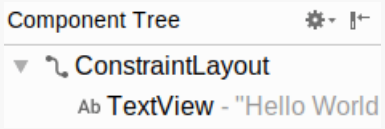


Рисунок 78: Дерево компонентов

Это предоставляет Вам полное дерево всех виджетов и контейнеров внутри данного ресурса макета. Что соответствует дереву элементов XML в самом ресурсе макета.

Щелчок на каком либо элементе в дереве компонентов подсвечивает его и в предпросмотре и в чертеже, плюс переключает на этот виджет или контейнер панель атрибутов.

Атрибуты

Когда виджет или контейнер выбран - через дерево компонентов, щелчком на предпросмотре или щелчком на чертеже - панель атрибутов справа позволит Вам изменять как виджет или контейнер выглядит или его поведение.

По умолчанию, это вызывает укороченный список самых важных атрибутов:

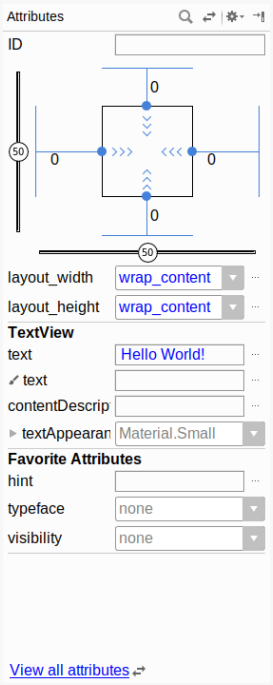


Рисунок 79: Панель атрибутов, Отображение короткого списка

Нажатие на ссылку“View all attributes” (отобразить все атрибуты) или кнопку панели инструментов с противоположными стрелками, переключит на список всех атрибутов:

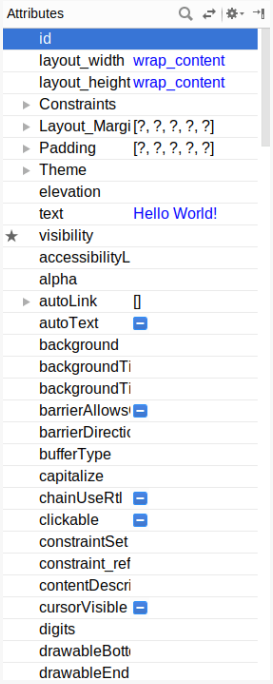


Рисунок 80:Панель атрибутов, Отображение полного списка

Вы также можете нажать на значок увеличительного стекла на панели инструментов данной панели для поиска доступных атрибутов по имени:

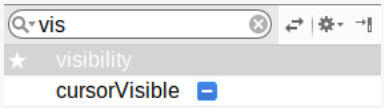


Рисунок 81: Панель атрибутов, Отображение результатов поиска.

Мы увидим что такое многие из этих атрибутов и как работать с ними в следующих нескольких главах.

Для атрибутов в полном списке, Вы можете начать на значок звезды слева и отметить их как “favorites” (избранные), как видно с атрибутом “visibility” (видимость) на скриншоте выше. Эти избранные атрибуты отображаются в коротком списке, в разделе с надписью “Favorite Attributes” (избранные атрибуты).

Вкладка текст

Снизу от инструмента графического макета, находится две подвкладки. Одна, "Design", охватывает все описанное выше. Другая “Text”, позволяет Вам редактировать сырой XML, который является действительным содержимым ресурса макета:



Рисунок 82: Под-вкладка Text редактора макетов.

По умолчанию всё пространство отведено текстовому редактору. Тем не менее, когда под-вкладка текст активна, появляется инструмент “Preview” прикреплённый справа в окне Android Studio. Нажатие на него отобразит предпросмотр из под-вкладки Design:

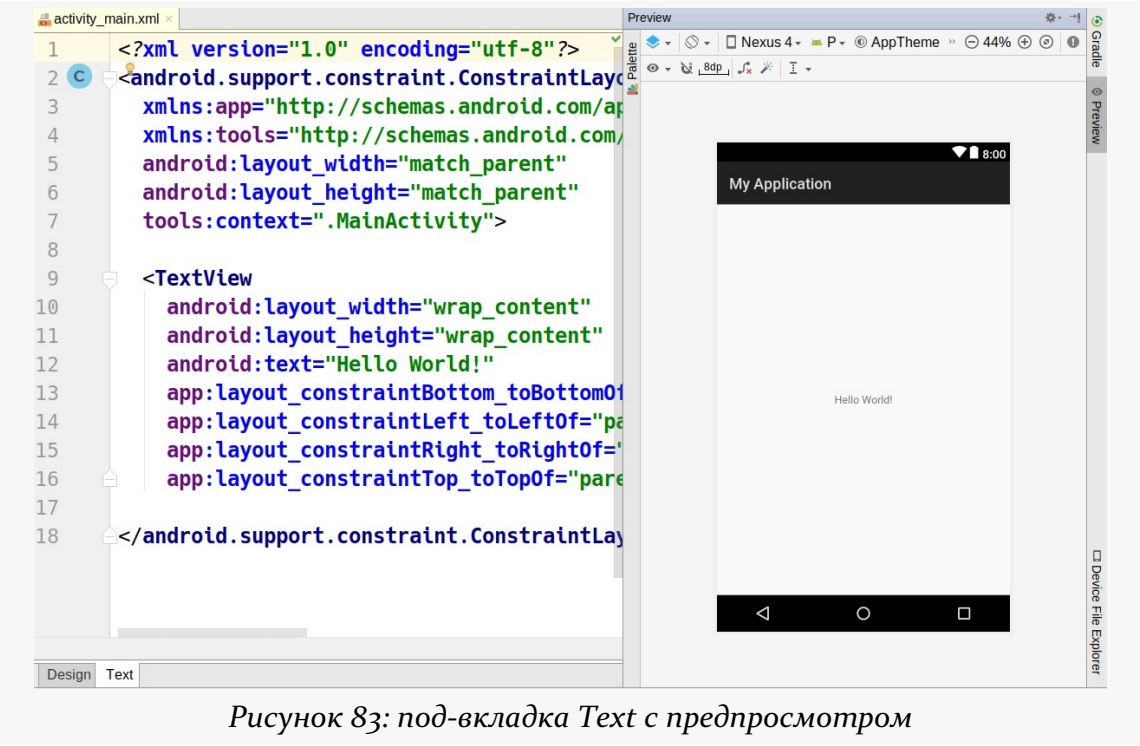


Рисунок 83: под-вкладка Text с предпросмотром

Нажатие на элемент в предпросмотре подсветит соответствующий элемент XML в текстовом редакторе.

И сейчас, несколько заметок о простых проектах в книге

Данная книга предоставляет сотни приложений примеров, демонстрирующих всё от того как отобразить текст на экране до того как сканировать NFC метки или работать с буфером обмена.

Данные приложения примеры хранятся в репозитории Git на GitHub. Все они с открытым исходным кодом и в *подавляющем* большинстве лицензированы по д лицензией Apache Software License 2.0, в случаи если Вы копируете какой-либо из кода.

Репозиторий Git использует метки, которые совпадают с номером версии книги (пример метка `v8.8` для Версии 8.8 книги). Следовательно для получения исходного кода, который соответствует версии Вашей книге

вы можете:

- Клонировать репозиторий (пример., `git clone https://github.com/commonsguy/cw- omnibus.git`) и после отметить определенную метку или
- Загрузить ZIP архив, ассоциируемый с определённой версией книги

Для тех проектов, которые Вы захотите изучить в Android Studio, Вы можете использовать File > New > Import Project для импорта проекта. Тем не менее, вы должны делать это проект за проектом, импортируя каждую директорию проекта по мере необходимости. Грубо говоря, каждая директория в которой есть `settings.gradle` внутри - это директория проекта, особенно в отношении примеров этой книги.

Начиная со следующего раздела, книга отображает код из этих проектов примеров. Ссылки укажут Вам необходимый проект примера или определённый файл в этом проекте, такой как ресурс макета или файл исходного кода Java. Эти ссылки ведут на репозиторий GitHub, но вы можете использовать эту информацию чтобы определить где соответствующий файл на Вашей машине разработчика, если Вы выбрали загрузить код.

Присвоение надписей.

Самый простой виджет - это надпись, упоминаемая в Андроид как `TextView`. Как и большинство инструментов ГИП, надпись является частью текста не редактируемая пользователем напрямую. Обычно, они применяются для идентификации прилегающего виджета (например надпись “Name:” перед полем где заполняется имя.)

В Java Вы можете создать надпись создавая экземпляр `TextView` . Более в общем, Вы можете создать надпись в файле макета XML добавляя элемент `TextView` в макет с атрибутом `android:text` для установки значения самой надписи. Если Вам необходимо менять надписи основываясь на определённых критериях, таких как интернационализация, ВЫ можете использовать строковый ресурс на который ссылаться в `android:text` (пример `@string/label`)

Для примера в нашем последнем руководстве мы использовали автоматически сгенерированный файл `res/ layout/main.xml` , содержащий среди других элементов `TextView`:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout android:id="@+id/main"
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
```

```
  android:layout_height="match_parent"
  android:paddingBottom="@dimen/activity_vertical_margin"
  android:paddingLeft="@dimen/activity_horizontal_margin"
  android:paddingRight="@dimen/activity_horizontal_margin"
  android:paddingTop="@dimen/activity_vertical_margin"
  tools:context="com.commonsware.empublite.EmPubLiteActivity">

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />

</RelativeLayout>
```

(из EmPubLite-AndroidStudio/T4-Resources/EmPubLite/app/src/main/res/layout/main.xml)

Графический редактор макетов Android Studio

Виджет `TextView` доступен в категории “Widgets” палитры в графическом редакторе макетов Android Studio:

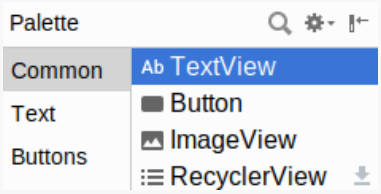


Рисунок 84: Палитра, “Plain TextView” в категории виджетов

Вы можете переместить `TextView` из палитры в файл макета в основной области редактирования чтобы добавить виджет в макет. Или переместить его над некоторым контейнером в панели дерева компонентов редактора, чтобы добавить его как потомка определённого контейнера.

Нажатие на новый `TextView` настроит панель атрибутов с различными атрибутами виджета, готовыми к изменениям по необходимости.

Редактирование теста

Атрибут “Text” позволяет Вам выбрать или определить строковый ресурс для отображения как текст:

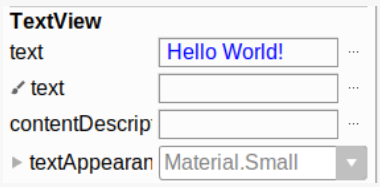


Рисунок 85: Панель атрибутов, Отображение атрибута TextView “text”

“text” со значком кисти позволяет Вам предоставить отдельную часть текста, которая отобразиться в предпросмотре, но не будет использована вашем приложением при работе.

Вы можете или ввести буквенную строку в строку панели атрибутов, или нажать кнопку “...” справа от поля для выбора строкового ресурса:

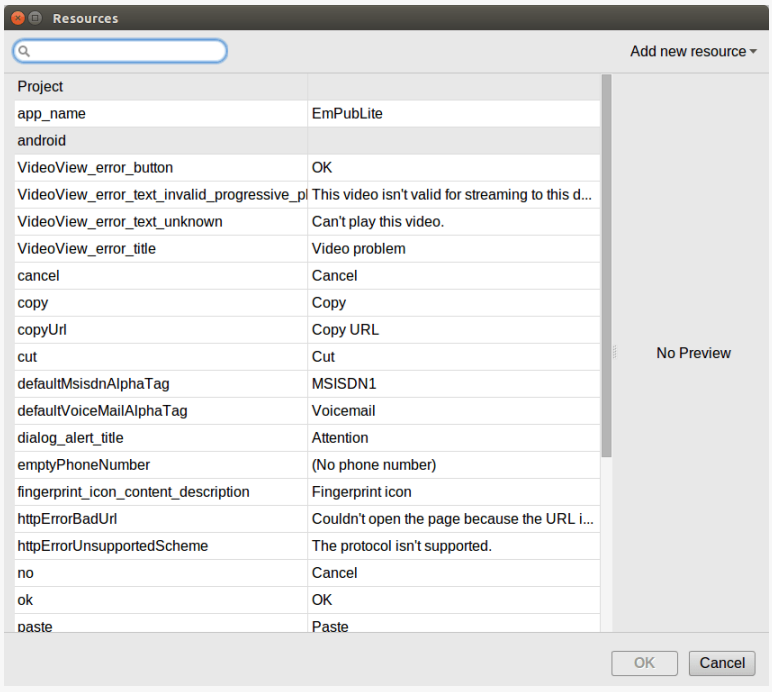


Рисунок 86: Диалог строкового ресурса

Вы можете выделить один из этих ресурсов и нажать “OK” для его использования. Или

в верхней правой части диалога находится выпадающее меню “Add new resource” (добавить новый ресурс). При просмотре строкового ресурса, выпадающее меню содержит единственную команду “New string Value...” (новое строковое значение). Выбор её позволит Вам определить новый строковый ресурс через другой диалог:

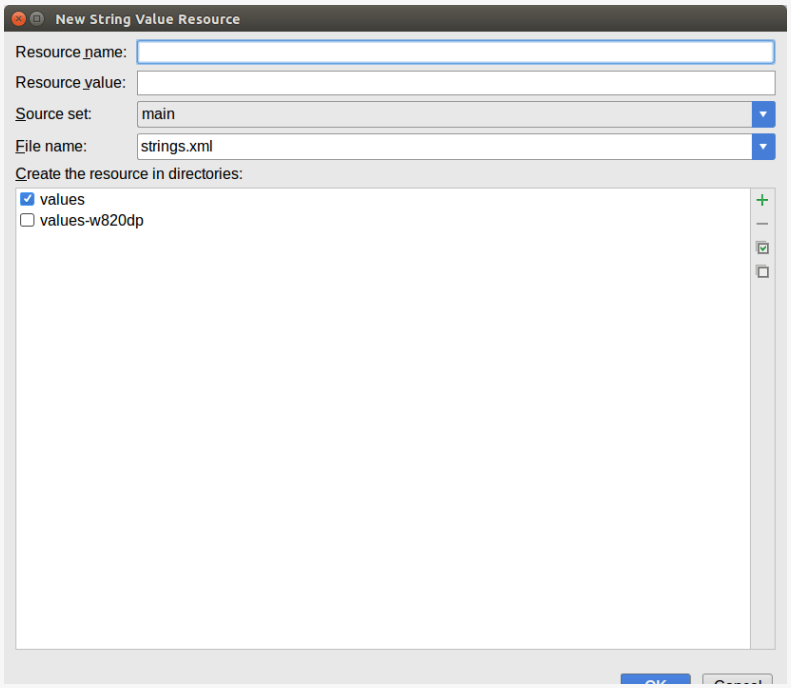


Рисунок 87: Диалог нового строкового ресурса

Вы можете присвоить новому строковому ресурсу название, непосредственно текст строки, название файла, где строковый ресурс должен находится (`strings.xml` по умолчанию), в какой директории `values/` будет находится строка (`values` по умолчанию). Вы также выбираете набор исходников - сейчас это будет просто `main`. Когда Вы подтвердите диалог, Ваш новый строковый ресурс будет добавлен к Вашему `TextView`.

Редактирование ID

Атрибут “ID” позволяет Вам изменять значение `android:id` виджета:



Рисунок 88: Панель атрибутов, Отображение поля ID

Значение которые вы заполняете здесь устанавливается после части `@+id/`

(пример `textView2`). Это работает для всех виджетов, не только `TextView`.

Некоторые атрибуты TextView

`TextView` содержит многие другие атрибуты относящиеся к надписи, такие как:

1. `android:typeface` для установки используемого шрифта надписи (пример `monospace`)
2. `android:textStyle` для указания что шрифт должен быть жирный (`bold`), курсив (`italic`), или жирный и курсив (`bold_italic`)
3. `android:textColor` для установки цвета текста надписи в формате шестнадцатеричном RGB (пример `#FF0000` для красного) или формате шестнадцатеричном ARGB (пример `#88FF0000` для прозрачного красного)

Данные атрибуты, как и большинство других, могут быть модифицированы через панель атрибутов.

К примеру в проекте примере `Basic/Label`, Вы найдёте следующий файл шаблон:

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:text="@string/profound"
  />
```

(из Basic/Label/app/src/main/res/layout/main.xml)

Только этот макет с исходным Java, которые представляют Вам приложение, вместе с соответствующими строковыми ресурсами, дают Вам:

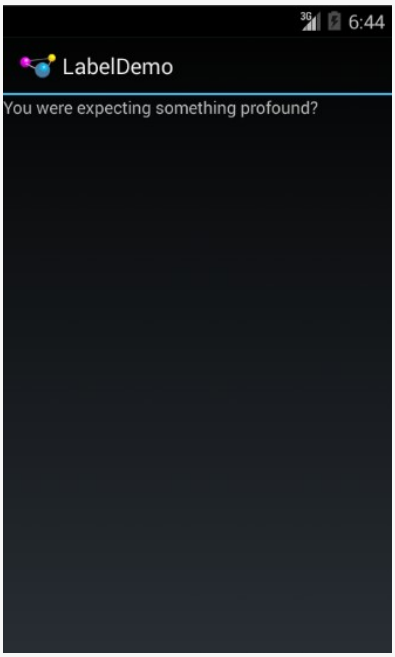


Рисунок 89: LabelDemo приложение пример

Командные кнопки

Андроид содержит виджет `Button` (кнопка), что является классическим виджетом кнопки-нажималки "нажми меня и что-то прикольное произойдёт". Как выясняется `Button` это подкласс `TextView`, так всё обсуждаемое в предыдущей части в терминах форматирования отображения кнопки сохраняется.

К примеру, в проекте примере `Basic/Button`, ВЫ найдёте следующий файл макета:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">

  <Button
    android:id="@+id/button1"
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/button"/>

</LinearLayout>
```

(из Basic/Button/app/src/main/res/layout/main.xml)

Только этот макет с исходным Java, которые представляют Вам приложение, вместе с соответствующими строковыми ресурсами, дают Вам:



Рисунок 90: Виджет Button

Графический редактор макетов Android Studio

Как и с виджетом `TextView` , виджет `Button` доступен в части “Buttons” (кнопки) палитры графического редактора макетов Android Studio:

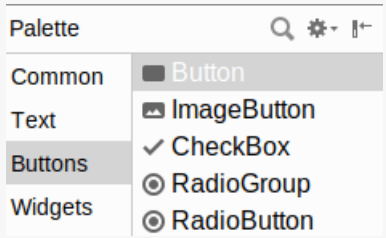


Рисунок 91: Палитра виджетов, Показана выделенная Button

Вы можете переместить `Button` из палитры в файл макета в основной области редактирования чтобы добавить виджет в макет. Панель атрибутов после позволяет Вам применить различные атрибуты данной `Button`. Так как `Button` наследуется от `TextView`, большинство опций те же (пример “Text”).

Отслеживание нажатий кнопки

Кнопки - это командные виджеты - когда пользователь нажимает кнопку, он ожидает что что-то произойдёт.

Для определения, что произойдёт, когда Вы нажимаете на кнопку, Вы можете:

1. Описать некоторый метод в Вашей `Activity` , которая содержит кнопку который содержит один параметр `View` возвращает `void` и `public`
2. В вашем файле макета XML, в элементе `Button` добавить `android:onClick` атрибут с названием метода, который вы описали в предыдущем

шаге Например мы можем иметь метод в нашей `Activity` который

```
public void someMethod(View theButton) {
  // делать что-то полезное здесь
}
```

выглядит как:

После, мы может использовать XML утверждение для самой `Button` включающее `android:onClick`:

```
<Button
  android:onClick="someMethod"
```

```
  ...
/>
```

Этого достаточно для Андроид, чтобы "связать вместе" `Button` с обработчиком нажатий. Когда пользователь нажимает кнопку вызовется `someMethod()`

Другая возможность пропустить `android:onClick`, вместо этого вызывать `setOnClickListener()` на объекте `Button` в коде Java Когда `кнопка` используется напрямую активностью, это обычно не используется - `android:onClick` немного чище. Тем не менее, когда мы начнём говорить об фрагментах Вы увидите, что `android:onClick` не работает так хорошо с фрагментами, и так мы будем использовать `setOnClickListener()` в этом случаи.

Быстрые рисунки

У Андроид есть два виджета, чтобы помочь Вам встроить рисунок в ваши активности: `ImageView` и `ImageButton`. Как предлагает название, это основанные на рисунке аналоги `TextView` и `Button`, соответственно.

Каждый виджет содержит атрибут `android:src` (в макете XML) для определения какой рисунок использовать. Они обычно ссылаются на ресурс drawable (пример, `@drawable/icon`)

`ImageButton`, подкласс `ImageView`, совмещает стандартное поведение `кнопки`, для отклика на нажатие и остальное.

Например, взгляните на, макет `main.xml` из проекта примера `Basic/ImageView`

```
<?xml version="1.0" encoding="utf-8"?>
<ImageView xmlns:android="http://schemas.android.com/apk/res/android"
  android:id="@+id/icon"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:adjustViewBounds="true"
  android:src="@drawable/molecule"/>
```

(из Basic/ImageView/app/src/main/res/layout/main.xml)

Результат, просто используя сгенерированную кодом активность, просто изображение:

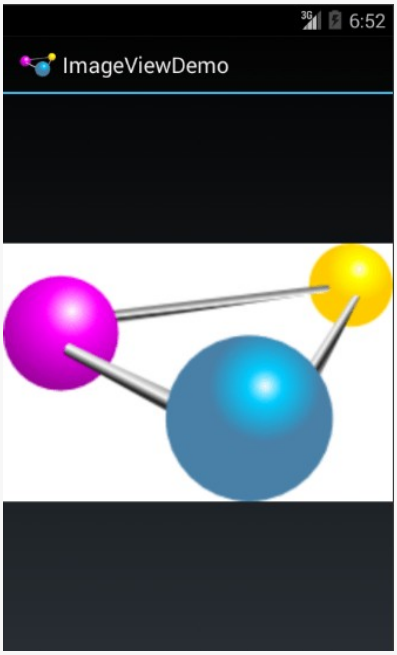


Рисунок 92: Приложение пример ImageViewDemo

Графический редактор макетов Android Studio

Виджет `ImageView` может быть найден в части “Widgets” палитры в графическом редакторе макетов Android Studio.

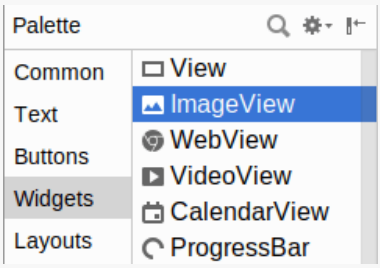


Рисунок 93: Палитра виджетов, Показана выделенная ImageView

`ImageButton` находится рядом с `ImageView` в этой палитре инструментов.

Когда Вы переносите одну из них на предпросмотр или чертёж, Вам сразу показывается диалог для выбора ресурса drawable или цвета для использования с рисунком:

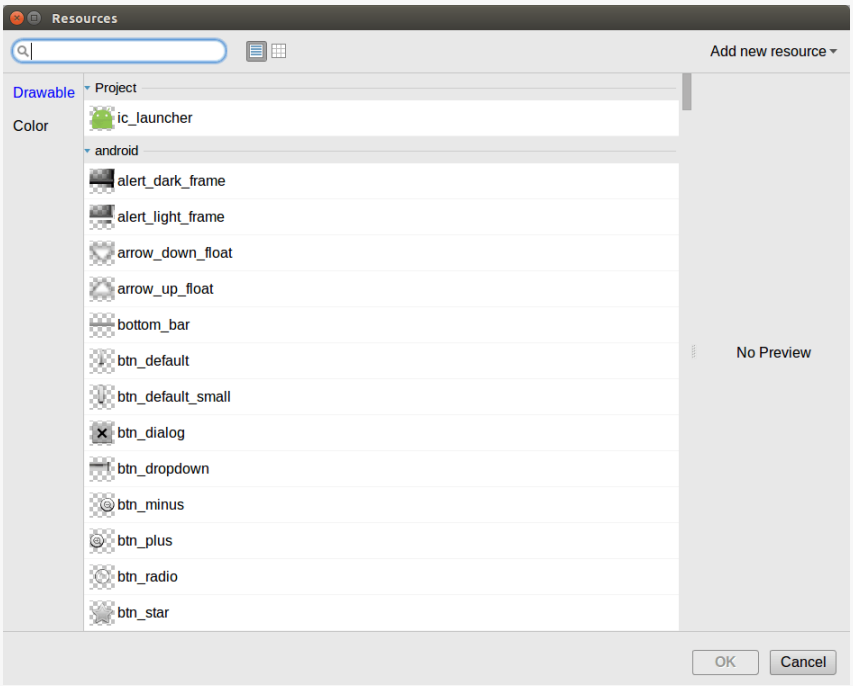


Figure 94: Диалог ресурса рисунка

К сожалению, у Вас нет выбора, кроме как выбрать один из них, в связи с ошибкой, если Вы нажмёте Cancel (отмена) для выхода из диалога, это также отменит все действия по переносу.

Вы можете перенести их в файл макета, после использовать панель атрибутов для установки их атрибутов. Как и у всех виджетов, есть опция “id” для установки значения `android:id` для виджета. Две других важных,и также, более уникальных для `ImageView` и `ImageButton` опции:

- “src” позволяет Вам выбрать ресурс drawable для использования как отображаемый рисунок, который будет заполняться так как Вы выберете в диалоге ресурса
- “contentDescription” представляет текст, который будет использован для описания рисунка для пользователей, у которых активирована функция специальных возможностей (accessibility services) (например TalkBack) для слабовидящих пользователей
- “scaleType” открывает выпадающее меню, где Вы можете выбрать как рисунок будет отмасштабирован:

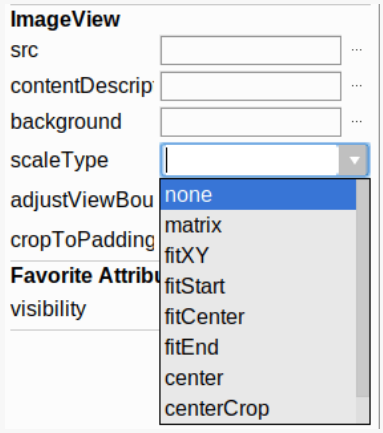


Рисунок 95: типы масштабирования в панели атрибутов Android Studio

Мы рассмотрим эти типы масштабирования в следующем разделе.

Масштабирование изображений

Возможно, или даже вероятно, что размер нашего `ImageView` точно не совпадает с размером изображения, который мы пытаемся отобразить. `ImageView` поддерживает разные “scale types” (способы масштабирования), которые пытаются справиться с несоответствием между размером/соотношением сторон изображения и размером/соотношением сторон самой `ImageView`.

Данные значения можно увидеть в JavaDocs в `ImageView.ScaleType` классе. Значение по умолчанию (`fitCenter`) просто увеличивает масштаб изображения для лучшего совпадения с доступным пространством.

Для заметки выбор “center” центрирует изображение в доступном пространстве, но не увеличивает изображение:

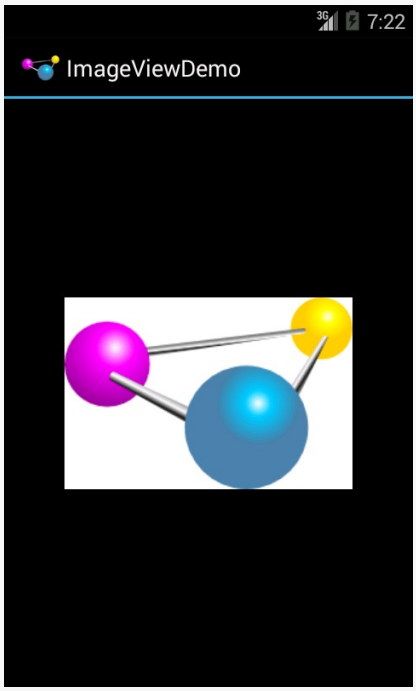


РИсунок 96: Пример ImageViewDemo, установлено центр

Выбор `centerCrop` масштабирует изображение так, что меньшая размерность совпадает с доступным пространством и отсекает остальное:

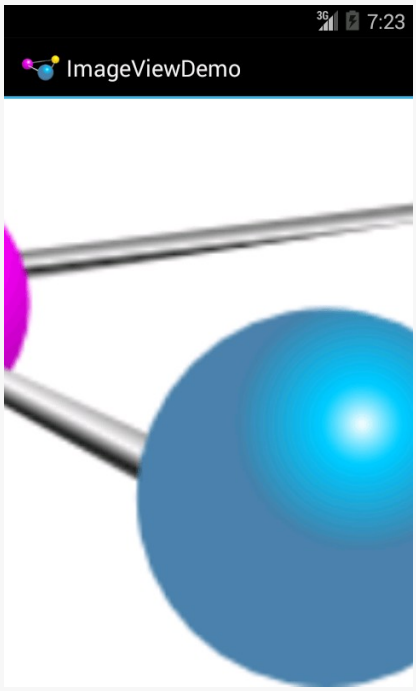


Рисунок 97: Пример ImageViewDemo, уставлен centerCrop

Выбор `fitXY` масштабирует изображение для заполнения пространства игнорируя соотношение сторон:

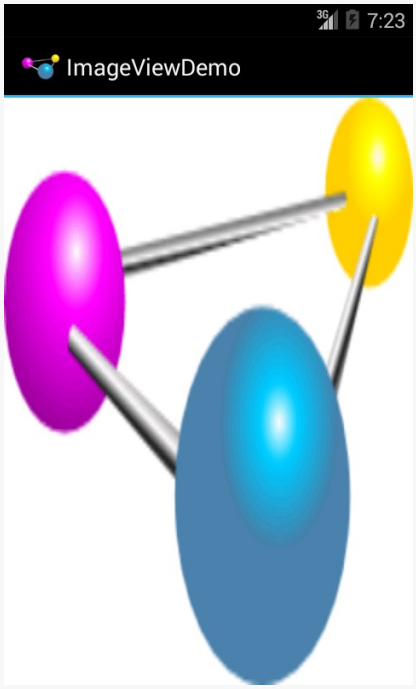


Рисунок 98: Пример ImageViewDemo, установлен fitXY

Поля зелёного Или других цветов.

Как с кнопками и надписями, поля - третий "якорь" основных инструментов ГИП. В Андроид они реализованы через виджет `EditText` , который является подклассом `TextView` используемом для надписей.

Вместе со стандартными атрибутами `TextView` (например `android:textStyle`) `EditText` содержит другие, которые будут полезны для Вас при создании полей, особенно `android:inputType`, для описания какой тип ввода Ваш `EditText` ожидает (цифры? адреса электронной почты? номера телефонов?). Подробное объяснение `android:inputType` и его взаимодействие с редакторами метода ввода (ещё “soft keyboards” (программные клавиатуры)) будет в последующих частях.

Для примера, из проекта примера `Basic/Field`, файл макета XML отображающий `EditText`:

```
<?xml version="1.0" encoding="utf-8"?>
<EditText xmlns:android="http://schemas.android.com/apk/res/android"
```

```
android:id="@+id/field"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:inputType="textMultiLine"
android:text="@string/license"
/>
```

(из Basic/Field/app/src/main/res/layout/main.xml)

Отметьте, что у нас `android:inputType="textMultiLine"`, так пользователи могут вводить несколько строк текста. Мы также определили начальный текст как значение строкового ресурса `license` .

Результат, после сборки и установки на эмулятор:

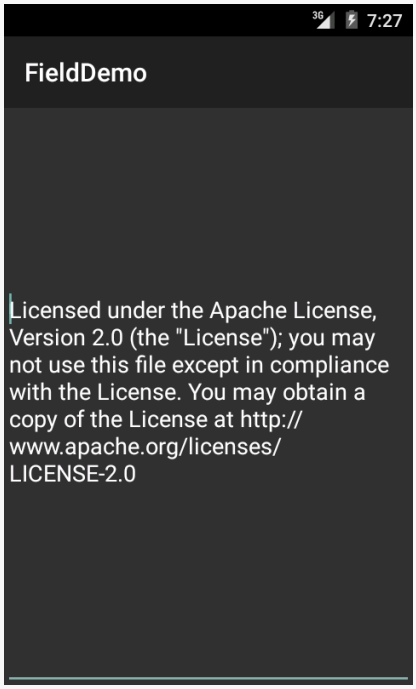


Рисунок 99: FieldDemo, в Theme.Material

Графический редактор макетов Android Studio

Палитра графических макетов Android Studio содержит целый раздел посвящённый в основном виджету `EditText` , с названием “Text”:

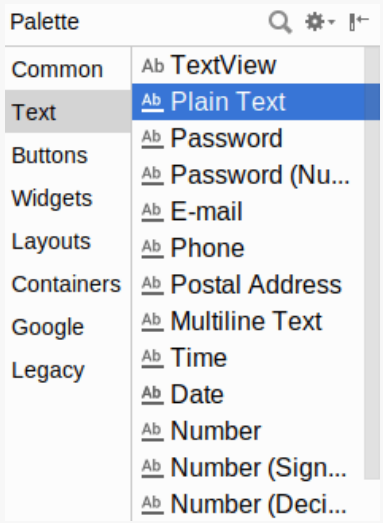


Рисунок 100: Палитра виджетов, показан выделенный “Plain Text”

Первая строка `TextView`. Вторая - (“Plain Text”) для общего назначения `EditText`. Остальные - предварительно сконфигурированные для различных сценариев, так как пароль.

Вы можете перенести любые из них в файл макета, после использовать панель атрибутов для установки соответствующих атрибутов. Атрибуты “Id” и “Text” те же, что в `TextView`, как и многие другие атрибуты, наследованные от `TextView`.

Некоторые атрибуты TextView

Элемент “Hint” в панели атрибутов позволяет Вам установить “hint” для данного `EditText`. Текст “hint” будет отображаться светло серым в виджете `EditText` , когда пользователь ещё ничего не ввёл. Когда пользователь начинает набирать в `EditText`, “hint” исчезает. Это позволит вам сохранить на экране пространство, заменяя отдельную надпись `TextView`.

Элемент “Input Type” (тип ввода) в панели атрибутов позволяет Вам определить какой тип ввода Вы ожидаете получить в этом `EditText`, подбирая из множества типов полей, которые Вы можете перенести из палитры в макет.

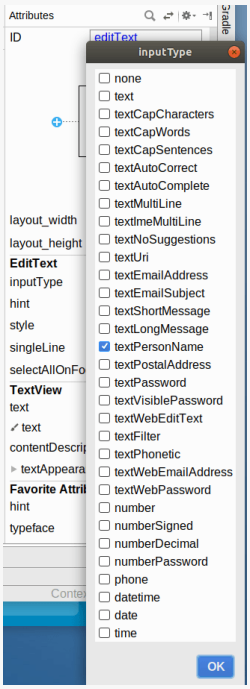


Рисунок 101: Список типов ввода тестовых полей Android Studio

Атрибут `inputType` будет более детально рассмотрен в последующей главе

Больше общих концепций

Все виджеты, включая показанные выше, наследуются от `View`. Базовый класс `View` предоставляет всем виджетам массив полезных атрибутов и методов, кроме уже описанных.

Padding (отступы)

Виджеты имеют минимальный размер, который может зависеть от того, что находится внутри них. Так к примеру, `Button` будет расширяться чтобы вместить размеры её надписи. Вы можете управлять размером используя отступы. Добавление отступов увеличит расстояние между содержимым (например надписи для `Button`) и границами виджета.

Отступ может быть установлен в XML сразу для всех сторон (`android:padding`) или для каждой стороны (`android:paddingLeft`, и т.д.). Отступы также могут быть установлены в Java коде через метод `setPadding()`. В Андроид 8.0+ также есть `android:paddingHorizontal` и `android:paddingVertical` для установки отступов с двух сторон одной оси.

Значение каждого из них размерность - комбинация из единиц измерения и значения. Так `10dip` это 10 независимых от разрешения пикселей, `2mm` это 2 миллиметра, и т.д.

Margins (поля)

По умолчанию, виджеты плотно упакованы, один за другим. Вы можете управлять этим через применение полей, концепция их напоминает отступы описанные ранее.

Разница между отступами и полями рассматривается в терминах фона. Для виджетов с прозрачным фоном - как вид по умолчанию `TextView` - отступы и поля имеют одинаковый визуальный эффект, увеличивая пространство между виджетом и соседними виджетами. Тем не менее, виджеты с непрозрачным фоном - такие как `Button` - отступ считается внутри фона, тогда как поля за пределами. Другими словами, добавление отступа увеличивает пространство между содержимым (например надписи `кнопки`) и границами, а добавление поля увеличивает пустое пространство между границами и соседними виджетами.

Поля могут быть установлены в XML, или на основе сторон (например `android:layout_marginTop`) или для всех сторон через `android:layout_margin`. Ещё раз, значение каждого из них размерность - комбинация из единицы измерения и числа, как `5dp` для 5 независимых от разрешения пикселей. В Андроид 8.0+ также есть `android:layout_marginHorizontal` и `android:layout_marginVertical`, для установки полей с двух сторон единичной оси.

Цвета

В Андроид виджетах есть два типа аттрибутов цвета. Некоторые, как `android:background`, принимают значение одного цвета (или drawable как фон). Другие как `android:textColor` в `TextView` (и подклассах) могут принимать `ColorStateList`, в том числе через сеттер Java (в данном случаи `setTextColor()`).

`ColorStateList` позволяет установить различные цвета для различных

состояний. Например, когда Вы перейдёте к виджетам селекторам в следующей главе, Вы увидите как `TextView` имеет различный цвет текста когда он выбранный элемент в списке

по сравнению когда он в списке но не выбран. Это управляется через стандартный `ColorStateList` связанный с `TextView`.

Если Вы хотите изменить цвет виджета `TextView` в коде Java у Вас есть два основных выбора:

- Использовать `ColorStateList.valueOf()`, который возвращает `ColorStateList` в котором все состояния определены одинаковым цветом, который Вы подаёте как параметр в метод `valueOf()` Это Java эквивалент выражению `android:textColor` , чтобы сделать `TextView` в определённом цвете всегда вне зависимости от состояний.
- Создать `ColorStateList` с различными значениями для различных состояний, или через конструктор или через drawable XML ресурс. Это будет рассмотрено более детально позже в книге.

Другие полезные атрибуты

Некоторые дополнительные атрибуты `View` которые наиболее вероятно будут использован включают:

1. `android:visibility`, которые управляет начальной видимостью виджета
2. `android:nextFocusDown`, `android:nextFocusLeft`, `android:nextFocusRight`, и `android:nextFocusUp`, который управляет порядком фокуса, когда пользователь использует D-Pad, трекбол или аналогичное устройство указания
3. `android:contentDescription`, который является грубым эквивалентом атрибута `alt` в HTML теге `<img>` , и используется в специальных возможностях чтобы помочь людям, которые не могут видеть экран управлять приложением - это очень важно для виджетов таких, как `ImageView`

Мы увидим больше про атрибуты фокуса и `android:contentDescription` в главе управление фокусом и специальные возможности далее в этой книге.

Полезные методы

Вы можете переключить включён или нет виджет через `setEnabled()` и увидеть что он включён через `isEnabled()`. Один частый паттерн использования этого отключение некоторых виджетов основываясь на состоянии `CheckBox` или `RadioButton` . Мы рассмотрим `CheckBox`, `RadioButton`, и похожие типы виджетов немного далее в книге.

Вы модете предоставить виджету фокус через `requestFocus()` и увидеть что он под фокусом через `isFocused()`. Вы можете использовать это в случаи с отключением виджетов, как указано выше, чтобы убедиться что нужный виджет имеет фокус, когда ваше действие по отключению

завершено.

Для помощи по навигации по дереву виджетов и контейнеров, которые создают общий вид активности, Вы можете использовать:

1. `getParent()` чтобы найти родительский виджет или контейнер
2. `findViewById()` чтобы найти виджет потомок с определенным ID
3. `getRootView()` для получения корневого элемента дерева (например., который Вы предоставляете активности через `setContentView()`)

Посетите пути!

Вы можете изучить больше о фреймворке методов ввода Андроид - о чем вы можете думать как о программных клавиатурах - в следующей главе.

Другая глава в путях рассматривает использование шрифтов для сшивки с Вашими виджетами `TextView` (и теми, что наследуются от них, как `Button`).

Ещё другая глава в путях рассматривает форматирование rich text и для представления отформатированного текста в `TextView` (например встроенный **жирный шрифт**) и для получения отформатированного текста от пользователя через настроенный `EditText`.

Отладка Вашего приложения

Сейчас, когда мы начали работать с изменениями в макетах и Java коде более существенно, повышается вероятность, что мы столкнёмся что что-то работает неправильно и наше приложение сбоит.

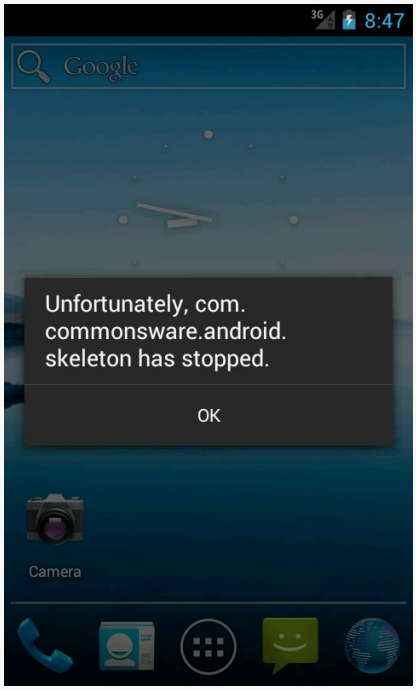


Рисунок 102: Диалог сбой на Андроид 4.0.3

В данной главе мы рассмотрим несколько советов как отлаживать данные типы ошибок.

Получение Stack Trace

Если Вы видите один из “Принудительное завершение” или “Остановлено” диалогов, первое что Вы захотите изучить это Java stack trace связанную с данным сбоем. Они регистрируются в объекте, известном как LogCat, на вашем устройстве или эмуляторе.

Чтобы просмотреть Logcat, у Вас есть два варианта:

1. Использовать команду `adb logcat` в командной строке (или что-то что использует `adb logcat`, как различные разноцветные скрипты доступные онлайн)
2. Использовать вкладку Logcat в Android Studio

Также в Play Store есть приложение Logcat, например aLogcat, которое отобразит содержимое Logcat. Тем не менее, по причинам безопасности и приватности, на устройствах с Андроид 4.1 и выше, такие приложения смогут отобразить Вам только *их* записи Logcat, а не от системы, вашего приложения, или ещё чего-то. Следовательно, при разработке, лучше использовать одну из альтернатив указанных выше.

Logcat в Android Studio

Просмотр Logcat доступен в любое время, практически отовсюду в Android Studio, по нажатию на элемент окна инструмента Андроид, обычно прикреплённому внизу Вашего окна IDE:

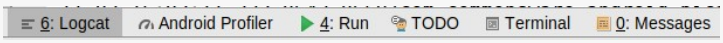


Рисунок 103: Минимизированные окна инструментов Android Studio,

Нажатие на него откроет некоторые специфические для Андроид журналы в окне инструмента “Android DDMS” с вкладками “Devices | logcat”:

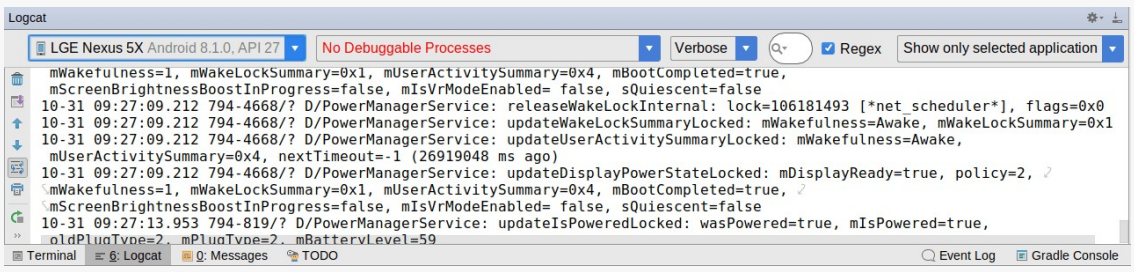


Рисунок 104: Окно инструмента Android DDMS, Отображающего Logcat

Logcat о отобразит Ваш stack traces, диагностическую информацию от операционной системы, и всё, что Вы добавите через вызов статических методов класса `android.util.Log` . Например `Log.e()` записывает в журнал сообщение с уровнем ошибка, что отобразит его красным.

Если Вы хотите отправить что-то из Logcat кому-то, как если через трекер ошибок, просто выделите текст и скопируйте его в буфер обмена, как Вы делаете с любым текстовым редактором.

Значок "мусорная корзина" сверху на полоске инструментов слева - инструмент "очистки журнала". Нажатие её приведёт к очистке Logcat. Это определённо очистить Ваше *отображение* Logcat, что Вы увидите только сообщения, которые добавлены в журнал после того, как Вы очистили его. Отметьте также, что это в действительности не очищает журналы из устройства или эмулятора.

В дополнение, Вы можете:

- Использовать выпадающее меню "Уровень логгирования" для фильтрации строк основываясь на важности, где будут отображаться сообщения выбранной Вами важности или выше
- Использовать поле поиска справа от выпадающего "Уровня логгирования" для фильтрации элементов основываясь на строке поиска
- Настроить больше постоянных фильтров через выпадающее меню справа от поля поиска

Случаи ошибок вызова класса

В случаи сбоя, stack trace может предложить что существует проблема с привязкой к Вашему ресурсу. Одна частая особенность этого `ClassCastException` когда Вы вызываете `findViewById()`. Например, Вы можете вызвать `(Button)findViewById(R.id.button)`, и получить `ClassCastException: android.widget.LinearLayout` в результате, указывающий, что когда Вы думаете что вызов `findViewById()` возвращает `Button`, в действительности он возвращает `LinearLayout`.

Часто это не Ваша вина. Иногда значения `R` выпадают из синхронизации с предскомпилированными классами из предыдущих сборок. Это часто происходит, сразу после изменения вашего набора ресурсов (например добавления нового макета).

Чтобы исправить это, Вам необходимо очистить Ваш проект. Чтобы сделать это, в Android Studio, выберите "Build > Clean Project" из главного меню.

Так, если Вы получаете странный сбой, который похоже относится к ресурсам очистите

Ваш проект Если проблема уйдёт, Вы справились - если проблема остаётся, Вам необходимо выполнить немного больше отладки.

Точки прерывания

Один из отличительных знаков Java IDE возможность выполнять отладку в реальном времени, применяя контрольные точки и тому подобное. В этом отношении, Android Studio работает для приложений Андроид тем же способом что IntelliJ IDEA и Eclipse для приложений Java. Вы можете проводить отладку на эмуляторе или любом Андроид устройстве для которого активирована отладка по USB (как Вы сделали в Руководстве №1).

При нехватки документации относящейся к Android Studio Вы можете перейти по ссылке документация для IntelliJ IDEA чтобы изучить как использовать отладчик.

С Android Studio, управление запуска в панели инструментов предоставляет Вам некоторые опции для отладки Вашего приложения:

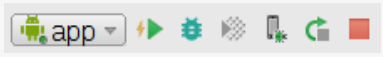


Рисунок 105: Android Studio управление запуска

Кнопка по форме жука справа от зелёного треугольника "запуск" запускает Ваше приложение и прикрепляет отладчик, так будут учитываться контрольные точки. Если Ваше приложение уже выполняется и Вы хотите начать отладку выполняющегося процесса, Вы можете сделать это через кнопку панели инструментов, которая выглядит как телефон с маленьким жуком в нижнем правом углу.

Классический контейнер классов

Контейнеры - иногда называемые менеджеры макета - выполняют организацию виджетов на экране. Контейнеры задаю местоположение и размер виджетов по правилам, которые Вы поддерживаете наряду с ключевыми характеристиками устройства, такими как доступный размер экрана.

Три контейнера преобладали в разработке Андроид приложений с внедрения в Андроид в конце 2007: `LinearLayout`, `RelativeLayout`, и `TableLayout`. Данная глава сфокусирована на них. Последние главы рассматривают другие классы контейнеров, как `ConstraintLayout` 2016 года.

Введение в приложение шаблон

проект пример `Containers/Sampler` содержит набор ресурсов макетов, которые используются в данной главе для иллюстрации как эти контейнеры работают.

Если вы запустите данное приложение пример, вы увидите серию вкладок, с одним макетом на вкладку:

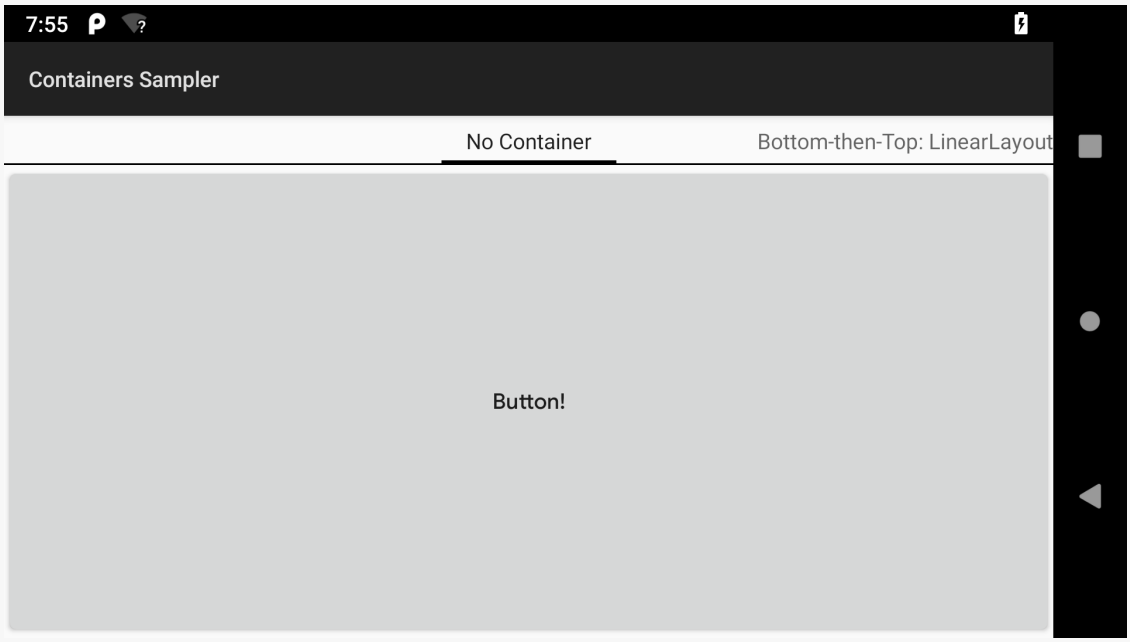


Рисунок 106: Приложение пример, Изначальный запуск

Мы не собираемся погружаться в Java код связанный с приложением примером в этой главе. Данный код опирается на другие темы, как фрагменты и виджет `ViewPager`, которые мы ещё не проходили. Мы вернёмся к данному приложению примеру и увидим как были реализованы вкладки после. В данный момент сфокусируемся на файлах макетах отображающих отдельные техники использования этих классических контейнеров.

RTL и Ваши макеты

Большинство мировых языков пишутся слева направо. Так, в данном параграфе, Вы читаете буквы и слова начиная с левой границы строки до правой границы.

Арабский и Иврит, среди других, пишутся справа налево. Аббревиатура “RTL” относится к данным языкам.

Изначально, Андроид фокусировался только на языках слева направо (LTR). Как результат, Вы видите атрибуты ссылающиеся на элементы относящиеся к лево и право (пример `android:paddingLeft`).

Медленно, Андроид улучшил поддержку RTL. В частности, начиная с уровня API 17 (Андроид 4.2), были добавлены аналогичные атрибуты, заменяя “left”(лево) на “start”(начало) на “right”(право) на “end”(конец). Когда используются атрибуты “start”/“end” (например `android:paddingStart`), “start” относиться к тому, где Вы начинаете читать строку, и “end” относиться к тому, где Вы заканчиваете читать строку текста:

Направление языка	“Start” означает...	“End” означает...
LTR	Слева	Справа
RTL	Справа	Слева

В общем, мы хотим, чтобы ГИП соотносился с направлением языка. Вещи, которые мы хотим слева в языке LTR обычно переходят направо в языке RTL, и так далее.

Так как, данные атрибуты “start” и “end” новые начиная с уровня API 17, Вы не можете использовать их на более старых устройствах. Иногда, мы будем использовать или старые атрибуты “left”/“right” или использовать оба типа атрибутов, чтобы покрыть все версии устройств.

Мы вернёмся к поддержке языков RTL немного позже в этой главе.

LinearLayout и модель Box

`LinearLayout` представляет Андроид реализацию коробочной модели - виджеты или контейнеры потомки располагаются в линию в столбец или строку, один за другим.

Концепт и атрибуты

Для настройки `LinearLayout`, у Вас есть четыре основное области управления кроме содержимого контейнера: ориентация, модель заполнения, веса, гравитация.

Ориентация

Ориентация указывает, представляет ли `LinearLayout` строку или столбец. Просто добавьте свойство `android:orientation` к элементу `LinearLayout` в

Вашем XML макете, устанавливаемые значения будут `horizontal` (горизонтально) для строки или `vertical` (вертикально) для столбца.

Ориентация может быть изменена во время выполнения через вызов `setOrientation()` на

LinearLayout, поддерживающий или HORIZONTAL или VERTICAL.

Модель заполнения

Ещё одна задача LinearLayout - или любого класса контейнеров Андроид - организовать множество виджетов. Часть организации этих виджетов - определить сколько пространства каждый занимает.

LinearLayout принимает реализацию "более старый потомок выигрывает" по отношение к выделяемому пространству. Так, если у нас есть LinearLayout с тремя потомками, первый потомок получает запрошенное пространство. Второй потомок получит запрашиваемое пространство если осталось достаточно пространства и также для третьего потомка. Так если первый потомок запрашивает всё пространство (например это горизонтальный LinearLayout и первый потомок содержит android:layout_width="match_parent"), второй и третий потомок будут развёрнуты с нулевой шириной.

Вес

Но, что случиться если у нас два и более виджета, которые должны разделить доступное свободное пространство? Например, допустим у нас есть два многострочный поля в столбце, и мы хотим чтобы они занимали оставшееся пространство в столбце после всех других виджетов уже расположенных в выделенном пространстве.

Чтобы это работало в дополнение к установке android:layout_width (для строк) или android:layout_height (для столбцов), Вы должны также установить android:layout_weight. Данное свойство указывает какая пропорция свободного пространства должна отойти этому виджету. Если Вы установите android:layout_weight одинаковое ненулевое значение для пары виджетов (например, 1), свободное пространство разделится поровну между ними. Если Вы установите 1 для одного виджета и 2 для другого виджета, второй виджет виджет будет использовать в два раза больше свободного пространства чем первый виджет. И так далее.

По умолчанию для виджетов вес нулевой.

Другой паттерн использования весов если вы хотите задать размеры на основе процентов. Для использования этой техники для, например, горизонтального макета:

1 Установите все значения `android:layout_width` на `0` для виджетов в макете
2 Установите значения `android:layout_weight` на соответствующий размер в процентах для каждого виджета в макете
3 Удостоверьтесь что сумма всех весов `100`

Если Вы хотите оставить пространство, не выделенное какому либо виджету, Вы можете добавить атрибут `android:weightSum` к `LinearLayout`, и обеспечить, чтобы сумма атрибутов `android:layout_weight` потомков меньше, чем эта сумма. Каждый потомок получит пространство основываясь на соотношение их `android:layout_weight` к `android:weightSum`, а не к сумме весов. И останется свободное пространство, которое займёт остаток пространства не назначенное потомкам.

Гравитация

По умолчанию всё в `LinearLayout` выровнено к началу и вверх. Так если Вы создаёте строку виджетов через горизонтальный `LinearLayout`, строка будет выровнена на стартовую сторону экрана (например слева для LTR языка).

Если это не то, что Вам нужно, Вам необходимо указать гравитацию. В отличии от физического мира, Андроид содержит два типа гравитации: гравитация виджета внутри `LinearLayout`, и гравитация содержимого виджета или контейнера.

Свойство `android:gravity` некоторых виджетов и контейнеров - которая также может быть указана через `setGravity()` в Java - указывает Андроид переместить содержимое виджета или контейнера в соответствующем направлении. Например `android:gravity="right"` говорит сместить содержимое виджета вправо; `android:gravity="right|bottom"` говорит сместить содержимое виджета вправо и вниз.

Здесь, “содержимое” различное. `TextView` поддерживает `android:gravity`, и "содержимое" - текст содержащийся внутри `TextView`. `LinearLayout` поддерживает `android:gravity`, и "содержимое" - виджеты внутри контейнера. И так далее.

Потомки `LinearLayout` также содержат опцию указывающую `android:layout_gravity`. Здесь, потомок говорит `LinearLayout` "если есть место, пожалуйста смести меня (и меня одного) в этом направлении". Тем не менее, это работает только в направлениях *противоположных* ориентации `LinearLayout` - потомки `вертикального LinearLayout` могут использовать `android:layout_gravity` для управления их расположением горизонтально (`начало` или `конец`) но не вертикально.

Для строки виджетов, по умолчанию они выровнены, так их текст выровнены по базовой линии (невидимая линия на которой буквы кажется "сидят") хотя Вы можете указать гравитацию `center vertical` для центрирования виджетов к

центральной по вертикали точке строки.

Графический редактор макетов Android Studio

Контейнер `LinearLayout` может быть найден в разделе “Layouts” палитры графического редактора макетов Android Studio:

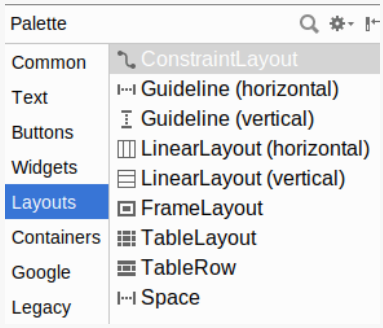


Рисунок 107: Палитра макетов графического редактора макетов

Вы можете перенести “LinearLayout (vertical (вертикальный))” или “LinearLayout (horizontal (горизонтальный))” в XML ресурс макета, после начать переносить потомков в контенер.

Когда Ваш `LinearLayout` - выбранный виджет, над предпросмотром появятся несколько новых кнопок:



Рисунок 108: Кнопки панели инструментов LinearLayout

Только одна из них работает, та что слева, которая переключает `LinearLayout` между вертикальной и горизонтальной ориентацией.

Когда выбранный виджет - один из *потомков* `LinearLayout` , панель инструментов изменяется:



Рисунок 109: Кнопки панели инструментов LinearLayout, для выбранного

Слева направо, Кнопки:

- Переключают родительский `LinearLayout` между горизонтальной и вертикальной ориентациями
- Назначает базовую линию виджетов потомков (где базовая линия - невидимая линия на которой располагается появляющийся текст)
- Удаляет атрибут `android:layout_weight` , если он существует
- Переключает ширину между `match_parent` и `wrap_content`
- Переключает высоту между `match_parent` и `wrap_content`

Пример: Низ-после-Верх

Данные правила станут более понятными, когда мы проработаем несколько примеров.

Первый пример, когда у нас два виджета. Один виджет должен занять его "естественное" количество пространства, привязан к низу экрана. Другой виджет должен занять оставшееся пространство:

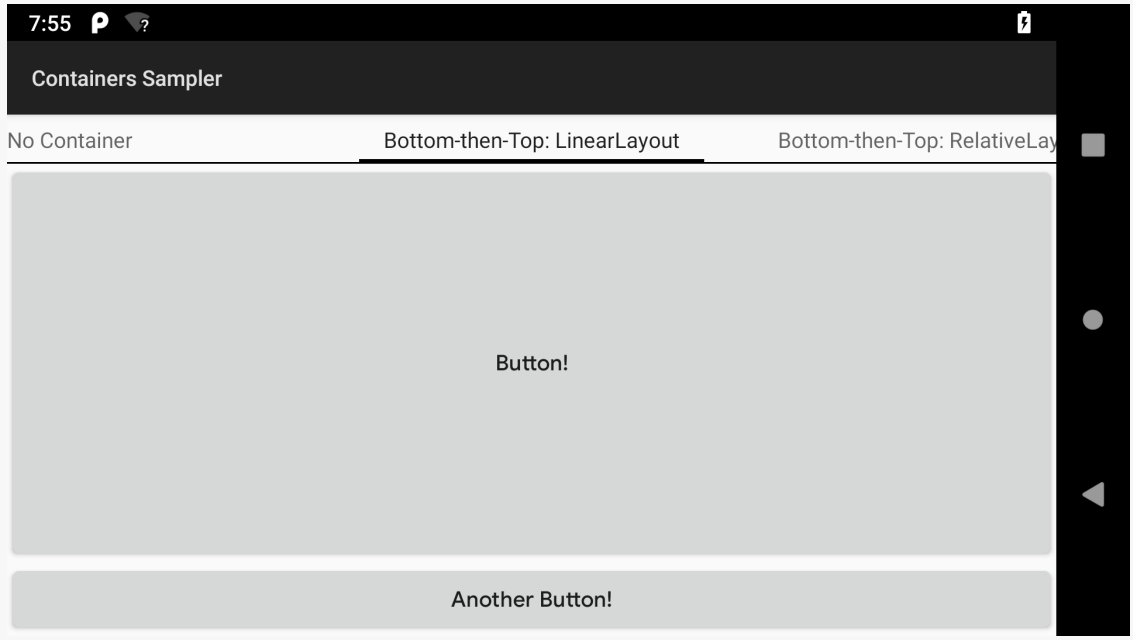


Рисунок 110: Низ-после-Верх макет, применяя LinearLayout

XML

XML макет, который генерирует тот снимок экрана состоящий из вертикального `LinearLay` и двух виджетов `Button`:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:baselineAligned="false">

  <Button
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:text="@string/button" />

  <Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/another_button" />
</LinearLayout>
```

(из Containers/Sampler/app/src/main/res/layout/bottom_then_top_ll.xml)

Размеры `LinearLayout` (`android:layout_width`, `android:layout_height`) установлен каждый на `match_parent`, так он заполняет все доступное для этого пространство. В данном приложении примере, все будет ниже вкладок.

Оба виджета `Button` с атрибутом `android:layout_width` также установленном на `match_parent`, так они будут заполнять все доступное на экране пространство горизонтальн

Атрибут `android:layout_height` кнопки `Button` установлен на `wrap_content`, так она будет занимать только столько места, сколько необходимо для рендеринга ее подписи и фона вокруг.

Верхняя `Button` содержит `android:layout_height` установленный на `0dp`. Если мы больше ничего не сделаем, это приведет к результату в невозможно-короткую кнопку. Тем не менее мы также указали `android:layout_weight="1"`.

Когда указанное будет рендериться на экране, `LinearLayout` произведет два прохода по потомкам. При первом проходе, он запрашивает сколько пространства каждый из потомком хочет, основываясь на значениях `android:layout_height` (так как это вертикальный `LinearLayout`). Первая кнопка запросит пространство для о пикселей, тогда как нижняя кнопка запросит сколько необходимо для ее содержимого. После, при втором проходе `LinearLayout` через его потомков запрашиваются их веса. Первая кнопка содержит вес 1, тогда как нижняя кнопка содержит вес по умолчанию в

0. Как результат верхняя кнопка получает 1/1 = 100% от всех оставшихся пикселей от использованных первым проходом. Это приводит к тому, что верхняя кнопка

становиться высокой, отображая нижнюю кнопку внизу экрана.

Графический редактор макетов Android Studio

Вы можете создать новый ресурс макета щелчком правой кнопки на директорией `res/layout/` и выбрав “New > Layout resource file” из контекстного меню правой кнопки мыши. Далее Вам предоставиться диалог, где Вы можете дать имя новому ресурсу и выбрать корневой контейнер для этого макета:

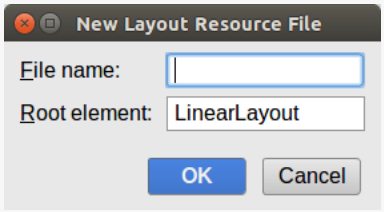


Рисунок 111: Диалог нового макета ресурса

Что Вы получаете по умолчанию немного зависит от Вашего проекта:

- Если Ваш проект содержит доступный для него `ConstraintLayout`, по умолчанию буд
  `ConstraintLayout`
- В другом случаи, по умолчанию будет `вертикальный LinearLayout`

Если Вы перемещаете виджеты `Button` в макет, изначально они получают ширину `match_parent` и высоту `wrap_content`:

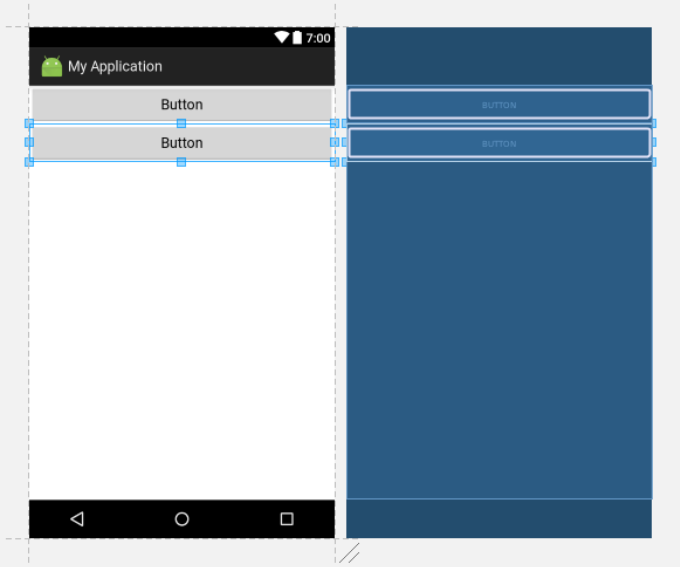


Рисунок 112: Вертикальный LinearLayout, с двумя виджетами Button

Нажмите на верхнюю кнопку, затем установите атрибут `layout_weight` на `1` в панели атрибутов, что даст желаемый вид:

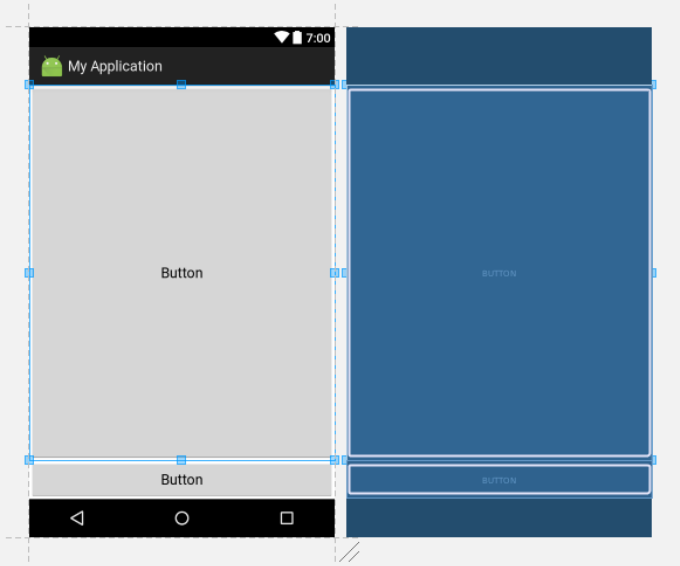


Рисунок 113: Вертикальный LinearLayout, с взвешенными виджетами Button

Отметьте, что `layout_weight` может появляться только при виде панели атрибутов “View all attributes”, а не в упрошенном списке.

Пример: Суммированные-Проценты

Веса могут быть более сложными, чем просто предоставлять всё доступное пространство одному виджету.

Сценарий суммированных-процентов выделяет пространство на основании всех потомков `LinearLayout`. В приложении примере, у нас файл макет с тремя кнопками, занимаемых 50%, 30%, и 20% пространства в вертикальном `LinearLayout`:



Рисунок 114: Макет суммированных-процентов, применяя LinearLayout

XML

В данный момент, XML ресурс макета содержит все три виджета `Button` с высотой в 0:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">

  <Button
```

```
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="50"
    android:text="@string/fifty_percent"/>

  <Button
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="30"
    android:text="@string/thirty_percent"/>

  <Button
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="20"
    android:text="@string/twenty_percent"/>

</LinearLayout>
```

(из Containers/Sampler/app/src/main/res/layout/stacked_percent_ll.xml)

Так, первый проход, который выполняет `LinearLayout` через его потомков, каждый потомок запрашивает о пикселей по высоте, что означает, что все пространство `LinearLayout` доступно для второго прохода.

При втором проходе, потомки рапортуют веса 50, 30 и 20 соответственно. 50+30+20=100, так верхняя `Button` получает 50/100 (50%) пространства, средняя `Button` получает 30/100(30%) пространства и нижняя `Button` получает 20/100 (20%) пространства.

Отметьте, что вы получите те же результаты с весами в 5, 3 и 2. В действительности имеют значение только соотношение весов. Тем не менее, если Вы предпочитаете думать в терминах процентов - возможно по причите прошлого опыта с другими инструментами ГИП - вы можете использовать целочисленные проценты по желанию.

Графический редактор макетов Android Studio

Создание этого применяя графический редактор макетов похоже на предыдущий пример:

- Создайте новый ресурс макет, выбрав `LinearLayout` как корневой контейнер.
- Переместите три виджета `Button` в `LinearLayout`.
- Используя панель атрибутов, установите значение `android:layout_height` в

трех виджетах `Button` на `0dp`

- Используя панель атрибутов установите значение `android:layout_weight` для трех виджетов `Button` . Отметьте, как и ранее, что веса могут не быть в сокращенном списке атрибутов. Вам может понадобиться отобразить все атрибуты для доступа к ним:

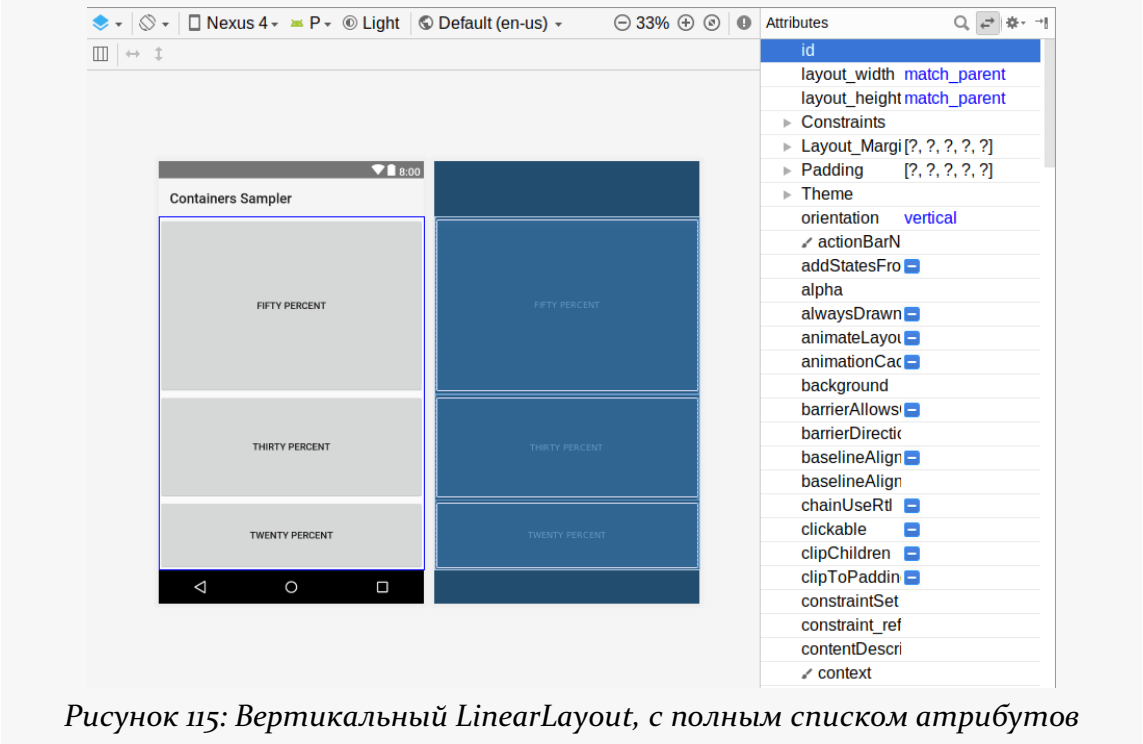


Рисунок 115: Вертикальный LinearLayout, с полным списком атрибутов

Пример: URL Диалог

Конечно, Вы не ограничены одной осью. Вы можете упорядочить виджеты `LinearLayout` в структуру по оси X и по оси Y.

Для примера, Вы можете вооружиться следующей формой диалогового стиля:

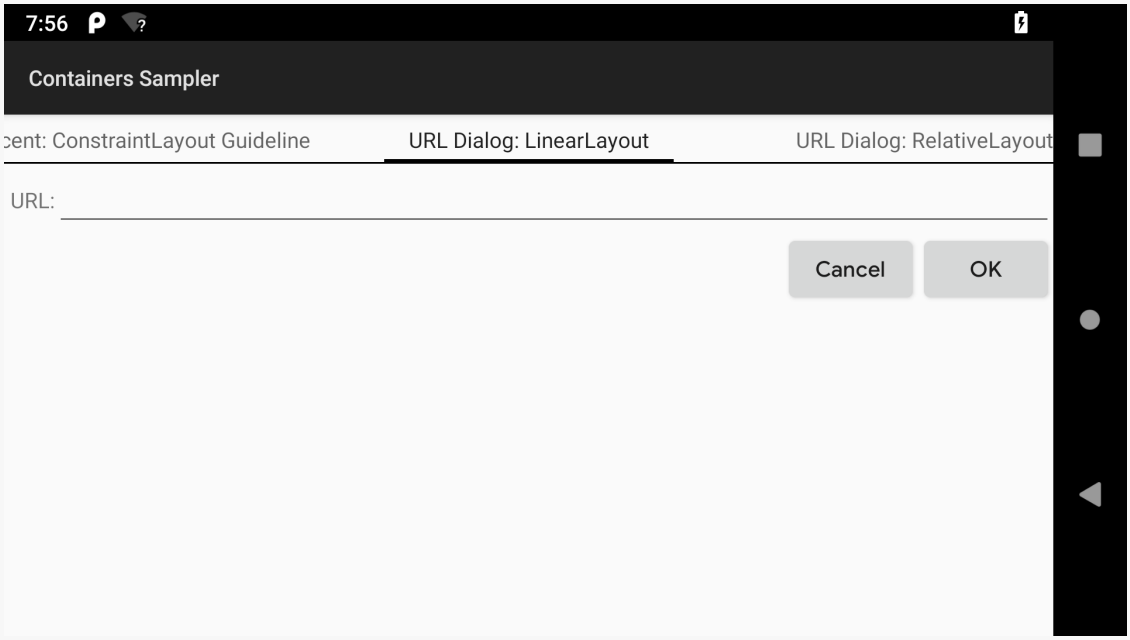


Рисунок 116: Макет URL-Диалога, используя LinearLayout

XML

Данный ресурс макета немного сложнее, сейчас у нас четыре виджета плюс три контейнера `LinearLayout` :

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="vertical">

  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginLeft="4dp"
      android:layout_marginStart="4dp"
```

```
      android:text="@string/url" />

    <EditText
      android:id="@+id/entry"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:inputType="text" />

  </LinearLayout>

  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="end"
    android:orientation="horizontal">

    <Button
      android:id="@+id/cancel"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/cancel" />

    <Button
      android:id="@+id/ok"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/ok" />
  </LinearLayout>

</LinearLayout>
```

(из Containers/Sampler/app/src/main/res/layout/url_dialog_ll.xml)

Внешний `LinearLayout` - вертикальный, служащий для хранения двух строк горизонтальных `LinearLayout`. Каждый из данных горизонтальных `LinearLayout` контейнеров содержит высоту `wrap_content` и ширину `match_parent`, так они простираются на ширину экрана, но занимают столько высоты, сколько необходимо для отображения содержимого их виджетов.

Первый из четырех виджетов `TextView`. Так как он в первом горизонтальном `LinearLayout`, он будет расположен с одной стороны (слева в LTR языках, как Английский). Кроме того у нас `4dp` поле используемое в старом `android:layout_marginLeft` и новом совместимом с RTL `android:layout_marginStart` атрибутах. Так виджет вставлен на небольшом расстоянии от границы.

Второй виджет `EditText`. Он содержит ширину `match_parent`, так он займет все

оставшееся пространство после `TextView`.

Нижний `LinearLayout` содержит `android:gravity="end"`, что приводит в смещению содержимого в направлении конечной стороны `LinearLayout` (вправо в языке LTR, таком как Английский). Он содержит два виджета `Button` , и это причина почему виджеты `Button` смещены в противоположный конец формы.

Графический редактор макетов Android Studio

Не удивительно, чем более сложный макет мы хотим создать, тем больше работы необходимо чтобы создать его. IDE только помогает указывать.

Так, для создания данного макета, Вам необходимо:

- Создайте новый ресурс макет, выбрав `LinearLayout` как корневой контейнер.
- Перенести элемент “LinearLayout (horizontal)” из палитры в вертикальный `LinearLayout`. К сожалению результат в том что новый `LinearLayout` занимает все пространство родителя, так как его высота и ширина по умолчанию `match_parent` :

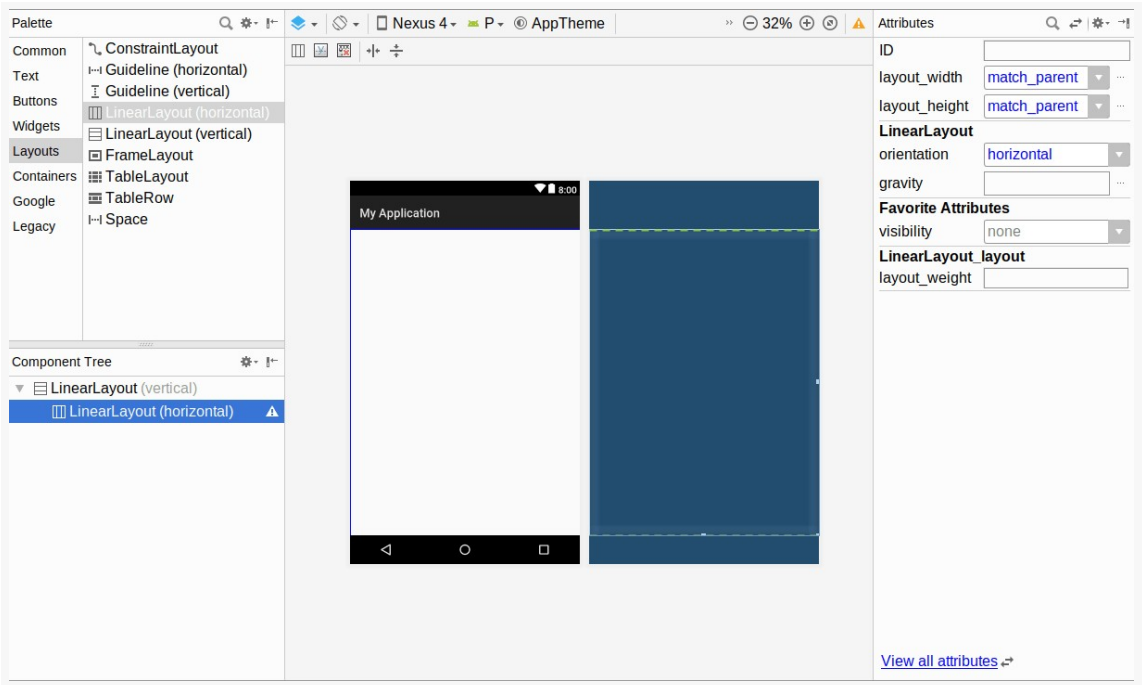


Рисунок 117: Вертикальный LinearLayout содержащий горизонтальный

- Используя панель атрибутов установите отступы на `8dp` со всех сторон:

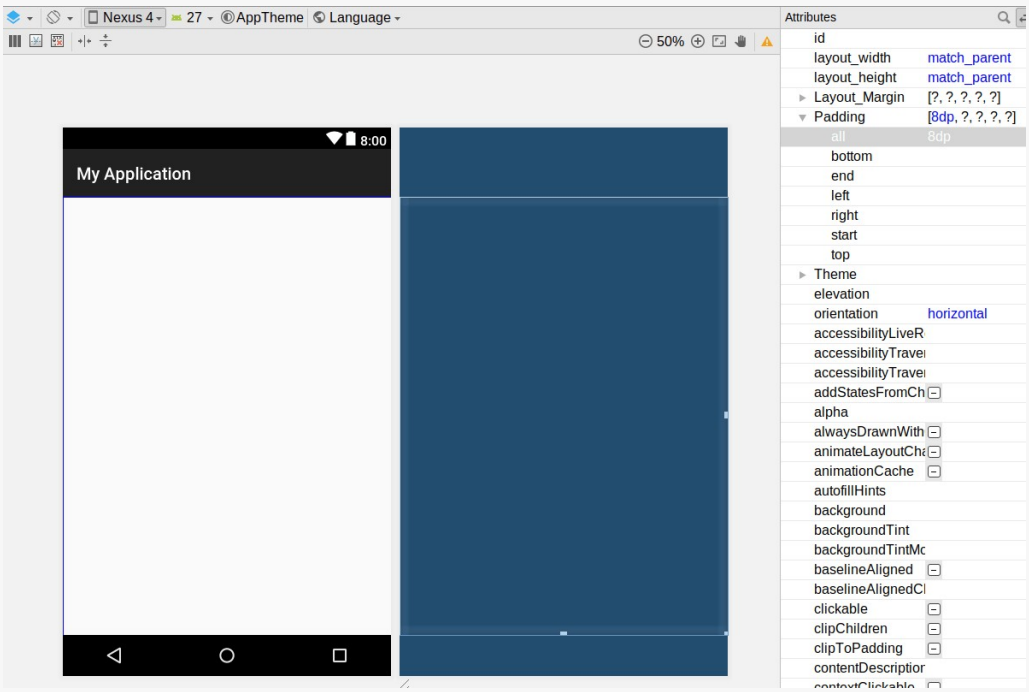


Рисунок 118: 8dp отступ в горизонтальном LinearLayout

- Перенесите `TextView` в `горизонтальный LinearLayout` *в дереве компонентов* и установите его `text` в `URL:`, ширину в `wrap_content` и удалите его значение `layout_weight` :

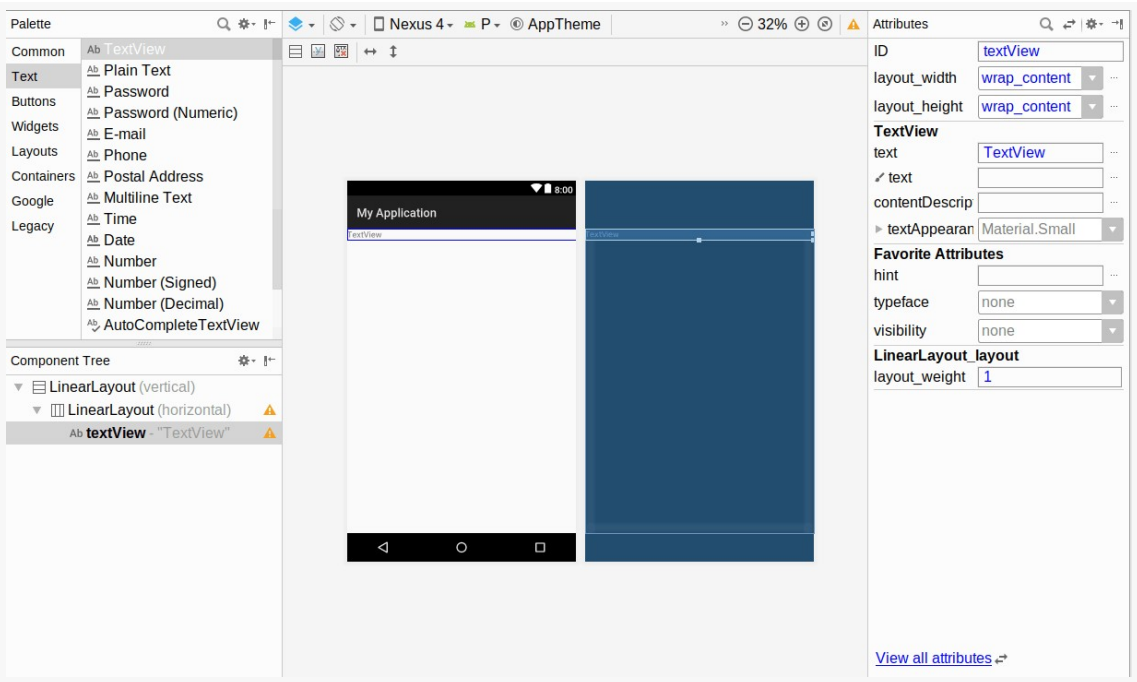


Рисунок 119: Горизонтальный LinearLayout с TextView

- Перенесите “Plain Text” `EditText` в `горизонтальный LinearLayout`, установите его ширину в `match_parent`, его `text` будет пустым, его `inputType` будет чем-то подходящим (например `textUri`, так как в теории данное поле должно содержать URL) и уберите его weight. Отметьте, Вы можете решить что проще переносить виджет в его контейнер в дереве компонентов, так Вы можете лучше в данном случаи контролировать порядок потомков:

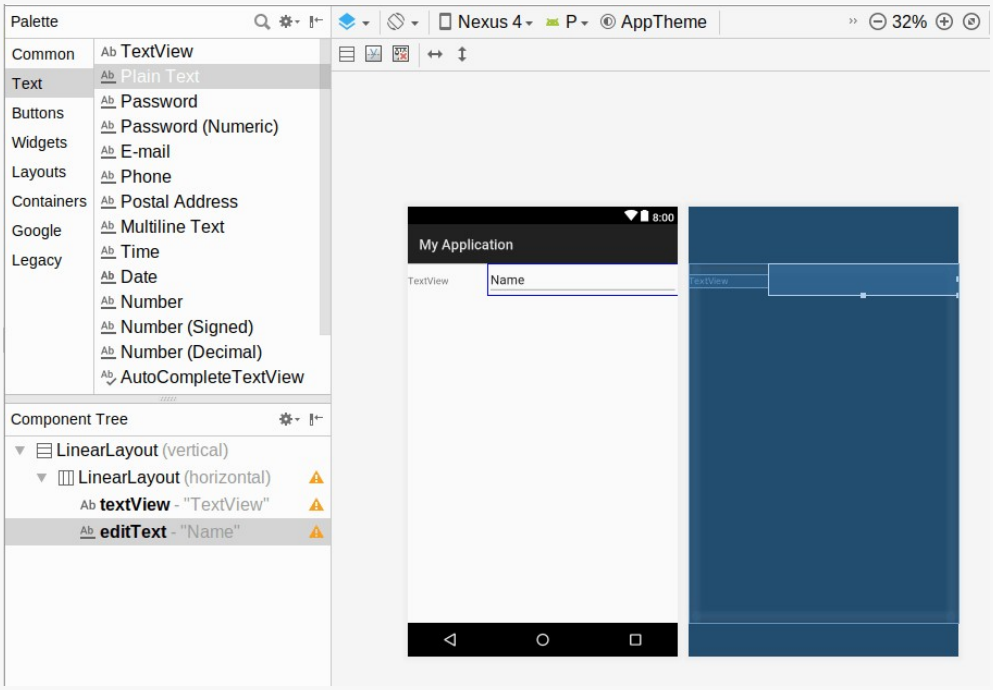


Рисунок 120: Горизонтальный LinearLayout с TextView и EditText

- Перенесите другой элемент “LinearLayout (горизонтальный)” из палитры, но в этот раз поместите его в “LinearLayout(вертикальный)” в дереве компонентов, после измените порядок в дереве компонентов положив новый `LinearLayout` снизу:

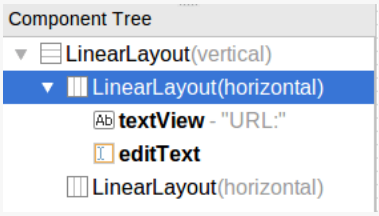


Рисунок 121: Дерево компонентов с двумя горизонтальными LinearLayout

- Отредактируйте высоту обеих виджетов `LinearLayout` до `wrap_content`.
- Переместите два виджета `Button` в тот нижний `LinearLayout` в дереве компонентов:

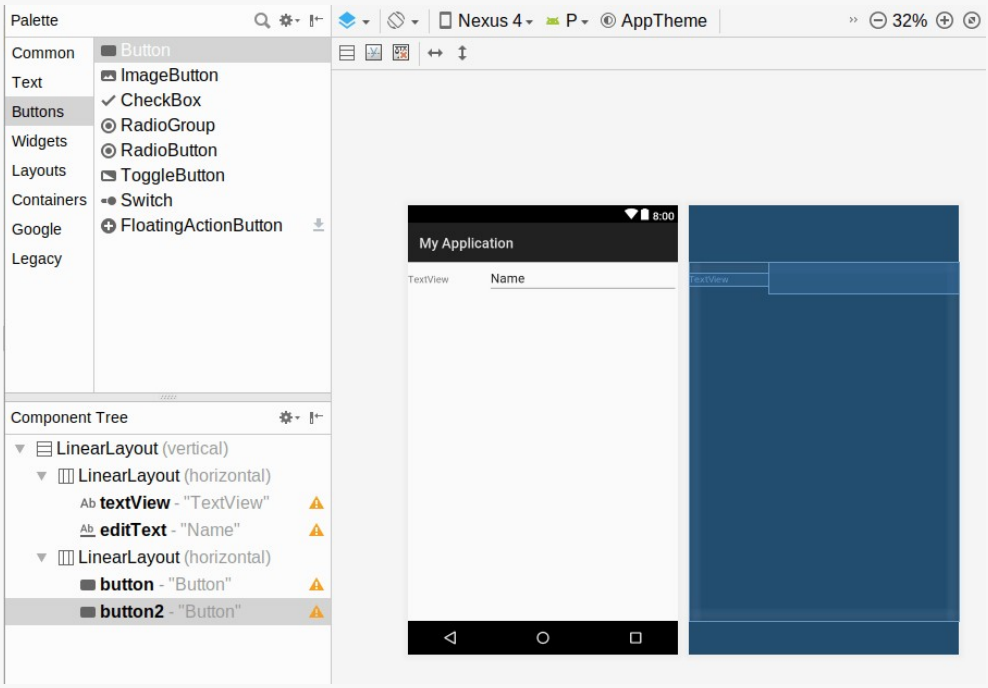


Рисунок 122: Две кнопки в нижнем горизонтальном LinearLayout

- Удалите веса из обеих виджетов `Button` через панель атрибутов:

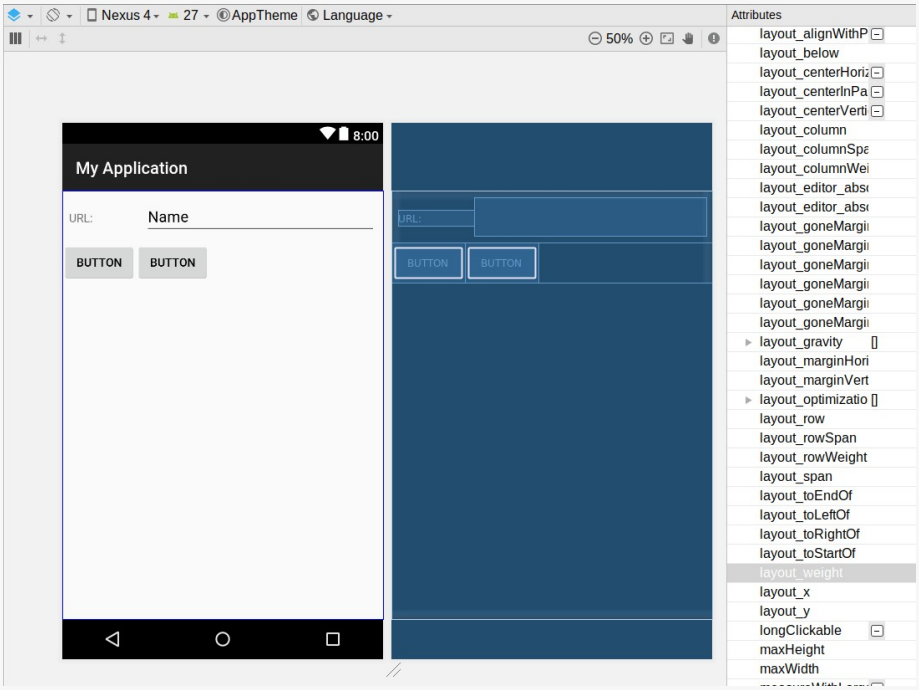


Рисунок 123: Две меньшие кнопки в нижнем горизонтальном LinearLayout

- Измените `gravity` нижнего `горизонтального LinearLayout` на `end`, используя панель атрибутов:

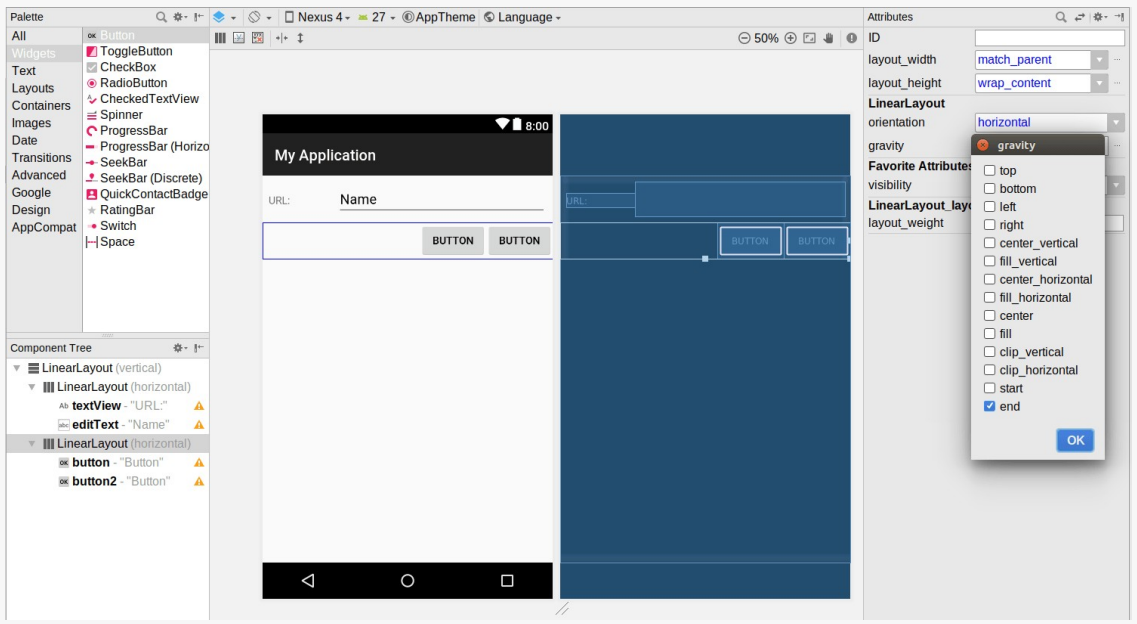


Рисунок 124: Две меньшие прикрепленные справа кнопки в нижнем

- Измените надписи на кнопках на “OK” и “Cancel”:

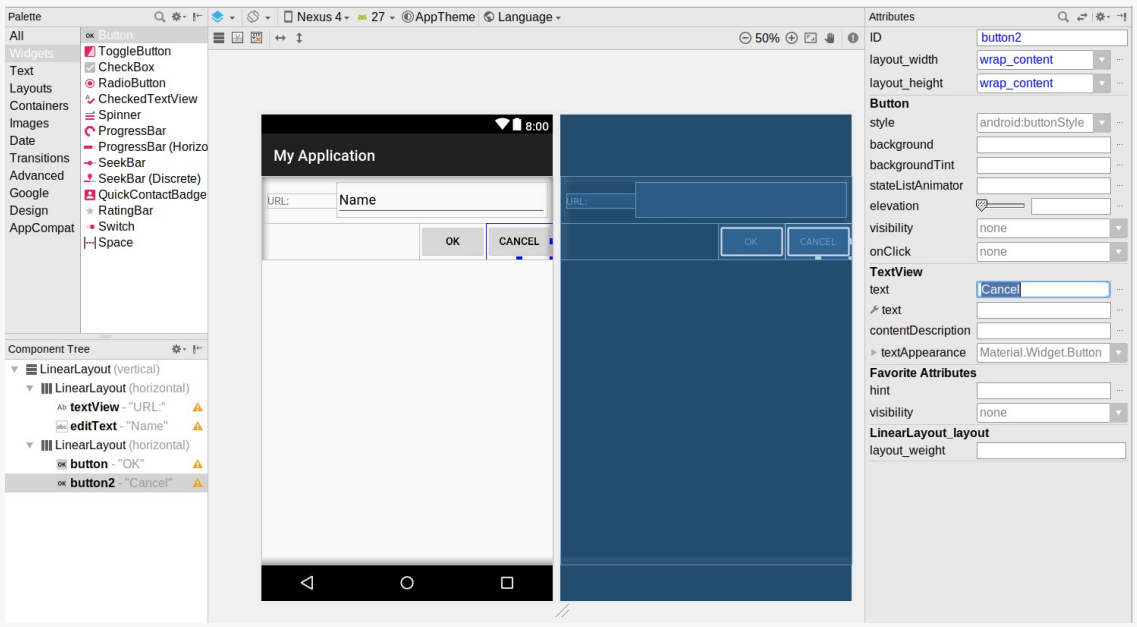


Рисунок 125: Кнопки с установленными надписями

Сейчас, выполнив все, самая правая кнопка панели инструментов над предпросмотром и чертежом будет желтым треугольником, указывающая на некоторые предупреждения в макеты который мы создали. Нажатие на этот значок отобразит панель с сообщениями объясняющими, что Android Studio не понравилось:

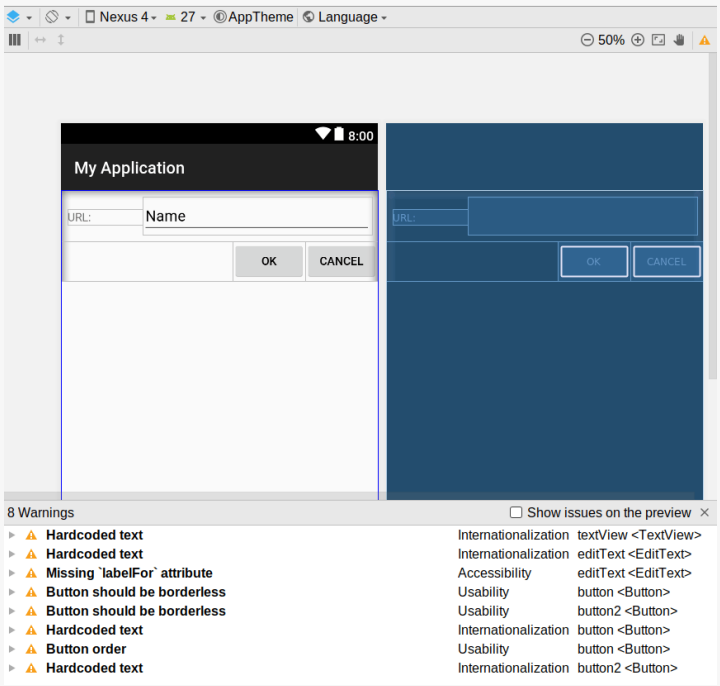


Рисунок 126: Сообщения о предупреждениях макета

Каждое из предупреждений в списке обычно содержит пояснения, некоторые предлагаемые исправления, и другую объясняющую информацию. Нажатие на кнопку для предложенного исправления применит это исправление и очистит это предупреждение.

Некоторые предупреждения могут происходить из надписей виджетов `TextView` и `Button` Если вы просто ввели строку в панели `атрибутов`, которая заполняет литеральную строку в макете, и идеально мы используем строковый ресурс. Для этого предлагаемое исправление “Extract string resource” (извлечь строковый ресурс) позволит Вам описать его как строковый ресурс.

Некоторые и этих предупреждений об общих элементах в стилях (например “Buttons in button bars should be borderless” (кнопки в панели кнопок должны быть без границ)). Хотя в долгосрочной перспективе Вы должны позаботиться

об этих предупреждениях, в краткосрочной, когда Вы изучаете Андроид, игнорирование предупреждений используя предлагаемые исправления - это хорошая идея.

Одним из этих предупреждений будет о пропущенном атрибуте `android:labelFor` , которы

часть системы дополнительных возможностей в Андроид, которую мы рассмотрим позже. Другая о порядке кнопок, с которым Google не согласен.

Пример: Большая форма

Не все в форме может быть внутри горизонтального `LinearLayout` который в свою очередь внутри вертикального `LinearLayout`. Вам нужен горизонтальный `LinearLayout` когда у Вас строка которая содержит два и более виджета. Для строк из одного виджета, они могут быть простыми потомками от вертикального `LinearLayout`.

К примеру, возможно у Вас есть более сложная форма в уме, с несколькими полями и другими виджетами, как следующая:

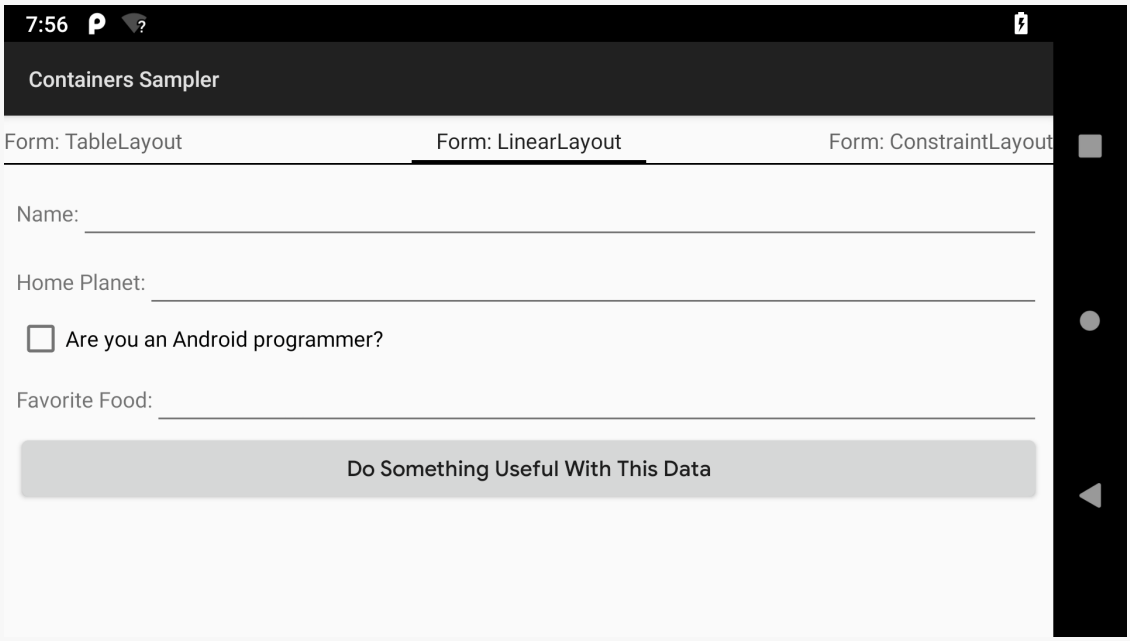


Рисунок 127: Макет формы, применяя LinearLayout

XML

Ресурс макета формы похожей на сценарий URL-диалога ранее:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
```

```
android:padding="8dp">

<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="horizontal">
  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/name" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="text" />
</LinearLayout>

<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="horizontal">
  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/home_planet" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="text" />
</LinearLayout>

<CheckBox
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:text="@string/android_programmer" />

<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="horizontal">
  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/favorite_food" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="text" />
</LinearLayout>
```

```
  <Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/do_something" />

</LinearLayout>
```

(из Containers/Sampler/app/src/main/res/layout/form_ll.xml)

Тем не менее отметьте, что `CheckBox` и `Button` сами не обернуты в контейнер `горизонтальный LinearLayout`. Они просто прямые потомки `вертикального LinearLayout`. В данном случаи, оба установлены с шириной `match_parent`, хотя это и не обязательно - Вы можете установить их в `wrap_content` по предпочтению.

Мы рассмотрим `CheckBox` более детально в последующей главе.

Графический редактор макетов Android Studio

Создание формы следует тому же простому рецепту, который мы использовали для более простой формы диалога ранее в этой главе:

- Установите корень ресурса макета в `вертикальный LinearLayout`
- Перенесите `горизонтальный LinearLayout` в `вертикальный LinearLayout`
- Перенесите `TextView` и `EditText` в `горизонтальный LinearLayout` и настройте как необходимо
- Повторите эти последние два шага для каждой из других пяти строк

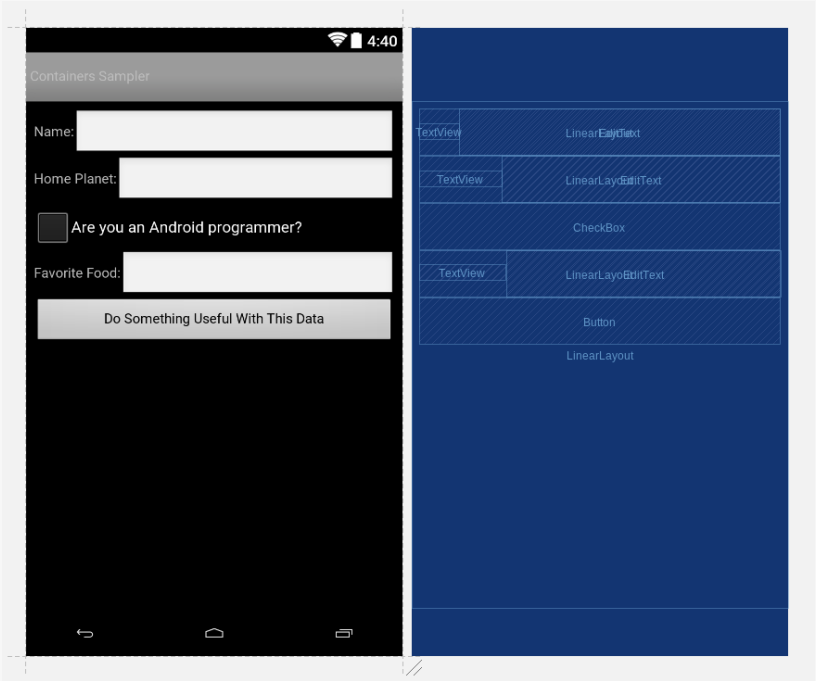


Рисунок 128: Большая форма, при отображении в Android Studio

Проблема

Если Вы посмотрите на снимок экрана, Вы заметите виджеты `EditText` с надписями неровные, с точки зрения их макета. Каждый `EditText` следует непосредственно после надписи `TextView` , несмотря на какой-либо тип "столбцов". Это потому, что каждый `LinearLayout` во многом независимы. Вы не можете изначально получить, чтобы одна строка зависела от других строк.

`TableLayout` будет лучшим выбором для данного типа формы, так как мы сможем получить отдельные столбцы для надписей и полей. Мы увидим как `TableLayout` описывает данную структуру позже в данной главе.

Все вещи относительны

`RelativeLayout`, как предполагает название, располагает виджеты основываясь на их взаимном расположении к другим виджетам в контейнере и к родительскому контейнеру. Вы можете расположить виджет X ниже и слева от виджета Y, или расположить нижнюю границу виджета Z на нижней границе контейнера и т.д.

Концепт и атрибуты

Чтобы все это работало, нам нужен способ ссылаться на другие виджеты внутри файла макета XML, плюс способ указать относительное положение этих виджетов.

Расположение относительно контейнера

Самое простое отношение для настройки - это привязка положения виджета к следующему его контейнера:

1. `android:layout_alignParentTop` говорит, что верх виджета должен совпадать с верхом контейнера
2. `android:layout_alignParentBottom` говорит, что низ виджета должен совпадать с низом контейнера
3. `android:layout_alignParentStart` говорит, что начальная сторона виджета должен совпадать с начальной стороной контейнера
4. `android:layout_alignParentEnd` говорит, что конечная сторона виджета должен совпадать с конечной стороной контейнера
5. `android:layout_centerHorizontal` говорит, что виджет должен быть расположен горизонтально по центру контейнера
6. `android:layout_centerVertical` говорит, что виджет должен быть расположен вертикально по центру контейнера
7. `android:layout_centerInParent` говорит, что виджет должен быть расположен горизонтально и вертикально по центру контейнера

Все данные атрибуты принимают простое булево значение (`true` или `false`) Также есть атрибуты `android:layout_alignParentLeft` и `android:layout_alignParentRight`, для устройств до Андроид 4.2 или в случаи когда неоходимо расположить независимо от направления языка.

Обратите внимание, что отступы виджета учитываются при выполнении этих различных выравниваний. Выравнивание базируется на общей ячейке

виджета (комбинации естественного размера плюс отступ)

Относительные нотации в атрибутах

Оставшиеся атрибуты относящиеся к `RelativeLayout` принимают значение идентичные виджету в контейнере. Чтобы сделать это:

- Установите идентификаторы (атрибуты `android:id`) на все элементы на которые необходимо ссылаться
- Ссылайтесь на эти виджеты из других виджетов с применением идентификаторов

Первый упоминание значения идентификатора должно содержать знак плюс (`@+id/widget_a`); второе и последующее использование идентификатора в файле макета должно пропускать знак плюс (`@id/widget_a`). Это позволяет инструменту сборки лучше помогать в отслеживании ошибок типов в значениях `идентификаторов` виджетов - если у Вас нет знака плюс для значения `идентификатора` виджета, которого не было ранее, это приведет к ошибке во время компиляции. Например, если виджет A появляется в `RelativeLayout` перед виджетом B и идентификатор виджета A `@+id/widget_a`, виджет B может ссылаться на виджет A в одном из своих атрибутов через идентификатор `@id/widget_a`.

Расположение относительно других виджетов

Существует четыре атрибута, которые управляют расположением виджета *по отношению к* другим виджетам:

1. `android:layout_above` указывает, что виджет должен располагаться над виджетом указанным в свойстве
2. `android:layout_below` указывает, что виджет должен располагаться под виджетом указанным в свойстве
3. `android:layout_toStartOf` указывает, что виджет должен располагаться с начала виджета указанного в свойстве
4. `android:layout_toEndOf` указывает, что виджет должен располагаться с конца виджета указанного в свойстве

Существуют также атрибуты `android:layout_toLeftOf` и `android:layout_toRightOf` для использования на более старых устройствах.

Кроме этих четырех, существует пять дополнительных атрибутов, которые могут контролировать выравнивание виджета по отношению к другим:

1. `android:layout_alignTop` указывает, что верх виджета должен быть выровнен с верхом виджета указанном в свойстве

2 `android:layout_alignBottom` указывает, что низ виджета должен быть выровнен с низом виджета указанном в свойстве

3 `android:layout_alignStart` указывает, что начальная граница виджета должна быть выровнена с начальной границей виджета указанном в свойстве
4 `android:layout_alignEnd` указывает, что конечная граница виджета должна быть выровнена с конечной границей виджета указанном в свойстве
5 `android:layout_alignBaseline` указывает, что базовая линия двух виджетов должна быть выровнена (где базовая линия - невидимая линия на которой располагается текст)

Последний удобный для выравнивания надписей и полей, чтобы текст появлялся "натурально". Так как поля имеют рамку вокруг них а надписи нет, `android:layout_alignTop` будет выравнивать верх рамки поля с верхом надписи, что приведет к тому, что текст надписи будет выше на экране чем текст вводимый в поле.

Графический редактор макетов Android Studio

Вы найдете `RelativeLayout` в разделе “Legacy” палитры графического редактора макетов Android Studio. Вы можете перенести его в Ваш XML ресурс макета.

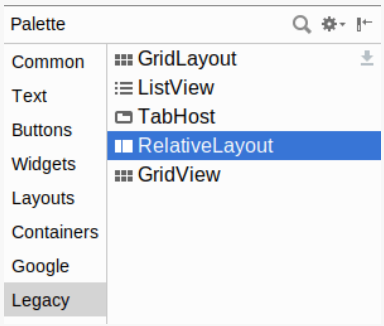


Рисунок 129: Раздел Legacy палитры, RelativeLayout подсвечен

Когда Вы переносите другие виджеты в Ваш `RelativeLayout`, Вы увидите подсказывающие стрелки для правил, которые будут применены если вы бросите виджет в текущем положении мыши:

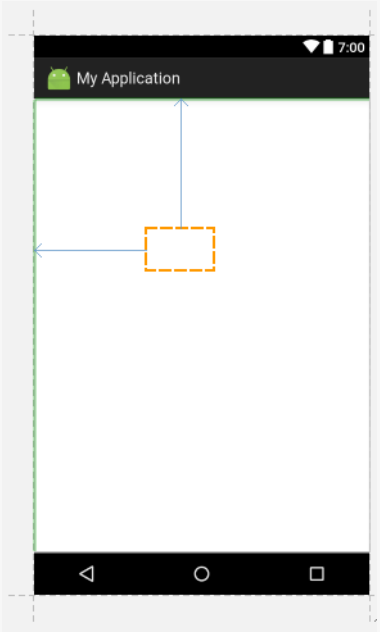


Рисунок 130: Перенос виджета в RelativeLayout

Получение правил, которые Вы *хотите* можно получить и можно и не получить через простое перетаскивание. Может понадобиться просто перенести виджет в `RelativeLayout` и вручную назначить правила, или используя панель атрибутов или напрямую редактируя XML.

Пример: Низ-после-Верх

Ранее в главе, мы видели как выполнить паттерн низ-после-верх, используя`LinearLayout`, где у нас была маленькая кнопка внизу и большая кнопка вверху. Большая кнопка была настроена занимать все пространство, которое не было необходимо для маленькой кнопки.

Мы можем получить те же результаты используя `RelativeLayout`.

XML

Как и при сценарии с `LinearLayout` у нас один контейнер плюс два виджета `Button`. В данном случаи контейнер `RelativeLayout`:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent">

  <Button
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_above="@+id/another_button"
    android:layout_alignParentTop="true"
    android:text="@string/button" />

  <Button
    android:id="@id/another_button"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true"
    android:text="@string/another_button" />
```

(из Containers/Sampler/app/src/main/res/layout/bottom_then_top_rl.xml)

Нижний виджет `Button` с таким же размером как ранее, с `android:layout_width` установленным в `match_parent` и `android:layout_height` установленным в `wrap_content`. Кроме того, также с `android:layout_alignParentBottom="true"`, прикрепляющим его внизу `RelativeLayout`. Так как `RelativeLayout` заполняет все доступное пространство, нижняя `Button` прикреплена к низу экрана, фактически.

Верхняя `Button` содержит два атрибута позиционирования `RelativeLayout`:

- `android:layout_above="@+id/another_button"`, так она расположена над нижней `Button`
- `android:layout_alignParentTop="true"`, так она прикреплена к верху `RelativeLayout`

С данными правила не имеет значения какие значения атрибутов `android:layout_height` `Button` будет растянута между этими двумя точками прикрепления: верхом `RelativeLayout` и верхом нижней `Button`.

Графический редактор макетов Android Studio

По умолчанию, при создании нового файла ресурса макета, Вы получаете или `ConstraintLayout` или `вертикальный LinearLayout` как корневой элемент. Вы можете изменить это заменив значение “Root element” (корневой элемент)

любым другим названием класса виджет или контейнер

как `RelativeLayout`:

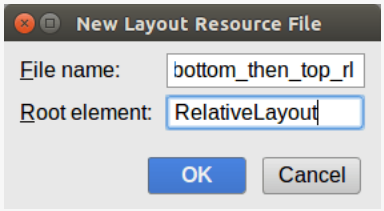


Рисунок 131: Новый ресурс макета с RelativeLayout

Тем не менее, начиная с этого, применение возможностей переместить-и-бросить становятся более болезненными чем они должны быть.

Вы можете перенести `Button` в `RelativeLayout`, к примеру:

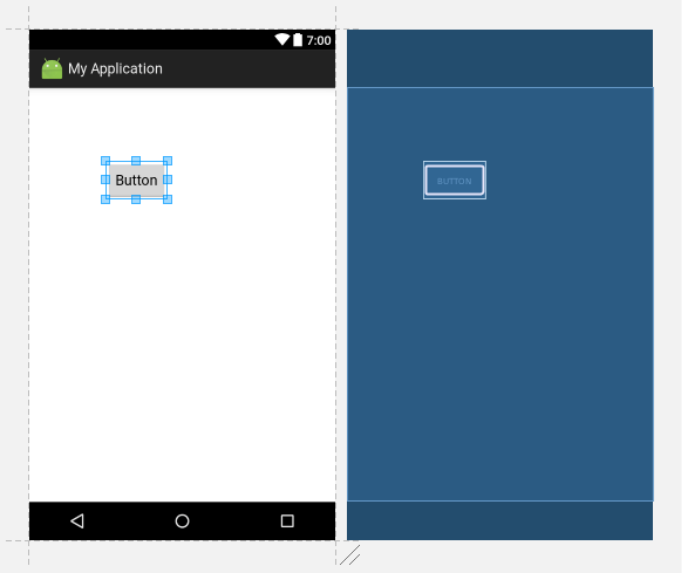


Рисунок 132: RelativeLayout и Button

Тем не менее, Вы не получите визуального отклика после переноса виджета какие правила выбрала IDE, основываясь на местоположение вашего переноса. Если Вы поищите в панели атрибутов, ВЫ увидите что правила изменили некоторые булевы атрибуты `RelativeLayout android:layout_` , плюс отступы основываясь как далеко от границ `RelativeLayout` Вы расположили виджет:

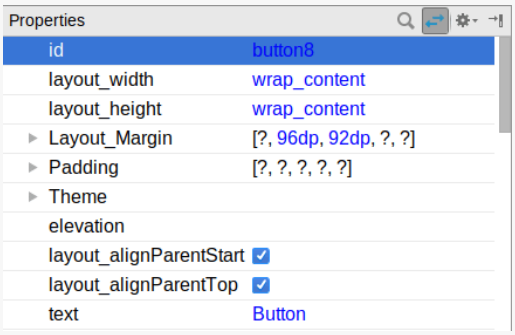


Рисунок 133: Атрибуты Button в RelativeLayout

Исправление этого через панель атрибутов не легче чем исправление их через редактор XML. Пожалуй, использование панели атрибутов медленнее.

Пример: URL Диалог

Мы использовали `LinearLayout` для создания интерфейса пользователя “URL dialog”, где у нас были поля с надписями вместе с кнопками “OK” и “Cancel” Такая же структура может быть построена с применением `RelativeLayout`.

С `LinearLayout`, нам необходимо было три контейнера: один вертикальный `LinearLayout` обернутый вокруг двух горизонтальных `LinearLayout`. `RelativeLayout` проще с этой точки зрения, так нам нужен только один `RelativeLayout`. Сложность вместо этого переходит к виджетам:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="wrap_content">

  <TextView
    android:id="@+id/label"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignBaseline="@+id/entry"
```

```
    android:layout_alignParentLeft="true"
    android:layout_alignParentStart="true"
    android:layout_marginLeft="4dp"
    android:layout_marginStart="4dp"
    android:text="@string/url"/>

  <EditText
    android:id="@id/entry"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:layout_toRightOf="@id/label"
    android:layout_toEndOf="@id/label"
    android:inputType="text"/>

  <Button
    android:id="@+id/ok"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignRight="@id/entry"
    android:layout_alignEnd="@id/entry"
    android:layout_below="@id/entry"
    android:text="@string/ok"/>

  <Button
    android:id="@+id/cancel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignTop="@id/ok"
    android:layout_toLeftOf="@id/ok"
    android:text="@string/cancel"/>

</RelativeLayout>
```

(из Containers/Sampler/app/src/main/res/layout/url_dialog_rl.xml)

Всё что нужно самому `RelativeLayout` это размеры, установленные такими же как в `вертикальном LinearLayout` в ранней версии примера - ширина в `match_parent` и высота в `wrap_content`.

Другие чем `android:layout_width` и `android:layout_height`, атрибуты виджета с `layout_` это правила используемые потомками `RelativeLayout` для позиционирования. Так, мы имеем:

Виджет	Горизонтальный якорь	Вертикальный якорь
TextView	левая/начальная сторона RelativeLayout, с отступом 4dp	базовая линии EditText
EditText	правая/конечная сторона TextView	верх RelativeLayout
“OK” Кнопки	правая/конечная сторона EditText	низ EditText
“Cancel” Кнопки	левая/начальная сторона кнопки “OK”	верх “OK” Кнопки

Так как ширина EditText установлена в match_parent, он будет заполнять всё пространство в "строке" после TextView. Так как горизонтальное расположение кнопки “OK” привязано к EditText, кнопка переместится к границе экрана, перемещая также присоединенную кнопку “Cancel”.

Как результат, мы получим некоторый базовый UI:

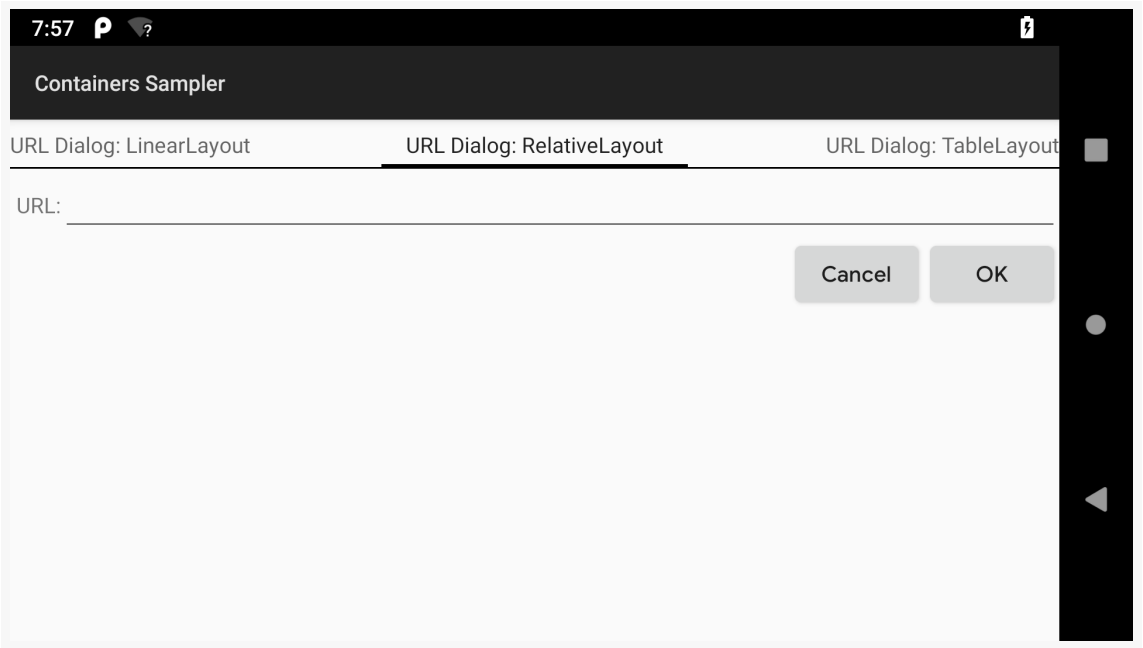


Рисунок 134: Макет URL-диалога, применяя RelativeLayout

Пример: Наложение

`RelativeLayout` также содержит особенность, которой нет у `LinearLayout` - возможность содержать виджеты, которые накладываются один на другого. Поздние потомки `RelativeLayout` "выше по оси Z" чем ранние потомки, что означает что поздние потомки накладываются на ранние потомки, если они настроены занимать то же пространство в макете.

Здесь у нас две кнопки, где кнопка “I am small” налаживается на кнопку “I am big”:



Рисунок 135: Макет с наложением, применяя RelativeLayout

Макет довольно прост:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent">

  <Button
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:text="@string/big"
    android:textSize="120dip"
```

```
  <Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerInParent="true"
    android:text="@string/small"/>

</RelativeLayout>
```

(из Containers/Sampler/app/src/main/res/layout/overlap_rl.xml)

Первая `Button` установлена заполнять экран. Вторая `Button` установлена в центре внутри родителя, но занимать только столько пространства сколько необходимо для её надписи. Следовательно, вторая `Button` отображается "плавающей" над первой `Button`.

Оба виджета `Button` могут быть нажаты, хотя нажатие на меньшую `Button` не нажимает большую `Button`. Ваши нажатия будут обработаны верхним виджетом в подобном случаи наложения.

На Андроид 5.0 и выше, возможно достигнуть похожего эффекта с `LinearLayout` используя атрибут `android:elevation` для управления осью Z, где большее значение превышения означает выше по оси Z, плавая над теми у которых меньшее значение по оси Z.

Tabula Rasa

Если Вам нравятся таблицы HTML, Вам понравиться Андроид `TableLayout`. Он позволяет Вам располагать Ваши виджеты в сетке с Вашими характеристиками. Вы управляете количеством строк и столбцов, столбцы могут сжиматься или растягиваться приспосабливаясь к их содержимому, и так далее.

`TableLayout` работает совместно с `TableRow`. `TableLayout` управляет общим поведением контейнера, с самими виджетами влитыми в один или более контейнеры `TableRow` , один на строку в сетке.

Концепт и атрибуты

Чтобы это все работало, нам необходимо понять как виджеты работают со строками и столбцами, плюс как управляться с виджетами, которые живут за пределами строк.

Расположение Ячеек в Строках

Строки объявляются Вами, разработчиком, располагая виджеты как потомки `TableRow` внутри общего `TableLayout`. Вы, следовательно, управляете напрямую сколько строк появиться в таблице.

Количество столбцов определяется Андроидом, Вы управляете количеством столбцов непрямым путем.

Сперва, должен быть как минимум один столбец на виджет в Вашей самой длинной строке. Так если у Вас три строки, одна с двумя виджетами, одна с тремя виджетами, и одна с четырьмя виджетами, должно быть как минимум четыре столбца.

Тем не менее, могут занимать более чем один столбец с добавлением свойства `android:layout_span` , указывающим количество столбцов, которое охватывает виджет. Это как свойство `colspan` , которое можно найти в ячейках таблицы в HTML:

```
<TableRow>
  <TextView android:text="URL:" />
  <EditText
    android:id="@+id/entry"
    android:layout_span="3"/>
</TableRow>
```

В указанном выше фрагменте макета XML, поле охватывает три столбца.

Обычно, виджеты располагаются в первом доступном столбце. В фрагменте выше надпись расположится в первом столбце (столбец `0`, так как столбцы считаются начиная с `0`), и поле расположится в охватывающем три столбца наборе (столбцы от `1` до `3`). Тем не менее, Вы можете расположить виджет в другой столбец через свойство `android:layout_column` , указывая начиная с `0` столбец которому принадлежит виджет:

```
<TableRow>
  <Button
    android:id="@+id/cancel"
    android:layout_column="2"
    android:text="Cancel" />
  <Button android:id="@+id/ok" android:text="OK" />
</TableRow>
```

В предшествующем фрагменте макета XML, кнопка Cancel размещается в

третьем столбце (столбец 2). Кнопка OK тогда размещается в следующем доступном столбце, который

четвертый столбец.

Потомки не-строки TableLayout

Обычно `TableLayout` содержит только элементы `TableRow` как прямые потомки. Тем не менее возможно расположить другие виджеты между строк. Для таких виджетов поведение `TableLayout` немного схоже с `LinearLayout` с вертикальной ориентировкой. Виджеты автоматически имеют установленную ширину на `match_parent`, так они заполняют тоже пространство что и самая длинная строка.

Растягивание, Сжимание и Коллапсирование

По умолчанию размер каждого столбца будет в соответствии с «естественным» размером самого широкого виджета в этом столбце (с учетом столбцов которые соединены). Иногда, также, это не работает очень хорошо, и Вам необходимо больше контроля над поведением столбцов.

Вы можете расположить свойство `android:stretchColumns` в `TableLayout`. Это перечисляет столбец или столбцы, которые должны включать любое дополнительное пространство в строке, если естественная ширина столбцов коллективно уже чем доступное горизонтальное пространство.
Вы можете:

- Перечислить единичный столбец для растягивания (например, `android:stretchColumns="0"` для растягивания первого столбца)
- Предоставить разделенный запятой список столбцов для растягивания (например, `android:stretchColumns="0,1"` для растягивания первых двух столбцов)
- Использовать `*` для указания того, что все столбцы должны быть растянуты, аналогично использованию значения `android:layout_weight` в горизонтальном `LinearLayout` (например, `android:stretchColumns="*"`)

Наоборот, Вы можете расположить свойство `android:shrinkColumns` в `TableLayout`. Опять, это должен быть единичный номер столбца, разделенный запятой список столбцов, или `*` как упрощенная запись для ссылки на все столбцы. Столбцы указанные в свойстве будут пытаться выполнить перенос слова их содержимого чтобы уменьшить эффективную ширину столбца - по умолчанию, виджеты без переноса слов. Это поможет если у Вас есть столбец с потенциально многословным содержимым, что может привести к тому, что некоторые столбцы будут выходить за правый край экрана.

Вы также можете использовать свойство `android:collapseColumns` в `TableLayout`, снова с номером столбца или с разделенным запятой списком столбцов (`*` не задокументирована как доступная опция). Эти столбцы начнут "коллапсировать",

означая, что они будут частью информации таблицы, но они будут невидимы. Программно, коллапсировать и де коллапсировать столбцы вызывая `setColumnCollapsed()` в `TableLayout`. Вы можете использовать это позволяя пользователям управлять какие столбцы им важны и должны быть показаны и какие менее важны и могут быть скрыты.

Вы можете также управлять растягиванием и сжиманием при выполнении программы чере `setColumnStretchable()` и `setColumnShrinkable()`.

Графический редактор макетов Android Studio

Вы найдете `TableLayout` и `TableRow` в разделе “Layouts” Палитры в графическом редакторе макетов Android Studio

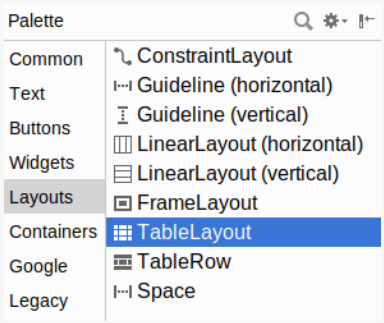


Рисунок 136: Раздел Layouts палитры, TableLayout выделен

Имея `TableLayout`, Вы можете переместить один или более контейнер `TableRow` в него, после начать перетаскивать виджеты в строки, так же, как Вы могли бы настроить вложенные контейнеры `Linearlayout`.

Пример: Большая форма

Одна область в которой `TableLayout` превосходит формы, особенно если Вы используете классическую структуру "надпись и виджет" для формы. Это потому `TableLayout` может предоставить действительно столбцы, тогда как `LinearLayout` и `RelativeLayout` нет.

При большинстве применений `TableLayout` непосредственными потомками в большинстве случаев являются
`TableRow` контейнеры, каждый предоставляя содержание для столбцов:

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:padding="8dp"
  android:stretchColumns="1">

  <TableRow>
    <TextView android:text="@string/name" />
    <EditText android:inputType="text" />
  </TableRow>

  <TableRow>
    <TextView android:text="@string/home_planet" />
    <EditText android:inputType="text" />
  </TableRow>

  <TableRow>
    <CheckBox
      android:layout_column="1"
      android:text="@string/android_programmer" />
  </TableRow>

  <TableRow>
    <TextView android:text="@string/favorite_food" />
    <EditText android:inputType="text" />
  </TableRow>

  <Button android:text="@string/do_something" />

</TableLayout>
```

(из Containers/Sampler/app/src/main/res/layout/form_tl.xml)

Сам `TableLayout` содержит `android:stretchColumns="1"`, поэтому все оставшееся пространство строк перейдет во второй столбец (первый столбец имеет индекс 0).

Первая, вторая и четвертая `TableRow` каждая имеет ту же структуру с надписью `TextView` предшествующей `EditText` где пользователь может заполнить данные. Третья `TableRow`, однако имеет только одного потомка: `CheckBox`. И, наш `Button` лежит за пределами какой-либо `TableRow`, как непосредственный потомок корневого `TableLayout`. Оба `CheckBox` и the `Button` будут существовать в их собственной строке. Разница в том что `CheckBox` переходит в второй столбец, как результат `android:layout_column="1"`, тогда как `Button` будет охватывать всю строку (как контейнеры `TableRow` охватывают всю ширину `TableLayout`).

Так, сравнивая с оригинальной версией `LinearLayout` этого примера, наш `TableLayout` столбцы аккуратны и выровнены:

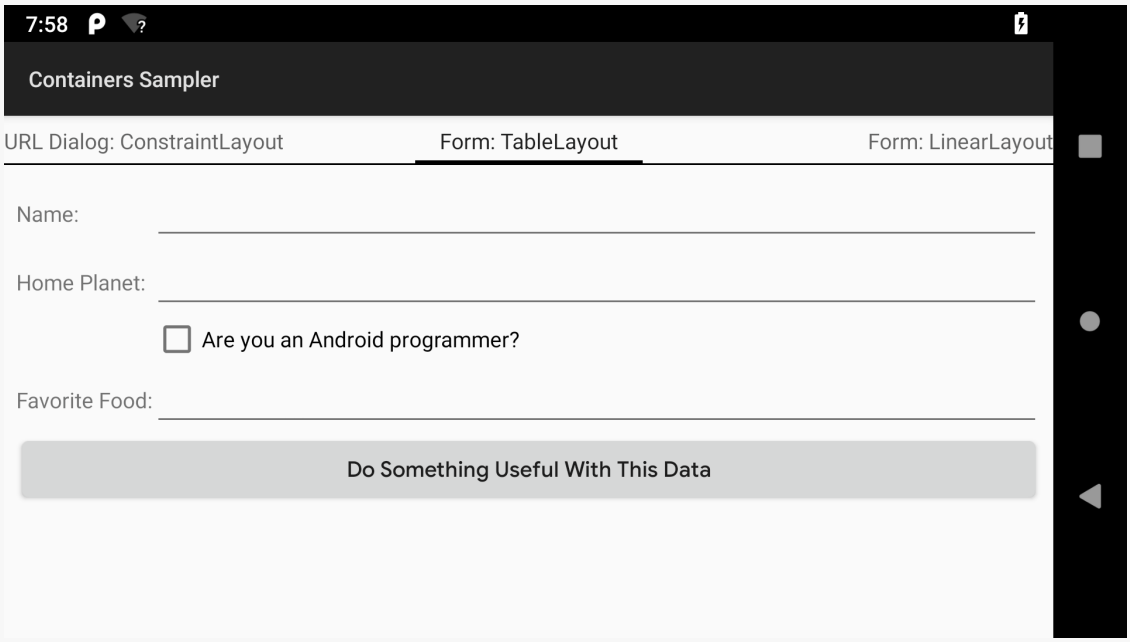


Рисунок 137: Макет Формы, используя TableLayout

Пример: URL Диалог

Макет “URL диалог”, указанный ранее реализован используя `LinearLayout` и `RelativeLayout`, может также быть реализован используя `TableLayout`. Это не наиболее обычное использование `TableLayout`, но Вы можете сделать это если захотите.

Как в примере формы выше, мы начинаем с корневого `TableLayout` имеющего `android:stretchColumns="1"` , чтобы предоставить все дополнительное пространство второму столбцу... несмотря на то, что на этот раз мы получим в общей сложности четыре столбца:

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:stretchColumns="1">

  <TableRow>

    <TextView
      android:layout_marginLeft="4dip"
```

```
      android:layout_marginStart="4dip"
      android:text="@string/url" />

    <EditText
      android:id="@+id/entry"
      android:layout_span="3"
      android:inputType="text" />
  </TableRow>

  <TableRow>

    <Button
      android:id="@+id/cancel"
      android:layout_column="2"
      android:text="@string/cancel" />

    <Button
      android:id="@+id/ok"
      android:text="@string/ok" />
  </TableRow>

</TableLayout>
```

(из Containers/Sampler/app/src/main/res/layout/url_dialog_tl.xml)

`EditText` в первой `TableRow` имеет `android:layout_span="3"`, указывая что он должен охватывать заполняя три столбца. Это, плюс наша одна `TextView`, означает что первая строка настроена для четырех столбцов в сумме.

Первая `Button` во второй `TableRow` имеет `android:layout_column="2"`, указывая что она должна перейти в третий столбец. Другая `Button` переходит в следующий столбец (четвертый столбец в нашем случаи), и первые два столбца пропускаются. Так, эта строка также настроена для четырех столбцов.

Так, когда применено `android:stretchColumns="1"` , дополнительное пространство будет предоставлено "содержимому" второго столбца:

- `EditText` в первой строке
- пустое пространство предшествует двум виджетам `Button` во второй строке

Эй, что о ConstraintLayout?

В 2016, Google представила `ConstraintLayout`, с видением того, что он станет четвертым из основных контейнеров и возможно тем, который Вы будете выбирать по умолчанию.

`ConstraintLayout` имеет свои преимущества, определенно. Тем не менее, он требует использования библиотеки, и мы еще не рассматривали как добавлять библиотеки в модуль Андроид.

Так, мы обсудим `ConstraintLayout` немного позже в книге.

Возвращение к RTL

Чтобы атрибуты “start”/“end” работали, Вам необходимо иметь `android:supportsRtl="true"` в Вашем элементе `<application>` в Вашем манифесте. Большинство ново-созданных проектов будут иметь этот атрибут уже установленным для Вас мастером новых проектов.

Чтобы увидеть как поведение вашего приложения с RTL - без необходимости изучать Арабский или Иврит, если у Вас нет образования в этих языках - Вы можете заставить Андроид использовать правила RTL макета с любым языком на устройствах Андроид 4.2+ Чтобы сделать это, перейдите в приложение Настройки устройства или эмулятора и выберите "Developer options". Там, прокрутите вниз к элементу “Force RTL layout direction” По умолчанию, он выключен, и поэтому направление макета определяется языком, выбранным пользователем:

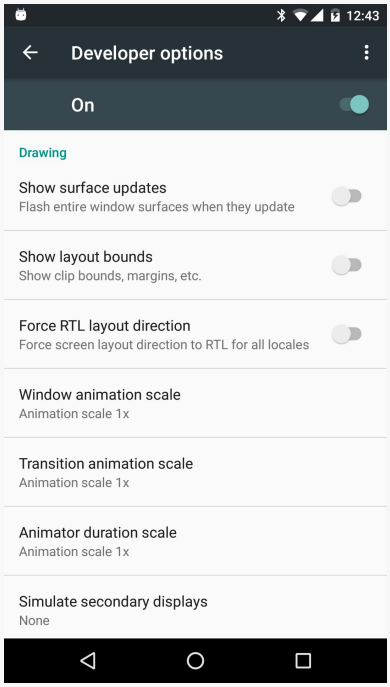


Рисунок 138: Опции разработчика в Настройках, Нормальный Режим

Нажатие, которое переключает правила макета RTL - когда “start” относиться к справа и “end” относиться к слева - для всех языков:

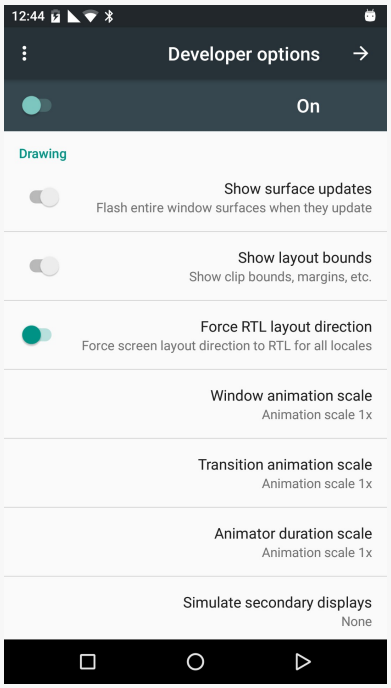


Рисунок 139: Опции разработчика в Настройках, принудительный режим

Как напоминание, если “Developer options” не в списке категорий Настройки, перейдите в категорию “About device”, найдите номер сборки и нажмите на него семь раз. Это активирует “Developer options” в основном списке категорий Настройки.

Другие Общие Виджеты и Контейнеры

В разделе базовых виджетов, мы пропустили все классические виджеты "двух-состояний", таких как checkbox и radio button. Мы рассмотрим эти и другие относящиеся виджеты в этой главе.

Кроме классических контейнеров общего назначения (`LinearLayout`, `RelativeLayout`, `TableLayout`), есть другие специализированные контейнеры, такие как `FrameLayout` и `RadioGroup`, которые Вы будете использовать время от времени. Мы также рассмотрим их в данной главе.

Просто Box to Check

Классический checkbox имеет два состояния: включен и выключен. Нажатие на checkbox переключает между этими состояниями для указывания выбора(например "Добавить срочную доставку к моему заказу").

В Андроид, есть виджет `CheckBox` для этих потребностей. Он содержит `TextView` , как предок, поэтому Вы можете использовать свойства `TextView` такие как `android:textColor` для форматирования виджета.

В Java, Вы можете вызвать:

1. `isChecked()` чтобы определить что checkbox включен
2. `setChecked()` для принудительной установки состояний включен или выключен
3. `toggle()` чтобы переключить checkbox как если

пользователь нажал на него. Также Вы можете

зарегистрировать объект listener (в данном случаи, экземпляр

`OnCheckedChangeListener`) чтобы быть уведомленным когда состояние checkbox изменено.

Для примера, из `Basic/CheckBox` проекта примера, здесь простой макет checkbox:

```
<?xml version="1.0" encoding="utf-8"?>
<CheckBox xmlns:android="http://schemas.android.com/apk/res/android"
  android:id="@+id/check"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:text="@string/unchecked"/>
```

(из Basic/CheckBox/app/src/main/res/layout/main.xml)

Соответствующий `CheckBoxDemo.java` извлекает и настраивает поведение checkbox:

```
package com.commonsware.android.checkbox;

import android.app.Activity;
import android.os.Bundle;
import
android.widget.CheckBox;
import android.widget.CompoundButton;

public class CheckBoxDemo extends Activity implements
    CompoundButton.OnCheckedChangeListener {
  CheckBox cb;

  @Override
  public void onCreate(Bundle state) {
    super.onCreate(state);
    setContentView(R.layout.main);

    cb=(CheckBox)findViewById(R.id.check);
    cb.setOnCheckedChangeListener(this);
  }

  public void onCheckedChanged(CompoundButton buttonView,
                               boolean isChecked) {
    if (isChecked) {
      cb.setText(R.string.checked);
    }
    else {
      cb.setText(R.string.unchecked);
    }
  }
```

(из Basic/CheckBox/app/src/main/java/com/commonsware/android/checkbox/CheckBoxDemo.java)

Отметьте, что активность служит как собственный listener для изменений состояния checkbox так как реализует интерфейс `OnCheckedChangeListener` (установленный через `cb.setOnCheckedChangeListener(this)`). Обратным вызовом для listener является `onCheckedChanged()`, который получает состояние тех checkbox которые изменены и каким является это состояние. В данном случаи мы обновляем текст checkbox чтобы реагировать что именно содержит контейнер.

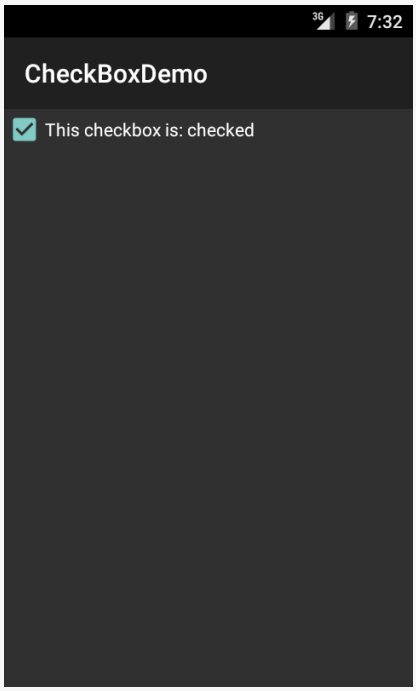


Рисунок 140: Приложение пример CheckBoxDemo, с включенным CheckBox

Графический редактор макетов Android Studio

Виджет `CheckBox` может быть найден в разделе “Buttons” палитры в графическом редакторе макетов Android Studio:

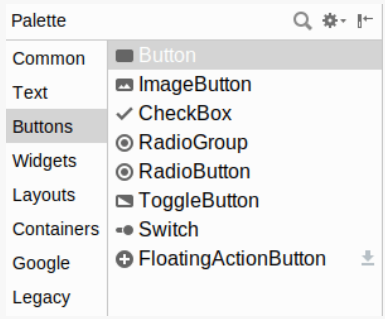


Рисунок 141: Палитра виджетов, CheckBox отображен выделением

Вы можете перетащить его в макет и настроить его как необходимо в панели атрибутов. Так как `CheckBox` наследуется от `TextView`, большинство настроек те же, что вы можете найти в стандартном `TextView`.

Вам не нравятся Checkboxes? Как насчёт Toggles и Switches

Похожий на `CheckBox` виджет это `ToggleButton`. Как и `CheckBox`, `ToggleButton` это виджет двух состояний который или включен или выключен. Тем не менее, `ToggleButton` имеет особое визуальное представление:

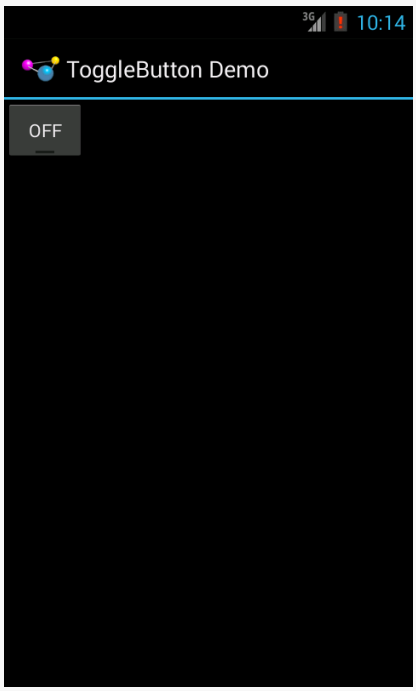


Рисунок 142:Пример ToggleButtonDemo, выключен

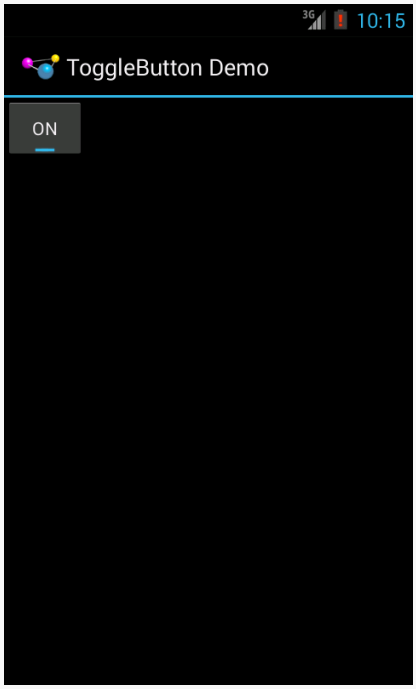


Рисунок 143:Пример ToggleButtonDemo, включен

В других отношениях поведение `ToggleButton` в большинстве такое же как и `CheckBox`. Вы можете добавить его в файл макета как показано в `Basic/ToggleButton` примере:

```
<?xml version="1.0" encoding="utf-8"?>
<ToggleButton xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/toggle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

(из Basic/ToggleButton/app/src/main/res/layout/main.xml)

Вы можете также настроить `OnCheckedChangeListener` чтобы быть уведомленным когда пользователь изменяет состояние `ToggleButton`.

Подобным образом, Андроид имеет виджет `Switch` , отображающий состояние через маленький слайдер “ON/OFF”

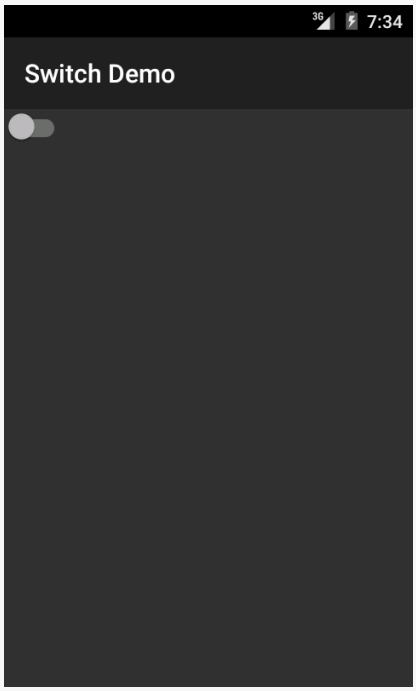


Рисунок 144:Пример SwitchDemo, выключен

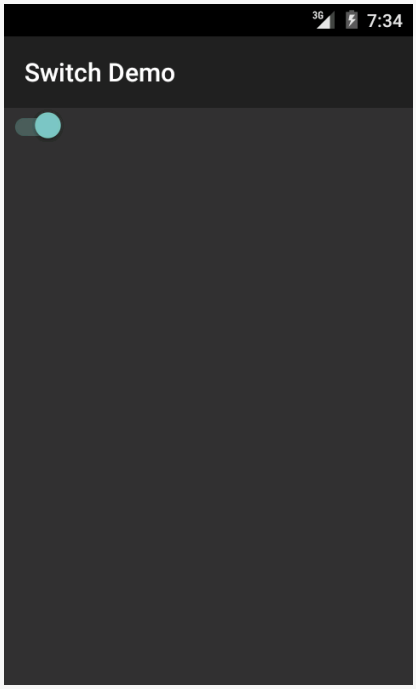


Рисунок 145:Пример SwitchDemo, включен

`Switch`, как `CheckBox` и `ToggleButton`, наследуется от `CompoundButton`, и потому разделяет общий API, для таких методов как `toggle()`, `isChecked()`, и `setChecked()`. И как и с другими Вы можете добавить его в файл макета, как показано в `Basic/Switch` примере:

```
<?xml version="1.0" encoding="utf-8"?>
<Switch xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/toggle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

(из Basic/Switch/app/src/main/res/layout/main.xml)

Наибольшее ограничение `Switch` в том что он был добавлен только в Android SDK с уровнем API 14. Если Ваше `minSdkVersion` установлено на 14 или выше, Вы можете использовать `Switch`. Если Ваше `minSdkVersion` установлен в ниже чем 14, так Вам либо нужно выбрать что-то другое или выбрать более сложный сценарий, такой как использование библиотеки, которая поддерживает обратную совместимость с `Switch`. Мы рассмотрим эти более сложные сценарии позже в книге; на данный момент, самое простое использовать `Switch` только если Ваше `minSdkVersion` установлено на 14 или выше.

Графический редактор макетов Android Studio

Виджеты `ToggleButton` и `Switch` могут быть найдены в части “Buttons” палитры в графическом редакторе макетов Android Studio.

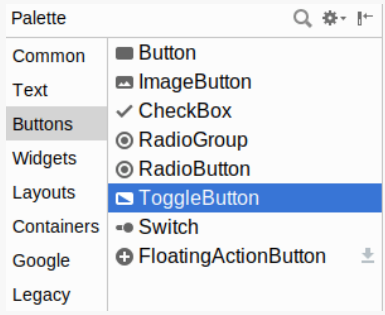


Рисунок 146: Палитра Виджетов, ToggleButton выделен

Вы также можете перетащить виджет в макет и настроить как необходимо используя панель атрибутов.

Включение Radio

Как и с другими реализациями радио кнопок в других наборах инструментах, радио кнопки Андроид имеет два состояния, как чекбоксы, но могут быть сгруппированы так, что только одна радио кнопка в группе может быть выбрана в определенный момент.

`CheckBox`, `ToggleButton`, `Switch`, и `RadioButton` наследуются от `CompoundButton`, которая в свою очередь наследуется от `TextView`. Следовательно, все стандартные свойства `TextView` для типа шрифта, стиля, цвета и т.д. доступны для управления отображением радио кнопок. Сходным образом, Вы можете вызвать `isChecked()` на `RadioButton` , чтобы увидеть, что она выбрана, `toggle()` чтобы изменить ее выбранное состояние, и так далее, как Вы можете с `CheckBox`.

В большинстве случаев Вы захотите расположить Ваш виджет `RadioButton` внутри `RadioGroup`. `RadioGroup` является `LinearLayout` который указывает на набор радио кнопок состояние которых связано, что означает, что только одна кнопка из группы может быть выбрана в какое-либо время. Если Вы назначите `android:id` Вашей `RadioGroup` в Вашем макете XML, вы можете иметь доступ к группе из Вашего Java кода и вызвать:

1. `check()` чтобы проверить определенную радио кнопку через ее ID (например, `group.check(R.id.radio1)`)
2. `clearCheck()` чтобы очистить все радио кнопки, так ничто в группе не выбрано

3 `getCheckedRadioButtonId()` чтобы получить ID текущей выбранной радио кнопки (или `-1` если ничего не выбрано)

Отметьте что функция взаимного исключения в `RadioGroup` применима только к виджетам `RadioButton` которые непосредственные потомки `RadioGroup`. Вы не можете иметь другие контейнера между `RadioGroup` и ее виджетов `RadioButton` .

Для примера, из проекта примера `Basic/RadioButton` , здесь макет XML отображающий `RadioGroup` который обертывает набор виджетов `RadioButton` :

```
<?xml version="1.0" encoding="utf-8"?>
<RadioGroup
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:orientation="vertical"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  >
    <RadioButton android:id="@+id/radio1"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/rock" />

    <RadioButton android:id="@+id/radio2"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/scissors" />

    <RadioButton android:id="@+id/radio3"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/paper" />
</RadioGroup>
```

(из Basic/RadioButton/app/src/main/res/layout/main.xml)

Используя стоковую сгенерированную Андроид Java для проекта и макета, Вы получите:

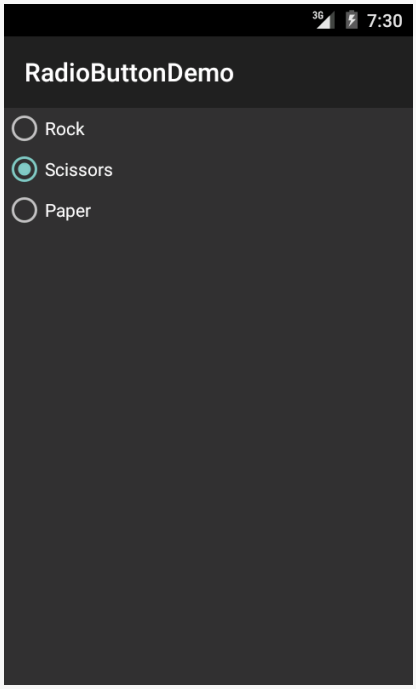


Рисунок 147: RadioButtonDemo, с выбранными "Scissors"

Отметьте, что группа радио кнопок изначально установлена в полностью не выбрано с самого начала. Чтобы предустановить одну из радио кнопок в выбрано, используйте либо `setChecked()` на `RadioButton` или `check()` на `RadioGroup` изнутри вашего вызова `onCreate()` вашей активности. Альтернативно, Вы можете использовать атрибут `android:checked` на одном из `RadioButton` виджетов в файле макета.

Графический редактор макетов Android Studio

Контейнер `RadioGroup` и виджет `RadioButton` может быть найден в разделе “Buttons” палитры в графическом редакторе макетов Android Studio:

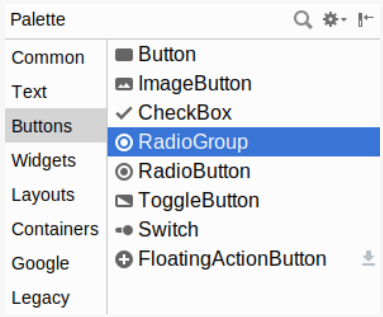


Рисунок 148: Палитра виджетов, подсвечена RadioGroup

Перетаскивание `RadioGroup` в предпросмотр работает в большинстве также как перетаскивание `LinearLayout` в предпросмотр. Вы получите контейнер в который вы можете переместить другие виджеты, такие как `RadioButton` находящиеся а разделе “Widgets” палитры.

Scrollwork

Экраны телефона имеют тенденцию быть маленькими, что требует от разработчиков применять некоторые трюки для предоставления большого количества информации в доступном ограниченном пространстве. Один из трюков, чтобы сделать это применить прокручивание, потому только часть информации видима одномоментно, остальная доступна через прокручивание вверх или вниз.

`ScrollView` это контейнер, который предоставляет прокручивание для своего содержимого. У Вас может быть макет, который слишком большой для некоторых экранов, оберните его в `ScrollView`, и все ещё используйте существующую логику макета. Так получается, что пользователь может видеть только часть Вашего макета одномоментно, остальное доступно через прокручивание.

Для примера, здесь `ScrollView` используемый в файле макета XML (из `Containers/ Scroll` demo):

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="wrap_content">
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:stretchColumns="0">
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#000000"/>
      <TextView android:text="#000000"
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#440000" />
      <TextView android:text="#440000"
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#884400" />
      <TextView android:text="#884400"
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#aa8844" />
      <TextView android:text="#aa8844"
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#ffaa88" />
      <TextView android:text="#ffaa88"
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#ffffaa" />
      <TextView android:text="#ffffaa"
```

```
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
    <TableRow>
      <View
        android:layout_height="80dip"
        android:background="#ffffff" />
      <TextView android:text="#ffffff"
        android:paddingLeft="4dip"
        android:layout_gravity="center_vertical" />
    </TableRow>
  </TableLayout>
</ScrollView>
```

(из Containers/Scroll/app/src/main/res/layout/main.xml)

Без `ScrollView`, таблица будет занимать как минимум 560 независимых от плотности пикселей (dip) (7 строк по 80 dip каждая, основываясь на описании `View`). Могут быть некоторые устройства с экранами, которые могут отображать столько информации, но многие будут меньше. `ScrollView` позволяет нам оставить таблицу как есть, но отображать только часть ее одномоментно.

На стоковом эмуляторе Андроид, когда активность отображена впервые, Вы увидите:

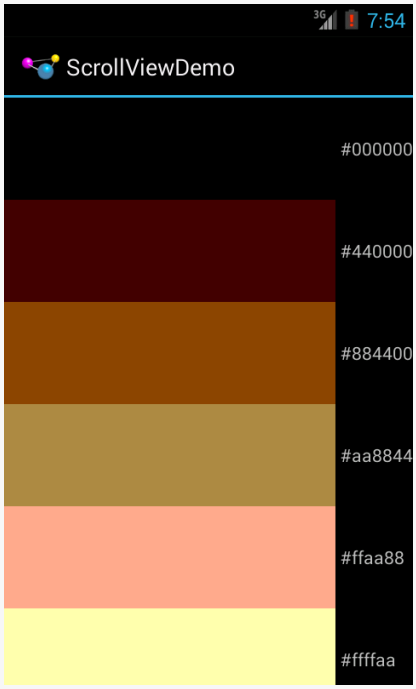


Рисунок 149: Приложение пример ScrollViewDemo

Отметьте, что только пять строк и часть шестой видимы. Вы можете прокрутить вверх и вниз чтобы увидеть оставшиеся строки. Также отметьте, как правая сторона содержимого обрезается полоской прокрутки - убедитесь что Вы добавили отступ на той стороне или наоборот убедитесь что Ваше содержимое не обрезается таким образом.

В Андроид также есть `HorizontalScrollView`, который работает как `ScrollView`... только горизонтально. Это будет хорошо для форм которые могут быть слишком широкие скорее чем слишком длинные. Отметьте что, `ScrollView` прокручивает только вертикально и `HorizontalScrollView` прокручивает только горизонтально.

Также, отметьте что вы не можете добавить элемент scrollable в `ScrollView`. Для примера виджет `ListView` - который мы просмотрим в следующей главе - также знает как прокручивать. Нет необходимости добавлять `ListView` в `ScrollView`, и если Вы попробуете, это не будет работать хорошо.

И `ScrollView` или `HorizontalScrollView` может иметь только одного потомка - если вы хотите больше одного, оберните потомков в подходящий класс

контейнер (например, а `LinearLayout`) и положите его внутрь `ScrollView` или `HorizontalScrollView`.

Графический редактор макетов Android Studio

Виджеты `ScrollView` и `HorizontalScrollView` находятся в разделе "Containers" палитры в графическом редакторе макетов. Вы можете перетащить один из них в ваш ресурсный XML макет, потом перетащить *один* потомок в него.

Создаём прогресс с ProgressBars

Если Вы собираетесь разделить выполнение работы в фоновых потоках на стороне пользователя, Вам захочется подумать об информировании пользователя о том, что работа выполняется. Это в особенности правдиво, если пользователь продуктивно ждет завершения той фоновой работы.

Типичная задача это информировать пользователя о прогрессе в какой-либо форме, используя прогресс-бар, как Вы можете видеть при копировании набора файлов с одного места в другое на многих операционных системах компьютера. Андроид поддерживает это через виджет `ProgressBar` .

`ProgressBar` отслеживает прогресс, определенный как целочисленное значение с `0` , указывающим на отсутствие сделанного прогресса. Вы можете указать максимальное значение конца интервала - которое показывает завершение прогресса - с помощью `setMax()`. По умолчанию `ProgressBar` начинается со значения прогресса `0`, хотя вы можете начать с некоторой другой позиции с помощью `setProgress()`. Андроид 8.0 добавляет `setMin()` и `android:min` для установки нижнего значения интервала в пользовательское значение вместо значения `0` по умолчанию.

Если вы предпочитаете чтобы Ваш прогресс бар был неопределенным - что означает что он будет отображать общий эффект анимации, а не определенное значение прогресса - используйте `setIndeterminate()` установленное в значение `true`.

В Вашем Java коде Вы можете или установить положительное значение прогресса, который был выполнен (через `setProgress()`) или увеличить прогресс от его текущего значения (через `incrementProgressBy()`). Вы можете узнать сколько прогресса было сделано через `getProgress()`.

Обрамление Scene

В Андроид есть класс `FrameLayout` . Как и `LinearLayout`,

`RelativeLayout`, и `TableLayout`, `FrameLayout` существует для того, чтобы задать размер и местоположение его потомков. Тем не менее `FrameLayout` имеет очень простые пару правил макета:

1. Все потомки располагаются в верхнем начальном углу (например, верхний левый дл LTR языков),

пока `android:gravity` не укажет расположение потомков где-либо

2. Более поздние потомки выше по оси Z, чем ранние потомки, как и с `RelativeLayout`, потомки могут накладываться

В результате все виджеты складываются один поверх другого.

Это может казаться бесполезным. И, по правде говоря, это почти не используется так, как обычно используются другие контейнеры.

В основном, `FrameLayout` используется в местах, где мы хотим зарезервировать пространство для чего-то, но мы не знаем для чего именно во время компиляции. Решение того, что такое "что-то" будет принято во время выполнения, где мы будем использовать Java код чтобы положить что-то в `FrameLayout`. Мы рассмотрим использование этого паттерна с фрагментами позже в этой книге.

Иногда `FrameLayout` непосредственно используется для "обрамления", когда мы хотим некоторый тип границы вокруг потомка. В этом случае фон `FrameLayout` (например, `android:background`) определяет как рамка должна выглядеть. Мы увидим использование этого подхода в нескольких местах, таких как глава о добавлении drag-and-drop в ваше приложение.

Посетите пути!

Часть пути этой книги содержит каталог виджетов, предоставляющий краткое описание и примеры набора виджетов, которые не описаны где-либо еще в этой книге.

Вы также можете заинтересоваться `GridLayout`, который является альтернативой классическим
контейнерам `LinearLayout`, `RelativeLayout`, и `TableLayout`.

Руководство №5 - Создание макета

Позже в этом руководстве мы позволим пользователю писать заметки относительно книги. Эти заметки будут сохранены в базе данных и могут быть просмотрены, изменены или удалены как пользователь считает нужным.

В этом руководстве мы настроим ресурс макета, что позволит пользователю заполнить эти заметки.

Это продолжение работы, выполненной в предыдущем руководстве.

Вы можете найти результаты предыдущего руководства и результаты данного руководства в GitHub репозитории книги.

Шаг №1: Создание нового ресурса макета

Сделайте правый щелчок над папкой `res/layout/` и выберите New > “Layout resource file” из контекстного меню. Это отобразит диалог New Layout Resource File:

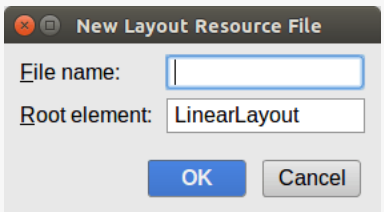


Рисунок 150: New Layout Resource File диалог в Android Studio

Заполните поле “Layout File Name” значением `editor` , оставьте остальное в диалоге как есть, и нажмите кнопку “OK”.

Шаг №2: Определение UI

Это откроет графический редактор макета для этого нового ресурса макета `editor` :

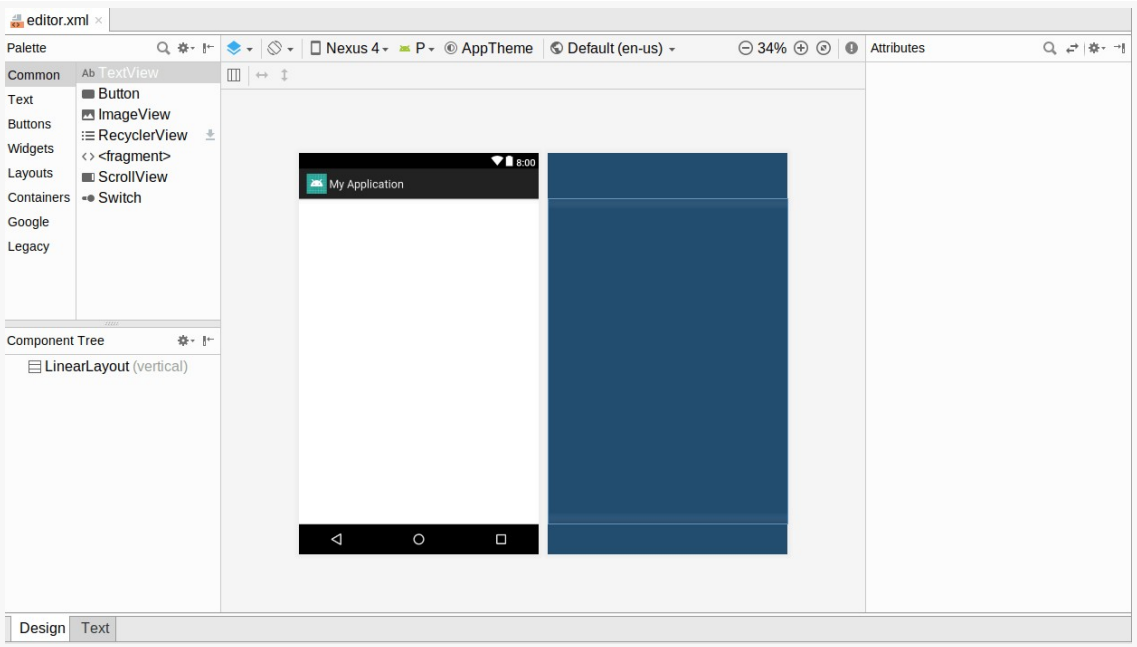


Рисунок 151: Графический редактор макетов

Перенесите виджет “Multiline Text” из раздела Text Палитры в область предпросмотра. В панели атрибутов, измените `android:layout_width` и `android:layout_height` каждый на `match_parent` и измените ID на `editor`:

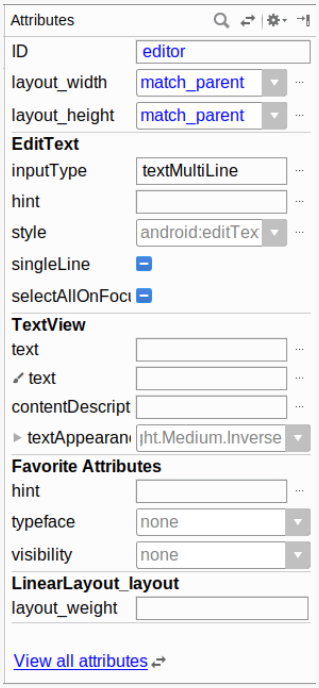


Рисунок 152: Графический редактор макетов Панель атрибутов

Далее, в Панели атрибутов щелкните на поле “hint”, затем щелкните на кнопку “…” справа от него. Это откроет диалог выбора строкового ресурса:

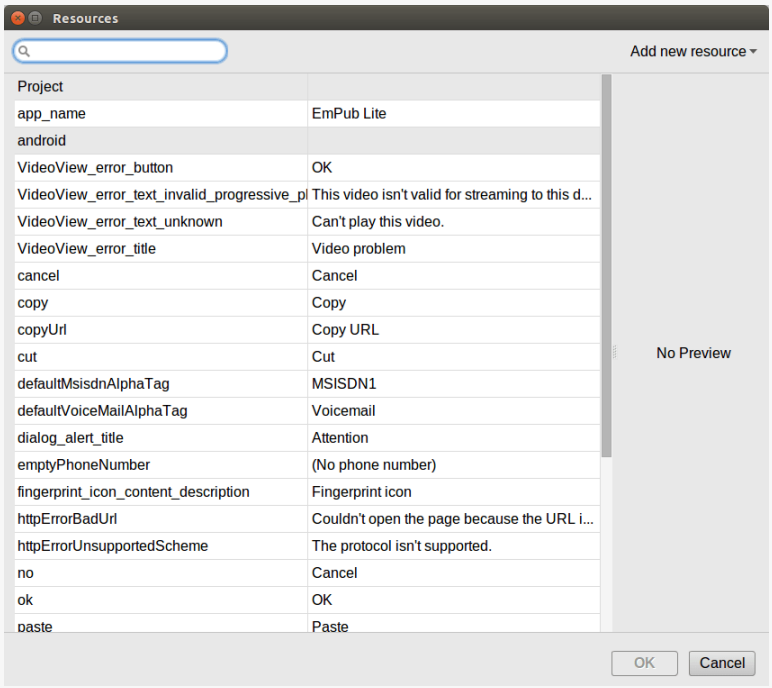


Рисунок 153: Диалог выбора строкового ресурса Android Studio

Справа вверху щелкните выпадающее меню “Add new resource” и выберите из него “New string Value...” чтобы отобразить диалог редактора строковых ресурсов:

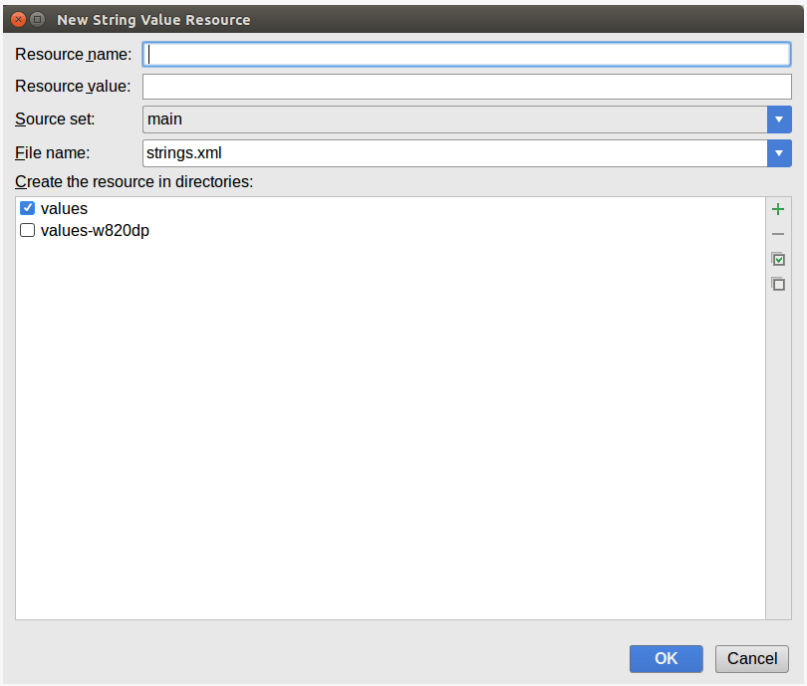


Рисунок 154: Диалог нового строкового ресурса Android Studio

Заполните название ресурса как `hint` и значение как `Enter notes here` Оставьте остальное в диалоге как есть и щелкните OK.

Затем, вернитесь в панель атрибутов, переключите на просмотр всех атрибутов, а не их части. Прокрутите вниз до свойства `gravity` , раскройте его и измените отмеченные значения на “top” и “start”.

Если Вы посмотрите на XML макета в подвкладке Text, Вы должны получить что-то похожее на это:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:orientation="vertical" android:layout_width="match_parent"
  android:layout_height="match_parent">

  <EditText
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:inputType="textMultiLine"
    android:ems="10"
    android:id="@+id/editor"
    android:hint="@string/hint"
    android:gravity="top|start" />
</LinearLayout>
```

(из EmPubLite-AndroidStudio/T5-Layout/EmPubLite/app/src/main/res/layout/editor.xml)

Ваш вспомогательный текст (hint) будет `"Enter notes here"`, как если бы Вы напрямую набрали его, а не создали строковый ресурс. Как было рассмотрено ранее, Android Studio обманывает Вас. Щелкните на `"Enter notes here"` чтобы увидеть фактическую ссылку на строковый ресурс.

В нашем следующем эпизоде...

... мы добавим сторонние библиотеки в наш учебный проект.

Построение GUI, продолжение

Если Вы используете IDE, и эксперементировали с графическим редактором макетов и построением GUI перетаскиванием, эта глава рассмотрит некоторые другие основные функции редактора, которые Вы возможно найдете полезными.

Даже если Вы не используете IDE, Вы можете хотя бы просмотреть эту главу, чтобы найти несколько трюков, которые также будут релевантны для Вас.

Создание собственных Selection

Нажатие на виджет делает его выбранным виджетом, что означает, что кнопки панели инструментов будут влиять на этот виджет (или, иногда, его контейнеру, в зависимости от кнопки).
Выбранные виджеты имеют тонкую синюю границу с синим квадратом “grab handles” для изменения их размера и местоположения.

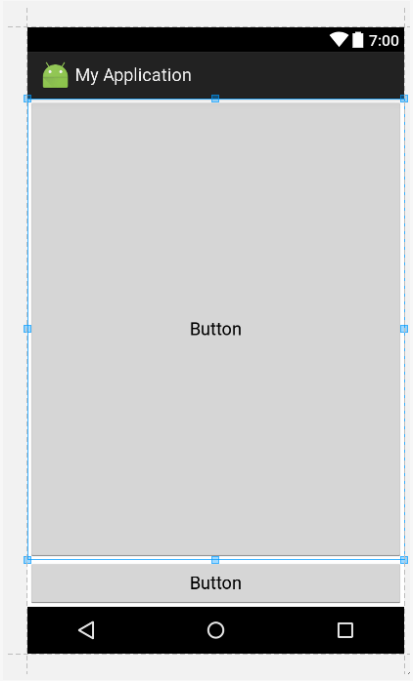


Рисунок 155: Android Studio, выбранный виджет в графическом

Нажатие на контейнер также выбирает его. Тем не менее, синей границы может быть или не быть - в частности, контейнеры, которые заполняют экран (`match_parent` для ширины и высоты) не имеют границы.

Иногда, также, Вы можете захотеть выбрать контейнер, до которого не сможете добраться, потому что его содержимое полностью заполнено виджетами. В этих случаях, нажмите на виджет или контейнер в панели дерево компонентов, чтобы выбрать его.

Включения включаются

Иногда, у Вас есть виджет или набор виджетов, который вы захотите

переиспользовать во многих других ресурсах XML макетов. Android поддерживает возможность добавить “include”

что позволяет это сделать. Просто создайте необходимый ресурс XML макета, который содержит виджет(ы) для переиспользования, затем добавьте их в Ваши основные макеты через элемент `<include>` :

```
<include layout="@layout/thing_we_are_reusing" />
```

Вы можете даже назначить элементу `<include>` ширину и высоту, если необходимо, как если бы это был просто виджет или контейнер.

IDE делает это более простым для Вас, взять виджеты из существующего ресурса XML макета и извлечь их в отдельный ресурс XML макета, заменяя их элементом
`<include>`. Выберите несколько виджетов на отображениях чертежа или дизайна, или в инструменте предпросмотра на подвкладке Text редактора макетов. Затем нажмите правую кнопку и выберите Refactor > Extract Layout из контекстного меню. Это отобразит диалог, где Вы можете заполнить название файла Вашего полученного в результате ресурса:

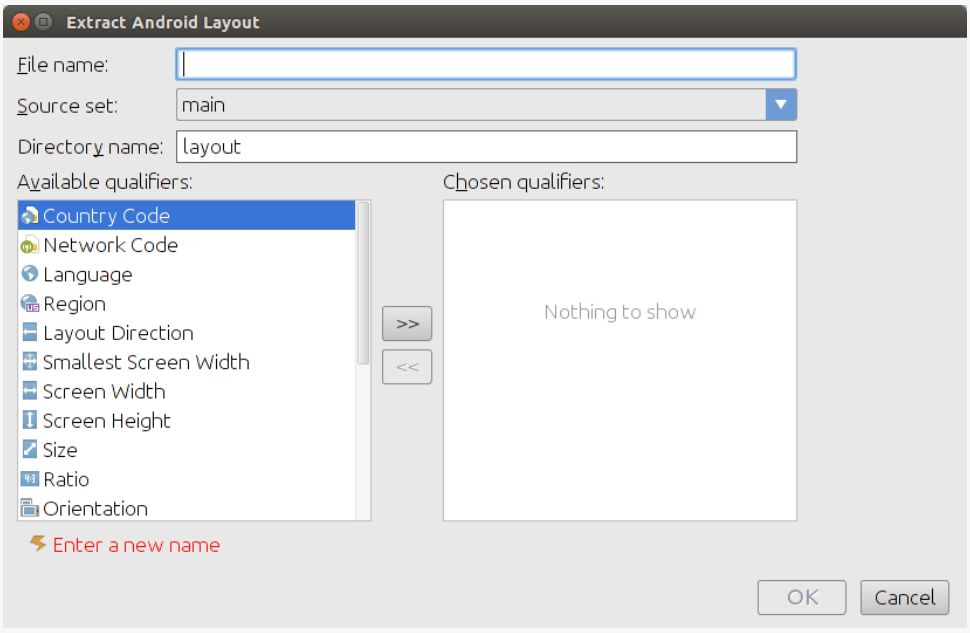


Рисунок 156: Диалог извлечения макета в Android Studio

Если Вы извлекаете несколько виджетов, которые не обернуты в их собственный контейнер, IDE автоматически обернет их элементом `<merge>` :

```
<?xml version="1.0" encoding="utf-8"?>
<merge xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- widgets go here -->
</merge>
```

Это необходимо исключительно с точки зрения XML — Вы не можете иметь несколько корневых элементов в файле XML. Когда `<merge>` добавлен в другой макет через
`<include>`, элемент `<merge>` сам испаряется, оставляя потомков.

Предварительный просмотр предстоящих Attractions

Вверху графического редактора макета, Вы найдете набор выпадающих меню, который позволит Вам установить как выглядит предпросмотр:

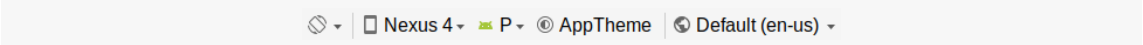


Рисунок 157: Управление предпросмотром в Android Studio

Ваш IDE выберет некоторые вероятные значения по умолчанию на основе настроек Вашего проекта, но Вы можете изменить их так, как Вы считаете нужным. Примечательные изменения включают в себя:

- Какая версия Android используется для предварительного просмотра (так как время от времени изменяется стиль виджетов в выпусках Android)
- Какой язык используется для ваших строковых ресурсов?
- Какой размер и разрешение экрана используется?
- Отображение в книжном формате или альбомном?

Это влияет только на предварительный просмотр, они показывают Вам (приблизительно) как Ваш макет будет выглядеть в этих условиях, но они ничего не изменяют в Вашем макете XML сами.

AdapterViews и Adapters

Если Вы хотите, чтобы пользователь выбирал что-то из коллекции чего-то, Вы можете использовать набор виджетов `RadioButton` . Тем не менее, Android имеет серии более гибких виджетов чем они, особенно для сценариев, где коллекция не известна когда Вы пишите Ваше приложение: результаты вызова веб сервиса, результаты запроса к базе данных, и т.д.

Популярные визуальные представления коллекций включают:

- Вертикально прокручиваемый список строк, где каждая строка представляет элемент
- Вертикально прокручиваемые сетки элементов, где каждая горизонтальная “строка” содержит назначенное количество элементов, поделенные на столбцы
- Выпадающие списки
- Деревья

У Android есть два основных решения для этого типа задач. `RecyclerView` — более новое решение. Для списков, сеток и деревьев Вы должны начать с рассмотрения `RecyclerView`.
Тем не менее, `RecyclerView` поставляется в виде библиотеки и Вы часто используете другие библиотеки, чтобы расширить его (например, для поддержки деревьев). Так как, мы еще пока не рассмотрели библиотеки, мы воздержимся от обсуждения `RecyclerView` до некоторых пор в этой книге.

Стандартное решение задействует подклассы `AdapterView`, такие как

- `ListView`, который является Вашим типичным “list box”
- `Spinner`, который (в большей или меньшей степени) является выпадающим списком
- `GridView`, предлагает двухмерный список вариантов
- `ExpandableListView`, ограниченный виджет “дерева”, поддерживающий два уровня в иерархии

и многие другие.

По их сути, это обычные виджеты. Вы найдете их в Вашей палитре инструментов Вашего графического редактора макетов IDE и можете перенести их и расположить их как Вам удобно.

Их базовый класс `AdapterView` так назван, потому что он находится в паре с объектами, которые имплементируют интерфейс `Adapter`, который определяет варианты, доступные для выбора пользователя.

`RecyclerView` также использует адаптеры, хотя и с немного другим API, чем тот, что использует `AdapterView` И существуют несколько сценариев - такие как выпадающий список - где `RecyclerView` на самом деле не вариант, и где `AdapterView` (особенно `Spinner`) будет лучшим выбором. Так, в данной главе, мы рассмотрим семейство `AdapterView` , частично из исторических причин, частично для основы для изучения `RecyclerView`, и частично для последующего использования в определенных сценариях.

Адаптация к условиям

`Adapter` это Ваш мост между данными вашей модели и визуальным представлением данных в `AdapterView`:

- `Adapter` должен "адаптировать" `Запрос` в `Отображение` которое будет служить как строка в `ListView`
- `Adapter` должен "адаптировать" `Книгу` в `Отображение` которое будет служить ячейко `GridView`
- и так далее.

Android поставляется с несколькими классами `Adapter` готовых для Вашего использования, где разные классы адаптера спроектированы для "адаптации" различных типов коллекций (например, массивы против результатов запросов в базу данных). Android также содержит класс `BaseAdapter` который может служить как основание для Вашей собственной имплементации `Adapter` , если Вам необходимо "адаптировать" набор данных, который не совпадает с какими либо классами `Adapter` предоставляемые Android.

Использование ArrayAdapter

Самый простой адаптер для использования это `ArrayAdapter` - все, что Вам нужно это обернуть один из них вокруг массива Java или экземпляра `java.util.List` , и Вы получите полностью работоспособный адаптер:

```
String[] items={"this", "is", "a", "really", "silly", "list"};
```

```
new ArrayAdapter<String>(this,
                         android.R.layout.simple_list_item_1,
                         items);
```

Одна из реализаций конструктора `ArrayAdapter` принимает три параметра:

1. `Context` для использования (обычно это будет экземпляр Вашей активности)
2. Ресурс ID отображения для использования (такой как встроенный ID ресурса системы, как показано выше)
3. Актуальный массив или список элементов для отображения

По умолчанию, `ArrayAdapter` вызывает `toString()` на объекте в списке и оборачивает каждую из этих строк в отображении назначенную предложенным ресурсом. `android.R.layout.simple_list_item_1` просто превращает эти строки в объекты`TextView` . Those `TextView` widgets, in turn, will be shown in the list or spinner or whatever widget uses this `ArrayAdapter`. If you want to see what `android.R.layout.simple_list_item_1` looks like, you can find a copy of it in your SDK installation — just search for `simple_list_item_1.xml`.

We will see in a later section how to subclass an `Adapter` and override row creation, to give you greater control over how rows and cells appear.

Список Naughty и Nice

The classic listbox widget in Android is known as `ListView`. Include one of these in your layout, invoke `setAdapter()` to supply your data and child views, and attach a listener via `setOnItemSelectedListener()` to find out when the selection has changed. With that, you have a fully-functioning listbox.

However, if your activity is dominated by a single list, you might well consider creating your activity as a subclass of `ListActivity`, rather than the regular `Activity` base class. If your main view is just the list, you do not even need to supply a layout — `ListActivity` will construct a full-screen list for you. If you do want to customize the layout, you can, so long as you identify your `ListView` as `@android:id/list`, so `ListActivity` knows which widget is the main list for the activity.

For example, here is a layout pulled from the `Selection/List` sample project:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
```

```
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:orientation="vertical"
  android:layout_width="match_parent"
  android:layout_height="match_parent" >
  <TextView
    android:id="@+id/selection"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
  <ListView
    android:id="@android:id/list"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    />
</LinearLayout>
```

(from Selection/List/app/src/main/res/layout/main.xml)

It is just a list with a label on top to show the current selection.

The Java code to configure the list and connect the list with the label is:

```
package com.commonsware.android.list;

import android.app.ListActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.TextView;

public class ListViewDemo extends ListActivity {
  private TextView selection;
  private static final String[] items={"lorem", "ipsum", "dolor",
          "sit", "amet",
          "consectetuer", "adipiscing", "elit", "morbi", "vel",
          "ligula", "vitae", "arcu", "aliquet", "mollis",
          "etiam", "vel", "erat", "placerat", "ante",
          "porttitor", "sodales", "pellentesque", "augue", "purus"};

  @Override
  public void onCreate(Bundle state) {
    super.onCreate(state);
    setContentView(R.layout.main);
    setListAdapter(new ArrayAdapter<String>(this,
                        android.R.layout.simple_list_item_1,
                        items));
    selection=(TextView)findViewById(R.id.selection);
```

```
  }

  @Override
  public void onListItemClick(ListView parent, View v, int position,
                              long id) {
    selection.setText(items[position]);
  }
}
```

(from Selection/List/app/src/main/java/com/commonsware/android/list/ListViewDemo.java)

With `ListActivity`, you can set the list adapter via `setListAdapter()` — in this case, providing an `ArrayAdapter` wrapping an array of Latin strings. To find out when the list selection changes, override `onListItemClick()` and take appropriate steps based on the supplied child view and position (in this case, updating the label with the text for that position).

The results?

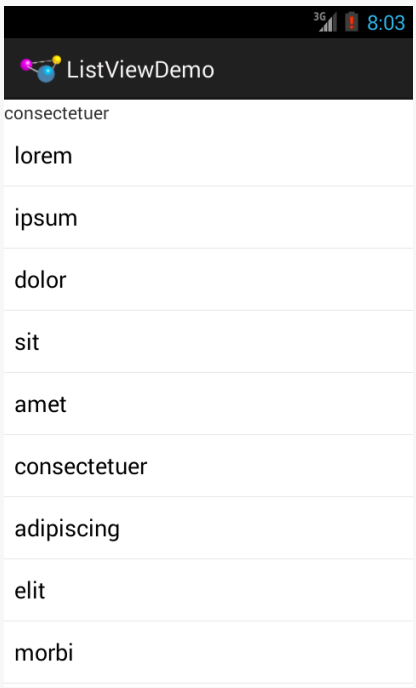


Figure 158: ListViewDemo, After User Taps on "consecteteur"

The second parameter to our `ArrayAdapter` — `android.R.layout.simple_list_item_1` — controls what the rows look like. The value used in the preceding example provides the standard Android list row: a big